

Основные понятия и определения

Термин «база данных» (database) был введен в обиход в области вычислительной техники примерно в 1962 году. Этот термин страдает от обилия различных интерпретаций. Примеры определения термина «баз данных» из авторитетных источников:

База данных (БД) - совокупность данных, организованных по определенным правилам, предусматривающая общие принципы описания, хранения и манипулирования данными, независимая от прикладных программ. Является информационной моделью предметной области.

[К. Дейт, Введение в системы баз данных, 1980]

База данных – совокупность хранимых операционных данных, используемых прикладными системами некоторого предприятия.

[К. Дейт, Введение в системы баз данных, 6-е издание, 2000]

База данных состоит из некоторого набора постоянных данных, которые используются прикладными системами для какого-либо предприятия.

[К. Дейт, Введение в системы баз данных, 7-е издание, 2001]

База данных – это некоторый набор перманентных (постоянных) данных, используемых прикладными системами какого-либо предприятия.

1. Операционные данные – мир OnLine Transaction Processing (Оперативная обработка транзакций) или **OLTP**.
2. Перманентные данные – мир OnLine Analitical Processing (Аналитическая обработка в реальном времени) или **OLAP**. OLAP используется в системах поддержки принятия решений (Decision Support System – DSS) и управленческих информационных системах (Executive Information System – EIS). Базу данных для поддержки принятия решения обычно называют хранилищем данных (data warehouse).

[Джефффри Д. Ульман, Дженнифер Уидом, Введение в системы баз данных, 2000]

По сути дела БД - это просто множество информации, существующее долгое время, часто в течении многих лет. Обычно термином "база данных" обозначается множество данных, управляемое СУБД, или просто СБД.

[Д. Кренке, Теория и практика построения баз данных, 2003]

База данных – это самодокументированное собрание интегрированных записей. Важно понять обе части этого определения.

1. БД является самодокументированной (self describing): она содержит описание собственной структуры. Это описание называется словарем данных (data dictionary), каталогом данных (data directory) или метаданными (metadata).
2. БД – это собрание интегрированных записей: она содержит:
 1. Файлы данных,
 2. Метаданные,
 3. Индексы (indexes), которые представляют связи между данными, а также служат для повышения производительности приложений базы данных.
 4. Может содержать метаданные приложений (application metadata).
3. БД является информационной моделью пользовательской модели (user model) предметной области.

База данных - совокупность взаимосвязанных данных некоторой предметной области, хранимых в памяти ЭВМ и организованных таким образом, что эти данные могут быть использованы для решения многих задач многими пользователями. Основные требования к организации данных:

1. **Неизбыточность данных** - каждое данное присутствует в БД в единственном экземпляре.
2. **Совместное использование данных** многими пользователями.
3. **Эффективность доступа к БД** - высокое быстродействие, т. е. малое время отклика на запрос.
4. **Целостность данных** - соответствие имеющейся в БД информации её внутренней логике, структуре и всем явно заданным правилам.
5. **Безопасность данных** – защита данных от преднамеренного или непреднамеренного искажения или разрушения данных.
6. **Восстановление данных** после программных и аппаратных сбоев.
7. **Независимость данных** от прикладных программ.

Существует множество других определений, отражающих скорее субъективное мнение тех или иных авторов о том, что означает база данных в их понимании, однако общепризнанная единая формулировка отсутствует.

Система управления базами данных (СУБД) - приложение, обеспечивающее создание, хранение, обновление и поиск информации в базах данных.

Система баз данных (или банк данных в отечественной терминологии) - совокупность одной или нескольких баз данных и комплекса информационных, программных и технических средств, обеспечивающих накопление, обновление, корректировку и многоаспектное использование данных в интересах пользователей. (См. Григорьев Ю.А., Ревунков Г.И. Банки данных. – М.: МГТУ, 2002.)

Состав системы баз данных

- Одна или несколько баз данных.
- Аппаратное обеспечение.
- Программное обеспечение.
 - СУБД.
 - Прочие компоненты: утилиты, средства разработки приложений, средства проектирования, генераторы отчетов, диспетчер транзакций и т. д.
- Пользователи.
 - Прикладные программисты, которые отвечают за написание прикладных пакетных и интерактивных программ.
 - Конечные пользователи, которые получают доступ к базе данных, применяя одно из интерактивных приложений, интерфейс, интегрированный в программное обеспечение самой СУБД, командный интерфейс, реализуемый процессором языка запросов или некомандный интерфейс, основанный на меню и формах.
 - Администраторы базы данных.

Основные функции СУБД

- **Непосредственное управление данными во внешней памяти.** В развитых СУБД пользователи не обязаны знать, как организованы файлы данных.
- **Управление буферами оперативной памяти.** Для увеличения скорости обмена данными с внешней памятью используется буферизация данных в оперативной памяти. В развитых СУБД поддерживается собственный набор буферов оперативной памяти с собственной дисциплиной замены буферов, даже если операционная система

производит общесистемную буферизацию. Существует даже отдельное направление СУБД, которое ориентировано на постоянное присутствие в оперативной памяти всей БД. Это направление основывается на предположении, что объем оперативной памяти компьютеров настолько велик, что можно не беспокоиться о буферизации.

- **Управление транзакциями.** Транзакция – это последовательность операций над данными, рассматриваемая СУБД как единое целое. Реализуется принцип «либо все, либо ничего». Поддержание механизма транзакций является обязательным условием даже однопользовательских СУБД. Но понятие транзакции гораздо более важно в многопользовательских СУБД. То свойство, что каждая транзакция начинается при целостном состоянии БД и оставляет это состояние целостным после своего завершения, делает удобным использование понятия транзакции как единицы активности пользователя по отношению к БД.
- **Журнализация.** Одним из основных требований к СУБД является требование надежного хранения данных во внешней памяти. Под надежностью хранения понимается то, что СУБД должна быть в состоянии восстановить последнее согласованное состояние БД после любого аппаратного или программного сбоя. Обычно рассматриваются два возможных вида аппаратных сбоев: так называемые мягкие сбои, которые можно трактовать как внезапную остановку работы компьютера (например, аварийное выключение питания), и жесткие сбои, характеризующиеся потерей информации на носителях внешней памяти. В любом случае для восстановления БД нужно располагать некоторой дополнительной информацией. Наиболее распространенным методом поддержания такой избыточной информации является ведение журнала изменений БД. Журнал – это особая часть БД, недоступная пользователям СУБД (иногда поддерживаются две копии журнала, располагаемые на разных физических носителях), в которую поступают записи обо всех изменениях основной части БД.
- **Поддержка языков БД.** Для работы с БД используются специальные языки, называемые языками баз данных. В ранних СУБД (иерархических и сетевых) поддерживалось несколько специализированных по своим функциям языков. В современных СУБД (реляционных) поддерживается язык SQL (Structured Query Language), содержащий все необходимые средства для работы с БД, начиная от ее создания, и обеспечивающий базовый пользовательский интерфейс.

Основные компонентами СУБД

- Ядро, которое отвечает за управление данными во внешней и оперативной памяти и журнализацию.
- Процессор языка базы данных, обеспечивающий оптимизацию запросов на извлечение и изменение данных, и создание, как правило, машинно-независимого исполняемого внутреннего кода,
- Подсистема поддержки времени исполнения, которая интерпретирует программы манипуляции данными, создающие пользовательский интерфейс с СУБД.
- Сервисные программы (внешние утилиты), обеспечивающие ряд дополнительных возможностей по обслуживанию информационной системы.

Классификация СУБД

Классификация СУБД может выполняться по нескольким признакам:

1. По модели данных:

1. Реляционные

1. **Инвертированные списки (файлы).** БД на основе инвертированных списков представляет собой совокупность файлов, содержащих записи (таблиц). Для записей в файле определен некоторый порядок, диктуемый физической организацией данных. Для каждого файла может быть определено произвольное число других упорядочений на основании значений некоторых полей записей (инвертированных списков). Обычно для этого используются индексы. В такой модели данных отсутствуют ограничения целостности как таковые. Все

ограничения на возможные экземпляры БД задаются теми программами, которые работают с БД. Одно из немногих ограничений, которое все-таки может присутствовать - это ограничение, задаваемое уникальным индексом.

2. **Иерархические.** Иерархическая модель БД состоит из объектов с указателями от родительских объектов к потомкам, соединяя вместе связанную информацию. Иерархические БД могут быть представлены как дерево.
3. **Сетевые.** К основным понятиям сетевой модели БД относятся: элемент (узел), связь. Узел — это совокупность атрибутов данных, описывающих некоторый объект. Сетевые БД могут быть представлены в виде графа. В сетевой БД логика процедуры выборки данных зависит от физической организации этих данных. Поэтому эта модель не является полностью независимой от приложения. Другими словами, если необходимо изменить структуру данных, то нужно изменить и приложение.
2. **Реляционные.** Реляционная модель данных включает следующие компоненты:
 1. Структурный (данные в базе данных представляют собой набор отношений),
 2. Целостностный (отношения (таблицы) отвечают определенным условиям целостности),
 3. Манипуляционный (манипулирование отношениями осуществляется средствами реляционной алгебры и/или реляционного исчисления). Кроме того, в состав реляционной модели данных включают теорию нормализации. В 1985 году доктор Кодд сформулировал двенадцать правил, которым должна соответствовать настоящая реляционная база данных. Они являются полуофициальным определением понятия реляционная база данных. Строгое изложение теории реляционных баз данных (реляционной модели данных) в современном понимании можно найти в книге К. Дж. Дейта.

3. Постреляционные

2. По архитектуре организации хранения данных

1. локальные СУБД (все части локальной СУБД размещаются на одном компьютере)
2. распределенные СУБД (части СУБД могут размещаться на двух и более компьютерах)

3. По способу доступа к БД (а может быть по местоположению БД):

1. **Файл-серверные.** При работе в архитектуре "файл-сервер" БД и приложение расположены на файловом сервере сети. Возможна многопользовательская работа с одной и той же БД, когда каждый пользователь со своего компьютера запускает приложение, расположенное на сетевом сервере. Тогда на компьютере пользователя запускается копия приложения. По каждому запросу к БД из приложения данные из таблиц БД перегоняются на компьютер пользователя, независимо от того, сколько реально нужно данных для выполнения запроса. После этого выполняется запрос. Каждый пользователь имеет на своем компьютере локальную копию данных, время от времени обновляемых из реальной БД, расположенной на сетевом сервере. При этом изменения, которые каждый пользователь вносит в БД, могут быть до определенного момента неизвестны другим пользователям, что делает актуальной задачу систематического обновления данных на компьютере пользователя из реальной базы данных. Другой актуальной задачей является блокирование записей, которые изменяются одним из пользователей; это необходимо для того, чтобы в это время другой пользователь не внес изменений в те же данные. В архитектуре "файл-сервер" вся тяжесть выполнения запросов к базе данных и управления целостностью базы данных ложится на приложение пользователя. База данных на сервере является пассивным источником данных.
2. **Клиент-серверные.** Клиент-сервер - сетевая архитектура, в которой устройства являются либо клиентами, либо серверами. Клиентом (front end) является запрашивающая машина, сервером (back end) - машина, которая отвечает на запрос. Оба термина (клиент и сервер) могут применяться как к физическим устройствам, так и к программному обеспечению. Характерной особенностью архитектуры "клиент-сервер" является перенос вычислительной нагрузки на сервер базы данных (sql-сервер) и максимальная разгрузка приложения клиента от вычислительной работы, а также существенное укрепление безопасности данных – как от злонамеренных, так и

просто ошибочных изменений. БД в этом случае помещается на сетевом сервере, как и в архитектуре "файл-сервер", однако прямого доступа к базе данных из приложений не происходит. Функцию прямого обращения к базе данных осуществляет СУБД. При этом ресурсы клиентского компьютера не участвуют в физическом выполнении запроса; клиентский компьютер лишь отправляет запрос к СУБД и получает результат, после чего интерпретирует его необходимым образом и представляет пользователю. Так как клиентскому приложению посылается результат выполнения запроса, по сети "путешествуют" только те данные, которые необходимы клиенту. В итоге снижается нагрузка на сеть. Кроме того, СУБД, если это возможно, оптимизирует полученный запрос таким образом, чтобы он был выполнен в минимальное время с наименьшими накладными расходами. При выполнении запросов сервером существенно повышается степень безопасности данных, поскольку правила целостности данных определяются в базе данных на сервере и являются едиными для всех приложений, использующих эту БД. Таким образом, исключается возможность определения противоречивых правил поддержания целостности. Аппарат транзакций, поддерживаемый СУБД, позволяет исключить одновременное изменение одних и тех же данных различными пользователями и предоставляет возможность откатов к первоначальным значениям при внесении в БД изменений, закончившихся аварийно.

3. **Встраиваемые.** Встраиваемая СУБД — библиотека, которая позволяет унифицированным образом хранить большие объёмы данных на локальной машине. Доступ к данным может происходить через SQL либо через особые функции СУБД. Встраиваемые СУБД быстрее обычных клиент-серверных и не требуют установки сервера, поэтому востребованы в локальном ПО, которое имеет дело с большими объёмами данных.
4. **Сервисно-ориентированные.** БД является хранилищем сообщений, промежуточных состояний, метаданных об очередях сообщений и сервисах. Отправка сообщений в очередь и прием сообщений из очереди производится в одной транзакции с изменением данных, что обеспечивает транзакционную целостность системы. Так как очереди сообщений и данные хранятся и обрабатываются в базе единообразно, это обеспечивает гарантированную доставку и обработку сообщений в случае сбоев оборудования или питания с таким же успехом, как и прочих данных, хранящихся в той же базе данных. Кроме этого, в базе данных хранится информация о самих сервисах и обрабатываемых ими очередях сообщений, что обеспечивает восстановление после сбоя состояний не только данных и сообщений, но и настроек сервисов и очередей сообщений.
5. **Прочие.** Пространственная (spatial database), Временная, или темпоральная (temporal database), Пространственно-временная (spatial-temporal database).

Семантическое моделирование данных

Реляционная модель данных достаточна для моделирования предметных областей. Однако проявляется ограниченность реляционной модели данных в следующих аспектах:

- Модель не предоставляет достаточных средств для представления смысла данных.
- Для многих приложений трудно моделировать предметную область на основе плоских таблиц.
- Хотя весь процесс проектирования происходит на основе учета зависимостей, реляционная модель не предоставляет каких-либо средств для представления этих зависимостей.
- Несмотря на то, что процесс проектирования начинается с выделения некоторых существенных для приложения объектов предметной области («сущностей») и выявления связей между этими сущностями, реляционная модель данных не предлагает какого-либо аппарата для разделения сущностей и связей.

Указанные ограничения вызвали к жизни направление семантических (концептуальных, инфологических) моделей данных. Любая развитая семантическая модель данных, как и реляционная модель, включает структурную, манипуляционную и целостную части. Главным назначением семантических моделей является обеспечение возможности выражения семантики данных. На практике семантическое моделирование используется на первой стадии проектирования базы данных. При этом в терминах семантической модели производится концептуальная схема базы данных, которая затем

- Либо вручную преобразуется к реляционной (или какой-либо другой) схеме.
- Либо реализуется автоматизированная компиляция концептуальной схемы в реляционную.
- Либо происходит работа с базой данных в семантической модели, т.е. под управлением СУБД, основанных на семантических моделях данных. (Третья возможность еще не вышла за пределы исследовательских и экспериментальных проектов.)

Наиболее известным представителем класса семантических моделей предметной области является модель «сущность-связь» или ER-модель, предложенная Питером Ченом в 1976 году. Модель сущность-связь — модель данных, позволяющая описывать концептуальные схемы предметной области. Предметная область — часть реального мира, рассматриваемая в пределах данного контекста. Под контекстом здесь может пониматься, например, область исследования или область, которая является объектом некоторой деятельности. ER-модель используется при высокоуровневом (концептуальном) проектировании баз данных. С её помощью можно выделить ключевые сущности и обозначить связи, которые могут устанавливаться между этими сущностями. Во время проектирования баз данных происходит преобразование ER-модели в конкретную схему базы данных на основе выбранной модели данных (реляционной, объектной, сетевой или др.). ER-модель представляет собой формальную конструкцию, которая сама по себе не предписывает никаких графических средств её визуализации. В качестве стандартной графической нотации, с помощью которой можно визуализировать ERM, была предложена диаграмма сущность-связь (entity-relationship diagram, ERD). На практике понятия ER-модель и ER-диаграмма часто не различают, хотя для визуализации ER-моделей предложены и другие графические нотации. Основными понятиями ER-модели являются сущность, связь и атрибут (свойство).

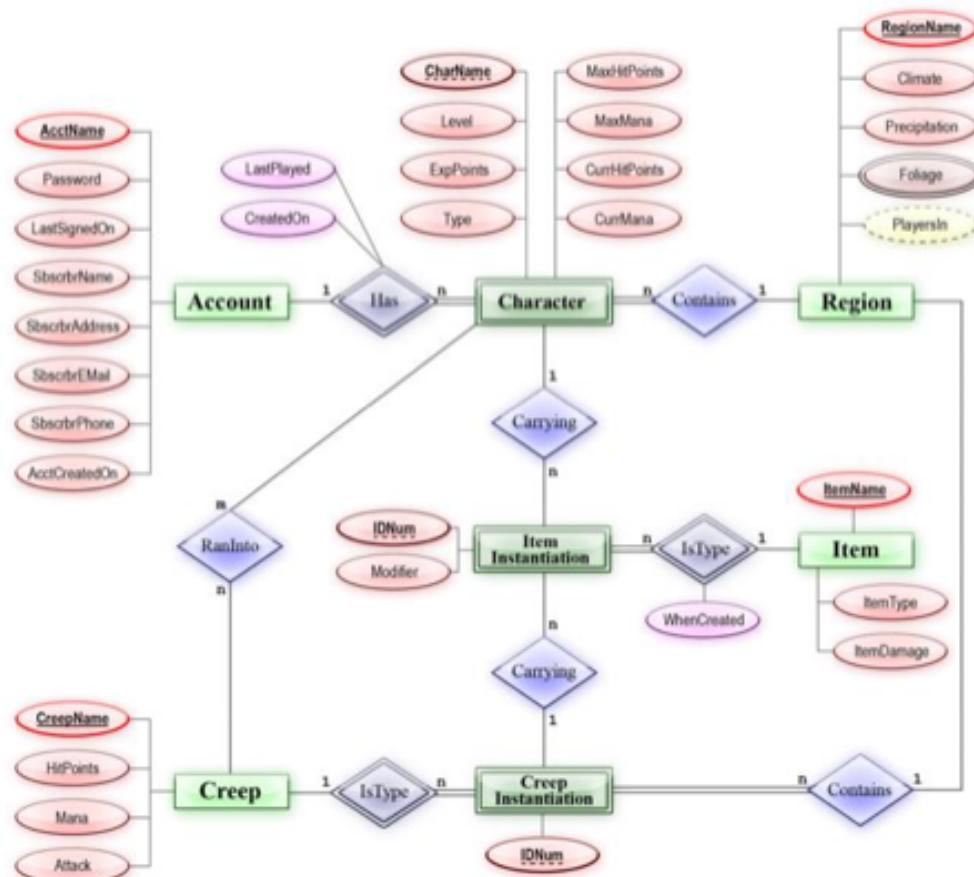
Сущность - это реальный или представляемый объект, информация о котором должна сохраняться и быть доступна. В диаграммах ER-модели сущность представляется в виде прямоугольника, содержащего имя сущности. При этом имя сущности - это имя типа, а не некоторого конкретного экземпляра этого типа. Для большей выразительности и лучшего понимания имя сущности может сопровождаться примерами конкретных объектов этого типа. Каждый экземпляр сущности должен быть отличим от любого другого экземпляра той же сущности (это требование в некотором роде аналогично требованию отсутствия кортежей-

дубликатов в реляционных таблицах). Сущности подразделяются на сильные и слабые. Сильные сущности существуют сами по себе, а существование слабых сущностей зависит от существования сильных.

Связь - это ассоциация, устанавливаемая между сущностями. Эта ассоциация может существовать между разными сущностями или между сущностью и ей же самой (рекурсивная связь). Сущности, включенные в связь, называются ее участниками, а количество участников связи называется ее степенью. Участие сущности в связи может быть как полным, так и частичным. Связи в ER-модели могут иметь тип «один к одному», «один ко многим», «многие ко многим». Именно тип связи «многие ко многим» является единственным типом, представляющим истинную связь, поскольку это единственный тип связи, который требует для своего представления отдельного отношения. Связи типа «один к одному» и «один ко многим» всегда могут быть представлены с помощью механизма внешнего ключа, помещаемого в одно из отношений.

Свойством сущности (и связи) является любая деталь, которая служит для уточнения, идентификации, классификации, числовой характеристики или выражения состояния сущности (или связи). Значения свойств каждого типа извлекаются из соответствующего множества значений, которое в реляционной терминологии называется доменом. Свойства могут быть простыми или составными, ключевыми, однозначными или многозначными, опущенными (т. е. «неизвестными» или «непредставленными»), базовыми или производными.

Более сложными элементами ER-модели являются подтипы и супертипы сущностей. Как в языках программирования с развитыми типовыми системами (например, в языках объектно-ориентированного программирования), вводится возможность наследования типа сущности, исходя из одного или нескольких супертипов.



На ER-диаграммах множества сущностей изображаются в виде прямоугольников, множества отношений изображаются в виде ромбов. Слабый тип сущности изображают в виде прямоугольника с двойным контуром. Слабый тип связи изображают в виде ромба с двойным контуром. Если сущность участвует в отношении, они связаны линией. Тип связи с частичным участием изображают двойной линией. Вид типа связи обозначается над линиями в виде соответствующих надписей возле типов сущностей. Например, если это вид бинарной связи «один ко многим», то делают надписи 1, n (или m), соответственно, возле соответствующих типов сущностей. Атрибуты изображаются в виде овалов и связываются линией с одним отношением или с одной сущностью. Именование сущности обычно выражается уникальным существительным, именование связи обычно выражается глаголом, именование атрибута обычно выражается существительным. Неизбыточный набор атрибутов, значения которых в совокупности являются уникальными для каждого экземпляра сущности, являются ключом сущности.

Существует множество инструментов для работы с ER-моделями, вот некоторые из них: Microsoft Visio, ERwin, Oracle Designer, PowerDesigner, Rational Rose. В справочниках приводятся сведения о 25 таких инструментах.

Получение реляционной схемы из ER-схемы осуществляется с помощью следующей пошаговой процедуры.

Шаг 1. Каждая простая сущность превращается в таблицу. Простая сущность - сущность, не являющаяся подтипом и не имеющая подтипов. Имя сущности становится именем таблицы.

Шаг 2. Каждое свойство (атрибут) становится возможным столбцом с тем же именем; может выбираться более точный формат. Столбцы, соответствующие необязательным атрибутам, могут содержать неопределенные значения; столбцы, соответствующие обязательным атрибутам, - не могут.

Шаг 3. Компоненты уникального идентификатора сущности превращаются в первичный ключ таблицы. Если имеется несколько возможных уникальных идентификаторов, выбирается наиболее используемый. Если в состав уникального идентификатора входят связи, к числу столбцов первичного ключа добавляется копия уникального идентификатора сущности, находящейся на дальнем конце связи (этот процесс может продолжаться рекурсивно). Для именования этих столбцов используются имена концов связей и/или имена сущностей.

Шаг 4. Связи «многие к одному» (и «один к одному») становятся внешними ключами. Т.е. делается копия уникального идентификатора с конца связи «один», и соответствующие столбцы составляют внешний ключ. Необязательные связи соответствуют столбцам, допускающим неопределенные значения; обязательные связи - столбцам, не допускающим неопределенные значения.

Шаг 5. Индексы создаются для первичного ключа (уникальный индекс), внешних ключей и тех атрибутов, на которых предполагается в основном базировать запросы.

Шаг 6. Если в концептуальной схеме присутствовали подтипы, то возможны два способа:

- все подтипы в одной таблице,
- для каждого подтипа - отдельная таблица.

Реляционная модель данных

Основоположником теории реляционных баз данных является британский учёный Эдгар Кодд, который в 1970 году опубликовал первую работу по реляционной модели данных. Наиболее распространенная трактовка реляционной модели данных принадлежит Кристоферу Дейту. Согласно Дейту, реляционная модель состоит из трех частей: структурной, целостностной и манипуляционной.

Структурная часть реляционной модели

Структурная часть реляционной модели описывает, из каких объектов состоит реляционная модель. Постулируется, что основной структурой данных, используемой в реляционной модели, являются нормализованные «n-арные» отношения. Основными понятиями структурной части реляционной модели являются тип данных, домен, атрибут, схема отношения, схема базы данных, кортеж, отношение, потенциальный, первичный и альтернативные ключи, реляционная база данных.

Понятие типа данных в реляционной модели полностью адекватно понятию типа данных в языках программирования.

Понятие домена можно считать уточнением типа данных. Домен можно рассматривать как подмножество значений некоторого типа данных, имеющих определенный смысл. Домен характеризуется следующими свойствами:

- домен имеет уникальное имя (в пределах базы данных),
- домен определен на некотором типе данных или на другом домене,
- домен может иметь некоторое логическое условие, позволяющее описать подмножество данных, допустимых для данного домена,
- домен несет определенную смысловую нагрузку.

Говорят, что домен отражает семантику, определенную предметной областью. Может быть несколько доменов, совпадающих как подмножества, но несущие различный смысл. Основное значение доменов состоит в том, что домены ограничивают сравнения. Некорректно, с логической точки зрения, сравнивать значения из различных доменов, даже если они имеют одинаковый тип.

Понятие домена помогает правильно моделировать предметную область. Не все домены обладают логическим условием, ограничивающим возможные значения домена. В таком случае множество возможных значений домена совпадает с множеством возможных значений типа данных.

Атрибут отношения – это пара вида <имя_атрибута, имя_домена>. Имена атрибутов должны быть уникальны в пределах отношения. Часто имена атрибутов отношения совпадают с именами соответствующих доменов.

Схема отношения – это именованное множество упорядоченных пар <имя_атрибута, имя_домена>. Степенью или «арностью» схемы отношения является мощность этого множества. Схема базы данных в реляционной модели – это множество именованных схем отношений. Понятие схемы отношения близко к понятию структурного типа в языках программирования (например, record в языке Pascal или struct в языке C).

Кортеж, соответствующий данной схеме отношения – это множество упорядоченных пар <имя_атрибута, значение_атрибута>, которое содержит одно вхождение каждого имени атрибута, принадлежащего схеме отношения. Значение атрибута должно быть допустимым значением домена, на котором определен данный атрибут. Степень или «арность» кортежа совпадает с «арностью» соответствующей схемы отношения.

Отношение, определенное на множестве из n доменов (не обязательно различных), содержит

две части: заголовок (схему отношения) и тело (множество из m кортежей). Значения n и m называются соответственно степенью и кардинальностью отношения. Отношения обладают следующими свойствами.

- В отношении нет одинаковых кортежей. Действительно, тело отношения есть множество кортежей и, как всякое множество, не может содержать неразличимые элементы.
- Кортежи не упорядочены (сверху вниз). Причина следующая – тело отношения есть множество, а множество не упорядочено.
- Атрибуты не упорядочены (слева направо). Т.к. каждый атрибут имеет уникальное имя в пределах отношения, то порядок атрибутов не имеет значения. В таблицах в отличие от отношений столбцы упорядочены.
- Каждый кортеж содержит ровно одно значение для каждого атрибута. Отношение, удовлетворяющее этому свойству, называется нормализованным или представленным в первой нормальной форме (1NF).
- Все значения атрибутов атомарны, т. е. не обладают структурой. Трактовка этого свойства в последнее время претерпела существенные изменения. Исторически в большинстве публикаций по базам данных считалось недопустимым использовать атрибуты со структурированными значениями. (А в большинстве изданий это считается таковым и поныне.) В настоящее время принимается следующая точка зрения: тип данных атрибута может быть произвольным, а, следовательно, возможно существование отношения с атрибутами, значениями которых также являются отношения. Именно так в некоторых постреляционных базах данных реализована работа со сколь угодно сложными типами данных, создаваемых пользователями.

В реляционной модели каждый кортеж любого отношения должен отличаться от любого другого кортежа этого отношения (т.е. любое отношение должно обладать уникальным ключом).

Непустое подмножество множества атрибутов схемы отношения будет потенциальным ключом тогда и только тогда, когда оно будет обладать свойствами уникальности (в отношении нет двух различных кортежей с одинаковыми значениями потенциального ключа) и неизбыточности (никакое из собственных подмножеств множества потенциального ключа не обладает свойством уникальности).

В реляционной модели по традиции один из потенциальных ключей должен быть выбран в качестве первичного ключа, а все остальные потенциальные ключи будут называться альтернативными.

Реляционная база данных – это набор отношений, имена которых совпадают с именами схем отношений в схеме базы данных.

Целостностная часть реляционной модели

В целостностной части реляционной модели фиксируются два базовых требования целостности, которые должны выполняться для любых отношений в любых реляционных базах данных. Это целостность сущностей и ссылочная целостность (или целостность внешних ключей).

Простой объект реального мира представляется в реляционной модели как кортеж некоторого отношения. Требование целостности сущностей заключается в следующем: каждый кортеж любого отношения должен отличаться от любого другого кортежа этого отношения (т.е. любое отношение должно обладать потенциальным ключом). Вполне очевидно, что если данное требование не соблюдается (т.е. кортежи в рамках одного отношения не уникальны), то в базе данных может храниться противоречивая информация об одном и том же объекте.

Поддержание целостности сущностей обеспечивается средствами СУБД. Это осуществляется с помощью двух ограничений:

- при добавлении записей в таблицу проверяется уникальность их первичных ключей,
- не допускается изменение значений атрибутов, входящих в первичный ключ.

Сложные объекты реального мира представляются в реляционной модели данных в виде кортежей нескольких нормализованных отношений, связанных между собой. При этом

- связи между данными отношениями описываются в терминах функциональных зависимостей,
- для отражения функциональных зависимостей между кортежами разных отношений используется дублирование первичного ключа одного отношения (родительского) в другое (дочернее). Атрибуты, представляющие собой копии ключей родительских отношений, называются внешними ключами.

Внешний ключ в отношении R2 – это непустое подмножество множества атрибутов FK этого отношения, такое, что: а) существует отношение R1 (причем отношения R1 и R2 необязательно различны) с потенциальным ключом СК; б) каждое значение внешнего ключа FK в текущем значении отношения R2 обязательно совпадает со значением ключа СК некоторого кортежа в текущем значении отношения R1.

Требование ссылочной целостности состоит в следующем: Для каждого значения внешнего ключа, появляющегося в дочернем отношении, в родительском отношении должен найтись кортеж с таким же значением первичного ключа.

Как правило, поддержание ссылочной целостности также возлагается на СУБД. Например, она может не позволить пользователю добавить запись, содержащую внешний ключ с несуществующим (неопределенным) значением.

Манипуляционная часть реляционной модели

Манипуляционная часть реляционной модели описывает два эквивалентных способа манипулирования реляционными данными – реляционную алгебру и реляционное исчисление. Принципиальное различие между реляционной алгеброй и реляционным исчислением заключается в следующем: реляционная алгебра в явном виде предоставляет набор операций, а реляционное исчисление представляет систему обозначений для определения требуемого отношения в терминах данных отношений. Формулировка запроса в терминах реляционной алгебры носит предписывающий характер, а в терминах реляционного исчисления – описательный характер. Говоря неформально, реляционная алгебра носит процедурный характер (пусть на очень высоком уровне), а реляционное исчисление – непроцедурный характер.

Реляционная алгебра

Реляционная алгебра является основным компонентом реляционной модели, опубликованной Коддом, и состоит из восьми операторов, составляющих две группы по четыре оператора:

- Традиционные операции над множествами: объединение (UNION), пересечение (INTERSECT), разность (MINUS) и декартово произведение (TIMES). Все операции модифицированы, с учетом того, что их операндами являются отношения, а не произвольные множества.
- Специальные реляционные операции: ограничение (WHERE) , проекция (PROJECT), соединение (JOIN) и деление (DIVIDE BY).

Результат выполнения любой операции реляционной алгебры над отношениями также является отношением. Эта особенность называется свойством реляционной замкнутости. Утверждается, что поскольку реляционная алгебра является замкнутой, то в реляционных выражениях можно использовать вложенные выражения сколь угодно сложной структуры. Если рассматривать свойство реляционной замкнутости строго, то каждая реляционная операция должна быть определена таким образом, чтобы выдавать результат с надлежащим типом отношения (в частности, с соответствующим набором атрибутов или заголовком). Для достижения этой цели вводится новый оператор переименование (RENAME), предназначенный для переименования атрибутов в определенном отношении.

В качестве основы для последующих обсуждений рассмотрим упрощенный синтаксис выражений реляционной алгебры в форме БНФ.

```
реляционное_выражение ::= унарное_выражение | бинарное_выражение
унарное_выражение ::= переименование | ограничение | проекция
переименование ::= терм RENAME имя_атрибута AS имя_атрибута
терм ::= имя_отношения | ( реляционное_выражение )
ограничение ::= терм WHERE логическое_выражение
проекция ::= терм | терм [ список_имен_атрибутов ]
бинарное_выражение ::= проекция бинарная_операция реляционное_выражение
бинарная_операция ::= UNION | INTERSECT | MINUS | TIMES | JOIN | DIVIDE BY
```

По приведенной грамматике можно сделать следующие замечания.

- Реляционные операторы UNION, INTERSECT и MINUS требуют, чтобы отношения были совместимыми по типу, т. е. имели идентичные заголовки.
- Реляционные операторы UNION, INTERSECT, TIMES и JOIN ассоциативны и коммутативны.
- Если отношения A и B не имеют общих атрибутов, то операция соединения A JOIN B эквивалентна операции A TIMES B, т. е. в таком случае соединение вырождается в декартово произведение. Такое соединение называют естественным.
- Другой допустимый синтаксис для синтаксической категории переименования таков: (терм RENAME список_переименований). Здесь каждый из элементов списка переименований представляет собой выражение имя_атрибута AS имя_атрибута.
- Несмотря на большие возможности, предоставляемые операторами реляционной алгебры, существует несколько типов запросов, которые нельзя выразить этими средствами. Для таких случаев необходимо использовать процедурные расширения реляционных языков.

В алгебре Кодда не все операторы являются независимыми, т. е. некоторые из реляционных операторов могут быть выражены через другие реляционные операторы.

Оператор естественного соединения по атрибуту Y определяется через оператор декартового произведения и оператор ограничения:

$A \text{ JOIN } B = ((A \text{ TIMES } (B \text{ RENAME } Y \text{ AS } Y1)) \text{ WHERE } Y=Y1)[X, Y, Z]$

Оператор пересечения выражается через вычитание следующим образом:

$A \text{ INTERSECT } B = A \text{ MINUS } (A \text{ MINUS } B)$

Оператор деления выражается через операторы вычитания, декартового произведения и проекции следующим образом:

$A \text{ DIVIDE BY } B = A[X] \text{ MINUS } ((A[X] \text{ TIMES } B) \text{ MINUS } A)[X]$

Оставшиеся реляционные операторы (объединение, вычитание, декартово произведение, ограничение, проекция) являются примитивными операторами – их нельзя выразить друг через друга.

В качестве примера рассмотрим запросы на языке реляционной алгебры для схемы базы данных «Поставщики и детали», представленной следующими схемами отношений:

S(Sno: integer, Sname: string, Status: integer, City: string)

P(Pno: integer, Pname: string, Color: string, Weight: real, City: string)

SP(Sno: integer, Pno: integer, Qty: integer)

В данном примере имена доменов представлены именами типов, имена типов отделяются от имен атрибутов двоеточием, первичные ключи выделены подчеркиванием, а имена внешних ключей схемы отношения SP (ПОСТАВКА) совпадают с именами первичных ключей схем отношений S (ПОСТАВЩИК) и P (ДЕТАЛЬ).

Получить имена поставщиков, которые поставляют деталь под номером 2	$((\text{ SP JOIN S }) \text{ WHERE } Pno = 2) [\text{ Sname }]$
Получить имена поставщиков, которые поставляют по крайней мере одну красную деталь	$(((\text{ P WHERE Color = 'Красный' }) \text{ JOIN SP }) [\text{ Sno }] \text{ JOIN S }) [\text{ Sname }]$ Другая формулировка того же запроса: $(((\text{ P WHERE Color = 'Красный' }) [\text{ Pno }] \text{ JOIN SP }) \text{ JOIN S }) [\text{ Sname }]$ Этот пример подчеркивает одно важное обстоятельство: возможность сформулировать один и тот же запрос несколькими способами.
Получить имена поставщиков, которые поставляют все детали	$((\text{ SP } [\text{ Sno, Pno}] \text{ DIVIDE BY P } [\text{ Pno }] \text{ JOIN S }) [\text{ Sname }]$

Получить номера поставщиков, поставляющих по крайней мере все те детали, которые поставяет поставщик под номером 2	SP [Sno, Pno] DIVIDE BY (SP WHEPE Sno = 2) [Pno]
Получить все пары номеров поставщиков, размещенных в одном городе	(((S RENAME Sno AS FirstSno) [FirstSno, City] JOIN (S RENAME Sno AS SecondSno) [SecondSno , City]) WHEPE FirstSno < SecondSno) [FirstSno, SecondSno]
Получить имена поставщиков, которые не поставяют деталь под номером 2	((S[Sno] MINUS (SP WHEPE Pno = 2) [Sno]) JOIN S) [Sname]

Вычислительные возможности реляционной алгебры можно увеличить путем введения дополнительных операторов. Дополнительные операторы реляционной алгебры

Многие авторы предлагали новые алгебраические операторы после определения Коддом первоначальных восьми. Рассмотрим несколько таких операторов:

SEMIJOIN (полусоединение), SEMIMINUS (полувычитание), EXTEND (расширение), SUMMARIZE (обобщение) и TCLOSE (транзитивное замыкание).

Синтаксис этих операторов выглядит следующим образом:

```

<полусоединение> ::= <реляционное выражение> SEMIJOIN <реляционное выражение>
<полувычитание> ::= <реляционное выражение> SEMIMINUS <реляционное выражение>
<расширение> ::= EXTEND <реляционное выражение> ADD
<список добавляемых расширений>
<добавляемое расширение> ::= <выражение> AS <имя атрибута>
<обобщение> ::= SUMMARIZE <реляционное выражение> PER <реляционное выражение>
ADD <список добавляемых обобщений>
<добавляемое обобщение> ::= <тип обобщения> [ ( <скалярное выражение> ) ] AS <имя атрибута>
<тип обобщения> ::= COUNT | SUM | AVG | MAX | MIN | ALL | ANY | COUNTD |
SUMD | AVGD
<транзитивное замыкание> ::= TCLOSE <реляционное выражение>

```

Операция расширения

С помощью операции расширения из определенного отношения (по крайней мере, концептуально) создается новое отношение, которое содержит дополнительный атрибут, значения которого получены посредством некоторых скалярных вычислений.

```

EXTEND S ADD 'Supplier' AS TAG
EXTEND P ADD (Weight * 454 ) AS GMWT
( EXTEND P ADD (Weight * 454 ) AS GMWT ) WHERE GMWT > Weight ( 10000.0 ) ) { ALL
BUT GMWT } EXTEND ( P JOIN SP ) ADD (Weight * Qty ) AS SHIPWT
( EXTEND S ADD City AS SCity ) { ALL BUT City }
EXTEND P ADD Weight * 454 AS GMW, Weight * 16 AS OZWT )
EXTEND S ADD COUNT ( ( SP RENAME SNo AS X ) WHERE X = SNo ) AS NP

```

Рассмотрим вкратце обобщающие функции. Общее назначение этих функций состоит в том, чтобы на основе значений некоторого атрибута определенного отношения получить скалярное значение. В языке Tutorial D параметр <вызов обобщающей функции> является особым случаем параметра <скалярное выражение> и в общем случае имеет следующий вид.

<имя функции> (<реляционное выражение> [, <имя атрибута>])

Если параметр <имя функции> имеет значение COUNT, то параметр <имя атрибута> недопустим и должен быть опущен. В остальных случаях параметр <имя атрибута> может быть опущен тогда и только тогда, когда параметр <реляционное выражение> задает отношение со степенью, равной единице.

Операция обобщения

В реляционной алгебре операция расширения позволяет выполнять "горизонтальные" вычисления в отношении отдельных строк. Оператор обобщения выполняет аналогичную функцию для "вертикальных" вычислений в отношении отдельного столбца. Примеры.

SUMMARIZE SP PER SP { PNo } ADD SUM (Qty) AS TOTQTY

В результате его вычисления создается отношение с заголовком {P#, TOTQTY}, содержащее один кортеж для каждого значения атрибута P# в проекции SP{P#}. Каждый из этих кортежей содержит значение атрибута P# и соответствующее общее количество деталей. Другими словами, концептуально исходное отношение P «перегруппировано» в множество групп кортежей (по одной группе для каждого уникального значения атрибута P#), после чего для каждой полученной группы сгенерирован один кортеж, помещаемый в окончательный результат.

В общем случае выражение выглядит: SUMMARIZE A PER B ADD <обобщение> AS

Определяется следующим образом:

- Отношение B должно иметь такой же тип, как и некоторая проекция отношения A, Т.е. каждый атрибут отношения B должен одновременно присутствовать в отношении A. Примем, что атрибутами этой проекции (или, что эквивалентно, атрибутами отношения B) являются атрибуты A1, A2, ... , An.
- Результатом вычисления данного выражения будет отношение с заголовком {A1, A2, ... , An, Z}, где Z является новым добавленным атрибутом.
- Тело результата содержит все кортежи t, где t является кортежем отношения B, расширенным значением нового атрибута Z. Это значение нового атрибута Z подсчитывается посредством вычисления обобщающего выражения по всем кортежам отношения A, которое имеет те же значения для атрибутов A1, A2, ... , An, что и кортеж t. (Разумеется, если в отношении A нет кортежей, принимающих те же значения для атрибутов A1, A2, ... , An, что и кортеж t, то обобщающее выражение будет вычислено для пустого множества.) Отношение B не должно содержать атрибут с именем Z, а обобщающее выражение не должно ссылаться на атрибут Z. Заметьте, что кардинальность результата равна кардинальности отношения B, а степень результата равна степени отношения B плюс единица. Типом переменной Z в этом случае будет тип обобщающего выражения.

Вот еще один пример:

SUMMARIZE (P JOIN SP) PER P { City } ADD COUNT AS NSP

Легко заметить, что оператор SUMMARIZE не примитивен – его можно моделировать с помощью оператора EXTEND. Рассмотрим следующее выражение

SUMMARIZE SP PER S { SNo } ADD COUNT AS NP

По сути, это сокращенная запись представленного ниже более сложного выражения

{ EXTEND S { SNo } ADD ((SP RENAME SNo AS X) WHERE X=SNo) AS Y, COUNT (Y) AS NP } { SNo, NP }

Группирование и разгруппирование

Поскольку значениями атрибутов отношений могут быть другие отношения, было бы желательным наличие дополнительных реляционных операторов, называемых операторами группирования и разгруппирования. Рассмотрим пример SP GROUP (PNo, Qty) AS PQ который можно прочесть как «сгруппировать отношение SP по атрибуту SNo», поскольку атрибут SNo является единственным атрибутом отношения SP, не упомянутым в предложении GROUP. В результате получится отношение, заголовок которого выглядит так

{ SNo SNo, PQ RELATION { PNo PNo, Qty Qty } }

Другими словами, он состоит из атрибута PQ, принимающего в качестве значений отношения (PQ, в свою очередь, имеет атрибуты PNo и Qty), а также из всех остальных атрибутов отношения SP (в нашем случае "все остальные атрибуты" - это атрибут SNo). Тело этого отношения содержит ровно по одному кортежу для всех различных значений атрибута SNo исходного отношения SP.

Перейдем теперь к операции разгруппирования. Пусть SPQ - это отношение, полученное в результате группирования. Тогда выражение SPQ UNGROUP PQ возвращает нас к отношению SP (как и следовало ожидать). Точнее, оно выдает в качестве результата отношение, заголовок которого выглядит так (SNo SNo, PNo PNo, Qty Qty)

Реляционные сравнения

Реляционная алгебра в том виде, в котором она была изначально определена, не поддерживает прямого сравнения двух отношений (например, проверки их равенства или того, является ли одно из них подмножеством другого). Это упущение легко исправляется следующим образом. Сначала определяется новый вид условия - реляционное сравнение - со следующим синтаксисом.

<реляционное выражение> <отношение> <реляционное выражение> <отношение> ::=
 > -- Собственное супермножество | >= -- Супермножество
 | < -- Собственное подмножество | <= -- Подмножество
 | = -- Равно
 | <> -- Не равно

Здесь параметр <реляционное выражение> в обоих случаях выражения реляционной алгебры, представляющие совместимые по типу отношения.

S (City) = P (City)

Смысл выражения: совпадает ли проекция отношения поставщиков S по атрибуту City с проекцией отношения деталей P по атрибуту City? S (SNo) > SPJ (SNo)

Смысл выражения: есть ли поставщики, вообще не поставляющие деталей?

На практике часто требуется определить, является ли данное отношение пустым.

Соответствующий оператор, возвращающий логическое значение имеет вид

IS_EMPTY (<реляционное выражение>)

Не менее часто требуется проверить, присутствует ли данный кортеж t в данном отношении

R . Для этой цели подойдет следующее реляционное сравнение.

RELATION { t } $\leq R$

Однако, с точки зрения пользователя, удобнее применять следующее сокращение: $t \text{ IN } R$

Реляционное исчисление

Реляционное исчисление основано на разделе математической логики, который называется исчислением предикатов. Реляционное исчисление существует в двух формах: исчисление кортежей и исчисление доменов. Основное различие между ними состоит в том, что переменные исчисления кортежей являются переменными кортежей (они изменяются на отношении, а их значения являются кортежами), в то время как переменные исчисления доменов являются переменными доменов (они изменяются на доменах, а их значения являются скалярами).

Исчисление кортежей

Реляционное исчисление является альтернативой реляционной алгебре. Внешне два подхода очень отличаются – исчисление описательное, а алгебра предписывающая, но на более низком уровне они представляют собой одно и то же, поскольку любые выражения исчисления могут быть преобразованы в семантически эквивалентные выражения в алгебре и наоборот.

Исчисление существует в двух формах: исчисление кортежей и исчисление доменов. Основное различие между ними состоит в том, что переменные исчисления кортежей являются переменными кортежей (они изменяются на отношении, а их значения являются кортежами), в то время как переменные исчисления доменов являются переменными доменов (они изменяются на доменах, а их значения являются скалярами; в этом смысле, действительно, "переменная домена" - не очень точный термин).

Выражение исчисления кортежей содержит заключенный в скобки список целевых элементов и выражение WHERE, содержащее формулу WFF ("правильно построенную формулу"). Такая формула WFF составляется из кванторов (EXISTS и FORALL), свободных и связанных переменных, литералов, операторов сравнения, логических (булевых) операторов и скобок. Каждая свободная переменная, которая встречается в формуле WFF, должна быть также перечислена в списке целевых элементов.

Упрощенный синтаксис выражений исчисления кортежей в форме БНФ имеет вид:

```
объявление-кортежной-переменной ::= RANGE OF переменная IS список-областей
область ::= отношение | реляционное-выражение
реляционное-выражение ::= (список-целевых-элементов)[WHERE wff]
целевой-элемент ::= переменная | переменная.атрибут [AS атрибут]
wff ::= условие | NOT wff | условие AND wff | условие OR wff | IF условие THEN wff |
      EXISTS переменная (wff) | FORALL переменная (wff) | (wff)
условие ::= (wff) | компаранд операция-отношения компаранд
```

По приведенной грамматике можно сделать следующие замечания.

- Квадратные скобки здесь указывают на компоненты, которые по умолчанию могут быть опущены.
- Категории отношение, атрибут и переменная – это идентификаторы (т. е. имена).
- Реляционное выражение содержит заключенный в скобки список целевых элементов и выражение WHERE, содержащее формулу wff («правильно построенную формулу»). Такая формула wff составляется из кванторов (EXISTS и FORALL), свободных и связанных переменных, констант, операторов сравнения, логических (булевых)

операторов и скобок. Каждая свободная переменная, которая встречается в формуле wff, должна быть также перечислена в списке целевых элементов.

- Категория условие представляет или формулу wff, заключенную в скобки, или простое скалярное сравнение, где каждый компаранд оператора сравнения – это либо скалярная константа, либо значение атрибута в форме переменная.атрибут.

Пусть кортежная переменная T определяются следующим образом:

RANGE OF T IS R1, R2, ..., Rn

Тогда отношения R1, R2, ..., Rn должны быть совместимы по типу т. е. они должны иметь идентичные заголовки, и кортежная переменная T изменяется на объединении этих отношений, т. е. её значение в любое заданное время будет некоторым текущим кортежем, по крайней мере одного из этих отношений.

Примеры объявлений кортежных переменных.

RANGE OF SX IS S RANGE OF SPX IS SP RANGE OF SY IS

(SX) WHERE SX.City = 'Смоленск',

(SX) WHERE EXISTS SPX (SPX.Sno = SX.Sno AND SPX.Pno = 1)

Здесь переменная кортежа SY может принимать значения из множества кортежей S для поставщиков, которые или размещены в Смоленске, или поставляют деталь под номером 1, или и то и другое.

Для сравнения с реляционной алгеброй рассмотрим некоторые запросы на языке исчисления кортежей, которые соответствуют рассмотренным ранее.

Для сравнения с реляционной алгеброй некоторые примеры соответствуют рассмотренным ранее. Любые примеры можно расширить, включив в них завершающий шаг присвоения, присвоив значение выражения некоторому именованному отношению; этот шаг для краткости опускается.

Получить номера поставщиков из Смоленска со статусом больше 20	(SX.Sno) WHERE SX.City = 'Смоленск' AND SX.Status > 20
Получить все такие пары номеров поставщиков, что два поставщика размещаются в одном городе	(SX.Sno AS FirstSno, SY.Sno AS SecondSno) WHERE SX.City = SY.City AND SX.Sno < SY.Sno <u>Замечание.</u> Спецификации «AS FirstSno» и «AS SecondSno» дают имена атрибутам результата; следовательно, такие имена недоступны для использования во фразе WHERE и потому второе сравнение во фразе WHERE «SX.Sno < SY.Sno», а не «FirstSno < SecondSno».
Получить имена поставщиков, которые поставляют деталь с номером 2	SX.Sname WHERE EXISTS SPX (SPX.Sno = SX.Sno AND SPX.Pno = 2)

Получить имена поставщиков, которые поставляют по крайней мере одну красную деталь	SX.Sname WHERE EXISTS SPX (SX.Sno = SPX.Sno AND EXISTS PX (PX.Pno = SPX.Pno AND PX.Color = 'Красный')) Или эквивалентная формула (но в предваренной нормальной форме, в которой все кванторы записываются в начале формулы WFF): SX.Sname WHERE EXISTS SPX (EXISTS PX (SX.Sno = SPX.Sno AND SPX. Pno = PX.Pno AND PX.Color = 'Красный'))
Получить имена поставщиков, которые поставляют по крайней мере одну деталь, поставляемую поставщиком под номером 2	SX.Sname WHERE EXISTS SPX (EXISTS SPY (SX.Sno = SPX.Sno AND SPX.Pno = SPX.Pno AND SPY.sno = 2))
Получить имена поставщиков, которые поставляют все детали	SX.SNAME WHERE FORALL PX (EXISTS SPX (SPX.Sno = SX.Sno AND SPX.Pno = PX.Pno)) Или равносильное выражение без использования квантора FORALL: SX. SNAME WHERE NOT EXISTS PX (NOT EXISTS SPX (SPX.Sno = SX.Sno AND SPX.Pno = PX.Pno)) Отметим, что содержание этого запроса – «имена поставщиков, которые поставляют все детали» - является точным на 100% (в отличие от случая с алгебраическим аналогом при использовании оператора DIVIDEBY).
Получить имена поставщиков, которые не поставляют деталь под номером 2	SX.Sname WHERE NOT EXISTS SPX (SPX.Sno = SX.Sno AND SPX.Pno = 2)
Получить номера поставщиков, которые поставляют по крайней мере все детали, поставляемые поставщиком с номером 2	SX.Sno WHERE FORALL SPY (SPY.Sno <> 2 OR EXISTS SPZ (SPZ.Sno = SX.Sno AND SPZ.Pno = SPY.Pno))

Предваренная нормальная форма не является более или менее правильной по сравнению с другими формами, но немного попрактиковавшись можно убедиться, что в большинстве случаев это наиболее естественная формулировка запросов. Кроме того, эта форма позволяет уменьшить количество скобок, как показано ниже.

Формулу WFF ::= квантор_1 переменная_1 (квантор_2 переменная_2 (wff)), где каждый из кванторов квантор_1 и квантор_2 представляет или квантор EXISTS, или квантор FORALL, переменная_1 и переменная_2 - имена переменных) по умолчанию можно однозначно сократить к виду: квантор_1 переменная_1 квантор_2 переменная_2 (wff)

Таким образом, приводимое в четвертом примере выражение исчисления можно переписать (при желании) следующим образом:

```
SX.Sname WHERE EXISTS SPX EXISTS PX ( SX.Sno = SPX.Sno AND SPX.Pno = PX.Pno
AND PX.Color = 'Красный' )
```

Вычислительные возможности исчисления кортежей

Добавить вычислительные возможности в исчисление довольно просто: необходимо расширить определение компарандов и целевых элементов так, чтобы они включали новую категорию - скалярные выражения, в которых операнды, в свою очередь, могут включать литералы, ссылки на атрибуты и (или) ссылки на итоговые функции. Для целевых элементов также требуется использовать спецификацию вида "AS attribute" для того, чтобы дать подходящее имя результирующему атрибуту, если нет очевидного наследуемого имени.

Т. к. что смысл скалярного выражения легко воспринимается, подробности опускаются. Однако синтаксис для ссылок на итоговые функции будет показан:

aggregate_function (expression [, attribute]), где

- aggregate_function - это COUNT, SUM, AVG, MAX или MIN (возможны, конечно, и некоторые другие функции),
- expression - это выражение исчисления кортежей (вычисляющее отношение),
- attribute - это такой атрибут результирующего отношения, по которому подсчитывается итог.

Для функции COUNT аргумент attribute не нужен и опускается; для других итоговых функций его можно опустить по умолчанию, если и только если вычисление аргумента expression дает отношение степени один и в таком случае единственный атрибут результата вычисления выражения expression подразумевается по умолчанию. Обратите внимание, что ссылка на итоговую функцию возвращает скалярное значение и поэтому допустима в качестве операнда скалярного выражения.

Из сказанного можно сделать следующие выводы:

- Итоговая функция действует в некоторых отношениях как новый тип квантора. В частности, если аргумент expression в данной ссылке на итоговую функцию представляется в виде "(tic) WHERE f", где tic- целевой элемент списка (target_item_commalist), а f - это формула WFF, и если экземпляр переменной кортежа T свободен в формуле f, то такой экземпляр переменной T связан в ссылке на итоговую функцию aggregate_function ((tic) WHERE f [, attribute])
- Пользователи, владеющие языком SQL, могут заметить, что с помощью двух следующих аргументов в ссылке на итоговую функцию, expression и attribute, можно избежать необходимых для SQL специальных приемов использования оператора DISTINCT для исключения дублирующих кортежей, если это требуется, перед выполнением итоговой операции. Вычисление аргумента expression дает отношение, из которого повторяющиеся кортежи всегда исключаются по определению. Аргумент attribute обозначает атрибут такого отношения, по которому выполняются итоговые вычисления, а дублирующие значения перед подсчетом итога из такого атрибута не удаляются. Конечно, атрибут может не содержать никаких дублирующих значений в любом случае, в частности, если такой атрибут является первичным ключом.

Получить номера деталей и их вес всех типов деталей, вес которых превышает 10 ед.	(PX.Pno, PX.Weight AS GMWT) WHERE PX.Weight > 10
Получить всех поставщиков, добавив для каждого литеральное значение "Поставщик"	(SX, 'Поставщик' AS TAG)
Получить каждую поставку с полными данными о входящих в нее деталях и общим весом поставки	(SPX.Sno, SPX.Qty, PX, PX.Weight * SPX.Qty AS SHIPWT) WHERE PX.Pno = SPX.Pno
Для каждой детали получить ее номер и общее поставляемое количество	(PX.Pno, SUM (SPX WHERE SPX.Pno = PX.Pno, Qty) AS TOTQTY)
Получить общее количество поставляемых деталей	(SUM (SPX, Qty) AS GRANDTOTAL))
Для каждого поставщика получить его номер и общее количество поставляемых им деталей	(SX.Sno, COUNT (SPX WHERE SPX.Sno = SX.Sno) AS NUMBER_OF_PARTS)
Получить города, в которых хранится а) более пяти красных деталей; б) не более пяти красных деталей	PX.City WHERE COUNT (PY WHERE PY.City = PX.City AND PY.Color = 'Красный') > 5

Исчисление доменов

Реляционное исчисление, ориентированное на домены (или исчисление доменов), отличается от исчисления кортежей тем, что в нем используются переменные доменов вместо переменных кортежей, т. е. переменные, принимающие свои значения в пределах домена, а не отношения. (Замечание. "Переменную домена" было бы лучше назвать "скалярной переменной", так как ее значения - это элементы домена, т.е. скаляры, а не сами домены.)

С практической точки зрения большинство очевидных различий между версиями исчисления для доменов и кортежей основано на том, что версия для доменов поддерживает дополнительную форму условия, которое мы будем называть условием принадлежности. В общем виде условие принадлежности можно записать так:

$R (pair, pair, \dots),$

где R - это имя отношения, а каждая пара $pair$ имеет вид $A : v$ (где A - атрибут отношения R , а v - или переменная домена, или литерал). Проверка условия дает значение истина, если и только если существует кортеж в отношении R , имеющий определенные значения для определенных атрибутов. Например, вычисление выражения

$SP (Sno : 1, Pno : 1)$ - дает значение истина, если и только если в отношении SP существует кортеж со значением Sno , равным 1, и значением Pno , равным 1. Аналогично, условие принадлежности

SP (Sno : SX, Pno : PX) - принимает значение истина, если и только если в отношении SP существует кортеж со значением Sno, эквивалентным текущему значению переменной домена SX (какому бы то ни было), и значением Pno, эквивалентным текущему значению переменной домена PX (опять же какому бы то ни было).

Далее будем подразумевать существование переменных доменов с именами: образуемыми добавлением букв X, Y, Z, ... к соответствующим именам доменов. Напомним, что в базе данных поставщиков и деталей каждый атрибут имеет такое же имя, как и соответствующий ему домен, за исключением атрибутов Sname и Pname, для которых соответствующий домен называется просто Name.

Множество всех номеров поставщиков.	(SX)
Множество всех номеров поставщиков отношения S.	(SX) WHERE S (Sno : SX)
Подмножество номеров поставщиков из города Смоленск.	(SX) WHERE S (Sno : SX, City : 'Смоленск')
Получить номера и города поставщиков, поставляющих деталь под номером 2. Обратите внимание, что для этого запроса, выраженного в терминах исчисления кортежей, требовался квантор существования; заметьте также, что это первый из приведенных здесь примеров, где действительно необходимы скобки для списка целевых элементов.	(SX, CityX) WHERE S (Sno : SX, City : CityX) AND SP (Sno : SX, Pno : 2)
Получить такие пары 'номер поставщика' – 'номер детали', что поставщики и детали размещены в одном городе".	(SX, PX) WHERE S (Sno : SX, City : CityX) AND P (Pno : PX, City : CityY) AND CityX <> CityY

Для сравнения с реляционной алгеброй некоторые примеры соответствуют рассмотренным ранее.

Получить номера поставщиков из Смоленска со статусом, большим 20	SX WHERE EXISTS StatusX (StatusX > 20 AND S (Sno : SX, Status : StatusX, City : 'Смоленск')) Обратите внимание, что кванторы все еще требуются. Этот пример несколько неуклюжий по сравнению с его аналогом, выраженным в терминах исчисления кортежей. С другой стороны, конечно, есть случаи, когда верно обратное; смотрите, в частности, более сложные примеры, приведенные ниже.
Получить все такие пары номеров поставщиков, что два поставщика размещаются в одном городе	(SX AS FirstSno, SY AS SecondSno) WHERE EXSIST CityZ (S (Sno : SX, City : CityZ) AND S (Sno : SY, City : CityZ) AND SX < SY)

Получить имена поставщиков, которые поставляют по крайней мере одну красную деталь	NAMEX WHERE EXISTS SX EXISTS PX (S (Sno : SX, Sname : NameX) AND SP (Sno : SX, Pno : PX) AND P (Pno : PX, Color : 'Красный'))
Получить имена поставщиков, которые поставляют все типы деталей	NameX WHERE EXISTS SX (S (Sno : SX, Sname : NameX) AND FORALL PX (IF P (Pno : PX) THEN SP (Sno : SX, Pno : PX)))
Получить имена поставщиков, которые не поставляют деталь с номером 2	NameX WHERE EXISTS SX (S (Sno : SX, Sname : NameX) AND NOT SP (Sno : SX, Pno : 2))
Получить номера поставщиков, которые поставляют по крайней мере все типы деталей, поставляемых поставщиком с номером 2	SX WHERE FORALL PX (IF SP (Sno : 2, Pno : PX) THEN SP (Sno : SX, Pno : PX))
Получить номера деталей, которые или весят более 16 фунтов, или поставляются поставщиком с номером 2, или и то, и другое	PX WHERE EXISTS WeightX (P (Pno : PX, Weight : WeightX) AND WeightX > 16) OR SP (Sno : 2, Pno : PX)

Исчисление доменов, как и исчисление кортежей, формально эквивалентно реляционной алгебре (т.е. оно реляционно полно). Для доказательства можно сослаться, например, на работы Ульмана (Ullman).

Реляционное исчисление и реляционная алгебра

Ранее утверждалось, что реляционная алгебра и реляционное исчисление в своей основе эквивалентны. Обсудим это утверждение более подробно. Вначале Кодд показал в своей статье, что алгебра, по крайней мере, мощнее исчисления. (Термин "исчисление" будет использоваться для обозначения исчисления кортежей.) Он сделал это, придумав алгоритм, называемый "алгоритмом редукции Кодда", с помощью которого любое выражение исчисления можно преобразовать в семантически эквивалентное выражение алгебры. Мы не станем приводить здесь полностью этот алгоритм, а ограничимся примером, иллюстрирующим в общих чертах, как этот алгоритм функционирует.

В качестве основы для нашего примера используется база данных поставщиков, деталей и проектов. Для удобства приводится набор примерных значений для этой базы данных.

Таблица Поставщики (S)

Sno	Same	Status	City
1	Алмаз	20	Смоленск
2	Циклон	10	Владимир
3	Дельта	30	Владимир
4	Орион	20	Смоленск
5	Аргон	30	Ярославль

Таблица Детали (P) Pno Pname

Pno	Pname	Color	Weight	City
1	Гайка	Красный	12	Смоленск
2	Болт	Зеленый	17	Владимир
3	Винт	Синий	17	Рязань
4	Винт	Красный	14	Смоленск
5	Шайба	Синий	12	Владимир
6	Шпунт	Зеленый	19	Смоленск

Таблица Проекты (J)

Jno	Jname	City
1	Ангара	Владимир
2	Алтай	Рязань
3	Енисей	Ярославль
4	Амур	Ярославль

Jno	Jname	City
5	Памир	Смоленск
6	Чегет	Тверь
7	Эльбрус	Смоленск

Таблица Поставки (SPJ)

Sno	Pno	Jno	Qty
1	1	1	200
1	1	4	700
2	3	1	400
2	3	2	200
2	3	3	200
2	3	4	500
2	3	5	600
2	3	6	400
2	3	7	800
2	5	2	100
3	3	1	200
3	4	2	500
4	6	3	300
4	6	7	300
5	2	2	200
5	2	4	100
5	5	5	500
5	5	7	100
5	6	2	200
5	1	4	100
5	3	4	200
5	4	4	800
5	5	4	400
5	6	4	500

Рассмотрим запрос: "Получить имена и города поставщиков, обеспечивающих по крайней мере один проект в городе Ярославль с поставкой по крайней мере 50 штук каждой детали". Выражение исчисления для этого запроса следующее:

(SX.Name, SX.City) WHERE EXISTS JX FORALL PX EXISTS SPJX (JX.City = 'Ярославль' AND JX.Jno = SPJX.Jno AND PX.Pno = SPJX.Pno AND SX.Sno = SPJX.Sno AND SPJX.Qty >= 50)

Здесь SX, PX, JX и SPJX - переменные кортежей, берущие свои значения из отношений S, P, J и SPJ соответственно. Теперь покажем, как вычислить это выражение, чтобы добиться необходимого результата.

Шаг 1. Для каждой переменной кортежа выбираем ее область значений (т.е. набор всех значений для этой переменной) если это возможно. Выражение «выбираем, если возможно» подразумевает, что существует условие выборки, встроенное в фразу WHERE, которую можно использовать, чтобы сразу исключить из рассмотрения некоторые кортежи. В нашем случае выбираются следующие наборы кортежей:

SX : Все кортежи отношения S 5 кортежей

PX : Все кортежи отношения P 6 кортежей

JX : Кортежи отношения J, в которых City = 'Ярославль' 2 кортежа SPJX : Кортежи отношения SPJ, в которых Qty >= 50 24 кортежа

Шаг 2. Строим декартово произведение диапазонов, выбранных на первом шаге. Получим ... Для экономии места таблица не приводится. Полное произведение содержит $5*6*2*24 = 1440$ кортежей.

Шаг 3. Осуществляем выборку из произведения, построенного на втором шаге в соответствии с частью «условие соединения» фразы WHERE. В нашем примере эта часть следующая:

JX.Jno = SPJX.Jno AND PX.Pno = SPJX.Pno AND SX.Sno = SPJX.Sno AND

Поэтому из произведения исключаются кортежи, для которых значение Sno поставщика не равно значению Sno поставки, значение Pno детали не равно значению Pno поставки, значение Jno проекта не равно значению Jno поставки, после чего получаем подмножество декартова произведения, состоящее только из 10 кортежей.

Шаг 4. Применяем кванторы справа налево следующим образом:

- Для квантора «EXISTS RX» (где RX - переменная кортежа, принимающая значение на не котором отношении R) проецируем текущий промежуточный результат, чтобы исключить все атрибуты отношения R.
- Для квантора «FORALL RX» делим текущий промежуточный результат на отношение «выбранной области значений», соответствующее RX, которое было получено выше. При выполнении этой операции также будут исключены все атрибуты отношения R.

В нашем примере имеем следующие кванторы: EXISTS JX FORALL PX EXISTS SPJX. Выполняем соответствующие операции.

- EXISTS SPJX. Проецируем, исключая атрибуты отношения SPJ (SPJ.Sno, SPJ.Pno, SPJ.Jno и SPJ.Qty). В результате получаем:

S no	Same	Status	City	P no	Pname	Color	Weight	City	Jno	Jname	City
1	Алмаз	20	Смоленск	1	Гайка	Красный	12	Смоленск	4	Амур	Ярославль
2	Циклон	10	Владимир	3	Винт	Синий	17	Рязань	3	Енисей	Ярославль
2	Циклон	10	Владимир	3	Винт	Синий	17	Рязань	4	Амур	Ярославль
4	Орион	20	Смоленск	6	Шпон	Красный	19	Смоленск	3	Енисей	Ярославль
5	Аргон	30	Ярославль	2	Болт	Зеленый	17	Владимир			Ярославль
5	Аргон	30	Ярославль	1	Гайка	Красный	12	Смоленск	4	Амур	Ярославль
5	Аргон	30	Ярославль	3	Винт	Синий	17	Рязань	4	Амур	Ярославль
5	Аргон	30	Ярославль	4	Винт	Красный	14	Смоленск	4	Амур	Ярославль
5	Аргон	30	Ярославль	5	Шайба	Синий	12	Владимир	4	Амур	Ярославль
5	Аргон	30	Ярославль	6	Шпунт	Красный	19	Смоленск	4	Амур	Ярославль

- FORALL PX. Делим на отношение P. В результате получаем: 17

Sno	Same	Status	City	Jno	Jname	City
5	Аргон	30	Ярославль	4	Амур	Ярославль

- EXISTS JX. Проецируем, исключая атрибуты отношения J (J.Jno, J.name и J.City). В результате получаем: Sname Status City

Sno	Same	Status	City
5	Аргон	30	Ярославль

Шаг 5. Проецируем результат шага 4 в соответствии со спецификациями в целевом списке элементов. В нашем примере целевым элементом списка будет: SX.Sname, SX.City. Следовательно, конечный результат таков:

Same	City
Аргон	Ярославль

Из сказанного выше следует, что начальное выражение исчисления семантически эквивалентно определенному вложенному алгебраическому выражению, а если быть более точным, то проекции от проекции деления проекции выборки из произведения четырех выборок (!). Этим завершаем пример. Конечно, можно намного улучшить алгоритм, хотя многие подробности скрыты в наших пояснениях; но вместе с тем, необходим адекватный пример, предлагающий общую идею.

Теперь можно объяснить одну из причин (и не только одну) определения Коддом ровно восьми алгебраических операторов. Эти восемь операторов обеспечивают соглашение целевого языка, как средства возможной реализации исчисления. Другими словами, для

данного языка, такого как QUEL, который основывается на исчислении, одно из возможных применений заключается в том, чтобы можно было брать запрос в том виде, в каком он предоставляется пользователем (являющийся, по существу, просто выражением исчисления), и применять к нему алгоритм получения эквивалентного алгебраического выражения. Это алгебраическое выражение, конечно, содержит множество алгебраических операций, которые согласно определению по своей природе выполнимы. (Следующий шаг состоит в продолжении оптимизации этого алгебраического выражения.)

Также необходимо отметить, что восемь алгебраических операторов Кодда являются мерой оценки выразительной силы любого языка баз данных (существующего или предлагаемого). Обсудим этот вопрос подробнее.

Во-первых, язык называется реляционно полным, если он по своим возможностям, по крайней мере, не уступает реляционному исчислению, т.е. любое отношение, которое можно определить с помощью реляционного исчисления, также можно определить и с помощью некоторого выражения рассматриваемого языка. «Реляционно полный» - значит, не уступающий по возможностям алгебре, а не исчислению, но это то же самое. По сути, из существования алгоритма преобразования Кодда немедленно следует, что реляционная алгебра обладает реляционной полнотой.)

Реляционную полноту можно рассматривать как основную меру возможностей выборки и выразительной силы языков баз данных вообще. В частности, так как исчисление и алгебра - реляционно полные, они могут служить базисом для проектирования языков, не уступающих им по выразительности, без необходимости пересортировки для использования циклов - особенно важное замечание, если язык предназначается конечным пользователям, хотя оно также применимо, если язык предназначается прикладным программистам.

Далее, поскольку алгебра реляционно полная, то, чтобы доказать, что некоторый язык L обладает реляционной полнотой, достаточно показать, что в языке L есть аналоги всех восьми алгебраических операций (на самом деле достаточно показать, что в нем есть аналоги пяти примитивных операций) и что операндами любой операции языка L могут быть любые выражения этого языка. Язык SQL - это пример языка, реляционную замкнутость которого можно показать таким способом. Язык QUEL - еще один такой пример. В действительности на практике часто проще показать, что в данном языке есть эквиваленты алгебраических операций, чем найти в нем эквиваленты выражений исчисления. Именно поэтому реляционная полнота обычно определяется в терминах алгебраических выражений, а не выражений исчисления.

Функциональные зависимости

Для демонстрации основных идей данной темы, будет использоваться несколько измененная версия отношения поставок SP, содержащая атрибут City, представляющий город соответствующего поставщика. Это измененное отношение будет называться SCP.

Sno	City	Pno	Qty
1	Смоленск	1	100
1	Смоленск	2	100
2	Владимир	1	200
2	Владимир	2	200
3	Владимир	2	300
4	Смоленск	2	400
4	Смоленск	4	400
4	Смоленск	5	400

Следует четко различать:

1. значение этого отношения в определенный момент времени;

2. и набор всех возможных значений, которые данное отношение может принимать в различные моменты времени.

Примеры ФЗ, которым удовлетворяет отношение SCP в данном состоянии:

$\{Sno\} \rightarrow \{City\}$

$\{Sno, Pno\} \rightarrow \{Qty\}$

$\{Sno, Pno\} \rightarrow \{City\}$

$\{Sno, Pno\} \rightarrow \{City, Qty\}$

$\{Sno, Pno\} \rightarrow \{Sno\}$

$\{Sno, Pno\} \rightarrow \{Sno, Pno, City, Qty\}$

$\{Sno\} \rightarrow \{Qty\}$

$\{Qty\} \rightarrow \{Sno\}$

Определение 1. Пусть R - это отношение, а X и Y - произвольные подмножества множества атрибутов отношения R. Тогда Y функционально зависит от X, что в символическом виде записывается как $X \rightarrow Y$ тогда и только тогда? Когда любое значение множества X связано в точности с одним значением множества Y.

Левая и правая стороны ФЗ будут называться детерминантом и зависимой частью соответственно.

Определение 2. Пусть R является переменной-отношением, а X и Y - произвольными подмножествами множества атрибутов переменной-отношения R. Тогда $X \rightarrow Y$ тогда и только тогда, когда для любого допустимого значения отношения R любое значение X связано в точности с одним значением Y.

Определение 2а. Пусть $R(A_1, A_2, \dots, A_n)$ - схема отношения. Функциональная зависимость, обозначаемая $X \rightarrow Y$ между двумя наборами атрибутов X и Y, которые являются подмножествами R определяет ограничение на возможность существования кортежа в некотором отношении r. Ограничение означает, что для любых двух кортежей t1 и t2 в r, для которых имеет место $t1[X] = t2[X]$, также имеет место $t1[Y] = t2[Y]$.

1. Если ограничение на схеме отношения R утверждает, что не может быть более одного кортежа со значением атрибутов X в любом отношении экземпляре отношения r, то X является потенциальным ключом R. Это означает, что $X \rightarrow Y$ для любого подмножества атрибутов Y из R. Если X является потенциальным ключом R, то $X \rightarrow R$.
2. Если $X \rightarrow Y$ в R, это не означает, что $Y \rightarrow X$ в R.

Функциональная зависимость является семантическим свойством, т. е. свойством значения атрибутов.

Примеры безотносительных ко времени ФЗ для переменной-отношения SCP:

$\{Sno, Pno\} \rightarrow \{Sno, Pno\} \rightarrow \{Sno, Pno\} \rightarrow \{Sno, Pno\} \rightarrow \{Sno, Pno\} \rightarrow \{Sno\} \rightarrow$
 $\{Qty\}$
 $\{City\}$
 $\{City, Qty\}$
 $\{Sno\}$
 $\{Sno, Pno, City, Qty\} \{City\}$

Следует обратить внимание на ФЗ, которые выполняются для отношения SCP, но не выполняются «всегда» для переменной-отношения SCP:

$\{Sno\} \rightarrow \{Qty\} \quad \{Qty\} \rightarrow \{Sno\}$

Если X является потенциальным ключом переменной-отношения R , то все атрибуты Y переменной-отношения R должны быть обязательно ФЗ от X (это следует из определения потенциального ключа). Если переменная-отношение R удовлетворяет ФЗ $X \rightarrow Y$ и X не является потенциальным ключом, то R будет характеризоваться некоторой избыточностью. Например, в случае отношения SCP сведения о том, что каждый данный поставщик находится в данном городе, будут повторяться много раз (это хорошо видно из таблицы).

Эта избыточность приводит к разным аномалиям обновления, получившим такое название по историческим причинам. Под этим понимаются определенные трудности, появляющиеся при выполнении операций обновления $INSERT$, $DELETE$ и $UPDATE$. Рассмотрим избыточность, соответствующую ФЗ ($Sno \rightarrow City$). Ниже поясняются проблемы, которые возникнут при выполнении каждой из указанных операций обновления.

1. Операция $INSERT$. Нельзя поместить в переменную-отношение SCP информацию о том, что некоторый поставщик находится в определенном городе, не указав сведения хотя бы об одной детали, поставляемой этим поставщиком.
2. Операция $DELETE$. Если из переменной-отношения SCP удалить кортеж, который является единственным для некоторого поставщика, будет удалена не только информация о поставке поставщиком некоторой детали, но также информация о том, что этот поставщик находится в определенном городе. В действительности проблема заключается в том, что в переменной-отношении SCP содержится слишком много собранной в одном месте информации, поэтому при удалении некоторого кортежа теряется слишком много информации.
3. Операция $UPDATE$. Название города для каждого поставщика повторяется в переменной-отношении SCP несколько раз, и эта избыточность приводит к возникновению проблем при обновлении.

Для решения всех этих проблем, как предлагалось выше, необходимо выполнить декомпозицию переменной-отношения SCP на две следующие переменные-отношения.

$S' \{ Sno, City \}$

$SP \{ Sno, Pno, Qty \}$

Даже если ограничиться рассмотрением ФЗ, которые выполняются «всегда», множество ФЗ, выполняющихся для всех допустимых значений данного отношения, может быть все еще очень большим. Поэтому встает задача сокращения множества ФЗ до компактных размеров. Важность этой задачи вытекает из того, что ФЗ являются ограничениями целостности. Поэтому при каждом обновлении данных в СУБД все они должны быть проверены. Следовательно, для заданного множества ФЗ S желательно найти такое множество T , которое (в идеальной ситуации) было бы гораздо меньшего размера, чем множество S , причем каждая ФЗ множества S могла бы быть заменена ФЗ множества T . Если бы такое множество T было найдено, то в СУБД достаточно было бы использовать ФЗ из множества T , а ФЗ из множества S подразумевались бы автоматически.

Очевидным способом сокращения размера множества ФЗ было бы исключение тривиальных зависимостей, т. е. таких, которые не могут не выполняться. Примером тривиальной ФЗ для отношения SCP может быть $\{ Sno, Pno \} \rightarrow \{ Sno \}$.

Определение 3. ФЗ ($X \rightarrow Y$) тривиальная тогда и только тогда, когда Y включается в X .

Одни ФЗ могут подразумевать другие ФЗ. Например, зависимость

$\{ Sno, Pno \} \rightarrow \{ City, Qty \}$

подразумевает следующие ФЗ

$\{ Sno, Pno \} \rightarrow City \quad \{ Sno, Pno \} \rightarrow Qty$

В качестве более сложного примера можно привести переменную-отношение R с атрибутами A , B и C , для которых выполняются ФЗ ($A \rightarrow B$) и ($B \rightarrow C$). В этом случае также выполняется ФЗ ($A \rightarrow C$), которая называется транзитивной ФЗ.

Определение 4. Множество всех ФЗ, которые задаются данным множеством ФЗ S , называется замыканием S и обозначается символом S^+ .

Из сказанного выше становится ясно, что для выполнения сформулированной задачи следует найти способ вычисления S^+ на основе S .

Первая попытка решить эту проблему принадлежит Армстронгу (Armstrong), который предложил набор правил вывода новых ФЗ на основе заданных (эти правила также называются аксиомами Армстронга).

Пусть в перечисленных ниже правилах A , B и C – произвольные подмножества множества атрибутов заданной переменной-отношения R , а символическая запись AB означает $\{A, B\}$. Тогда правила вывода определяются следующим образом.

1. Правило **рефлексивности**: (B принадлежит A) следовательно ($A \rightarrow B$)
2. Правило **дополнения**: ($A \rightarrow B$) следовательно $AC \rightarrow BC$
3. Правило **транзитивности**: ($A \rightarrow B$) и ($B \rightarrow C$) следовательно ($A \rightarrow C$)

Каждое из этих правил может быть непосредственно доказано на основе определения ФЗ. Более того, эти правила являются полными в том смысле, что для заданного множества ФЗ S минимальный набор ФЗ, которые подразумевают все зависимости из множества S , может быть выведен из S на основе этих правил. Они также являются исчерпывающими, поскольку никакие дополнительные ФЗ (т.е. ФЗ, которые не подразумеваются ФЗ множества S) с их помощью не могут быть выведены. Иначе говоря, эти правила могут быть использованы для получения замыкания S^+ .

Из трех описанных выше правил для упрощения задачи практического вычисления замыкания S^+ можно вывести несколько дополнительных правил. (Примем, что D – это другое произвольное подмножество множества атрибутов R .)

4. Правило **самоопределения**: $A \rightarrow A$
5. Правило **декомпозиции**: ($A \rightarrow BC$) следовательно ($A \rightarrow B$) и ($A \rightarrow C$)
6. Правило **объединения**: ($A \rightarrow B$) и ($A \rightarrow C$) следовательно ($A \rightarrow BC$)
7. Правило **композиции**: ($A \rightarrow B$) и ($C \rightarrow D$) следовательно ($AC \rightarrow BD$)

Кроме того, Дарвен (Darwen) доказал следующее правило, которое назвал общей теоремой объединения:

8. ($A \rightarrow B$) и ($C \rightarrow D$) следовательно ($A(C \rightarrow B) \rightarrow BD$).

Упражнение. Пусть дана некоторая переменная-отношение R с атрибутами A, B, C, D, E, F и следующими ФЗ:

$A \rightarrow BC$

$B \rightarrow E$

$CD \rightarrow EF$

Показать, что для переменной-отношения R также выполняется ФЗ ($AD \rightarrow F$), которая вследствие этого принадлежит замыканию заданного множества ФЗ.

1. $A \rightarrow BC$ (дано)
2. $A \rightarrow C$ (следует из п. 1 согласно правилу декомпозиции)
3. $AD \rightarrow CD$ (следует из п. 2 согласно правилу дополнения)
4. $CD \rightarrow EF$ (дано)
5. $AD \rightarrow EF$ (следует из п. 3 и 4 согласно правилу транзитивности)
6. $AD \rightarrow F$ (следует из п. 5 согласно правилу декомпозиции)

Хотя эффективный алгоритм для вычисления замыкания S^+ на основе множества ФЗ S так и не сформулирован, можно описать алгоритм, который определяет, будет ли данная ФЗ находиться в данном замыкании. Для этого, прежде всего, следует дать определение понятию **суперключ**.

Определение 5. Суперключ переменной-отношения R - это множество атрибутов переменной-отношения R , которое содержит в виде подмножества (но не обязательно собственного подмножества), по крайней мере, один потенциальный ключ.

Из этого определения следует, что суперключи для данной переменной-отношения R - это такие подмножества K множества атрибутов переменной-отношения R , что ФЗ ($K \rightarrow A$) истинна для каждого атрибута A переменной-отношения R .

Предположим, что известны некоторые ФЗ, выполняющиеся для данной переменной-отношения, и требуется определить потенциальные ключи этой переменной-отношения. По определению потенциальными ключами называются неприводимые суперключи. Таким образом, выясняя, является ли данное множество атрибутов K суперключом, можно в значительной степени продвинуться к выяснению вопроса, является ли K потенциальным ключом. Для этого нужно определить, будет ли набор атрибутов переменной-отношения R множеством всех атрибутов, функционально зависящих от K . Таким образом, для заданного множества зависимостей S , которые выполняются для переменной-отношения R , необходимо найти способ определения множества всех атрибутов переменной-отношения R , которые функционально зависимы от K , т.е. так называемое замыкание K^+ множества K для S .

Алгоритм вычисления этого замыкания имеет вид.

```
function Closure(K: SetOfAttr; S: SetOfFD): SetOfAttr;
var
    Jold, Jnew: SetOfAttr;
begin
    Jnew := K;
    repeat
        Jold := Jnew;
        foreach ((X  $\rightarrow$  Y in S) do
            if (X принадлежит Jnew) then Jnew=Jnew + Y;
        until (Jold = Jnew);
    return Jold;
end; // of Closure
```

Определение. Суперключ в схеме отношения $R(A_1, A_2, \dots, A_n)$ - это набор атрибутов S из R со свойством, что ни для каких двух кортежей t_1 и t_2 в любом отношении над схемой R не будет выполняться равенство $t_1[S] = t_2[S]$.

Определение. Потенциальный ключ K - это суперключ с дополнительным свойством: удаление любого атрибута из K приведет к тому, что K перестанет быть суперключом.

Определение. Атрибут в схеме отношения R называется первичным атрибутом R , если он является членом некоторого потенциального ключа R . Атрибут называется непровинным, если он не является первичным атрибутом, то есть, если он не является членом какого-либо потенциального ключа.

Алгоритм нахождения ключа K схемы отношения R для заданного множество функциональных зависимостей F

Вход: Схема отношения R и множество функциональных зависимостей F на атрибутах R .

1. Пусть $K := R$.
2. Для каждого атрибута из K
 {
 вычислить Closure($\{K - A\}$, F);
 если замыкание содержит все атрибуты из R , то установите $K := K - \{A\}$
 };

Заметьте, что алгоритм определяет только один ключ из множества возможных ключей на R ; возвращаемый ключ зависит от порядка, в котором атрибуты удаляются из R на шаге 2.

Упражнение. Пусть дана переменная-отношение R с атрибутами A, B, C, D, E и F и следующими ФЗ:

$A \rightarrow BC$

$E \rightarrow CF$

$B \rightarrow E$

$CD \rightarrow EF$

Вычислить замыкание $\{A, B\}^+$ множества атрибутов $\{A, B\}$, исходя из заданного множества ФЗ.

1. Присвоим $Jold$ и $Jnew$ начальное значение - множество $\{A, B\}$.
2. Выполним внутренний цикл четыре раза - по одному разу для каждой заданной ФЗ. На первой итерации (для зависимости $A \rightarrow BC$) будет обнаружено, что левая часть действительно является подмножеством замыкания $Jnew$. Таким образом, к $Jnew$ можно добавить атрибуты B и C . $Jnew$ теперь представляет собой множество $\{A, B, C\}$.
3. На второй итерации (для зависимости $E \rightarrow CF$) обнаруживается, что левая часть не является подмножеством полученного до этого момента результата, который, таким образом, остается неизменным.
4. На третьей итерации (для зависимости $B \rightarrow E$) к $Jnew$ будет добавлено множество E , которое теперь будет иметь вид $\{A, B, C, E\}$.
5. На четвертой итерации (для зависимости $CD \rightarrow EF$) $Jnew$ останется неизменным.
6. Далее внутренний цикл выполняется еще четыре раза. На первой итерации результат останется прежним, на второй он будет расширен до $\{A, B, C, E, F\}$, а на третьей и четвертой - снова не изменится.
7. Наконец после еще одного четырехкратного прохождения цикла замыкание $Jnew$ останется неизменным и весь процесс завершится с результатом $\{A, B\}^+ = \{A, B, C, E, F\}$. // Конец упр.

Выводы.

1. Для заданного множества ФЗ S легко можно указать, будет ли заданная ФЗ ($X \rightarrow Y$) следовать из S , поскольку это возможно тогда и только тогда, когда множество Y является подмножеством замыкания X^+ множества X для заданного множества S . Таким образом, представлен простой способ определения, будет ли данная ФЗ ($X \rightarrow Y$) включена в замыкание S^+ множества S .
2. Напомним определение понятия суперключа. Суперключ переменной-отношения R - это множество атрибутов переменной-отношения R , которое в виде подмножества (но необязательно собственного подмножества) содержит по крайней мере один потенциальный ключ. Из этого определения прямо следует, что суперключи для данной переменной-отношения R - это такие подмножества K множества атрибутов переменной-отношения R , что ФЗ ($K \rightarrow A$) будет истинна для каждого атрибута A переменной-отношения R . Другими словами, множество K является суперключом тогда и только тогда, когда замыкание K^+ для множества K в пределах заданного множества ФЗ является множеством абсолютно всех атрибутов переменной-отношения R . (Кроме того, множество K является потенциальным ключом тогда и только тогда, когда оно является неприводимым суперключом).

Определение 6. Два множества ФЗ S_1 и S_2 эквивалентны тогда и только тогда, когда они являются покрытиями друг для друга, т. е. $S_1^+ = S_2^+$.

Упражнение.

Эквивалентны ли следующие множества ФЗ: $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ and $G = \{A \rightarrow CD, E \rightarrow AH\}$.

Проверим, что G покрывается множеством F и F покрывается множеством G

G покрывается множеством F :

$\{A\}^+ = \{A, C, D\}$ (относительно F), покрывается $A \rightarrow CD$ из G

$\{E\}^+ = \{E, A, D, H, C\}$ (относительно F), покрывается $E \rightarrow AH$ в G

F покрывается множеством G :

$\{A\}^+ = \{A, C, D\}$ (относительно G), покрывается $A \rightarrow C$ в F

$\{A, C\}^+ = \{A, C, D\}$ (относительно G), покрывается $AC \rightarrow D$ в F
 $\{E\}^+ = \{E, A, H, C, D\}$ (относительно G), покрывается $E \rightarrow AD$ и $E \rightarrow H$ в F

Каждое множество ФЗ эквивалентно, по крайней мере, одному неприводимому множеству.

Определение 7. Множество ФЗ является неприводимым тогда и только тогда, когда оно обладает всеми перечисленными ниже свойствами.

1. Каждая ФЗ этого множества имеет одноэлементную правую часть.
2. Ни одна ФЗ множества не может быть устранена без изменения замыкания этого множества.
3. Ни один атрибут не может быть устранен из левой части любой ФЗ данного множества без изменения замыкания множества.

Если I является неприводимым множеством, которое эквивалентно множеству S , то проверка выполнения ФЗ из множества I автоматически обеспечит выполнение ФЗ из множества S .

Пример. Рассмотрим переменную-отношение деталей P с функциональными зависимостями, перечисленными ниже.

$Pno \rightarrow Pname$
 $Pno \rightarrow Color$
 $Pno \rightarrow Weight$
 $Pno \rightarrow City$

Нетрудно заметить, что это множество функциональных зависимостей является неприводимым:

1. правая часть каждой зависимости содержит только один атрибут,
2. левая часть, очевидно, является неприводимой,
3. ни одна из функциональных зависимостей не может быть опущена без изменения замыкания множества (т. е. без утраты некоторой информации).

В противоположность этому приведенные ниже множества функциональных зависимостей не являются неприводимыми.

1. $Pno \rightarrow \{Pname, Color\}$
 $Pno \rightarrow Weight$
 $Pno \rightarrow City$

(Правая часть первой ФЗ не является одноэлементным множеством.)

2. $\{Pno, Pname\} \rightarrow Color$
 $Pno \rightarrow Pname$
 $Pno \rightarrow Weight$
 $Pno \rightarrow City$

(Первую ФЗ можно упростить, опустив атрибут $Pname$ в левой части без изменения замыкания, т. е. она не является неприводимой слева.)

3. $Pno \rightarrow Pno$
 $Pno \rightarrow Pname$
 $Pno \rightarrow Color$
 $Pno \rightarrow Weight$
 $Pno \rightarrow City$

(Здесь первую ФЗ можно опустить без изменения замыкания.)

Можно сделать утверждение, что для любого множества ФЗ существует, по крайней мере, одно эквивалентное множество, которое является неприводимым. Это достаточно легко продемонстрировать на следующем примере. Пусть дано исходное множество ФЗ S . Тогда благодаря правилу декомпозиции можно без утраты общности предположить, что каждая ФЗ в этом множестве S имеет одноэлементную правую часть. Далее для каждой ФЗ f из этого множества S следует проверить каждый атрибут A в левой части зависимости f . Если

множество S и множество зависимостей, полученное в результате устранения атрибута A в левой части зависимости f , эквивалентны, значит этот атрибут следует удалить. Затем для каждой оставшейся во множестве S зависимости f , если множества S и $S \setminus \{f\}$ эквивалентны, следует удалить зависимость f из множества S . Получившееся в результате таких действий множество S является неприводимым и эквивалентно исходному множеству S .

Алгоритм поиска минимального покрытия F для множества функциональных зависимостей E

Вход: Множество функциональных зависимостей E .

1. Пусть $F := E$.
2. Заменить каждую функциональную зависимость $X \rightarrow \{A_1, A_2, \dots, A_n\}$ из F на n функциональных зависимостей $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$.
3. Для каждой функциональной зависимости $X \rightarrow A$ из F
 1. Для каждого атрибута B , который является элементом X
 1. если $\{F - \{X \rightarrow A\}\}$ объединить $\{(X - \{B\}) \rightarrow A\}$ эквивалентно F заменить $X \rightarrow A$ на $(X - \{B\}) \rightarrow A$ в F .
4. Для каждой оставшейся функциональной зависимости $X \rightarrow A$ из F
 1. если $\{F - \{X \rightarrow A\}\}$ эквивалентно F , удалить $X \rightarrow A$ из F .

Упражнение. Пусть дана переменная-отношение R с атрибутами A, B, C, D и следующими функциональными зависимостями.

$A \rightarrow BC$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C$
 $AC \rightarrow D$

Найти неприводимое множество функциональных зависимостей эквивалентное данному множеству.

1. Прежде всего, следует переписать заданные ФЗ таким образом, чтобы каждая из них имела одноэлементную правую часть.

$A \rightarrow B$
 $A \rightarrow C$
 $B \rightarrow C$
 $A \rightarrow B$
 $AB \rightarrow C$
 $AC \rightarrow D$

Нетрудно заметить, что зависимость $A \rightarrow B$ записана дважды, так что одну из них можно удалить.

2. Затем в левой части зависимости $AC \rightarrow D$ может быть опущен атрибут C , поскольку дана зависимость $A \rightarrow C$, из которой по правилу дополнения можно получить зависимость $A \rightarrow AC$. Кроме того, дана зависимость $AC \rightarrow D$, из которой по правилу транзитивности можно получить зависимость $A \rightarrow D$. Таким образом, атрибут C в левой части исходной зависимости $AC \rightarrow D$ является избыточным.
3. Далее можно заметить, что зависимость $AB \rightarrow C$ может быть исключена, поскольку дана зависимость $A \rightarrow C$, из которой по правилу дополнения можно получить зависимость $AB \rightarrow CB$, а затем по правилу декомпозиции - зависимость $AB \rightarrow C$.
4. Наконец зависимость $A \rightarrow C$ подразумевается зависимостями $A \rightarrow B$ и $B \rightarrow C$, так что она также может быть отброшена. В результате получается неприводимое множество зависимостей.

$A \rightarrow B$
 $B \rightarrow C$
 $A \rightarrow D$

Множество ФЗ I , которое неприводимо и эквивалентно другому множеству ФЗ S , называется неприводимым покрытием множества S . Таким образом, с тем же успехом в системе вместо

исходного множества ФЗ S может использоваться его неприводимое покрытие I (здесь следует повторить, что для вычисления неприводимого эквивалентного покрытия I необязательно вычислять замыкание S^+). Однако необходимо отметить, что для заданного множества ФЗ не всегда существует уникальное неприводимое покрытие.

Проектирование базы данных может быть выполнено с использованием двух подходов: снизу вверх (bottom-up) или сверху вниз (top-down).

1. Методология проектирования bottom-up (также называемая методологией синтеза) рассматривает основные связи между отдельными атрибутами в качестве отправной точки и использует их, чтобы построить схемы отношений схем. Этот подход не пользуется популярностью на практике, потому что она страдает от проблемы того, чтобы собрать большое число бинарных связей между атрибутами в качестве отправной точки. На практике это сделать почти невозможно.
2. Методология проектирования top-down (также называемая методологией анализа) начинается с некоторого набора отношений, состоящих из атрибутов. Затем отношения анализируются отдельно или совместно, в результате чего происходит их декомпозиция до тех пор, пока не будут достигнуты все желаемые свойства. Неявными целями обеих методологий являются сохранение информации и минимизация избыточности.

Процесс нормализации схем отношений, основанный на операции декомпозиции, должен обладать следующими свойствами:

1. неаддитивностью JOIN или JOIN без потерь информации (NJP), которая гарантирует, что в результате декомпозиции не появятся лишние кортежи.
2. свойством сохранения зависимостей (DPP), которое гарантирует, что каждая функциональная зависимость будет представлена в каком-либо отдельном отношении после декомпозиции.

Свойство NJP является очень критическим и должно быть достигнуто любой ценой. Свойство DPP желательно, но не всегда достижимо.

Теория проектирования реляционных баз данных

При проектировании базы данных решаются две основные проблемы:

1. Каким образом отобразить объекты предметной области в абстрактные объекты модели данных, чтобы это отображение не противоречило семантике предметной области, и было, по возможности, лучшим (эффективным, удобным и т. д.)? Часто эту проблему называют проблемой логического проектирования баз данных.
2. Как обеспечить эффективность выполнения запросов к базе данных? Эту проблему обычно называют проблемой физического проектирования баз данных.

В случае реляционных баз данных нет общих рецептов по части физического проектирования. Здесь слишком много зависит от используемой СУБД. Поэтому ограничимся только существенными вопросами логического проектирования реляционных баз данных. Более того, не будем касаться определения ограничений целостности общего вида, а ограничимся ограничениями первичного и внешнего ключей. Будем считать, что проблема проектирования реляционной базы данных состоит в обоснованном принятии решений о том, из каких отношений должна состоять база данных, и какие атрибуты должны быть у этих отношений.

Классический подход к проектированию реляционных баз данных заключается в том, что сначала предметная область представляется в виде одного или нескольких отношений, а далее осуществляется процесс нормализации схем отношений, причем каждая следующая нормальная форма обладает свойствами лучшими, чем предыдущая. Каждой нормальной форме соответствует некоторый определенный набор ограничений, и отношение находится в некоторой нормальной форме, если удовлетворяет свойственному ей набору ограничений. Примером набора ограничений является ограничение первой нормальной формы – значения всех атрибутов отношения атомарны. Поскольку требование первой нормальной формы является базовым требованием классической реляционной модели данных, будем считать, что исходный набор отношений уже соответствует этому требованию.

В теории реляционных баз данных обычно выделяется следующая последовательность нормальных форм:

1. первая нормальная форма (1НФ или 1NF);
2. вторая нормальная форма (2НФ или 2NF);
3. третья нормальная форма (3НФ или 3NF);
4. нормальная форма Бойса-Кодда (НФБК или BCNF);
5. четвертая нормальная форма (4НФ или 4NF);
6. пятая нормальная форма, или нормальная форма проекции-соединения (5НФ или 5NF или PJ/NF).

Основные свойства нормальных форм такие:

1. каждая следующая нормальная форма в некотором смысле лучше предыдущей;
2. при переходе к следующей нормальной форме свойства предыдущих нормальных свойств сохраняются.

Процесс проектирования реляционной базы данных на основе метода нормализации преследует две основные цели:

1. избежать избыточности хранения данных;
2. устранить аномалии обновления отношений.

Эти цели являются актуальными для информационных систем оперативной обработки транзакций (On-Line Transaction Processing – OLTP), которым свойственны частые обновления базы данных, и потому аномалии обновления могут сильно вредить

эффективности приложения. В информационных системах оперативной аналитической обработки (On-Line Analytical Processing – OLAP), в частности, в системах поддержки принятия решений, базы данных в основном используются для выборки данных. Поэтому аномалиями обновления можно пренебречь. Из этого не следует, что принципы нормализации непригодны при проектировании баз данных OLAP-приложений. Даже если схема такой базы данных должна быть денормализована по соображениям эффективности, то чтобы получить правильную денормализованную схему, нужно сначала понять, как выглядит нормализованная схема.

В основе метода нормализации лежит декомпозиция отношения, находящегося в предыдущей нормальной форме, в два или более отношения, удовлетворяющих требованиям следующей нормальной формы. Считаются правильными такие декомпозиции отношения, которые обратимы, т. е. имеется возможность собрать исходное отношение из декомпозированных отношений без потери информации.

Наиболее важные на практике нормальные формы отношений основываются на фундаментальном в теории реляционных баз данных понятии функциональной зависимости.

Декомпозиция без потерь

Декомпозицией отношения R называется замена R на совокупность отношений $\{R_1, R_2, \dots, R_n\}$ такую, что каждое из них есть проекция R , и каждый атрибут R входит хотя бы в одну из проекций декомпозиции.

Например, для отношения R с атрибутами $\{a, b, c\}$ существуют следующие основные варианты декомпозиции:

- $\{a\}, \{b\}, \{c\}$
- $\{a\}, \{b, c\}$
- $\{a, b\}, \{c\}$
- $\{b\}, \{a, c\}$
- $\{a, b\}, \{b, c\}$
- $\{a, b\}, \{a, c\}$
- $\{b, c\}, \{a, c\}$
- $\{a, b\}, \{b, c\}, \{a, c\}$

Рассмотрим теперь отношение R' , которое получается в результате операции естественного соединения (NATURAL JOIN), применённой к отношениям, полученным в результате декомпозиции R .

Декомпозиция называется декомпозицией без потерь, если R' в точности совпадает с R .

Неформально говоря, при декомпозиции без потерь отношение «разделяется» на отношения-проекции таким образом, что из полученных проекций возможна «сборка» исходного отношения с помощью операции естественного соединения.

Далеко не всякая декомпозиция является декомпозицией без потерь. Проиллюстрируем это на примере отношения R с атрибутами $\{a, b, c\}$, приведённом выше. Пусть отношение R имеет вид:

a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

Декомпозиция $R_1 = \{a\}$, $R_2 = \{b, c\}$ имеет вид:

a	b	c
Москва	Россия	столица
Томск	Россия	не столица
Берлин	Германия	столица

Результат операции соединения этих отношений: $R' = R_1 \text{ NATURAL JOIN } R_2$

a	b	c
Москва	Россия	столица
Москва	Россия	не столица
Москва	Германия	столица
Томск	Россия	столица
Томск	Россия	не столица
Томск	Германия	столица
Берлин	Россия	столица
Берлин	Россия	не столица
Берлин	Германия	столица

Очевидно, что R' не совпадает с R , а значит такая декомпозиция не является декомпозицией без потерь. Рассмотрим теперь декомпозицию $R_1 = \{a, b\}$, $R_2 = \{a, c\}$:

a	b	a	c
Москва	Россия	Москва	столица
Томск	Россия	Томск	не столица
Берлин	Германия	Берлин	столица

Такая декомпозиция является декомпозицией без потерь, в чём читатель может убедиться самостоятельно.

В некоторых случаях отношение вообще невозможно декомпозировать без потерь. Существуют также примеры отношений, для которых нельзя выполнить декомпозицию без потерь на две проекции, но которые можно подвергнуть декомпозиции без потерь на три или большее количество проекций.

Первая нормальная форма

Отношение находится в 1НФ, если все его атрибуты являются простыми, все используемые домены должны содержать только скалярные значения. Не должно быть повторений строк в таблице. Например, есть таблица «Автомобили»:

Фирма	Модели
BMW	M5, X5M, M1
Nissan	GT-R

Нарушение нормализации 1НФ происходит в моделях BMW, т.к. в одной ячейке содержится список из 3 элементов: M5, X5M, M1, т.е. он не является атомарным. Преобразуем таблицу к 1НФ:

Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

Вторая нормальная форма

Отношение находится во 2НФ, если оно находится в 1НФ и каждый не ключевой атрибут неприводимо зависит от Первичного Ключа(ПК).

Неприводимость означает, что в составе потенциального ключа отсутствует меньшее подмножество атрибутов, от которого можно также вывести данную функциональную зависимость.

Например, дана таблица:

Модель	Фирма	Цена	Скидка
M5	BMW	5500000	5 %
X5M	BMW	6000000	5 %
M1	BMW	2500000	5 %
GT-R	Nissan	5000000	10 %

Таблица находится в первой нормальной форме, но не во второй. Цена машины зависит от модели и фирмы. Скидка зависит от фирмы, то есть зависимость от первичного ключа неполная. Исправляется это путем декомпозиции на два отношения, в которых не ключевые атрибуты зависят от ПК.

Модель	Фирма	Цена	Фирма	Скидка
M5	BMW	5500000	BMW	5 %
X5M	BMW	6000000	Nissan	10 %
M1	BMW	2500000		
GT-R	Nissan	5000000		

Третья нормальная форма

Отношение находится в 3НФ, когда находится во 2НФ и каждый не ключевой атрибут нетранзитивно зависит от первичного ключа. Проще говоря, второе правило требует выносить все не ключевые поля, содержимое которых может относиться к нескольким записям таблицы в отдельные таблицы.

Рассмотрим таблицу:

Модель	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Таблица находится во 2НФ, но не в 3НФ.

В отношении атрибут «Модель» является первичным ключом. Личных телефонов у автомобилей нет, и телефон зависит исключительно от магазина.

Таким образом, в отношении существуют следующие функциональные зависимости: Модель → Магазин, Магазин → Телефон, Модель → Телефон.

Зависимость Модель → Телефон является транзитивной, следовательно, отношение не находится в 3НФ.

В результате разделения исходного отношения получаются два отношения, находящиеся в 3НФ:

Магазин	Телефон	Модель	Магазин
Риал-авто	87-33-98	BMW	Риал-авто
Некст-Авто	94-54-12	Audi	Риал-авто
		Nissan	Некст-Авто

Нормальная форма Бойса-Кодда (НФБК) (частная форма третьей нормальной формы)

Определение 3НФ не совсем подходит для следующих отношений:

- 1) отношение имеет два или более потенциальных ключа;
- 2) два и более потенциальных ключа являются составными;
- 3) они пересекаются, т.е. имеют хотя бы один общий атрибут.

Для отношений, имеющих один потенциальный ключ (первичный), НФБК является 3НФ.

Отношение находится в НФБК, когда каждая нетривиальная и неприводимая слева функциональная зависимость обладает потенциальным ключом в качестве детерминанта.

Предположим, рассматривается отношение, представляющее данные о бронировании стоянки на день:

Номер стоянки	Время начала	Время окончания	Тариф
1	09:30	10:30	Бережливый
1	11:00	12:00	Бережливый
1	14:00	15:30	Стандарт
2	10:00	12:00	Премиум-В
2	12:00	14:00	Премиум-В
2	15:00	18:00	Премиум-А

Тариф имеет уникальное название и зависит от выбранной стоянки и наличии льгот, в частности:

- «Бережливый»: стоянка 1 для льготников
- «Стандарт»: стоянка 1 для не льготников
- «Премиум-А»: стоянка 2 для льготников
- «Премиум-В»: стоянка 2 для не льготников.

Таким образом, возможны следующие составные первичные ключи: {Номер стоянки, Время начала}, {Номер стоянки, Время окончания}, {Тариф, Время начала}, {Тариф, Время окончания}.

Отношение находится в 3НФ. Требования второй нормальной формы выполняются, так как все атрибуты входят в какой-то из потенциальных ключей, а неключевых атрибутов в отношении нет. Также нет и транзитивных зависимостей, что соответствует требованиям третьей нормальной формы. Тем не менее, существует функциональная зависимость Тариф → Номер стоянки, в которой левая часть (детерминант) не является потенциальным ключом отношения, то есть отношение не находится в нормальной форме Бойса — Кодда.

Недостатком данной структуры является то, что, например, по ошибке можно приписать тариф «Бережливый» к бронированию второй стоянки, хотя он может относиться только к первой стоянке.

Можно улучшить структуру с помощью декомпозиции отношения на два и добавления атрибута Имеет льготы, получив отношения, удовлетворяющие НФБК (подчёркнуты атрибуты, входящие в первичный ключ.):

Тариф	Время начала	Время окончания
Бережливый	09:30	10:30
Бережливый	11:00	12:00
Стандарт	14:00	15:30
Премиум-В	10:00	12:00
Премиум-В	12:00	14:00
Премиум-А	15:00	18:00

Тариф	Номер	Имеет
Бережливый	1	Да
Стандарт	1	Нет
Премиум-А	2	Да
Премиум-В	2	Нет

Четвертая нормальная форма

Отношение находится в 4НФ, если оно находится в НФБК и все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей.

В отношении R (A, B, C) существует многозначная зависимость $R.A \twoheadrightarrow R.B$ в том и только в том случае, если множество значений B, соответствующее паре значений A и C, зависит только от A и не зависит от C.

Предположим, что рестораны производят разные виды пиццы, а службы доставки ресторанов работают только в определенных районах города. Составной первичный ключ соответствующей переменной отношения включает три атрибута: {Ресторан, Вид пиццы, Район доставки}.

Ресторан	Вид пиццы	Район доставки
A1 Пицца	Толстая корка	Springfield
A1 Пицца	Толстая корка	Shelbyville
A1 Пицца	Толстая корка	Столица
A1 Пицца	Фаршированная корочка	Springfield
A1 Пицца	Фаршированная корочка	Shelbyville
A1 Пицца	Фаршированная корочка	Столица
Элитная пицца	Толстая корка	Столица
Элитная пицца	Фаршированная корочка	Столица
Пицца Винченцо	Толстая корка	Springfield
Пицца Винченцо	Толстая корка	Shelbyville
Пицца Винченцо	Толстая корка	Springfield
Пицца Винченцо	Толстая корка	Shelbyville

Такая переменная отношения не соответствует 4НФ, так как существует следующая многозначная зависимость:

{Ресторан} $\twoheadrightarrow$ {Вид пиццы}

{Ресторан} $\twoheadrightarrow$ {Район доставки}

То есть, например, при добавлении нового вида пиццы придется внести по одному новому кортежу для каждого района доставки. Возможна логическая аномалия, при которой определенному виду пиццы будут соответствовать лишь некоторые районы доставки из обслуживаемых рестораном районов.

Для предотвращения аномалии нужно декомпонировать отношение, разместив независимые факты в разных отношениях. В данном примере следует выполнить декомпозицию на {Ресторан, Вид пиццы} и {Ресторан, Район доставки}.

Однако, если к исходной переменной отношения добавить атрибут, функционально зависящий от потенциального ключа, например цену с учётом стоимости доставки ({Ресторан, Вид

пиццы, Район доставки} → Цена), то полученное отношение будет находиться в 4НФ и его уже нельзя подвергнуть декомпозиции без потерь.

Ресторан	Район доставки	Ресторан	Вид пиццы
A1 Пицца	Springfield	A1 Пицца	Толстая корка
A1 Пицца	Shelbyville	A1 Пицца	Фаршированная корочка
A1 Пицца	Столица	Элитная пицца	Толстая корка
Элитная пицца	Столица		Фаршированная корочка
Пицца Винченцо	Springfield	Пицца Винченцо	Толстая корка
Пицца Винченцо	Shelbyville		

Пятая нормальная форма

Отношение находится в пятой нормальной форме (иначе — в проекционно-соединительной нормальной форме) тогда и только тогда, когда каждая нетривиальная зависимость соединения в нём определяется потенциальным ключом (ключами) этого отношения.

Это очень жесткое требование, которое можно выполнить лишь при дополнительных условиях. На практике трудно найти пример реализации этого требования в чистом виде. Предположим, что нужно хранить данные об ассортименте нескольких продавцов, торгующих продукцией нескольких фирм (номенклатура товаров фирм может пересекаться):

Продавец	Фирма	Товар
Иванов	Рога и Копыта	Пылесос
Иванов	Рога и Копыта	Хлебница
Петров	Безенчук&Ко	Сучкорез
Петров	Безенчук&Ко	Пылесос
Петров	Безенчук&Ко	Хлебница
Петров	Безенчук&Ко	Зонт
Сидоров	Безенчук&Ко	Пылесос
Сидоров	Безенчук&Ко	Телескоп
Сидоров	Рога и Копыта	Пылесос
Сидоров	Рога и Копыта	Лампа
Сидоров	Геркулес	Вешалка

Если дополнительных условий нет, то данное отношение, которое находится в 4-й нормальной форме, является корректным и отражает все необходимые ограничения.

Теперь предположим, что нужно учесть следующее ограничение: каждый продавец имеет в своём ассортименте ограниченный список фирм и ограниченный список типов товаров и предлагает товары из списка товаров, производимые фирмами из списка фирм.

То есть продавец не имеет право торговать какими угодно товарами каких угодно фирм. Если продавец П имеет право торговать товарами фирмы Ф, и если продавец П имеет право

торговать товарами типа Т, то в этом случае в ассортимент продавца П входят товары типа Т фирмы Ф при условии, что фирма Ф производит товары типа Т.

Такое ограничение может быть вызвано, например, тем, что список типов товаров продавца ограничен имеющимися у него лицензиями, либо знаниями и квалификацией, необходимыми для их продажи, а список фирм каждого продавца определён партнёрскими соглашениями.

В рассматриваемом примере, в частности, предполагается, что продавец Иванов имеет право торговать товарами только фирмы «Рога и Копыта», продавец Петров — товарами только фирмы «Безенчук & Ко», зато продавец Сидоров не имеет права торговать хлебницами и сучкорезами и т. д.

Предложенное выше отношение не может исключить ситуации, при которых данное ограничение будет нарушено. Ничто не препятствует занести данные о торговле товаром, который данная фирма вообще не выпускает, либо данные о торговле товарами той фирмы, которую данный продавец не обслуживает, либо данные о торговле таким типом товара, который данный продавец не имеет право продавать.

Отношение не находится в 5NF, поскольку в нём есть нетривиальная зависимость соединения $*\{\{\text{Продавец, Фирма}\}, \{\text{Фирма, Товар}\}, \{\text{Продавец, Товар}\}\}$, однако подмножества $\{\text{Продавец, Фирма}\}, \{\text{Фирма, Товар}\}, \{\text{Продавец, Товар}\}$ не являются суперключами исходного отношения.

В данном случае для приведения к 5NF отношение должно быть разбито на три: $\{\text{Продавец, Фирма}\}, \{\text{Фирма, Товар}\}, \{\text{Продавец, Товар}\}$.

Продавец	Фирма	Фирма	Товар	Продавец	Товар
Иванов	Рога и Копыта	Рога и Копыта	Пылесос	Иванов	Пылесос
Петров	Безенчук&Ко	Рога и Копыта	Хлебница	Иванов	Хлебница
Сидоров	Безенчук&Ко	Рога и Копыта	Лампа	Петров	Сучкорез
Сидоров	Рога и Копыта	Безенчук&Ко	Сучкорез	Петров	Пылесос
Сидоров	Геркулес	Безенчук&Ко	Пылесос	Петров	Хлебница
		Безенчук&Ко	Хлебница	Петров	Зонт
		Безенчук&Ко	Зонт	Сидоров	Телескоп
		Безенчук&Ко	Телескоп	Сидоров	Пылесос
		Геркулес	Вешалка	Сидоров	Лампа
				Сидоров	Вешалка

Шестая нормальная форма

Переменная отношения находится в шестой нормальной форме тогда и только тогда, когда она удовлетворяет всем нетривиальным зависимостям соединения. Из определения следует, что переменная находится в 6НФ тогда и только тогда, когда она неприводима, то есть не может быть подвергнута дальнейшей декомпозиции без потерь. Каждая переменная отношения, которая находится в 6НФ, также находится и в 5НФ.

Идея «декомпозиции до конца» выдвигалась до начала исследований в области хронологических данных, но не нашла поддержки. Однако для хронологических баз данных максимально возможная декомпозиция позволяет бороться с избыточностью и упрощает поддержание целостности базы данных.

Для хронологических баз данных определены U_операторы, которые распаковывают отношения по указанным атрибутам, выполняют соответствующую операцию и упаковывают полученный результат. В данном примере соединение проекций отношения должно производится при помощи оператора U_JOIN.

Таб.№	Время	Должность	Домашний адрес
6575	01-01-2000:10-02-2003	слесарь	ул.Ленина,10
6575	11-02-2003:15-06-2006	слесарь	ул.Советская,22
6575	16-06-2006:05-03-2009	бригадир	ул.Советская,22

Переменная отношения «Работники» не находится в БНФ и может быть подвергнута декомпозиции на переменные отношения «Должности работников» и «Домашние адреса работников».

Таб.№	Время	Должность	Таб.№	Время	Домашний адрес
6575	01-01-2000: 10-02-2003	слесарь	6575	01-01-2000: 10-02-2003	ул.Ленина,10
6575	16-06-2006: 05-03-2009	бригадир	6575	11-02-2003: 15-06-2006	ул.Советская, 22

Транзакции

Транзакция — это последовательность операций, выполняемая как единое целое. В составе транзакций можно исполнять почти все операторы языка Transact-SQL. Если при выполнении транзакции не возникает никаких ошибок, то все модификации базы данных, сделанные во время выполнения транзакции, становятся постоянными. Транзакция выполняется по принципу «все или ничего». Транзакция не оставляет данные в промежуточном состоянии, в котором база данных не согласована. Транзакция переводит базу данных из одного целостного состояния в другое.

В качестве примера транзакции рассмотрим последовательность операций по приему заказа в коммерческой компании. Для приема заказа от клиента приложение ввода заказов должно:

- выполнить запрос к таблице товаров и проверить наличие товара на складе;
- добавить заказ к таблице счетов;
- обновить таблицу товаров, вычтя заказанное количество товаров из количества товара, имеющегося в наличии;
- обновить таблицу продаж, добавив стоимость заказа к объему продаж служащего, принявшего заказ;
- обновить таблицу офисов, добавив стоимость заказа к объему продаж офиса, в котором работает данный служащий.

Для поддержания целостности транзакция должна обладать четырьмя свойствами АСИД: атомарность, согласованность, изоляция и долговечность. Эти свойства называются также ACID-свойствами (от англ., atomicity, consistency, isolation, durability).

- Атомарность (Atomicity). Транзакция должна представлять собой атомарную (неделимую) единицу работы (исполняются либо все модификации, из которых состоит транзакция, либо ни одна).
- Согласованность (или Непротиворечивость) (Consistency). По завершении транзакции все данные должны остаться в согласованном состоянии. Чтобы сохранить целостность всех данных, необходимо выполнение модификации транзакций по всем правилам, определенным в реляционных СУБД.
- Изоляция (Isolation). Модификации, выполняемые одними транзакциями, следует изолировать от модификаций, выполняемых другими транзакциями параллельно. Уровни изоляции транзакции могут изменяться в широких пределах. На каждом уровне изоляции достигается определенный компромисс между степенью распараллеливания и степенью непротиворечивости. Чем выше уровень изоляции, тем выше степень непротиворечивости данных. Но чем выше степень непротиворечивости, тем ниже степень распараллеливания и тем ниже степень доступности данных.
- Долговечность (или Устойчивость) (Durability). По завершении транзакции ее результат должен сохраниться в системе, несмотря на сбой системы, либо (что касается незафиксированных транзакций) может быть полностью отменен вслед за сбоем системы.

Способы определения границ транзакций

Замечания:

- В данной теме не будут учитываться особенности режима MARS (Multiple Active Result Sets). Режим MARS — это функция, которая появилась в SQL Server 2005 и ADO.NET 2.0 для выполнения нескольких пакетов по одному соединению. По умолчанию режим MARS отключен. Включить его можно, добавив в строку соединения параметр «MultipleActiveResultSets=True».

- В данной теме не будут рассматриваться распределенные транзакции.

По признаку определения границ различают автоматические, неявные и явные транзакции.

Автоматические транзакции. Режим автоматической фиксации транзакций является режимом управления транзакциями SQL Server по умолчанию. В этом режиме каждая инструкция T-SQL выполняется как отдельная транзакция. Если выполнение инструкции завершается успешно, происходит фиксация; в противном случае происходит откат. Если возникает ошибка компиляции, то план выполнения пакета не строится и пакет не выполняется.

В следующем примере ни одна из инструкций INSERT во втором пакете не выполнится из-за ошибки компиляции. При этом произойдет откат первых двух инструкций INSERT, и они не будут выполняться:

```
CREATE TABLE Tab1 (  
    Col1 int NOT NULL PRIMARY KEY,  
    Col2 char(3)  
);  
  
GO  
INSERT INTO Tab1 VALUES (1, 'aaa');  
INSERT INTO Tab1 VALUES (2, 'bbb');  
INSERT INTO Tab1 VALUSE (3, 'ccc'); -- Синтаксическая ошибка.  
  
GO  
SELECT * FROM Tab1; -- Результат пустой. GO
```

В следующем примере третья инструкция INSERT вызывает ошибку дублирования первичного ключа во время выполнения. Поэтому первые две инструкции INSERT выполняются успешно и фиксируются, а третья инструкция INSERT вызывает ошибку времени выполнения и не выполняется.

```
CREATE TABLE Tab1 (  
    Col1 int NOT NULL PRIMARY KEY,  
    Col2 char(3)  
);  
  
GO  
INSERT INTO Tab1 VALUES (1, 'aaa');  
INSERT INTO Tab1 VALUES (2, 'bbb');  
INSERT INTO Tab1 VALUES (1, 'ccc'); -- Ошибка времени исполнения. GO  
SELECT * FROM Tab1; -- Возвращаются две строки.  
GO
```

Неявные транзакции. Если соединение работает в режиме неявных транзакций, то после фиксации или отката текущей транзакции SQL Server автоматически начинает новую транзакцию. В этом режиме явно указывается только граница окончания транзакции с помощью инструкций COMMIT TRANSACTION и ROLLBACK TRANSACTION. Для ввода в действие поддержки неявных транзакций применяется инструкция SET IMPLICIT_TRANSACTION ON. В конце каждого пакета необходимо отключать этот режим. По умолчанию режим неявных транзакций в SQL Server отключен.

В следующем примере сначала создается таблица Tab1, затем включается режим неявных транзакций, после чего начинаются две транзакции. После их исполнения режим неявных транзакций отключается:

```
CREATE TABLE Tab1 (  
    Col1 int NOT NULL PRIMARY KEY,  
    Col2 char(3) NOT NULL  
)  
  
GO  
SET IMPLICIT_TRANSACTIONS ON  
-- Первая неявная транзакция, начатая оператором INSERT  
INSERT INTO Tab1 VALUES (1, 'aaa')  
INSERT INTO Tab1 VALUES (2, 'bbb')  
COMMIT TRANSACTION -- Фиксация первой транзакции  
-- Вторая неявная транзакция, начатая оператором INSERT  
INSERT INTO Tab1 VALUES (3, 'ccc')  
SELECT * FROM Tab1  
COMMIT TRANSACTION -- Фиксация второй транзакции  
SET IMPLICIT_TRANSACTIONS OFF  
GO
```

Явные транзакции. Для определения явных транзакций используются следующие инструкции:

- BEGIN TRANSACTION – задает начальную точку явной транзакции для соединения;
- COMMIT TRANSACTION или COMMIT WORK – используется для успешного завершения транзакции, если не возникла ошибка;
- ROLLBACK TRANSACTION или ROLLBACK WORK – используется для отмены транзакции, во время которой возникла ошибка.
- SAVE TRANSACTION – используется для установки точки сохранения или маркера внутри транзакции. Точка сохранения определяет место, к которому может возвратиться транзакция, если часть транзакции условно отменена. Если транзакция откатывается к точке сохранения, то ее выполнение должно быть продолжено до завершения с обработкой дополнительных инструкций языка T-SQL, если необходимо, и инструкции COMMIT TRANSACTION, либо транзакция должна быть полностью отменена откатом к началу. Для отмены всей транзакции следует использовать инструкцию ROLLBACK TRANSACTION; в этом случае отменяются все инструкции транзакции.

В следующем примере демонстрируется использование точки сохранения транзакции для отката изменений:

```
USE pubs  
GO  
  
BEGIN TRANSACTION royaltychange  
    UPDATE titleauthor  
    SET royaltyper = 65  
    FROM titleauthor, titles  
    WHERE royaltyper = 75 AND titleauthor.title_id = titles.title_id  
        AND title = 'The Gourmet Microwave'
```

```

UPDATE titleauthor
SET royaltyper = 35
FROM titleauthor, titles
WHERE royaltyper = 25 AND titleauthor.title_id = titles.title_id
AND title = 'The Gourmet Microwave'
SAVE TRANSACTION percentchanged

```

/* После того, как обновлено royaltyper для двух авторов, вставляется точка сохранения percentchanged, а затем определяется, насколько изменится заработок авторов после увеличения на 10 процентов цены книги */

```

UPDATE titles
SET price = price * 1.1
WHERE title = 'The Gourmet Microwave'

```

```

SELECT (price * royalty * ytd_sales) * royaltyper
FROM titles, titleauthor
WHERE title = 'The Gourmet Microwave' AND titles.title_id = titleauthor.title_id

```

```

/* Откат транзакции до точки сохранения и фиксация транзакции в целом */
ROLLBACK TRANSACTION percentchanged
COMMIT TRANSACTION

```

Режим явных транзакций действует только на протяжении данной транзакции. После завершения явной транзакции соединение возвращается в режим, заданный до запуска этого режима, то есть в неявный или автоматический.

Функции для обработки транзакций

- @@TRANCOUNT возвращает число активных транзакций для текущего соединения. Инструкция BEGIN TRANSACTION увеличивает значение @@TRANCOUNT на 1, а инструкция ROLLBACK TRANSACTION уменьшает его до 0 (исключение — инструкция ROLLBACK TRANSACTION имя_точки_сохранения, которая не влияет на значение @@TRANCOUNT). Инструкции COMMIT TRANSACTION уменьшают значение @@TRANCOUNT на 1.
- XACT_STATE () сообщает о состоянии пользовательской транзакции текущего выполняемого запроса в соответствии с данными, представленными в таблице.

Возвращаемое значение	Пояснение
1	Текущий запрос содержит активную пользовательскую транзакцию и может выполнять любые действия, включая запись данных и фиксацию транзакции.
0	У текущего запроса нет активной пользовательской транзакции.
-1	В текущем запросе есть активная транзакция, однако произошла ошибка, из-за которой транзакция классифицируется как не фиксируемая. Запросу не удастся зафиксировать транзакцию или выполнить откат до точки сохранения; можно только запросить полный откат транзакции.

Ограничения.

- Функция @@TRANCOUNT не может использоваться для определения фиксируемости транзакции.
- Функция XACT_STATE не может использоваться для определения наличия вложенных транзакций.

Ошибки, возникающие в процессе обработки транзакций

Если ошибка делает невозможным успешное выполнение транзакции, SQL Server автоматически выполняет ее откат и освобождает ресурсы, удерживаемые транзакцией. Если сетевое соединение клиента с SQL Server разорвано, то после того, как SQL Server получит уведомление от сети о разрыве соединения, выполняется откат всех необработанных транзакций для этого соединения. В случае сбоя клиентского приложения, выключения либо перезапуска клиентского компьютера соединение также будет разорвано, а SQL Server выполнит откат всех необработанных транзакций после получения уведомления о разрыве от сети. Если клиент выйдет из приложения, выполняется откат всех необработанных транзакций.

В случае ошибки (нарушения ограничения целостности) во время выполнения инструкции в пакете по умолчанию SQL Server выполнит откат только той инструкции, которая привела к ошибке. Это поведение можно изменить с помощью инструкции SET XACT_ABORT. После выполнения инструкции SET XACT_ABORT ON любая ошибка во время выполнения инструкции приведет к автоматическому откату текущей транзакции.

На случай возникновения ошибок код приложения должен содержать исправляющее действие: COMMIT или ROLLBACK. Эффективным средством для обработки ошибок, включая ошибки транзакций, является конструкция языка Transact-SQL TRY...CATCH.

Пример

```
USE MyDB;
GO
IF OBJECT_ID ( N'dbo.SaveTranExample', N'P' ) IS NOT NULL DROP PROCEDURE
dbo.SaveTranExample;
GO

CREATE PROCEDURE dbo.SaveTranExample ... -- Список параметров AS
-- Надо проверить, была ли процедура вызвана из активной транзакции, и если да,
-- то установить точку сохранения для последующего использования.
-- @TranCounter = 0 означает, что активной транзакции нет и процедура явно начинает
-- локальную транзакцию.
-- @TranCounter > 0 означает, что активная транзакция началась еще до вызова процедуры.
DECLARE @TranCounter INT;
SET @TranCounter = @@TRANCOUNT;
IF @TranCounter > 0
    SAVE TRANSACTION ProcedureSave;
ELSE
    BEGIN TRANSACTION;
-- Пытаемся модифицировать базу данных.
BEGIN TRY
-- INSERT, UPDATE, DELETE
```

```

...
-- Здесь мы окажемся, если не произойдет никакой ошибки.
-- Если транзакция началась внутри процедуры, то выполнить COMMIT TRANSACTION.
-- Если транзакция началась до вызова процедуры, то выполнять COMMIT TRANSACTION
нельзя.
IF @TranCounter = 0
-- @TranCounter = 0 означает, что никакая транзакция до вызова процедуры не начиналась. --
Процедура должна завершить начатую в ней транзакцию.
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
-- Здесь мы окажемся, если произошла ошибка.
-- Надо определить, какой тип отката транзакции применять.
IF @TranCounter = 0
-- Транзакция началась в теле процедуры. -- Полный откат транзакции.
    ROLLBACK TRANSACTION;
    ELSE

-- Транзакция началась еще до вызова процедуры.
-- Нельзя отменять изменения, сделанные до вызова процедуры.
IF XACT_STATE() <> -1
-- Если транзакция активна, то выполнить откат до точки сохранения.
ROLLBACK TRANSACTION ProcedureSave;
-- Если транзакция активна, однако произошла ошибка, из-за которой транзакция
-- классифицируется как нефиксируемая,
-- то вернуться в точку вызова процедуры, где должен произойти откат внешней транзакции.
-- В точку вызова процедуры передается информация об ошибках.
DECLARE @ErrorMessage NVARCHAR(4000); DECLARE @ErrorSeverity INT;
DECLARE @ErrorState INT;
SELECT @ErrorMessage = ERROR_MESSAGE(); SELECT @ErrorSeverity =
ERROR_SEVERITY(); SELECT @ErrorState = ERROR_STATE();
RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState); END CATCH
GO

```

Пояснения к примеру

Для получения сведений об ошибках в области блока CATCH конструкции TRY...CATCH можно использовать следующие системные функции:

- ERROR_LINE() - возвращает номер строки, в которой произошла ошибка.
- ERROR_MESSAGE() - возвращает текст сообщения, которое будет возвращено приложению. Текст содержит значения таких подставляемых параметров, как длина, имена объектов или время.
- ERROR_NUMBER() - возвращает номер ошибки.
- ERROR_PROCEDURE() - возвращает имя хранимой процедуры или триггера, в котором произошла ошибка. Эта функция возвращает значение NULL, если данная ошибка не была совершена внутри хранимой процедуры или триггера.
- ERROR_SEVERITY() - возвращает уровень серьезности ошибки.
- ERROR_STATE() - возвращает состояние.

Инструкция RAISERROR позволяет вернуть приложению сообщение в формате системных ошибок или предупреждений.

Управлением параллельным выполнением транзакций

Когда множество пользователей одновременно пытаются модифицировать данные в базе данных, необходимо создать систему управления, которая защитила бы модификации, выполненные одним пользователем, от негативного воздействия модификаций, сделанных другими. Выделяют два типа управления параллельным выполнением:

- **Пессимистическое** управление параллельным выполнением.
- **Оптимистическое** управление параллельным выполнением.

Пессимистическое управление реализуется с помощью технологии блокировок, оптимистическое управления реализуется с помощью технологии версии строк.

Система блокировок не разрешает пользователям выполнить модификации, влияющие на других пользователей. Если пользователь выполнил какое-либо действие, в результате которого установлена блокировка, то другим пользователям не удастся выполнять действия, конфликтующие с установленной блокировкой, пока владелец не освободит ее. Пессимистическое управление используется главным образом в средах, где высока конкуренция за данные.

В случае управления на основе версий строк пользователи не блокируют данные при чтении. Во время обновления система следит, не изменил ли другой пользователь данные после их прочтения. Если другой пользователь модифицировал данные, генерируется ошибка. Как правило, получивший ошибку пользователь откатывает транзакцию и повторяет операцию снова. Этот способ управления в основном используется в средах с низкой конкуренцией за данные.

SQL Server поддерживает различные механизмы оптимистического и пессимистического управления параллельным выполнением. Пользователю предоставляется право определить тип управления параллельным выполнением, установив уровень изоляции транзакции для соединения и параметры параллельного выполнения для курсоров.

Если в случае пессимистического управления в СУБД не реализованы механизмы блокирования, то при одновременном чтении и изменении одних и тех же данных несколькими пользователями могут возникнуть следующие проблемы одновременного доступа:

проблема последнего изменения возникает, когда несколько пользователей изменяют одну и ту же строку, основываясь на ее начальном значении; тогда часть данных будет потеряна, т.к. каждая последующая транзакция перезапишет изменения, сделанные предыдущей. Выход из этой ситуации заключается в последовательном внесении изменений;

проблема "грязного" чтения (Dirty Read) возможна в том случае, если пользователь выполняет сложные операции обработки данных, требующие множественного изменения данных перед тем, как они обретут логически верное состояние. Если во время изменения данных другой пользователь будет считывать их, то может оказаться, что он получит логически неверную информацию. Для исключения подобных проблем необходимо производить считывание данных после окончания всех изменений;

проблема неповторяемого чтения (Non-repeatable or Fuzzy Read) является следствием неоднократного считывания транзакцией одних и тех же данных. Во время выполнения первой транзакции другая может внести в данные изменения, поэтому при повторном чтении первая транзакция получит уже иной набор данных, что приводит к нарушению их целостности или логической несогласованности;

проблема чтения фантомов (Phantom) появляется после того, как одна транзакция выбирает данные из таблицы, а другая вставляет или удаляет строки до завершения первой. Выбранные из таблицы значения будут некорректны.

Для решения перечисленных проблем в стандарте ANSI SQL определены четыре уровня блокирования. Уровень изоляции транзакции определяет, могут ли другие (конкурирующие) транзакции вносить изменения в данные, измененные текущей транзакцией, а также может ли текущая транзакция видеть изменения, произведенные конкурирующими транзакциями, и наоборот. Каждый последующий уровень поддерживает требования предыдущего и налагает дополнительные ограничения:

уровень 0 – запрещение «загрязнения» данных. Этот уровень требует, чтобы изменять данные могла только одна транзакция; если другой транзакции необходимо изменить те же данные, она должна ожидать завершения первой транзакции;

уровень 1 – запрещение «грязного» чтения. Если транзакция начала изменение данных, то никакая другая транзакция не сможет прочитать их до завершения первой;

уровень 2 – запрещение неповторяемого чтения. Если транзакция считывает данные, то никакая другая транзакция не сможет их изменить. Таким образом, при повторном чтении они будут находиться в первоначальном состоянии;

уровень 3 – запрещение фантомов. Если транзакция обращается к данным, то никакая другая транзакция не сможет добавить новые или удалить имеющиеся строки, которые могут быть считаны при выполнении транзакции. Реализация этого уровня блокирования выполняется путем использования блокировок диапазона ключей. Подобная блокировка накладывается не на конкретные строки таблицы, а на строки, удовлетворяющие определенному логическому условию.

Проблемы, связанные с параллельным выполнением транзакций, разрешают, используя уровни изоляции транзакции. От уровня изоляции зависит то, в какой степени транзакция влияет на другие транзакции и испытывает влияние других транзакций. Более низкий уровень изоляции увеличивает возможность параллельного выполнения, но за это приходится расплачиваться согласованностью данных. Напротив, более высокий уровень изоляции гарантирует согласованность данных, но при этом страдает параллельное выполнение.

Стандарт ISO определяет следующие уровни изоляции:

- **read uncommitted** (самый низкий уровень, при котором транзакции изолируются до такой степени, чтобы только уберечь от считывания физически поврежденных данных);
- **read committed** (уровень по умолчанию);
- изоляция повторяющегося чтения **repeatable read**;
- изоляция упорядочиваемых транзакций **serializable** (самый высокий уровень, при котором транзакции полностью изолированы друг от друга).

SQL Server также поддерживает еще два уровня изоляции транзакций, использующих управление версиями строк.

- **read committed** с использованием управления версиями строк;
- уровень изоляции моментальных снимков **snapshot**.

Следующая таблица показывает побочные эффекты параллелизма, допускаемые различными уровнями изоляции.

Уровень изоляции	«Грязное» чтение	Неповторяющееся чтение	Фантомное чтение
read uncommitted	Да	Да	Да
read committed	Нет	Да	Да
repeatable read	Нет	Нет	Да
snapshot	Нет	Нет	Нет
serializable	Нет	Нет	Нет

Для установки уровня изоляции используется следующая инструкция SET TRANSACTION ISOLATION LEVEL, имеющая следующий синтаксис:

SET TRANSACTION ISOLATION LEVEL

{ READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SNAPSHOT |
SERIALIZABLE }

[;]

Описание аргументов

READ UNCOMMITTED - указывает, что инструкции могут считывать строки, которые были изменены другими транзакциями, но еще не были зафиксированы.

READ COMMITTED - указывает, что инструкции не могут считывать данные, которые были изменены другими транзакциями, но еще не были зафиксированы. Это предотвращает чтение «грязных» данных. Данные могут быть изменены другими транзакциями между отдельными инструкциями в текущей транзакции, результатом чего будет неповторяемое чтение или недействительные данные.

Напоминание. Поведение READ COMMITTED зависит от настройки аргумента базы данных READ_COMMITTED_SNAPSHOT (находится в состоянии OFF по умолчанию). REPEATABLE READ - указывает на то, что инструкции не могут считывать данные, которые были изменены, но еще не зафиксированы другими транзакциями, а также на то, что другие транзакции не могут изменять данные, читаемые текущей транзакцией, до ее завершения.

SNAPSHOT - указывает на то, что данные, считанные любой инструкцией транзакции, будут согласованы на уровне транзакции с версией данных, существовавших в ее начале. Транзакция распознает только те изменения, которые были зафиксированы до ее начала. Инструкции, выполняемые текущей транзакцией, не видят изменений данных, произведенных другими транзакциями после запуска текущей транзакции. Таким образом достигается эффект получения инструкциями в транзакции моментального снимка зафиксированных данных на момент запуска транзакции. Перед запуском транзакции, использующей уровень изоляции моментальных снимков, необходимо установить параметр базы данных ALLOW_SNAPSHOT_ISOLATION в ON. Если транзакция с уровнем изоляции моментального снимка обращается к данным из нескольких баз данных, аргумент ALLOW_SNAPSHOT_ISOLATION должен быть включен в каждой базе данных.

SERIALIZABLE - указывает следующее:

- Инструкции не могут считывать данные, которые были изменены другими транзакциями, но еще не были зафиксированы.
- Другие транзакции не могут изменять данные, считываемые текущей транзакцией, до ее завершения.
- Другие транзакции не могут вставлять новые строки со значениями ключа, которые входят в диапазон ключей, считываемых инструкциями текущей транзакции, до ее завершения.

Замечания.

- Одновременно может быть установлен только один параметр уровня изоляции, который продолжает действовать для текущего соединения до тех пор, пока не будет явно изменен.
- Уровни изоляции транзакции определяют тип блокировки, применяемый к операциям считывания.
- В любой момент транзакции можно переключиться с одного уровня изоляции на другой, однако есть одно исключение. Это смена уровня изоляции на уровень изоляции SNAPSHOT. Такая смена приводит к ошибке и откату транзакции. Однако для транзакции, которая была начата с уровнем изоляции SNAPSHOT, можно установить любой другой уровень изоляции.
- Когда для транзакции изменяется уровень изоляции, ресурсы, которые считываются после изменения, защищаются в соответствии с правилами нового уровня. Ресурсы, которые считываются до изменения, остаются защищенными в соответствии с правилами предыдущего уровня. Например, если для транзакции уровень изоляции изменяется с READ COMMITTED на SERIALIZABLE, то совмещаемые блокировки, полученные после изменения, будут удерживаться до завершения транзакции.
- Если инструкция SET TRANSACTION ISOLATION LEVEL использовалась в хранимой процедуре или триггере, то при возврате управления из них уровень изоляции будет изменен на тот, который действовал на момент их вызова. Например, если уровень изоляции REPEATABLE READ устанавливается в пакете, а пакет затем вызывает хранимую процедуру, которая меняет уровень изоляции на SERIALIZABLE, при возвращении хранимой процедурой управления пакету, настройки уровня изоляции меняются назад на REPEATABLE READ.
- Определяемые пользователем функции и типы данных среды CLR не могут выполнять инструкцию SET TRANSACTION ISOLATION LEVEL. Однако уровень изоляции можно переопределить с помощью табличной подсказки.

Блокировки

Блокировка — это механизм, с помощью которого компонент Database Engine синхронизирует одновременный доступ нескольких пользователей к одному фрагменту данных. Прежде чем транзакция сможет распоряжаться текущим состоянием фрагмента данных, например, для чтения или изменения данных, она должна защититься от изменений этих данных другой транзакцией. Для этого транзакция запрашивает блокировку фрагмента данных. Существует несколько режимов блокировки, например общая или монопольная. Режим блокировки определяет уровень подчинения данных транзакции. Ни одна транзакция не может получить блокировку, которая противоречит другой блокировке этих данных, предоставленной другой транзакцией. Если транзакция запрашивает режим блокировки, противоречащий предоставленной ранее блокировке тех же данных, экземпляр компонента Database Engine приостанавливает ее работу до тех пор, пока первая блокировка не освободится.

При изменении фрагмента данных транзакция удерживает блокировку, защищая изменения до конца транзакции. Продолжительность блокировки, полученной для защиты операций чтения, зависит от уровня изоляции транзакции. Все блокировки, удерживаемые транзакцией, освобождаются после ее завершения (при фиксации или откате).

Приложения обычно не запрашивают блокировку напрямую. За управление блокировками отвечает внутренний компонент Database Engine, называемый диспетчером блокировок. Когда экземпляр компонента Database Engine обрабатывает инструкцию Transact-SQL, обработчик запросов компонента Database Engine определяет, к каким ресурсам требуется доступ. Обработчик запросов определяет, какие типы блокировок требуются для защиты каждого ресурса, в зависимости от типа доступа и уровня изоляции транзакции. Затем обработчик запросов запрашивает соответствующую блокировку у диспетчера блокировок. Диспетчер блокировок предоставляет блокировку, если она не противоречит блокировкам, удерживаемым другими транзакциями. Использование блокировки как механизма управления транзакциями может разрешить проблемы параллелизма. Блокировка позволяет запускать все транзакции в полной изоляции друг от друга, что позволяет одновременно выполнять несколько транзакций. Уровень, на котором транзакция готова к принятию противоречивых данных, называется уровнем изоляции. Чем выше уровень изоляции, тем ниже вероятность непоследовательности данных, однако при этом появляется такой недостаток, как сокращение параллелизма.

Каждая блокировка обладает тремя свойствами: гранулярностью (или размером блокировки), режимом (или типом блокировки) и продолжительностью.

Гранулярность блокировок и иерархии блокировок

SQL Server поддерживает мультигранулярную блокировку, позволяющую транзакции блокировать различные типы ресурсов. Чтобы уменьшить издержки применения блокировок, компонент Database Engine автоматически блокирует ресурсы на соответствующем уровне. Блокировка при меньшей гранулярности, например на уровне строк, увеличивает параллелизм, но в то же время увеличивает и накладные расходы на обработку, поскольку при большом количестве блокируемых строк требуется больше блокировок. Блокировки на большем уровне гранулярности, например на уровне таблиц, обходятся дорого в отношении параллелизма, поскольку блокировка целой таблицы ограничивает доступ ко всем частям таблицы других транзакций. Однако накладные расходы в этом случае ниже, поскольку меньше количество поддерживаемых блокировок.

Компонент Database Engine часто получает блокировки на нескольких уровнях гранулярности одновременно, чтобы полностью защитить ресурс. Такая группа блокировок на нескольких уровнях гранулярности называется иерархией блокировки. Например, чтобы полностью защитить операцию чтения индекса, экземпляру компоненту Database Engine может потребоваться получить разделяемые блокировки на строки и намеренные разделяемые блокировки на страницы и таблицу.

Следующая таблица содержит перечень ресурсов, которые могут блокироваться компонентом Database Engine.

Ресурс	Описание
RID	Идентификатор строки, используемый для блокировки одной строки в куче
KEY	Блокировка строки в индексе, используемая для защиты диапазонов значений ключа в сериализуемых транзакциях.
PAGE	8-килобайтовая (КБ) страница в базе данных, например страница данных или индекса.
EXTENT	Упорядоченная группа из восьми страниц, например страниц данных или индекса.
HOBT	Куча или сбалансированное дерево. Блокировка, защищающая индекс или кучу страниц данных в таблице, не имеющей кластеризованного индекса.
TABLE	Таблица полностью, включая все данные и индексы.
FILE	Файл базы данных.
APPLICATION	Определяемый приложением ресурс.
METADATA	Блокировки метаданных.
ALLOCATION_UNIT	Единица размещения.
DATABASE	База данных, полностью.

По умолчанию блокировки укрупняются с уровня отдельных строк до целых страниц даже до уровня таблицы из соображений повышения производительности. Хотя в общем случае укрупнение считается хорошей штукой, оно может создавать проблемы, например, когда один SPID блокирует всю таблицу, препятствуя другому SPID работать ней. Можно настроить параметры блокировки на уровне строк и страниц. Такие блокировки по умолчанию разрешены для индексов.

Режимы блокировки

SQL Server блокирует ресурсы с помощью различных режимов блокировки, которые определяют доступ одновременных транзакций к ресурсам. В следующей таблице показаны режимы блокировки ресурсов, применяемые компонентом Database Engine.

Режим блокировки	Описание
Совмещаемая блокировка (S)	Совмещаемая. Используется для операций считывания, которые не меняют и не обновляют данные, такие как инструкция SELECT.
Блокировка обновления (U)	Применяется к тем ресурсам, которые могут быть обновлены. Предотвращает возникновение распространенной формы взаимоблокировки, возникающей тогда, когда несколько сеансов считывают, блокируют и затем, возможно, обновляют ресурс.
Монопольная блокировка (X)	Используется для операций модификации данных, таких как инструкции INSERT, UPDATE или DELETE. Гарантирует, что несколько обновлений не будет выполнено одновременно для одного ресурса.
Блокировка с намерением	Используется для создания иерархии блокировок. Типы намеренной блокировки: с намерением совмещаемого доступа (IS), с намерением монопольного доступа (IX), а также совмещаемая с намерением монопольного доступа (SIX).
Блокировка схемы	Используется во время выполнения операции, зависящей от схемы таблицы. Типы блокировки схем: блокировка изменения схемы (Sch-S) и блокировка стабильности схемы (Sch-M).
Блокировка массового обновления (BU)	Используется, если выполняется массовое копирование данных в таблицу и указана подсказка TABLOCK.
Диапазон ключей	Защищает диапазон строк, считываемый запросом при использовании уровня изоляции сериализуемой транзакции. Запрещает другим транзакциям вставлять строки, что помогает запросам сериализуемой транзакции уточнять, были ли запросы запущены повторно.

Совмещаемые блокировки. Совмещаемые (S) блокировки позволяют одновременным транзакциям считывать (SELECT) ресурс под контролем пессимистичного параллелизма. Пока для ресурса существуют совмещаемые (S) блокировки, другие транзакции не могут изменять данные. Совмещаемые блокировки (S) ресурса снимаются по завершении операции считывания, если только уровень изоляции транзакции не установлен на повторяющееся чтение или более высокий уровень, а также если совмещаемые блокировки (S) не продлены на все время транзакции с помощью указания блокировки.

Блокировки обновления. Блокировки обновления (U) предотвращают возникновение распространенной формы взаимоблокировки. В сериализуемой транзакции или транзакции операций чтения с возможностью повторения транзакция считывает данные, запрашивает совмещаемую (S) блокировку на ресурс (страницу или строку), затем выполняет изменение данных, что требует преобразование блокировки в монопольную (X). Если две транзакции запрашивают совмещаемую блокировку на ресурс и затем пытаются одновременно обновить

данные, то одна из транзакций пытается преобразовать блокировку в монопольную (X). Преобразование совмещаемой блокировки в монопольную потребует некоторого времени, поскольку монопольная блокировка для одной транзакции несовместима с совмещаемой блокировкой для другой транзакции. Начнется ожидание блокировки. Вторая транзакция попытается получить монопольную (X) блокировку для обновления. Поскольку обе транзакции выполняют преобразование в монопольную (X) блокировку и при этом каждая из транзакций ожидает, пока вторая снимет совмещаемую блокировку, то в результате возникает взаимоблокировка.

Чтобы избежать этой потенциальной взаимоблокировки, применяются блокировки обновления (U). Блокировку обновления (U) может устанавливать для ресурса одновременно только одна транзакция. Если транзакция изменяет ресурс, то блокировка обновления (U) преобразуется в монопольную (X) блокировку.

Монопольные блокировки. Монопольная (X) блокировка запрещает транзакциям одновременный доступ к ресурсу. Если ресурс удерживается монопольной (X) блокировкой, то другие транзакции не могут изменять данные. Операции считывания будут допускаться только при наличии указания NOLOCK или уровня изоляции незафиксированной операции чтения. Изменяющие данные инструкции, такие как INSERT, UPDATE или DELETE, соединяют как операции изменения, так и операции считывания. Чтобы выполнить необходимые операции изменения данных, инструкция сначала получает данные с помощью операций считывания. Поэтому, как правило, инструкции изменения данных запрашивают как совмещаемые, так и монопольные блокировки. Например, инструкция UPDATE может изменять строки в одной таблице, основанной на соединении данных из другой таблицы. В этом случае инструкция UPDATE кроме монопольной блокировки обновляемых строк запрашивает также совмещаемые блокировки для строк, считываемых в соединенной таблице.

Блокировки с намерением. В компоненте Компонент Database Engine блокировки с намерением применяются для защиты размещения совмещаемой (S) или монопольной (X) блокировки ресурса на более низком уровне иерархии. Блокировки с намерением называются так потому, что их получают до блокировок более низкого уровня, то есть они обозначают намерение поместить блокировку на более низком уровне.

Блокировка с намерением выполняет две функции:

- предотвращает изменение ресурса более высокого уровня другими транзакциями таким образом, что это сделает недействительной блокировку более низкого уровня;
- повышает эффективность компонента Компонент Database Engine при распознавании конфликтов блокировок на более высоком уровне гранулярности.

Например, в таблице требуется блокировка с намерением совмещаемого доступа до того, как для страниц или строк этой таблицы будет запрошена совмещаемая (S) блокировка. Если задать блокировку с намерением на уровне таблицы, то другим транзакциям будет запрещено получать монопольную (X) блокировку для таблицы, содержащей эту страницу.

Блокировка с намерением повышает производительность, поскольку компонент Компонент Database Engine проверяет наличие таких блокировок только на уровне таблицы, чтобы определить, может ли транзакция безопасно получить для этой таблицы совмещаемую блокировку. Благодаря этому нет необходимости проверять блокировки в каждой строке и на каждой странице, чтобы убедиться, что транзакция может заблокировать всю таблицу.

Блокировки схем. В компоненте Компонент Database Engine блокировка изменения схемы (Sch-M) применяется с операциями языка DDL для таблиц, например при добавлении столбца или очистке таблицы. Пока удерживается блокировка изменения схемы (Sch-M),

одновременный доступ к таблице запрещен. Это означает, что любые операции вне блокировки изменения схемы (Sch-M) будут запрещены до снятия блокировки.

- Блокировка изменения схемы (Sch-M) применяется с некоторыми операциями языка обработки данных, например усечением таблиц, чтобы предотвратить одновременный доступ к таблице.
- Блокировка стабильности схемы (Sch-S) применяется компонентом Компонент Database Engine при компиляции и выполнении запросов. Блокировка стабильности схемы (Sch-S) не влияет на блокировки транзакций, включая монопольные (X) блокировки. Поэтому другие транзакции (даже транзакции с монопольной блокировкой (X) для таблицы) могут продолжать работу во время компиляции запроса. Однако одновременные операции DDL и DML, которые запрашивают блокировки изменения схемы (Sch-M), не могут выполняться над таблицей.

Блокировки массового обновления. Блокировка массового обновления (BU) позволяет поддерживать несколько одновременных потоков массовой загрузки данных в одну и ту же таблицу и при этом запрещать доступ к таблице любым другим процессам, отличным от массовой загрузки данных. Компонент Database Engine использует блокировки массового обновления (BU), если выполняются два следующих условия. Используется инструкция Transact-SQL BULK INSERT, функция OPENROWSET(BULK) или одна из таких команд массовой вставки API, как .NET SqlBulkCopy, OLEDB Fast Load APIs или ODBC Bulk Copy APIs, для массового копирования данных в таблицу. Выделено указание TABLOCK или установлен параметр таблицы table lock on bulk load с помощью хранимой процедуры sp_tableoption.

Блокировки диапазона ключа. Блокировки диапазона ключей защищают диапазон строк, неявно включенный в набор записей, считываемый инструкцией Transact-SQL при использовании уровня изоляции сериализуемых транзакций. Блокировка диапазона ключей предотвращает фантомные чтения. Кроме того, защита диапазона ключей между строк предотвращает фантомную вставку или удаление из набора записи, к которому получает доступ транзакция.

Совместимость блокировок

Совместимость блокировок определяет, могут ли несколько транзакций одновременно получить блокировку одного и того же ресурса. Если ресурс уже блокирован другой транзакцией, новая блокировка может быть предоставлена только в том случае, если режим запрошенной блокировки совместим с режимом существующей. В противном случае транзакция, запросившая новую блокировку, ожидает освобождения ресурса, пока не истечет время ожидания существующей блокировки.

Например, с монопольными блокировками не совместим ни один из режимов блокировки. Пока удерживается монопольная (X) блокировка, больше ни одна из транзакций не может получить блокировку ни одного из типов (разделяемую, обновления или монопольную) на этот ресурс, пока не будет освобождена монопольная (X) блокировка. И наоборот, если к ресурсу применяется разделяемая (S) блокировка, другие транзакции могут получать разделяемую блокировку или блокировку обновления (U) на этот элемент, даже если не завершилась первая транзакция. Тем не менее, другие транзакции не могут получить монопольную блокировку до освобождения разделяемой. Полная матрица совместимости блокировок приводится в справочниках.

Продолжительность блокировки

Продолжительность блокировок также определяется типами запросов. Когда в рамках транзакции запрос не выполняется, и не используются подсказки блокировки, блокировки для выполнения инструкций SELECT выполняются только на время чтения ресурса, но не во время запроса. Блокировки для инструкций INSERT, UPDATE и DELETE сохраняются на все время выполнения запроса. Это помогает гарантировать согласованность данных и позволяет SQL Server откатывать запросы в случае необходимости.

Когда запрос выполняется в рамках транзакции, продолжительность блокировки определяется тремя факторами:

- типом запроса;
- уровнем изоляции транзакции;
- наличием или отсутствием подсказок блокировки.

Кратковременные (locking) и обычные (blocking) блокировки — нормальное явление в реляционных базах данных, но они могут ухудшать производительность, если блокировки ресурсов сохраняются на протяжении длительного времени. Производительность также страдает, когда, заблокировав ресурс, SPID не в состоянии освободить его.

В первом случае проблема обычно разрешается через какое-то время, так как SPID, в конечном счете, освобождает блокировку, но угроза деградации производительности остается вполне реальной. Проблемы с блокировкой второго типа могут вызвать серьезное падение производительности, но, к счастью, они легко обнаруживаются при мониторинге SQL Server на предмет кратковременных и обычных блокировок.

Эскалация блокировок и их влияние на работу системы

Эскалация (lock escalation) связана с тем, что по мере увеличения, количества отдельных малых заблокированных объектов накладные расходы, связанные с их поддержкой, начинают значительно сказываться на производительности. Блокировки длятся дольше, что приводит к спорным ситуациям — чем дольше существует блокировка, тем выше вероятность обращения к заблокированному объекту со стороны другой транзакции. Очевидно, что на некотором этапе потребуется выполнить объединение (увеличение масштаба) блокировок, чем собственно и занимается диспетчер блокировок. В инструкции ALTER TABLE предусмотрена опция вида SET (LOCK_ESCALATION = { AUTO | TABLE | DISABLE }), которая указывает разрешенные методы укрупнения блокировки для таблицы.

Задание определенного типа блокировки в запросе

Для повышения эффективности исполнения запросов и получения дополнительного контроля над блокировками в запросе можно давать подсказки (hints или хинты), указывая их непосредственно за именем таблицы, которая нуждается в том или ином типе блокировки. Существуют следующие параметры оптимизатора запросов:

SERIALIZABLE/HOLDLOCK
READUNCOMMITTED/NOLOCK
READCOMMITTED
REPEATABLEREAD
READPAST
ROWLOCK
PAGLOCK
TABLOCK
TABLOCKX
UPDLOCK

XLOCK

Пример задания эксклюзивной табличной блокировки для таблицы Orders (вместо блокировки на уровне ключей или строк, которую может предложить оптимизатор) может быть таким:

```
SELECT *  
FROM Orders AS o WITH (TABLOCKX) JOIN [Order Details] AS od  
ON o.OrderID = od.OrderID
```

Поскольку оптимизатор запросов обычно выбирает лучший план выполнения запроса, использовать подсказки рекомендуется только опытным разработчикам и администраторам баз данных в самом крайнем случае. Сведения об активных блокировках на текущий момент времени можно получить с помощью системной хранимой процедуры `sp_lock`.

Взаимоблокировки транзакций

Взаимоблокировки или тупиковые ситуации (deadlocks) возникают тогда, когда одна из транзакций не может завершить свои действия, поскольку вторая транзакция заблокировала нужные ей ресурсы, а вторая в то же время ожидает освобождения ресурсов первой транзакцией. На рисунке транзакция T1 зависит от транзакции T2 для ресурса блокировки таблицы Детали. Аналогично транзакция T2 зависит от транзакции T1 для ресурса блокировки таблицы Поставщик. Так как эти зависимости из одного цикла, возникает взаимоблокировка транзакций T1 и T2.



Транзакция T1 не может завершиться до того, как завершится транзакция T2, а транзакция T2 заблокирована транзакцией T1. Такое условие также называется циклической зависимостью: транзакция T1 зависит от транзакции T2, а транзакция T2 зависит от транзакции T1 и этим замыкает цикл. Обе транзакции находятся в состоянии взаимоблокировки и будут всегда находиться в состоянии ожидания, если взаимоблокировка не будет разрушена внешним процессом.

Взаимоблокировки часто путают с обычными блокировками. Если транзакция запрашивает блокировку на ресурс, заблокированный дугой транзакцией, то запрашивающая транзакция ожидает до тех пор, пока блокировка не освобождается. По умолчанию время ожидания транзакций SQL Server не ограничено, если только не установлен параметр `LOCK_TIMEOUT`.

Значение -1 (по умолчанию) указывает на отсутствие времени ожидания (то есть инструкция будет ждать всегда). Когда ожидание блокировки превышает значение времени ожидания, возвращается ошибка. Значение «0» означает, что ожидание отсутствует, а сообщение

возвращается, как только встречается блокировка. Функция @@LOCK_TIMEOUT возвращает значение времени ожидания блокировки в миллисекундах для текущего сеанса.

Запрашивающая транзакция блокируется, но не устанавливается в состояние взаимоблокировки, потому что запрашивающая транзакция ничего не сделала, чтобы заблокировать транзакцию, владеющую блокировкой. Наконец, владеющая транзакция завершится и освободит блокировку, и затем запрашивающая транзакция получит блокировку и продолжится.

Как SQL Server обнаруживает взаимоблокировки?

Обнаружение взаимоблокировки выполняется потоком диспетчера блокировок, который периодически производит поиск по всем задачам в экземпляре компонента Database Engine. Следующие пункты описывают процесс поиска:

- Значение интервала поиска по умолчанию составляет 5 секунд.
- Если диспетчер блокировок находит взаимоблокировки, интервал обнаружения взаимоблокировок снижается с 5 секунд до 100 миллисекунд в зависимости от частоты взаимоблокировок.
- Если поток диспетчера блокировок прекращает поиск взаимоблокировок, компонент Database Engine увеличивает интервал до 5 секунд.
- Если взаимоблокировка была только что найдена, предполагается, что следующие потоки, которые должны ожидать блокировки, входят в цикл взаимоблокировки. Первая пара элементов, ожидающих блокировки, после того как взаимоблокировка была обнаружена, запускает поиск взаимоблокировок вместо того, чтобы ожидать следующий интервал обнаружения взаимоблокировки. Например, если текущее значение интервала равно 5 секунд и была обнаружена взаимоблокировка, следующий ожидающий блокировки элемент немедленно приводит в действие детектор взаимоблокировок. Если этот ожидающий блокировки элемент является частью взаимоблокировки, она будет обнаружена немедленно, а не во время следующего поиска взаимоблокировок.
- Компонент Database Engine обычно выполняет только периодическое обнаружение взаимоблокировок. Так как число взаимоблокировок, произошедших в системе, обычно мало, периодическое обнаружение взаимоблокировок помогает сократить издержки от взаимоблокировок в системе.
- Если монитор блокировок запускает поиск взаимоблокировок для определенного потока, он идентифицирует ресурс, ожидаемый потоком. После этого монитор блокировок находит владельцев определенного ресурса и рекурсивно продолжает поиск взаимоблокировок для этих потоков до тех пор, пока не найдет цикл. Цикл, определенный таким способом, формирует взаимоблокировку.
- После обнаружения взаимоблокировки компонент Database Engine завершает взаимоблокировку, выбрав один из потоков в качестве жертвы взаимоблокировки. Компонент Database Engine прерывает выполняемый в данный момент пакет потока, производит откат транзакции жертвы взаимоблокировки и возвращает приложению ошибку 1205. Откат транзакции жертвы взаимоблокировки снимает все блокировки, удерживаемые транзакцией. Это позволяет транзакциям потоков разблокироваться, и продолжить выполнение. Ошибка 1205 жертвы взаимоблокировки записывает в журнал ошибок сведения обо всех потоках и ресурсах, затронутых взаимоблокировкой.

Как выбирается жертва взаимоблокировки?

По умолчанию в качестве жертвы взаимоблокировки выбирается сеанс, выполняющий ту транзакцию, откат которой потребует меньше всего затрат. В качестве альтернативы

пользователь может указать приоритет сеансов, используя инструкцию SET DEADLOCK_PRIORITY. DEADLOCK_PRIORITY может принимать значения LOW, NORMAL или HIGH или в качестве альтернативы может принять любое целочисленное значение на отрезке [-10..10]. По умолчанию DEADLOCK_PRIORITY устанавливается на значение NORMAL. Если у двух сеансов имеются различные приоритеты, то в качестве жертвы взаимоблокировки будет выбран сеанс с более низким приоритетом. Если у обоих сеансов установлен одинаковый приоритет, то в качестве жертвы взаимоблокировки будет выбран сеанс, откат которого потребует наименьших затрат. Если сеансы, вовлеченные в цикл взаимоблокировки, имеют один и тот же приоритет и одинаковую стоимость, то жертва взаимоблокировки выбирается случайным образом.

Журнализация

Базы данных SQL Server содержат файлы трех типов:

- **Первичные файлы данных.** Первичный файл данных является отправной точкой базы данных. Он указывает на остальные файлы базы данных. В каждой базе данных имеется один первичный файл данных. Для имени первичного файла данных рекомендуется использовать расширение MDF.
- **Вторичные файлы данных.** Ко вторичным файлам данных относятся все файлы данных, за исключением первичного файла данных. Некоторые базы данных могут вообще не содержать вторичных файлов данных, тогда как другие содержат несколько вторичных файлов данных. Для имени вторичного файла данных рекомендуется использовать расширение NDF.
- **Файлы журналов.** Файлы журналов содержат все сведения журналов, используемые для восстановления базы данных. В каждой базе данных должен быть, по меньшей мере, один файл журнала, но их может быть и больше. Для имен файлов журналов рекомендуется использовать расширение LDF. Журнал транзакций нельзя ни удалять, ни изменять, если только не известны возможные последствия. Кэш журнала управляется отдельно от буферного кэша для страниц данных.

Журнал транзакций поддерживает следующие операции:

- **Восстановление отдельных транзакций.** Если приложение выполняет инструкцию ROLLBACK или если компонент Database Engine обнаруживает ошибку, такую как потеря связи с клиентом, записи журнала используются для отката изменений, сделанных незавершенной транзакцией.
- **Восстановление всех незавершенных транзакций при запуске SQL Server.** Если на сервере, где работает SQL Server, происходит сбой, базы данных могут остаться в таком состоянии, в котором часть изменений не была переписана из буферного кэша в файлы данных, и могут быть изменения в файлах данных, совершенные незаконченными транзакциями. Когда экземпляр SQL Server будет запущен, он выполнит восстановление каждой базы данных. Будет выполнен накат каждого записанного в журнал изменения, которое, возможно, не было переписано в файл данных. Чтобы сохранить целостность базы данных, будет также произведен откат каждой незавершенной транзакции, найденной в журнале транзакций.
- **Нкат восстановленной базы данных, файла, файловой группы или страницы до момента сбоя.** После потери оборудования или сбоя диска, затрагивающего файлы базы данных, можно восстановить базу данных на момент, предшествующий сбою. Сначала восстановите последнюю полную резервную копию и последнюю дифференциальную резервную копию базы данных, затем восстановите последующую серию резервных копий журнала транзакций до момента возникновения сбоя. Поскольку восстанавливается каждая резервная копия журнала, компонент Database Engine повторно применяет все модификации, записанные в журнале, для наката всех транзакций. Когда последняя резервная копия журнала будет восстановлена, тогда компонент Database Engine начнет

использовать данные журнала для отката всех транзакций, которые не были завершены на момент сбоя.

- **Поддержка репликации транзакций.** Агент чтения журнала следит за журналами транзакций всех баз данных, которые настроены на репликацию транзакций, и копирует отмеченные для репликации транзакции из журнала транзакций в базу данных распространителя. Дополнительные сведения см. в разделе Как работает репликация транзакций.
- **Поддержка решений с резервными серверами.** Решения резервного сервера, зеркальное отображение базы данных и доставка журналов в значительной степени полагаются на журнал транзакций. В сценарии доставки журналов основной сервер отправляет активный журнал транзакций основной базы данных одному или более адресатам. Каждый сервер-получатель восстанавливает журнал в свою локальную базу данных-получатель.

Операции резервного копирования и восстановления выполняются в контексте модели восстановления. Модель восстановления — это свойство базы данных, которое управляет процессом регистрации транзакций, определяет, требуется ли для журнала транзакций резервное копирование, а также определяет, какие типы операций восстановления доступны. Есть три модели восстановления:

- **Модель полного восстановления (FULL).** Обеспечивает модель обслуживания для баз данных, в которых необходима поддержка длительных транзакций. Требуются резервные копии журналов. При использовании этой модели выполняется полное протоколирование всех транзакций, и сохраняются записи журнала транзакций до момента их резервного копирования. Модель полного восстановления позволяет восстановить базу данных до точки сбоя при условии, что после сбоя возможно создание резервной копии заключительного фрагмента журнала. Кроме того, модель полного восстановления поддерживает восстановление отдельных страниц данных.
- **Модель восстановления с неполным протоколированием (BULK_LOGGED).** Эта модель восстановления обеспечивает неполное протоколирование большинства массовых операций. Она предназначена для работы только в качестве дополнения к полной модели полного восстановления. Для ряда масштабных массовых операций (массовый импорт, создание индекса и т.п.) временное переключение на модель восстановления с неполным протоколированием повышает производительность и уменьшает место, необходимое для журналов. Тем не менее, для работы этой модели требуются резервные копии журналов. Как и в модели полного восстановления, в модели восстановления с неполным протоколированием сохраняются записи журнала транзакций после его резервного копирования. Это увеличивает объем резервных копий журналов и повышает риск потери результатов работы, поскольку модель восстановления с неполным протоколированием не поддерживает восстановление до заданного момента времени.
- **Простая модель восстановления (SIMPLE).** Сводит к минимуму административные затраты, связанные с журналом транзакций, поскольку его резервная копия не создается. При использовании простой модели восстановления в случае повреждения базы данных возникает риск потери значительной части результатов работы. Данные могут быть восстановлены только до момента последнего резервного копирования. Поэтому при использовании простой модели восстановления интервалы между резервными

копированием должны быть достаточно короткими, чтобы предотвратить потерю значительного объема данных. В то же время они должны быть велики настолько, чтобы затраты на резервное копирование не влияли на производительность. Снизить затраты поможет использование разностного резервного копирования.

Обычно в базе данных используется модель полного восстановления или простая модель восстановления. Модели восстановления оказывают влияние на поведение журнала транзакций и способ регистрации операций, либо на и на то, и на другое.

Модель восстановления FULL подразумевает, что регистрируется каждая часть каждой операции, и это называется полной регистрацией. После выполнения полного резервного копирования базы данных в модели восстановления FULL в журнале транзакций не будет проводиться автоматическое усечение до тех пор, пока не будет выполнено резервное копирование журнала. Если вы не намерены использовать резервные копии журнала и возможность восстановления состояния базы данных на конкретный момент времени, не следует использовать модель восстановления FULL. Однако, если вы предполагаете использовать зеркальное отображение базы данных, тогда у вас нет выбора, поскольку оно поддерживает только модель восстановления FULL.

Модель восстановления BULK_LOGGED обладает такой же семантикой усечения журнала транзакций, как и модель восстановления FULL, но допускает частичную регистрацию некоторых операций, что называется минимальной регистрацией. Примерами являются повторное создание индекса и некоторые операции массовой загрузки — в модели восстановления FULL регистрируется вся операция.

Но в модели восстановления BULK_LOGGED регистрируются только изменения распределения, что радикально сокращает число создаваемых записей журнала и, в свою очередь, сокращает потенциал разрастания журнала транзакций.

Модель восстановления SIMPLE, фактически ведет себя с точки зрения ведения журнала так же, как и модель восстановления BULK_LOGGED, но имеет совершенно другую семантику усечения журнала транзакций. В модели восстановления SIMPLE невозможны резервные копии журнала, что означает, что журнал может быть усечен (если ничто не удерживает записи журнала в активном состоянии) при возникновении контрольной точки.

SQL Server поддерживает восстановление данных на следующих уровнях:

- **База данных** (полное восстановление базы данных). Вся база данных возвращается в прежнее состояние и восстанавливается, при этом база данных находится в автономном режиме во время операций возврата и восстановления.
- **Файл данных** (восстановление файла). Файл данных или набор файлов данных возвращается в исходное состояние и восстанавливается. Во время восстановления файлов файловые группы, содержащие обрабатываемые файлы, автоматически переводятся в автономный режим на время восстановления. Любые попытки подключения и работы с недоступной файловой группой приведут к ошибке.
- **Страница данных** (восстановление страницы). При использовании модели полного восстановления или модели восстановления с неполным протоколированием можно

восстановить отдельные базы данных. Восстановление страниц может применяться для любой базы данных вне зависимости от числа файловых групп.

Логическая архитектура журнала транзакций

На логическом уровне журнал транзакций состоит из последовательности записей. Каждая запись содержит:

- Log Sequence Number: регистрационный номер транзакции (LSN). Каждая новая запись добавляется в логический конец журнала с номером LSN, который больше номера LSN предыдущей записи.
- Prev LSN: обратный указатель, который предназначен для ускорения отката транзакции.
- Transaction ID number: идентификатор транзакции.
- Type: тип записи Log-файла.
- Other information: Прочая информация.

Двумя основными типами записи Log-файла являются:

- Transaction Log Operation Code: код выполненной логической операции, либо
- Update Log Record: исходный и результирующий образ измененных данных. Исходный образ записи – это копия данных до выполнения операции, а результирующий образ – копия данных после ее выполнения.

Действия, которые необходимо выполнить для восстановления операции, зависят от типа Log-записи:

- Зарегистрирована логическая операция.
 - Для наката логической операции выполняется эта операция.
 - Для отката логической операции выполняется логическая операция, обратная зарегистрированной.
- Зарегистрированы исходный и результирующий образы записи.
 - Для наката операции применяется результирующий образ.
 - Для отката операции применяется исходный образ.

В журнал транзакций записываются различные типы операций:

0 BEGINXACT: Begin Transaction

4 Insert

5 Delete

6 Indirect Insert

7 Index Insert

8 Index Delete

9 MODIFY: Modify the record on the page

11 Deferred Insert (NO-OP)

12 Deferred Delete (NO-OP)

13 Page Allocation

15 Extent Allocation

16 Page Split

17 Checkpoint

20 DEXTENT

30 End Transaction (Either a commit or rollback)

38 CHGSYSINDSTAT — A change to the statistics page in sysindexes

39 CHGSYSINDPG — Change to a page in the sysindexes table

Каждая транзакция резервирует в журнале транзакций место, чтобы при выполнении инструкции отката или возникновения ошибки в журнале было достаточно места для регистрации отката. Объем резервируемого пространства зависит от выполняемых в транзакции операций, но обычно он равен объему, необходимому для регистрации каждой из операций. Все это пространство после завершения транзакции освобождается.

Раздел журнального файла, который начинается от первой записи, необходимой для успешного отката на уровне базы данных, до последней зарегистрированной записи называется активной частью журнала, или активным журналом. Именно этот раздел необходим для выполнения полного восстановления базы данных. Активный журнал не может быть усечен.

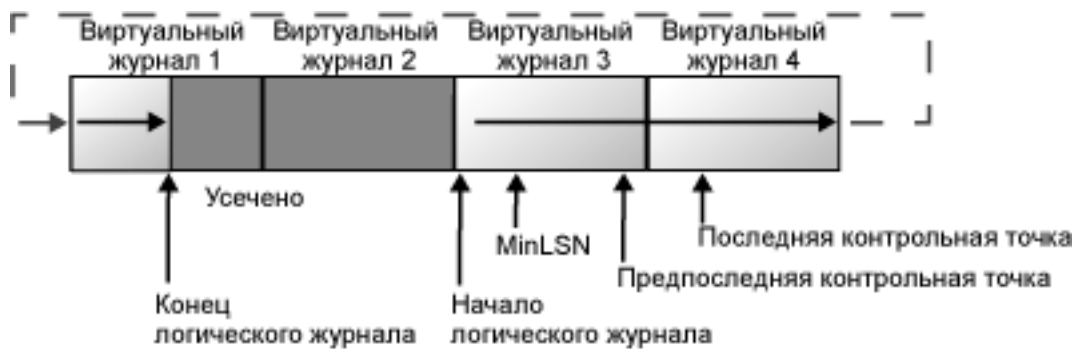
Физическая архитектура журнала транзакций

На физическом уровне журнал транзакций состоит из одного или нескольких физических файлов. Каждый физический файл журнала разбивается на несколько виртуальных файлов журнала (VLF). VLF не имеет фиксированного размера. Не существует также и определенного числа VLF, приходящихся на один физический файл журнала. Компонент Database Engine динамически определяет размер VLF при создании или расширении файлов журнала. Компонент Database Engine стремится обслуживать небольшое число VLF. Администраторы не могут настраивать или устанавливать размеры и число VLF.

Журнал транзакций является оборачиваемым файлом. Рассмотрим пример. Пусть база данных имеет один физический файл журнала, разделенный на четыре виртуальных файла журнала. При создании базы данных логический файл журнала начинается в начале физического файла журнала. Новые записи журнала добавляются в конце логического журнала и приближаются к концу физического файла журнала. Усечение журнала освобождает любые виртуальные журналы, все записи которых находятся перед минимальным регистрационным номером восстановления в журнале транзакций (MinLSN). MinLSN является регистрационным номером самой старой записи, которая необходима для успешного отката на уровне всей базы данных. Журнал транзакций рассматриваемой в данном примере базы данных будет выглядеть примерно так же, как на следующей иллюстрации.



Когда конец логического журнала достигнет конца физического файла журнала, новые записи журнала будут размещаться в начале физического файла журнала.



Этот цикл повторяется бесконечно, пока конец логического журнала не совмещается с началом этого логического журнала. Если старые записи журнала усекаются достаточно часто, так что при этом всегда остается место для новых записей журнала, созданных с новой контрольной точки, журнал постоянно остается незаполненным. Однако, если конец логического журнала совмещается с началом этого логического журнала, происходит одно из двух событий, перечисленных ниже:

- Если для данного журнала применена установка FILEGROWTH и на диске имеется свободное место, файл расширяется на величину, указанную в `growth_increment`, и новые записи журнала добавляются к этому расширению. Дополнительные сведения о настройке FILEGROWTH см. в разделе ALTER DATABASE (Transact- SQL).
- Если установка FILEGROWTH не применяется или диск, на котором размещается файл журнала, имеет меньше свободного места, чем это указано в `growth_increment`, формируется ошибка 9002.
- Если в журнале содержится несколько физических файлов журнала, логический журнал будет продвигаться по всем физическим файлам журнала до тех пор, пока он не вернется на начало первого физического файла журнала.

Просмотр журнала транзакций

Вариант А. Недокументированная команда DBCC LOG(имя_базы_данных, тип_вывода)

где тип_вывода принимает значение из диапазона 0..4. По умолчанию принят 0 (Current LSN, Operation, Context, Transaction ID). 3 выдает полный информационный дамп каждой операции.

Вариант В. Недокументированная функция fn_dblog().

Пример.

```
create table t1 (id int, name varchar(20));
```

```
insert into t1 values (1, 'test1');
```

```
checkpoint;
```

```
insert into t1 values (2, 'test2');
```

```
checkpoint;
```

```
select * from t1;
```

```
drop table t1;
```

```
-- DBCC LOG(N'dbtest', 3)
```

```
-- Выводится таблица 45*102
```

```
-- select top 1 * from fn_dblog(null, null)
```

```
-- Выводится таблица 1*97
```


Так как все изменения страниц данных происходят в страничных буферах, то изменения данных в памяти не обязательно отражаются в этих страницах на диске. Процесс кэширования происходит по алгоритму последней использованной страницы, поэтому страница, подверженная постоянным изменениям, помечается как последняя использованная, и она не записывается на диск. Чтобы эти страницы были записаны на диск применяется контрольная точка. Все грязные страницы должны быть сохранены на диске в обязательном порядке.

Контрольная точка выполняет в базе данных следующее:

- Записывает в файл журнала запись, отмечающую начало контрольной точки.
- Сохраняет данные, записанные для контрольной точки в цепи записей журнала контрольной точки. Одним из элементов данных, регистрируемых в записях контрольной точки, является номер LSN первой записи журнала, при отсутствии которой успешный откат в масштабе всей базы данных невозможен. Такой номер LSN называется минимальным номером LSN восстановления (MinLSN). Номер MinLSN является наименьшим значением из:
 - номера LSN начала контрольной точки;
 - номера LSN начала старейшей активной транзакции;
 - номера LSN начала старейшей транзакции репликации, которая еще не была доставлена базе данных распространителя.
 - Записи контрольной точки содержат также список активных транзакций, изменивших базу данных.
- Если база данных использует простую модель восстановления, помечает для повторного использования пространство, предшествующее номеру MinLSN.
- Записывает все измененные страницы журналов и данных на диск.
- Записывает в файл журнала запись, отмечающую конец контрольной точки.
- Записывает в страницу загрузки базы данных номер LSN начала соответствующей цепи.

Действия, приводящие к срабатыванию контрольных точек

Контрольные точки срабатывают в следующих ситуациях:

- При явном выполнении инструкции CHECKPOINT. Контрольная точка срабатывает в текущей базе данных соединения.
- При выполнении в базе данных операции с минимальной регистрацией, например при выполнении операции массового копирования для базы данных, на которую распространяется модель восстановления с неполным протоколированием.
- При добавлении или удалении файлов баз данных с использованием инструкции ALTER DATABASE.
- При остановке экземпляра SQL Server с помощью инструкции SHUTDOWN или при остановке службы SQL Server (MSSQLSERVER). И в том, и в другом случае будет создана контрольная точка каждой базы данных в экземпляре SQL Server.
- Если экземпляр SQL Server периодически создает в каждой базе данных автоматические контрольные точки для сокращения времени восстановления базы данных.
- При создании резервной копии базы данных.

- При выполнении действия, требующего отключения базы данных. Примерами могут служить присвоение параметру AUTO_CLOSE значения ON и закрытие последнего соединения пользователя с базой данных или изменение параметра базы данных, требующее перезапуска базы данных.

Автоматические контрольные точки

Компонент Database Engine создает контрольные точки автоматически. Интервал между автоматическими контрольными точками определяется на основе использованного места в журнале и времени, прошедшего с момента создания последней контрольной точки. Интервал между автоматическими контрольными точками колеблется в широких пределах и может быть довольно длительным, если база данных изменяется редко. При крупномасштабных изменениях данных частота автоматических контрольных точек может быть гораздо выше.

Можно использовать параметр конфигурации сервера `recovery interval` для вычисления интервала между автоматическими контрольными точками для всех баз данных на экземпляре сервера. Значение этого параметра определяет максимальное время, отводимое компоненту Database Engine на восстановление базы данных при перезапуске системы. Компонент Database Engine оценивает количество записей журнала, которые он может обработать за время `recovery interval` при выполнении операции восстановления.

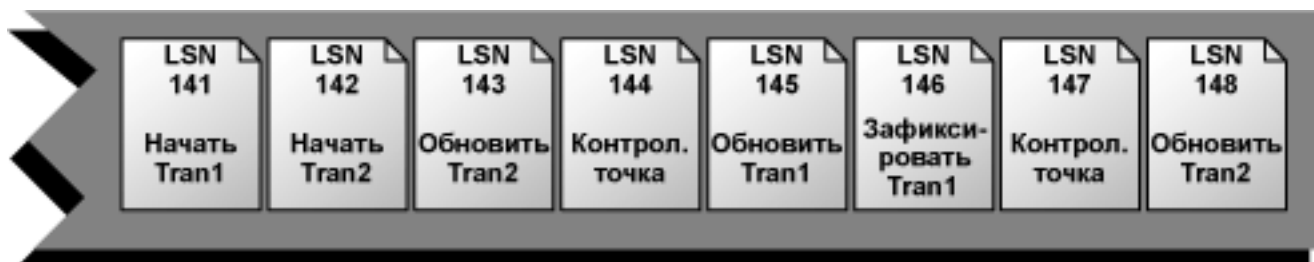
Если используется простая модель восстановления базы данных, автоматическая контрольная точка создается каждый раз, когда число записей в журнале достигает меньшего из двух предельных условий:

- журнал заполняется на 70 процентов;
- число записей в журнале достигает значения, определенного компонентом Database Engine в качестве количества записей, которое он может обработать за время, заданное параметром `recovery interval`.

Активный журнал

Часть журнала, начинающаяся с номера MinLSN и заканчивающаяся последней записью, называется активной частью журнала, или активным журналом. Этот раздел журнала необходим для выполнения полного восстановления базы данных. Ни одна часть активного журнала не может быть усечена. Все записи журнала до номера MinLSN должны быть удалены из частей журнала.

На следующем рисунке изображена упрощенная схема журнала завершения транзакций, содержащего две активные транзакции. Записи контрольных точек были сжаты в одну запись.



Последней записью в журнале транзакций является запись с номером LSN, равным 148. На момент обработки записанной контрольной точки с номером LSN 147 транзакция 1 уже зафиксирована и единственной активной транзакцией является транзакция 2. В результате первая запись журнала, созданная для транзакции 2, становится старейшей записью активной транзакции на момент последней контрольной точки. Таким образом, номером MinLSN становится номер LSN, равный 142 и соответствующий записи начала транзакции 2.

Длительные транзакции

Активный журнал должен включать в себя все элементы всех незафиксированных транзакций. Приложение, инициирующее транзакцию и не выполняющее ее фиксацию или откат, не позволяет компоненту Database Engine повышать MinLSN.

Это может привести к проблемам двух типов.

- Если система будет выключена после того, как транзакцией было выполнено много незафиксированных изменений, этап восстановления при последующем перезапуске может занять гораздо больше времени, чем указано параметром `recovery interval`.
- Журнал может достичь очень большого объема, потому что после номера MinLSN усесть его нельзя. Это справедливо даже в том случае, если используется простая модель восстановления, когда журнал транзакций обычно усекается при каждой автоматической контрольной точке.

Журнал транзакций с упреждающей записью

SQL Server использует журнал с упреждающей записью, который гарантирует, что до занесения на диск записи, связанной с журналом, никакие изменения данных записаны не будут. Таким образом обеспечиваются свойства ACID для транзакции. Для понимания принципов работы журнала с упреждающей записью важно знать принципы записи измененных данных на диск. SQL Server поддерживает буферный кэш, из которого система считывает страницы данных при извлечении необходимых данных. Изменения данных не заносятся непосредственно на диск, а записываются на копии страницы в буферном кэше. Изменение не записывается на диск, пока в базе данных не возникает контрольная точка, или же изменение должно быть записано на диск таким образом, чтобы для хранения новой страницы мог использоваться буфер. Запись измененной страницы данных из буферного кэша на диск называется сбросом страницы на диск. Страница, измененная в кэше, но еще не записанная на диск, называется грязной страницей.

Во время изменения страницы в буфере запись журнала строится в кэше журнала, который записывает изменение. Данная запись журнала должна быть перенесена на диск до того, как соответствующая «грязная» страница будет записана из буферного кэша на диск. Если «грязная» страница переносится на диск до записи журнала, эта страница создает изменение на диске, которое не может быть откачено, если сервер выйдет из строя до переноса записи журнала на диск. SQL Server обладает алгоритмом, который защищает «грязную» страницу от записи на диск до переноса на него соответствующей записи журнала. Содержимое журнала запишется на диск после того, как будут зафиксированы транзакции.

Управление журналом транзакций

Чтобы логический журнал не увеличивался до размера физических файлов журнала, следует периодически выполнять его усечение. Процесс усечения журнала уменьшает размер файла

логического журнала, пометая виртуальные файлы журнала, которые не содержат частей логического журнала, как неактивные. В некоторых случаях может оказаться полезным физическое сжатие или расширение размера реального файла журнала.

Время усечения журнала зависит от модели восстановления базы данных. Есть три модели восстановления: простая модель восстановления, модель полного восстановления и модель восстановления с неполным протоколированием. Обычно в базе данных используется полная модель восстановления или простая модель восстановления. В качестве примера рассмотрим усечение журнала в простой модели восстановления.

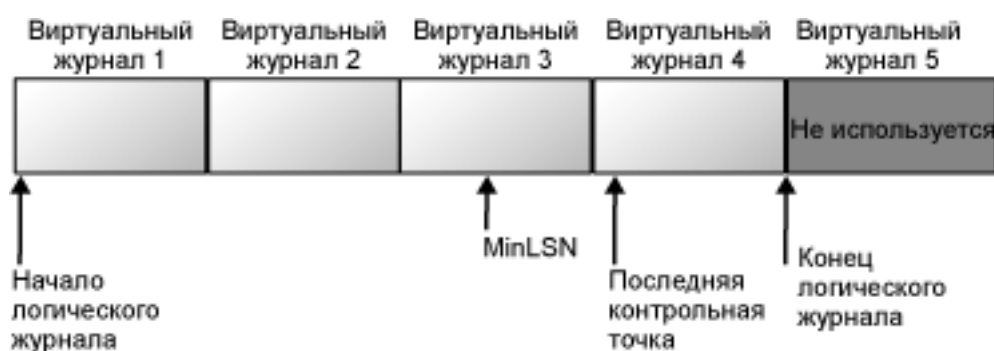
При использовании простой модели восстановления усечение журналов выполняется автоматически. Если все записи в виртуальном файле журнала неактивны, то этот логический журнал усекается обычно после контрольной точки. При этом освобождается место для повторного использования. Это относится и к контрольным точкам инструкции CHECKPOINT, и к неявным контрольным точкам, сформированным системой. Однако усечение журнала может быть отложено, если виртуальные файлы журнала остаются активными вследствие выполнения долгой транзакции или резервного копирования.

Эта модель регистрирует только минимальные сведения, необходимые для обеспечения согласованности базы данных после сбоя системы или для восстановления данных из резервной копии. Это сводит к минимуму расход места на диске под журнал транзакций по сравнению с другими моделями восстановления. Чтобы предотвратить переполнение журнала, базе данных требуется достаточно места для записи в случае задержки его усечения.

Как работает усечение журнала

Кроме прочих данных, в контрольной точке записывается номер LSN первой записи журнала, которую необходимо сохранить для успешного отката на уровне базы данных. Этот номер LSN называется минимальным номером LSN восстановления (MinLSN). Начало активной части журнала занято VLF, содержащим MinLSN. При усечении журнала транзакций освобождаются только те записи, которые находятся перед этим VLF.

На следующем рисунке показан журнал транзакций до усечения и после. На первом рисунке показан журнал транзакций, который никогда не усекался. В настоящий момент логический журнал состоит из четырех виртуальных файлов. Логический журнал начинается с начала первого файла виртуального журнала и заканчивается виртуальным файлом журнала 4. Запись MinLSN находится в виртуальном журнале 3. Виртуальные журналы 1 и 2 содержат только неактивные записи журнала. Эти записи можно усесть. Виртуальный журнал 5 пока не используется и не является частью текущего логического журнала.



На втором рисунке показан журнал после усечения. Виртуальные журналы 1 и 2 усечены и могут использоваться повторно. Логический журнал теперь начинается с начала виртуального журнала 3. Виртуальный журнал 5 все еще не используется и не является частью текущего логического журнала.



Управление размером файла журнала транзакций

Контролировать используемое пространство журнала можно с помощью процедуры DBCC SQLPERF (LOGSPACE). Она возвращает сведения об объеме пространства, используемого журналом в данный момент, и указывает, если необходимо провести усечение журнала транзакций. Для получения сведений о текущем размере файла журнала, его максимальном размере и параметре автоматического увеличения файла можно использовать столбцы size, max_size и growth для данного файла журнала в представлении sys.database_files.

Сжатие файла журнала

Усечение журнала освобождает место на диске для повторного использования, но не уменьшает размер физического файла журнала. Для уменьшения физического размера файл журнала должен быть сжат с целью удаления одного или более неактивных VLF. VLF, хранящий какие-либо активные записи журнала, удалить нельзя. При сжатии файла журнала транзакций в конце файла журнала удаляется достаточное количество неактивных виртуальных файлов журнала, чтобы журнал уменьшился до приблизительного целевого размера.

Примечание. Если журнал транзакций не усекался в последнее время, его сжатие может быть невозможным до тех пор, пока не выполнится усечение.

Система безопасности SQL Server

SQL Server обеспечивает защиту данных от неавторизованного доступа и от фальсификации. Основными функциями безопасности SQL Server являются:

- **проверка подлинности (аутентификация)** – процедура проверки соответствия некоего лица и его учетной записи в компьютерной системе. Один из способов аутентификации состоит в задании пользовательского идентификатора, в просторечии называемого «логином» (login – регистрационное имя пользователя) и пароля – некой конфиденциальной информации, знание которой обеспечивает владение определенным ресурсом. Аутентификацию не следует путать с идентификацией. Идентификация – это установление личности самого физического лица (а не его виртуальной учетной записи, коих может быть много).
- **авторизация** – это предоставление лицу прав на какие-то действия в системе.

Дополнительными функциями безопасности SQL Server являются:

- шифрование,
- контекстное переключение,
- олицетворение,
- встроенные средства управления ключами.

В основе системы безопасности SQL Server лежат три понятия:

- участники системы безопасности (Principals);
- защищаемые объекты (Securables) и
- система разрешений (Permissions).

Участники системы безопасности

Участники системы безопасности или принципалы – это сущности, которые могут запрашивать ресурсы SQL Server. Принципалы могут быть иерархически упорядочены. Область влияния принципала зависит

- от области его определения: Windows, SQL Server, база данных,
- от того, коллективный это участник или индивидуальный. Имя входа Windows является примером индивидуального (неделимого) участника, а группа Windows — коллективного.

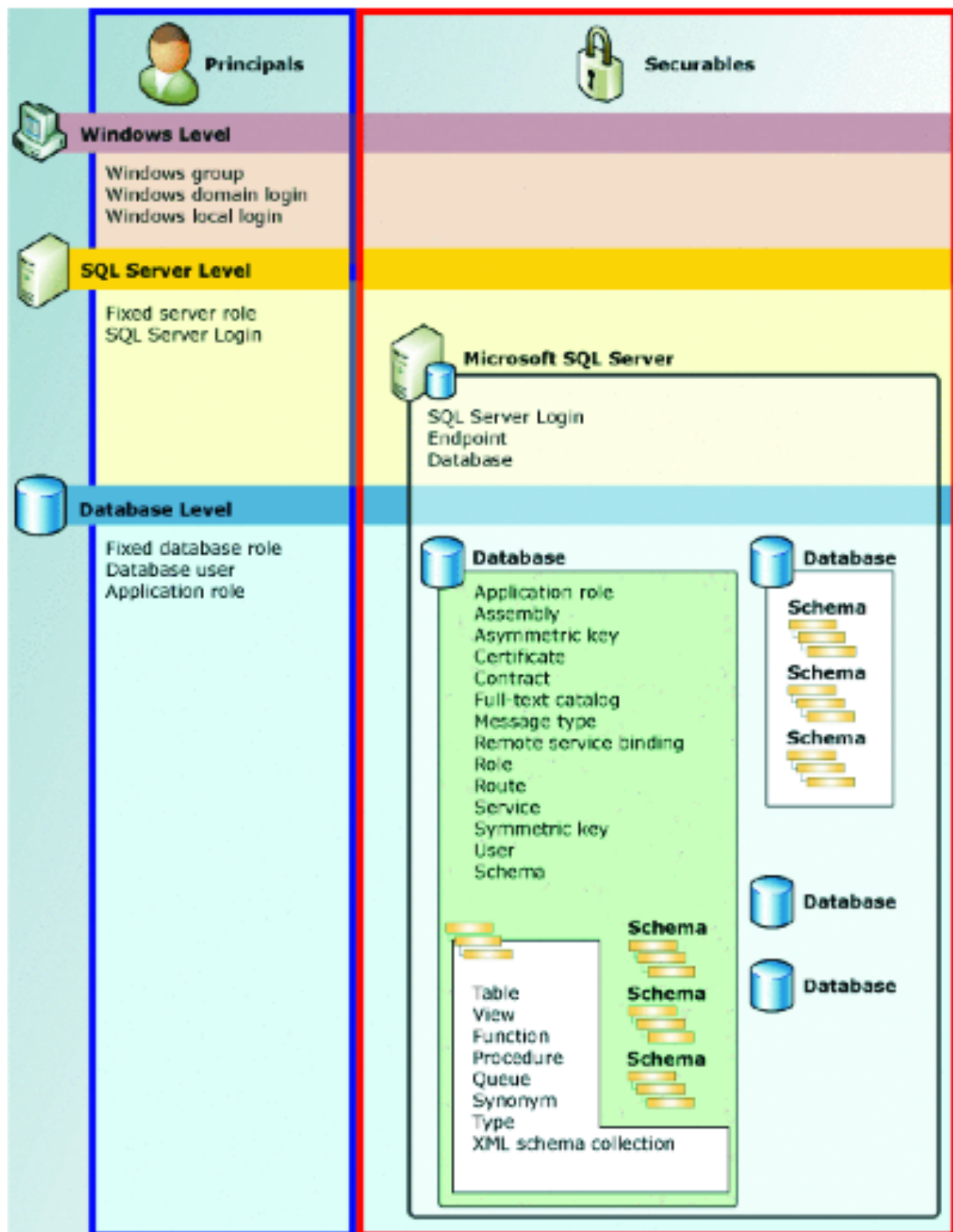
При создании учетной записи SQL Server ей назначается идентификатор и идентификатор безопасности. В представлении каталога sys.server_principals они отображаются в столбцах principal_id и SID.

Участники уровня Windows – это

А. Имя входа домена Windows,

В. Локальное имя входа Windows. Участники уровня SQL Server – это а) Имя входа SQL Server, б) Роль сервера.

Участники уровня базы данных – это а) Пользователь базы данных, б) Роль базы данных, в) Роль приложения.



Замечания.

1. Имя входа «sa» SQL Server. Имя входа sa SQL Server является участником уровня сервера. Оно создается по умолчанию при установке экземпляра. В SQL Server для имени входа sa базой данных по умолчанию будет master.
2. Роль базы данных public. Каждый пользователь базы данных является членом роли базы данных public. Если пользователю не были предоставлены или запрещены особые разрешения на защищаемый объект, то он наследует на него разрешения роли public.
3. Пользователи INFORMATION_SCHEMA и sys. Каждая база данных включает в себя две сущности, которые отображены в представлениях каталога в виде пользователей: INFORMATION_SCHEMA и sys. Они необходимы для работы SQL Server; эти пользователи не являются участниками и не могут быть изменены или удалены.

Защищаемые объекты

Защищаемые объекты – это ресурсы, доступ к которым регулируется системой авторизации. Некоторые защищаемые объекты могут храниться внутри других, создавая иерархии «областей», которые сами могут защищаться. К областям защищаемых объектов относятся (a) сервер, (b) база данных и (c) схема.

Далее введем следующие ограничения:

1. для области «сервер» ограничимся рассмотрением вопросами управления учетными записями подключения (login),
2. для области «база данных» ограничимся рассмотрением вопросами управления
 1. учетными записями пользователей (user),
 2. ролями (role),
 3. ролями приложений (application role).
3. для области «схема» ограничимся рассмотрением вопросами управления разрешениями на
 1. функции (function),
 2. процедуры (stored procedure),
 3. таблицы (table),
 4. представления (view).

Разрешения

У субъекта системы есть только один путь получения доступа к объектам - иметь назначенные непосредственно или опосредовано разрешения. При непосредственном управлении разрешениями они назначаются субъекту явно, а при опосредованном разрешения назначаются через членство в группах, ролях или наследуются от объектов, лежащих выше по цепочке иерархии. Управление разрешениями производится путем выполнения инструкций языка DCL (Data Control Language): GRANT (разрешить), DENY (запретить) и REVOKE (отменить).

Управление учетными данными сервера (credential)

Поддерживаются два вида учетных записей подключения к серверу

- учетные записи сервера, создаваемые на основании учетных записей операционной системы
- учетные записи сервера, создаваемые для прямого подключения к серверу.

Проверка подлинности Windows означает, что для подключения к SQL Server проверка подлинности полностью выполняется операционной системой Microsoft Windows. В этом

случае клиент идентифицируется на основании учетной записи Windows. Проверка подлинности SQL Server означает, что для подключения к SQL Server проверка подлинности выполняется путем сравнения имени пользователя и пароля с хранящимся на сервере SQL Server списком действительных имен пользователей и паролей). Учетные данные сервера создаются при помощи инструкции CREATE CREDENTIAL. После создания учетных данных можно сопоставить их имени входа SQL Server, используя инструкцию CREATE LOGIN или ALTER LOGIN.

Управление именами входа SQL Server (login)

Сами имена входа не имеют доступа ни к одной конкретной базе данных на сервере, они позволяют только подключиться к SQL Server. Имена входа – это объекты, которым может быть дано разрешение в масштабе сервера на выполнение определенных действий. Эти действия собираются в фиксированные серверные роли: bulkadmin, dbcreator, diskadmin, processadmin, public, securityadmin, serveradmin, setupadmin, sysadmin. Все имена входа SQL Server являются обладателями роли public. Добавление имени входа в качестве члена предопределенной роли сервера выполняется при помощи хранимой процедуры sp_addsrvrolemember. Имя входа сервера может быть создано в SSMS, при помощи инструкции CREATE LOGIN или при помощи хранимой процедуры sp_addlogin.

Замечание. В SQL Server 2012 при помощи инструкции CREATE SERVER ROLE можно создать новую, определяемую пользователем роль сервера, а для изменения членства в роли сервера можно использовать инструкцию ALTER SERVER ROLE.

Примеры.

```
--Creating a new SQL login
CREATE LOGIN Carol
WITH PASSWORD = 'Th1sI$!VlyP@ssw0rd'; GO

--Creating a credential based on a Windows user
CREATE CREDENTIAL StreetCred
WITH IDENTITY = 'AughtFive\CarolStreet', SECRET = 'P@ssw0rd';
--Associating a login with a credential
ALTER LOGIN Carol WITH CREDENTIAL = StreetCred; GO

--Creating a login, and adding the user to a fixed server role
CREATE LOGIN Ted WITH PASSWORD = 'P@ssw0rd'; GO
EXEC sp_addsrvrolemember 'Ted', 'securityadmin'; GO
```

Управление учетными записями пользователей базы данных (user)

После создания имени входа в SQL Server можно предоставить этому имени доступ к конкретной базе данных. Для этого сначала надо создать пользователя базы данных для определенного ранее имени входа. Это можно сделать в SSMS, при помощи инструкции CREATE USER или при помощи хранимой процедуры sp_adduser. При входе в SQL Server под именем входа база данных запросит имя и идентификатор созданного пользователя базы данных. Если при создании пользователя базы данных не будет указано имя входа SQL Server, то новый пользователь базы данных будет сопоставлен с именем входа SQL Server, имеющим такое же имя. После создания пользователя базы данных можно (но это необязательно) предоставить ему одну из фиксированных ролей базы данных: db_accessadmin, db_backupoperator, db_datareader, db_datawriter, db_ddladmin, db_denydatareader,

db_denydatawriter, db_owner, db_securityadmin. При желании можно определить дополнительные роли базы данных при помощи инструкции CREATE ROLE. Для добавления пользователя базы данных к роли текущей базы данных используется инструкция ALTER ROLE или хранимая процедура sp_addrolemember.

Примеры

```
-- Managing Server RoleMembers
CREATE LOGIN Veronica
WITH PASSWORD = 'PalmTree1' GO
EXEC master..sp_addsrvrolemember 'Veronica', 'sysadmin' GO
USE TestDB
GO

CREATE USER Veronica
GO

-- Т. к. предложение FOR LOGIN не указано, то новый пользователь базы данных будет ----
сопоставлен с именем входа SQL Server, имеющим такое же имя.
CREATE ROLE HR_ReportSpecialist AUTHORIZATION db_owner GO
EXEC sp_addrolemember 'HR_ReportSpecialist', 'Veronica' GO
```

Замечание. В SQL Server 2012 многие системные хранимые процедуры, используемые для управления безопасностью, считаются устаревшими, но остаются доступными для обратной совместимости. Для устаревших процедур рекомендуется использовать инструкции Transact-SQL, например, ALTER LOGIN, ALTER SERVER ROLE, ALTER USER, ALTER ROLE и др.

Управление разрешениями

После добавления пользователя следует определить разрешения, управляющие действиями, которые пользователь может выполнять, с помощью инструкций GRANT, DENY и REVOKE. Необходимо, чтобы пользователь был владельцем базы данных.

А. Предоставление разрешений на объекты (инструкция GRANT)

Инструкция GRANT предоставляет разрешения на таблицу, представление, функцию, хранимую процедуру, очередь обслуживания, синоним. Синтаксис инструкции GRANT:

```
GRANT { ALL [ PRIVILEGES ] | список_разрешений } ON список_объектов
TO список_принципалов
[ WITH GRANT OPTION ]
[ AS принципал ]
```

Пояснения к инструкции GRANT

1. Ключевое слово ALL с необязательным словом PRIVILEGES не включает все возможные разрешения, оно эквивалентно предоставлению всех разрешений ANSI-92, применимых к указанному объекту. Значение ALL различается для разных типов объектов. Ниже перечислены главные классы разрешений и защищаемых объектов, к которым эти разрешения могут применяться:
 1. разрешения на скалярные функции: EXECUTE, REFERENCES;

2. разрешения на функции, возвращающие табличное значение: DELETE, INSERT, REFERENCES, SELECT, UPDATE;
3. разрешения на хранимые процедуры: EXECUTE;
4. разрешения на таблицы: DELETE, INSERT, REFERENCES, SELECT, UPDATE;
5. разрешения на представления: DELETE, INSERT, REFERENCES, SELECT, UPDATE.

2. Полный список разрешений содержит 195 пунктов.

1. Если разрешение предоставляется на таблицу, представление или функцию, возвращающую табличное значение, то справа от разрешения в круглых скобках могут указываться имена столбцов. На столбец могут быть предоставлены только разрешения SELECT, REFERENCES и UPDATE.
2. Объект, на который предоставляется разрешение, имеет следующее описание: [OBJECT ::] [имя_схемы]. имя_объекта. Фраза OBJECT необязательна, если указан аргумент имя_схемы. Если же она указана, указание квалификатора области (::) обязательно. Если не указан аргумент имя_схемы, подразумевается схема по умолчанию. Если указан аргумент имя_схемы, обязательно указание квалификатора области схемы (.).

3. Принципом может быть:

1. пользователь базы данных,
2. роль базы данных,
3. роль приложения.

4. Необязательная фраза WITH GRANT OPTION указывает, что принципалу также дается возможность предоставлять указанное разрешение другим принципалам.

5. Необязательная фраза AS принципал определяет принципала, у которого другой принципал, выполняющий данный запрос, наследует право предоставлять данное разрешение.

Пример. Предоставление разрешения EXECUTE на хранимую процедуру HumanResources.uspUpdateEmployeeHireInfo роли приложения Role03.

```
USE AdventureWorks;
GRANT EXECUTE ON OBJECT::HumanResources.uspUpdateEmployeeHireInfo TO Role03;
GO
```

Пример. Предоставление разрешения REFERENCES на столбец EmployeeID в представлении HumanResources.vEmployee пользователю User02 с параметром GRANT OPTION.

```
USE AdventureWorks;
GRANT REFERENCES (EmployeeID) ON OBJECT::HumanResources.vEmployee TO User02
WITH GRANT OPTION;
GO
```

В. Отмена разрешений на объекты (инструкция REVOKE)

Инструкция REVOKE отменяет разрешения, ранее предоставленные инструкцией GRANT. Синтаксис инструкции REVOKE:

```
REVOKE [ GRANT OPTION FOR ] список_разрешений ON список_объектов  
{ FROM | TO } список_принципалов  
[ CASCADE ]  
[ AS принципал ]
```

Пояснения к инструкции REVOKE.

1. Необязательная фраза GRANT OPTION FOR показывает, что право на предоставление заданного разрешения другим принципалам будет отменено. Само разрешение отменено не будет.
2. Необязательное ключевое слово CASCADE показывает, что отменяемое разрешение также отменяется для других принципалов, для которых оно было предоставлено или запрещено этим принципалом. Каскадная отмена разрешения, предоставленного с помощью параметра WITH GRANT OPTION, приведет к отмене прав GRANT и DENY для этого разрешения.
3. Необязательная фраза AS принципал указывает принципала, от которого принципал, выполняющий данный запрос, получает право на отмену разрешения.

Пример. Отмена разрешения EXECUTE для хранимой процедуры

```
USE AdventureWorks;  
REVOKE EXECUTE ON OBJECT::HumanResources.uspUpdateEmployeeHireInfo FROM  
Role03;  
GO
```

С. Запрет разрешений на объекты (инструкция DENY)

Инструкция DENY запрещает разрешения на члены класса OBJECT защищаемых объектов. Синтаксис инструкции DENY:

```
DENY список_разрешений ON список_объектов  
TO список_принципалов  
[ CASCADE ]  
[ AS принципал ]
```

Пример. Запрет разрешения REFERENCES на представление с CASCADE

```
USE AdventureWorks;  
DENY REFERENCES (EmployeeID) ON OBJECT::HumanResources.vEmployee TO User02  
CASCADE; GO
```

Цепочки владения

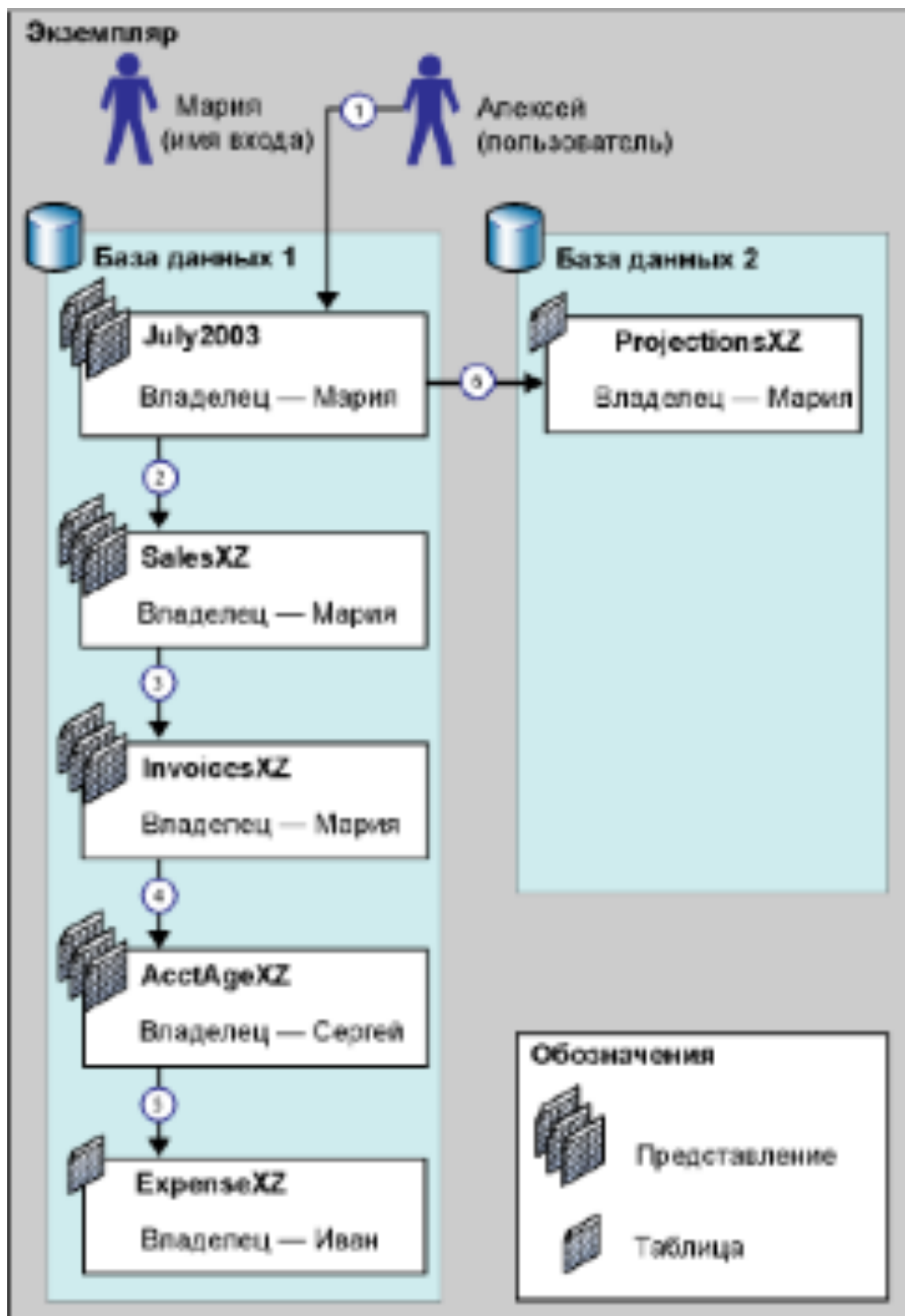
Если несколько объектов базы данных последовательно обращаются друг к другу, то такая последовательность известна как цепочка. Такие цепочки не могут существовать независимо, но когда SQL Server проходит по звеньям цепи, то SQL Server проверяет разрешения составляющих объектов иначе, нежели при раздельном доступе к объектам. Эти различия имеют важные последствия для обеспечения безопасности. Цепочки владения позволяют управлять доступом к нескольким объектам, таким как таблицы, назначая разрешения одному объекту, например представлению. Цепочки владения также обеспечивают небольшое

повышение производительности в случаях, когда позволено пропускать проверку наличия разрешений. Проверка разрешений в цепи выполняется так:

1. SQL Server сначала сравнивает владельца вызываемого объекта с владельцем вызывающего объекта.
2. Если оба объекта имеют одного владельца, то разрешения для ссылаемого объекта не проверяются.

Пример цепочки владения

1. Алексей выполняет инструкцию `SELECT *` в представлении `July2003`. SQL Server проверяет разрешения в представлении и подтверждает, что Алексей имеет разрешение выбирать.
2. Представление `July2003` требует данные из представления `SalesXZ`. SQL Server проверяет владение `SalesXZ`. Владелец этого представления (Мария) такой же, как у вызывающего представления, поэтому разрешения для `SalesXZ` не проверяются. Возвращаются необходимые данные.
3. Представление `SalesXZ` требует данные из представления `InvoicesXZ`. SQL Server проверяет владение представления `SalesXZ`. Владелец этого представления такой же, как у предшествующего объекта, поэтому разрешения для `InvoicesXZ` не проверяются. Возвращаются необходимые данные. До этого этапа все элементы последовательности имели одного владельца (Мария). Это известно как неразрывная цепочка владения.
4. Представление `InvoicesXZ` требует данные из представления `AcctAgeXZ`. SQL Server проверяет владение представления `AcctAgeXZ`. Владелец этого представления иной, чем у предшествующего объекта (Сергей, а не Мария), поэтому должны быть получены полные сведения о разрешениях на это представление. Если на представление `AcctAgeXZ` имеются разрешения, которые обеспечивают доступ со стороны пользователя Алексей, сведения будут возвращены.
5. Представление `AcctAgeXZ` требует данные из представления `ExpenseXZ`. SQL Server проверяет владение таблицы `ExpenseXZ`. Владелец этой таблицы иной, чем у предшествующего объекта (Иван, а не Сергей), поэтому должны быть получены полные сведения о разрешениях на эту таблицу. Если таблица `ExpenseXZ` имеет разрешения, которые обеспечивают доступ со стороны пользователя Алексей, сведения возвращаются.
6. Если представление `July2003` пытается получить данные из таблицы `ProjectionsXZ`, то сервер сначала проверяет наличие цепочечных связей между базами данных `Database 1` и `Database 2`. Если цепочечные связи между базами данных активны, то сервер проверяет владение для таблицы `ProjectionsXZ`. Владелец этой таблицы такой же, как у вызывающего представления (Мария), поэтому разрешения для этой таблицы не проверяются. Возвращаются необходимые данные.



По умолчанию межбазовые цепочки владения отключены. SQL Server можно настроить так, чтобы разрешить цепочку владения между конкретными базами данных или между всеми базами данных внутри одного экземпляра SQL Server.

Пример потенциальных опасностей при использовании цепочек владения

Включены межбазовые цепочки владения между базой данных А и базой данных В. В этом случае член предопределенной роли базы данных db_owner любой из этих баз данных может незаконно проникнуть в другую базу данных. Процедура выглядит так:

- Член предопределенной роли базы данных db_owner в базе данных А по имени Х создает пользователя Y в базе данных А, который уже существует как пользователь в базе данных В.
- Затем Х создает в базе данных А объект, владельцем которого является Y.
- Пользователь Y из базы данных А вызывает любой объект, принадлежащий пользователю Y в базе данных В.
- Вызывающий и вызываемый объекты имеют общего владельца, поэтому разрешения для объекта в базе данных В не будут проверяться, когда Х обратится к ним через созданный ею объект. Контекст выполнения и переключение контекста

Контекст выполнения определяется подключенным к сеансу именем входа, пользователем или выполняющимся модулем. Он устанавливает идентификатор пользователя или имени входа, чьи разрешения на выполнение инструкций или совершение действий проверяются. В SQL Server контекст выполнения может переключаться на другое имя входа или пользователя при помощи выполнения инструкции EXECUTE AS или предложения EXECUTE AS в модуле. После переключения контекста SQL Server проверяет разрешения у имени входа и пользователя этой учетной записи, а не у того, кто вызвал инструкцию EXECUTE AS, или модуля. Имя входа или пользователь олицетворяется на оставшееся время выполнения сеанса или модуля либо до того момента, когда происходит явное восстановление переключения контекста (сеанс удаляется, контекст переключен на другое имя входа или на другого пользователя с помощью новой инструкции EXECUTE AS, контекст восстановлен до контекста предыдущего выполнения с помощью инструкции REVERT).

Синтаксис инструкции EXECUTE AS имеет вид:

```
EXECUTE AS {LOGIN | USER} = 'имя' [WITH {NO REVERT | COOKIE INTO @varbinary_переменная}] | CALLER
```

- LOGIN указывает, что область олицетворения — это уровень сервера.
- USER указывает, область олицетворения ограничена текущей базой данных.
- 'имя' - допустимое имя входа или имя пользователя.
- NO REVERT указывает, что переключение контекста нельзя вернуть к предыдущему контексту.
- COOKIE INTO @varbinary_переменная указывает, что контекст выполнения можно переключить к предыдущему контексту, если при вызове инструкция REVERT WITH COOKIE содержит правильное значение @varbinary_переменная. SQL Server передает куки-файл в @varbinary_переменная.
- CALLER указывает, что инструкции модуля выполняются в контексте вызывающей стороны. Вне модуля инструкция не действует.

Синтаксис предложения EXECUTE AS имеет вид:

```
EXECUTE AS { CALLER | SELF | OWNER | 'имя_пользователя' }
```

- EXECUTE AS CALLER указывает, что инструкции, содержащиеся в модуле, выполняются в контексте пользователя, вызывающего этот модуль.
- EXECUTE AS SELF указывает, что модуль выполняется от имени того пользователя, который последним модифицировал модуль.
- EXECUTE AS OWNER указывает, что инструкции, содержащиеся в модуле, выполняются в контексте текущего владельца этого модуля.
- EXECUTE AS 'имя_пользователя' указывает, что инструкции, содержащиеся в модуле, выполняются в контексте пользователя, указываемого аргументом 'имя_пользователя'.

Пример контекста выполнения

```
USE master
GO
-- Создаем имена входов
CREATE LOGIN User1 WITH PASSWORD='^*ahfn2@^(K' GO
CREATE LOGIN User2 WITH PASSWORD='*HABa7s7aas' GO
CREATE LOGIN User3 WITH PASSWORD='zxd837&^gqF' GO
CREATE DATABASE MyDB
GO
USE MyDB
GO
-- Создаем пользователей и схемы
CREATE USER User3 WITH DEFAULT_SCHEMA=User3
GO
CREATE SCHEMA User3 AUTHORIZATION User3
GO
CREATE USER User2 WITH DEFAULT_SCHEMA=User2
GO
CREATE SCHEMA User2 AUTHORIZATION User2
GO
CREATE USER User1 WITH DEFAULT_SCHEMA=User1
GO
CREATE SCHEMA User1 AUTHORIZATION User1
GO
-- Пользователь User3 имеет право создавать GRANT CREATE TABLE TO User3
GO
-- Пользователь User2 имеет право создавать GRANT CREATE PROC TO User2
GO
EXECUTE AS LOGIN='User3'
GO
CREATE TABLE User3.CustomerInformation
(
    CustomerName nvarchar(50)
```



```

GO
INSERT INTO CustomerInformation VALUES('Bryan's Bowling Alley') INSERT INTO
CustomerInformation VALUES('Tammie's Tavern') INSERT INTO CustomerInformation
VALUES('Frank's Fresh Produce') GO
GRANT SELECT ON CustomerInformation TO User2
GO
REVERT
GO
EXECUTE AS LOGIN='User2'
GO
--create a stored proc that will return the rows in our table CREATE PROC ViewCustomerNames
AS

BEGIN
SELECT * FROM User3.CustomerInformation

END GO

GRANT EXECUTE ON ViewCustomerNames TO User1 GO
REVERT
GO

```

```

EXECUTE AS LOGIN='User1'
-- User1 cannot access table directly

```

```

SELECT * FROM User3.CustomerInformation

```

таблицу

процедуру

Msg 229, Level 14, State 5, Line 3

The SELECT permission was denied on the object 'CustomerInformation', database 'myDB', schema 'User3'.

--User1 can execute the procedure but does not have permissions on the underlying table

```

EXEC User2.ViewCustomerNames

```

Msg 229, Level 14, State 5, Procedure ViewCustomerNames, Line 5

The SELECT permission was denied on the object 'CustomerInformation', database 'myDB', schema 'User3'.

```

GO
REVERT
GO
EXECUTE AS LOGIN='User2'
GO
ALTER PROCEDURE ViewCustomerNames WITH EXECUTE AS OWNER AS

BEGIN
SELECT * FROM User3.CustomerInformation

```

```
END
GO
REVERT
GO
EXECUTE AS LOGIN='User1'
--User1 still cannot access table directly SELECT * from User3.CustomerInformation
```

Msg 229, Level 14, State 5, Line 1

The SELECT permission was denied on the object 'CustomerInformation', database 'myDB', schema 'User3'

--User1 can execute a procedure that uses the CustomerInformation table

```
EXEC User2.ViewCustomerNames GO
REVERT
GO
```

Аудит SQL Server

Начиная с версии SQL Server 2008 в редакции Enterprise вводится аудит SQL Server Audit – функциональность системы безопасности, которая может отслеживать практически любое действие с сервером или базой данных (выполняемое пользователями) и записывать эти действия в файловую систему или системный журнал Windows.

Система управления на основе политик

Одна из новых возможностей, появившаяся в SQL Server 2008, – система управления на основе политик (Policy-Based Management), которая позволяет создавать политики для обеспечения соответствия нормативам управления базой данных.

Система управления на основе политик позволяет администратору баз данных (АБД) создавать политики для управления объектами и экземплярами базы данных. Эти политики дают АБД возможность устанавливать правила создания и изменения объектов и их свойств. С помощью новой системы можно, например, создать политику уровня БД, запрещающую использование для БД свойства AutoShrink. Другой пример – политика, в соответствии с которой все имена табличных триггеров в таблице БД начинаются с tr_.

Система управления на основе политик предусматривает использование новых терминов и понятий. Основными из них являются:

1. Политика (Policy) – набор условий, определенных аспектами цели управления. Другими словами, политика — это набор правил для свойств БД или серверных объектов.
2. Цель управления (Target) – объект, управляемый данной системой. Сюда относятся такие объекты, как экземпляр БД, база данных, таблица, хранимая процедура, триггер, индекс.
3. Аспект (Facet)– свойство объекта (цели управления), которое используется системой управления на основе политик. Например, имя триггера или свойство базы данных AutoShrink.
4. Условие (Condition) – критерий для аспектов цели управления. Например, можно создать для факта условие, по которому все имена хранимых процедур в схеме «Banking» должны начинаться с bnk_.

Кроме того, политику можно связать с определенной категорией, что позволяет осуществлять управление набором политик, привязанных к той же самой категории. Политика может принадлежать только к одной категории.

Режим оценки политик

Существует несколько режимов оценки политик:

1. По запросу (On demand) – оценку политики запускает непосредственно администратор.
2. При изменении: запретить (On change: prevent) – для предотвращения нарушения политики используются триггеры DDL.
3. При изменении: только внесение в журнал (On change: log only) – для проверки политики при изменении используются уведомления о событиях.
4. По расписанию (On schedule) – для проверки политики на нарушения используется задание агента SQL (SQL Agent).

Преимущества системы управления на основе политик

Система управления на основе политик позволяет АБД в полной мере контролировать процессы, происходящие в базе данных. Администратор получает возможность реализовать

принятые в компании политики на уровне БД. Политики, принятые только на бумаге, помогают определить основные принципы управления базой данных и могут служить прекрасным руководством к действию, но воплощать их в жизнь очень нелегко. Для обеспечения строгого соответствия БД принятым нормативом АБД приходится пристально следить за ее повседневным использованием и функционированием. Система управления на основе политик позволяет раз и навсегда выработать политики управления и быть уверенным в том, что они будут применяться постоянно и в полном объеме.

Шифрование

SQL Server поддерживает три механизма шифрования:

1. на основе сертификатов (стандарт X. 509v3),
2. на основе асимметричных ключей (алгоритм RSA),
3. на основе симметричных ключей (алгоритмы шифрования RC4, RC2, DES, AES).

Для работы с этими сущностями используются следующие операторы T-SQL:

```
CREATE/ALTER/DROP/BACKUP CERTIFICATE CREATE/ALTER/DROP/OPEN/CLOSE  
ASYMMETRIC KEY CREATE/ALTER/DROP/OPEN/CLOSE SYMMETRIC KEY
```

Детальное описание иерархической схемы шифрования данных в SQL Server 2005 см. в разделе «Encryption Hierarchy» MSDN.

Контекстное переключение

SQL Server позволяет задать контекст выполнения хранимых процедур и пользовательских функций с помощью выражения EXECUTE AS, помещенного в заголовок определения модуля. Данный механизм позволяет одному пользователю выполнять действия внутри модуля так, будто он аутентифицирован как другой пользователь. Параметр EXECUTE AS имеет четыре возможных значения:

- CALLER. Если программист указывает EXECUTE AS CALLER, операторы внутри модуля выполняются в контексте пользователя, вызвавшего процедуру. Поэтому пользователь, выполняющий процедуру, должен иметь соответствующие разрешения не только на запуск процедуры, но и на любые объекты базы данных, на которые она ссылается.
- USER. Если используется значение EXECUTE AS USER имя_пользователя, процедура выполняется в контексте того пользователя, чье имя указано в параметре. При выполнении процедуры SQL Server сначала проверяет, имеет ли пользователь разрешение EXECUTE на данную процедуру, затем проверяет разрешения на операторы внутри процедуры для пользователя, указанного в параметре EXECUTE AS USER. Для того чтобы иметь возможность указать AS имя_пользователя, необходимо иметь специальные разрешения (например, IMPERSONATE) или быть членом специальной роли (sysadmin или db_owner).
- SELF. EXECUTE AS SELF, аналогично EXECUTE AS USER имя_пользователя, где имя_пользователя является именем человека, создающего или изменяющего модуль. Система сохраняет идентификатор пользователя (UID) вместо самого значения SELF. Пользователь, указанный в параметре SELF, не обязательно должен быть владельцем объекта. На самом деле объекты в SQL Server 2005 не имеют владельцев, но можно думать о владельцах схем так, как будто они владеют всеми объектами в схемах.

- **OWNER.** Применение EXECUTE AS OWNER хорошо подходит в ситуациях, если пользователь создает хранимые процедуры и таблицы в ходе своей работы, тогда, переходя на другое место, он сможет передать их в собственность другому пользователю, будучи уверенным, что они всегда будут выполняться в контексте безопасности собственника.

Олицетворение

Можно настроить SQL Server и Windows таким образом, чтобы экземпляр SQL Server мог подключаться к другому экземпляру SQL Server в контексте пользователя Windows, прошедшего проверку подлинности. Это называется олицетворением или делегированием. При использовании делегирования экземпляр SQL Server, к которому подключается пользователь Windows с проверкой подлинности Windows, выполняет олицетворение этого пользователя при обмене данными с другим экземпляром SQL Server или поставщиком SQL Server. Этот второй экземпляр или поставщик могут располагаться на том же компьютере или на удаленном компьютере в том же домене Windows, что и первый экземпляр.

Встроенная поддержка прозрачного шифрования

Механизм прозрачного шифрования основан на использовании мастер-ключа, сертификата, защищенного мастер-ключом, и ключа для шифрования базы данных. Для работы с мастер-ключом используются следующие операторы T-SQL:

CREATE/ALTER/DROP/OPEN/CLOSE MASTER KEY

Прозрачное шифрование позволяет создавать индексы и выполнять поиск по зашифрованным данным без использования каких-либо дополнительных функций.

Распределенные вычисления. Hadoop

Hadoop не так уж сложен, ядро состоит из файловой системы HDFS и MapReduce фреймворка для обработки данных из этой файловой системы.

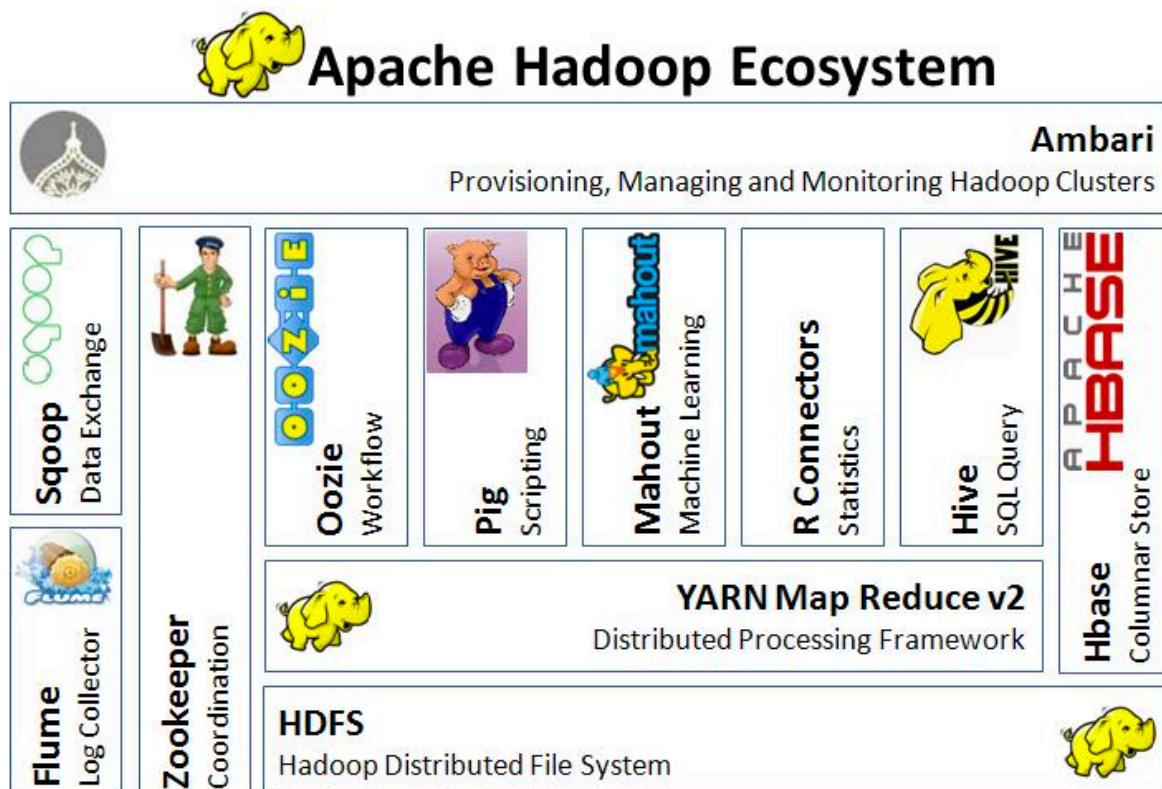
Hadoop следует использовать, если:

- Вычисления должны быть компонентными, другими словами, вы должны иметь возможность запустить вычисления на подмножестве данных, а затем слить результаты.
- Вы планируете обрабатывать большой объем неструктурированных данных — больше, чем можно уместить на одной машине (> нескольких терабайт данных).

Hadoop не следует использовать:

- Для некомпонуемых задач — например, для задач рекуррентных.
- Если весь объем данных уместается на одной машине. Существенно сэкономяте время и ресурсы.
- Hadoop в целом — система для пакетной обработки и не подходит для анализа в режиме реального времени (здесь на помощь приходит система Storm).

Архитектура HDFS и типичный Hadoop кластер



HDFS подобна другим традиционным файловым системам: файлы хранятся в виде блоков, существует маппинг между блоками и именами файлов, поддерживается древовидная структура, поддерживается модель доступа к файлам основанная на правах и т. п.

Отличия HDFS:

- Предназначена для хранения большого количества огромных (>10GB) файлов. Одним Следствие — большой размер блока по сравнению с другими файловыми системами (>64MB)
- Оптимизирована для поддержки потокового доступа к данным (high-streaming read), соответственно производительность операций произвольного чтения данных начинает хромать.
- Ориентирована на использование большого количество недорогих серверов. В частности, серверы используют JBOV структуру (Just a bunch of disk) вместо RAID — зеркалирование и репликация осуществляются на уровне кластера, а не на уровне отдельной машины.
- Многие традиционные проблемы распределенных систем заложены в дизайн — уже по дефолту все выход отдельных нод из строя является совершенно нормальной и естественной операцией, а не чем-то из ряда вон.

Hadoop-кластер состоит из нод трех типов: NameNode, Secondary NameNode, Datanode.

namenode — мозг системы. Как правило, одна нода на кластер (больше в случае Namenode Federation, но мы этот случай оставляем за бортом). Хранит в себе все метаданные системы — непосредственно маппинг между файлами и блоками. Если нода 1 то она же и является Single Point of Failure. Эта проблема решена во второй версии Hadoop с помощью Namenode Federation.

Secondary NameNode — 1 нода на кластер. Принято говорить, что «Secondary NameNode» — это одно из самых неудачных названий за всю историю программ. Действительно, Secondary NameNode не является репликой NameNode. Состояние файловой системы хранится непосредственно в файле fsimage и в лог файле edits, содержащим последние изменения файловой системы (похоже на лог транзакций в мире РСУБД). Работа Secondary NameNode заключается в периодическом мерже fsimage и edits — Secondary NameNode поддерживает размер edits в разумных пределах. Secondary NameNode необходима для быстрого ручного восстановления NameNode в случае выхода NameNode из строя.

В реальном кластере NameNode и Secondary NameNode — отдельные сервера, требовательные к памяти и к жесткому диску. А заявленное “commodity hardware” — уже случай DataNode.

DataNode — Таких нод в кластере очень много. Они хранят непосредственно блоки файлов. Нода регулярно отправляет NameNode свой статус (показывает, что еще жива) и ежечасно — репорт, информацию обо всех хранимых на этой ноде блоках. Это необходимо для поддержания нужного уровня репликации.

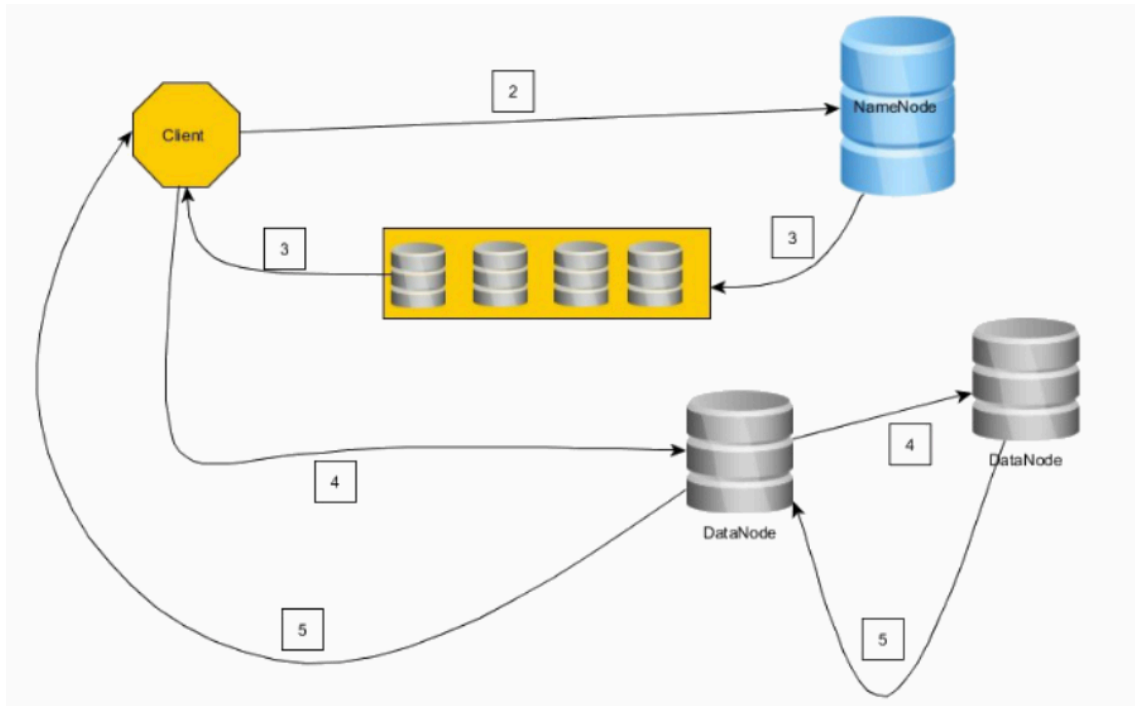
Посмотрим, как происходит запись данных в HDFS:

1. Клиент разрезает файл на цепочки блокового размера.
2. Клиент соединяется с NameNode и запрашивает операцию записи, присылая количество блоков и требуемый уровень репликации
3. NameNode отвечает цепочкой из DataNode.

4. Клиент соединяется с первой нодой из цепочки (если не получилось с первой, со второй и т. д. не получилось совсем — откат). Клиент делает запись первого блока на первую ноду, первая нода — на вторую и т. д.

5. По завершении записи в обратном порядке (4 -> 3, 3 -> 2, 2 -> 1, 1 -> клиенту) присылаются сообщения об успешной записи.

6. Как только клиент получит подтверждение успешной записи блока, он оповещает NameNode о записи блока, затем получает цепочку DataNode для записи второго блока и т.д.



Клиент продолжает записывать блоки, если сумеет записать успешно блок хотя бы на одну ноду, т. е. репликация будет работать по хорошо известному принципу «eventual», в дальнейшем NameNode обязуется компенсировать и таки достичь желаемого уровня репликации.

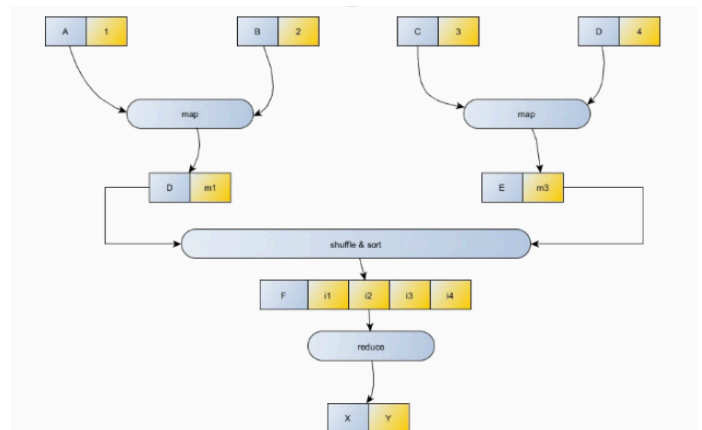
Завершая обзор HDFS и кластера, обратим внимание на еще одну замечательную особенность Hadoop'a — rack awareness. Кластер можно сконфигурировать так, чтобы NameNode имел представление, какие ноды на каких rack'ах находятся, тем самым обеспечив лучшую защиту от сбоев.

MapReduce

Единица работы job — набор map (параллельная обработка данных) и reduce (объединение выводов из map) задач. Map-задачи выполняют mapper'ы, reduce — reducer'ы. Job состоит минимум из одного mapper'a, reducer'ы опциональны. Здесь разобран вопрос разбиения задачи на map'ы и reduce'ы. Если слова «map» и «reduce» вам совсем непонятны, можно посмотреть классическую статью на эту тему.

Модель MapReduce

- Ввод/вывод данных происходит в виде пар (key, value)
- Используются две функции map: $(K1, V1) \rightarrow (K2, V2), (K3, V3), \dots$ — отображающая пару ключ-значение на некое множество промежуточных пар ключей и значений, а также reduce: $(K1, (V2, V3, V4, \dots, VN)) \rightarrow (K1, V1)$, отображающая некоторое множество значений, имеющих общий ключ на меньшее множество значений.
- Shuffle and sort нужна для сортировки ввода в reducer по ключу, другими словами, нет смысла отправлять значение $(K1, V1)$ и $(K1, V2)$ на два разных reducer'а. Они должны быть обработаны вместе.

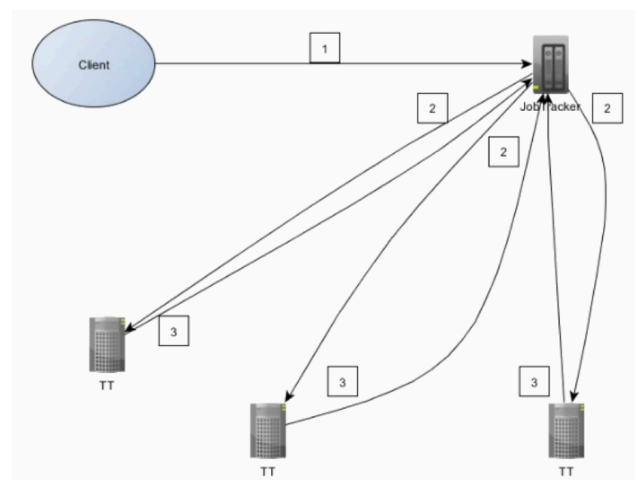


Посмотрим на архитектуру MapReduce

1. Для начала расширим представление о hadoop-кластере, добавив в кластер два новых элемента — JobTracker и TaskTracker. JobTracker непосредственно запросы от клиентов и управляет map/reduce задачами на TaskTracker'ах. JobTracker и NameNode разносится на разные машины, тогда как DataNode и TaskTracker находятся на одной машине.

Взаимодействие клиента и кластера выглядит следующим образом:

1. Клиент отправляет job на JobTracker. Job представляет из себя jar-файл.
2. JobTracker ищет TaskTracker'ы с учетом локальности данных, т.е. предпочитая те, которые уже хранят данные из HDFS. JobTracker назначает map и reduce задачи TaskTracker'ам
3. TaskTracker'ы отправляют отчет о выполнении работы JobTracker'у.



Неудачное выполнение задачи — ожидаемое поведение, провалившиеся таски автоматически перезапускаются на других машинах.

В Map/Reduce 2 (Apache YARN) больше не используется терминология «JobTracker/TaskTracker». JobTracker разделен на **ResourceManager** — управление ресурсами и **Application Master** — управление приложениями (одним из которых и является непосредственно MapReduce). MapReduce v2 использует новое API

Базы Данных

Семинар 1

БД - самодокументированное собрание интегрированных записей.

Самодокументированное - Содержит описание собственной структуры -> Это словарь данных, каталог данных, метаданные.

Интегрированных - Файлы данных, метаданные, индексы

Требования к организации БД:

1. Не избыточность данных. Каждый элемент данных присутствует в БД в единственном экземпляре.
2. Совместное использование данных многими пользователями.
3. Эффективность доступа к БД. Высокое быстродействие, малое время отклика.
4. Целостность данных. Сохранение внутренней логики, структуры, соблюдение явно заданных правил.
5. Безопасность данных. Защита от преднамеренного или непреднамеренного искажения или разрушения.
6. Восстановление данных после программных и аппаратных сбоев.
7. Независимость данных от других прикладных программ.

СУБД - приложение, обеспечивающее создание, хранение, обновление и поиск информации в БД.

Основные функции СУБД:

1. Управление данными во внешней памяти
2. Управление операционной памятью - буферизация данных в ОП
3. Управление транзакциями
4. Журнализация
5. Поддержка языков БД (SQL)

Основные компоненты СУБД:

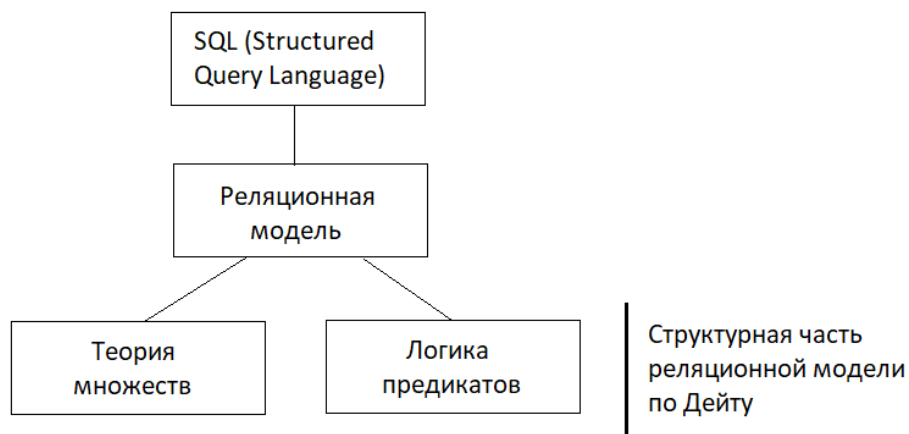
1. Ядро. Управление данными во внешней и оперативной памяти + журнализация.
2. Процессор языка БД. Оптимизация запросов на изменение и извлечение данных и создание исполняемого кода.

3. Подсистема поддержки времени использования. Интерпретирует программы манипуляции данными, создающие пользовательский интерфейс с БД.
4. Сервисные программы. Дополнительные возможности по обслуживанию ИС.

ОСНОВЫ ЯЗЫКА T-SQL

SQL - стандарт международной организации по стандартизации ISO (SQL:2011) и американского национального института стандартов (ANSI).

T-SQL - диалект стандартного языка SQL.



В основе стандарта лежит реляционная модель - математическая модель для управления и обработки данных. Реляционная модель оперирует понятием «Отношение». Отношение имеет заголовок и тело.

Заголовок - набор атрибутов (в SQL - столбцы), каждый из которых имеет определенный тип. Атрибут - совокупность имени и типа данных.

Тело - множество картежей (в SQL - строки). Заголовок кортежа - заголовок отношения.

Заголовок отношения				
Атрибут				
1	2	3	4	5
aaa	aba	abb	bba	bab
bbb	bcb	bcc	ccb	cbc
ccc	cdc	cdd	ddc	dcd
ddd	ded	dee	eed	ede

Кортеж

Тело
отношения

Логику работы с данными можно разделить на три основных языка:

1. DDL (Data Definition Language). Служит для описания структуры БД:
 - Создание объекта (create)
 - Изменение объекта (alter)
 - Удаление объекта (drop)
2. DML (Data Manipulation Language). Служит для выполнения операций с данными:
 - Выбор информации (select)
 - Добавление данных (insert)
 - Обновление данных (update)
 - Удаление данных (delete)
3. DCL (Data Control Language). Служит для осуществления администраторских действий
 - Выдача прав доступа к объекту (grant)
 - Удаление прав доступа на объект (revoke)

DDL

Создание и изменение таблиц.



Временные таблицы. Существуют на протяжении сессии БД (например, пока не закрыли окно запросов). Работа с командами аналогична обычным таблицам. Название начинается с символа «#». Если используется 1 знак «#», то создается локальная таблица, доступная только в текущей сессии. Если используется 2 знака, создается глобальная временная таблица, доступная всем открытым сессиям БД.

```
CREATE TABLE #table1
```

Табличные переменные. Переменные с особым типом данных TABLE, используются для временного хранения результирующего набора данных в виде строк таблицы. Создаются с помощью инструкции DECLARE. Переменные такого типа служат альтернативой временным таблицам. Табличные переменные нельзя изменить после их создания или создать с помощью выборки из другой таблицы, также не изменяются в результате откатов транзакция, т.к. не являются частью постоянных данных БД.

```
DECLARE @Table2 (...);
```

Представление. Виртуальные таблицы, содержащие запросы, динамически извлекающие используемые данные. Изменение данных в реальной таблице БД отображается в содержимом представления при первом же обращении.

```
CREATE VIEW View1 () AS ...;
```

Индексированное представление. Представления, для которых создан кластеризованный индекс. Создание такого индекса означает, что система материализует динамические данные в страницах узлов структуры индекса. Результирующий набор, возвращаемый представлением с кластеризованным индексом, сохраняется в БД таким же образом как и таблица с кластеризованным индексом.

```
CREATE VIEW View2 () AS ...;  
CREATE UNIQUE CLUSTERED INDEX index_name ON View2;
```

Таблица. Основной способ хранения данных.

Схема обращения к таблице: [название БД].[название схемы].Название таблицы.

Явный способ создания таблиц.

```

CREATE TABLE [database_name.[schema_name].|schema_name. ] table_name [ AS
FileTable ](
    { <column_definition>
    | <computed_column_definition>
    | <column_set_definition>
    | [<table_constraint>]
      | [<table_index> ] [ ,...n ]
    }
)
[ON {partition_scheme_name (partition_column_name) | "default"}]
[ WITH ( <table_option> [ ,...n ] ) ]
[ ; ]

```

Атрибут – название столбца, его тип + дополнительные настройки: допускаются/не допускаются NULL значения или в это столбец-счетчик + могут присутствовать дополнительные ограничения на столбец (уникальность/первичный ключ/...). Например, `MyColumn int not null`

```

<column_definition> ::= column_name <data_type>
    [NULL | NOT NULL]
    | [IDENTITY [(seed, increment)] [NOT FOR REPLICATION]]
    [<column_constraint> [ ...n ] ]

```

Тип - название типа (если нужно, еще и название схемы), размер, точность, максимальное значение, XML-коллекция. Например, `varchar(100)`, `decimal(15,3)`

```

<data _type> ::= [type_schema_name.] type_name
    [ (precision [ ,scale]
        | max
        | [{ CONTENT | DOCUMENT }] xml_schema_collection)
    ]

```

Ограничения - правила, принудительно применяющиеся к компонентам БД (таблицы/столбцы/...). Могут иметь имя, могут содержать логическое выражение для проверки содержимого атрибута. Бывают кластеризованные и не кластеризованные. Например: `PRIMARY KEY (contact_id, group_id)`, `FOREIGN KEY (contact_id) REFERENCES contacts (contact_id) ON DELETE CASCADE ON UPDATE NO ACTION`

```

<column_constraint> ::= [CONSTRAINT constraint_name]
{ {PRIMARY KEY | UNIQUE} [CLUSTERED | NONCLUSTERED]
  [ WITH FILLFACTOR = fillfactor
    |WITH (< index_option > [ , ...n ])
  ]
  [ ON {partition_scheme_name (partition_column_name)
      | filegroup
      | "default" }
  ]
  | [ FOREIGN KEY ] REFERENCES [schema_name.]
    referenced_table_name [(ref_column)]
  [ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}]
  [ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}]
  [NOT FOR REPLICATION]
  |CHECK [NOT FOR REPLICATION] (logical_expression)
}

```

```

< table_constraint > ::= [CONSTRAINT constraint_name]
{
  { PRIMARY KEY | UNIQUE }
  [CLUSTERED | NONCLUSTERED]
  (column [ASC | DESC] [ ,...n ] )
  [
    WITH FILLFACTOR = fillfactor
    | WITH ( <index_option> [ , ...n ])
  ]
  [ON {partition_scheme_name (partition_column_name)
      |filegroup
      |"default"
    }
  ]
  | FOREIGN KEY (column [ ,...n ])
    REFERENCES referenced_table_name [(ref_column [ ,...n ])]
  [ON DELETE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}]
  [ON UPDATE {NO ACTION|CASCADE|SET NULL|SET DEFAULT}]
  [ NOT FOR REPLICATION ]
  | CHECK [NOT FOR REPLICATION] ( logical_expression )
}

```


Свойство вычисляемого столбца. Определяется как название атрибута и способ его получения. Вычисляемый столбец является виртуальным столбцом, который физически не хранится в таблице, а вычисляется с помощью некоторого выражения, использующего другие столбцы данной таблицы. Например: `cost AS price*qty`

```
<computed_column_definition> ::= column_name AS
    computed_column_expression
    [PERSISTED [NOT NULL]]
    [ [CONSTRAINT constraint_name] {PRIMARY KEY | UNIQUE}
      [CLUSTERED | NONCLUSTERED]
      [ WITH FILLFACTOR = fillfactor
        | WITH (<index_option> [ , ...n ] )
      ]
    [ON { partition_scheme_name (partition_column_name)
          | filegroup
          | "default"
        }
      ]
    | [FOREIGN KEY]
      REFERENCES referenced_table_name [ (ref_column ) ]
      [ON DELETE {NO ACTION|CASCADE}]
      [ON UPDATE {NO ACTION}]
      [NOT FOR REPLICATION]
      [CHECK [ NOT FOR REPLICATION ] (logical_expression )
    ]
```

Дополнительные опции при создании таблицы: **сжатие** (основная задача, которая является оптимизацией хранения и доступа к таблице. Позволяет сжимать данные, применять сжатие для конкретных блоков таблицы (партиции), указывать специфические имена при использовании первичных ограничений, ограничений уникальности.

```

<table_option> ::= {
    [DATA_COMPRESSION = {NONE|ROW|PAGE}
        [ON PARTITIONS ({<partition_number_expression>|
<range>}
            [ , ...n ]))
    ]
    [FILETABLE_DIRECTORY = <directory_name>]
    [FILETABLE_COLLATE_FILENAME = {<collation_name>
database_default}]
    [FILETABLE_PRIMARY_KEY_CONSTRAINT_NAME =
<constraint_name>]
    [FILETABLE_STREAMID_UNIQUE_CONSTRAINT_NAME =
<constraint_name>]
    [FILETABLE_FULLPATH_UNIQUE_CONSTRAINT_NAME =
<constraint_name>]
}

```

Пример создания таблицы:

```

CREATE TABLE dbo.EmployeePhoto
(
    Id int IDENTITY(1, 1),
    EmployeeId int NOT NULL PRIMARY KEY,
    Photo varbinary(max) FILESTREAM NULL,
    MyRowGuidColumn uniqueidentifier NOT NULL ROWGUIDCOL
    UNIQUE DEFAULT NEWID()
);

```

Базы Данных

Семинар 2

DDL (Продолжение)

Мы рассмотрели создание объектов (create). Следующий блок инструкций — alter. Изменение уже существующих объектов в системе.

Что можно сделать с помощью инструкции ALTER. Обратите внимание, на примере изменения типа атрибута, что синтаксис для разных БД отличается:

1. Добавить атрибут (столбец)

```
ALTER TABLE table_name ADD column_name datatype;  
ALTER TABLE table_name ADD (column_name_1 datatype,  
                                column_name_2 datatype,  
                                ...  
                                column_name_n datatype);
```

2. Создание ограничений

```
ALTER TABLE table_name_1 ADD CONSTRAINT constant_name  
        FOREIGN KEY (column_name_1, ..., column_name_n)  
        REFERENCES table_name_1 (column_name_1, ...,  
column_name_n)  
        ON Update Cascade  
        ON Delete Cascade;  
ALTER TABLE table_name ADD CONSTRAINT primary_key_name  
        PRIMARY KEY (column_name_1, ..., column_name_n);
```

3. Удалить атрибут/ограничения

```
ALTER TABLE table_name DROP COLUMN column_name;  
ALTER TABLE table_name DROP CONSTRAINT constraint_name;
```

3. Изменить тип атрибута

```
ALTER TABLE table_name ALTER COLUMN column_name datatype; (SQL Server)
```

```
ALTER TABLE table_name MODIFY COLUMN column_name datatype; (MySQL, Oracle)
```

```
ALTER TABLE table_name ALTER COLUMN column_name TYPE datatype; (PostgreSQL)
```

Работает не для всех типов, например, пока в таблице пустая можно изменять типы атрибутов как угодно. Если в таблице уже есть данные, например, при изменении varchar на number, будет ошибка, ddl не обновится. Oracle(11) также не допускает и обратного преобразования из number в varchar.

4. Переименование атрибута

```
ALTER TABLE table_name CHANGE column_old column_new; (MySQL)
```

```
ALTER TABLE table_name RENAME COLUMN column_old TO column_new; (Oracle)
```

5. Переименование объекта

```
ALTER TABLE table_name RENAME TO new_table_name;
```

Для удаления объектов, таких как таблицы, представления, индексы, базы БД используется инструкция DROP.

```
DROP TABLE table_name;
```

DML

Язык для проведения манипуляций с данными. Используется для вставки/обновления/удаления данных. При этом сами объекты не изменяются. Языки DML могут существенно различаться у различных производителей СУБД. Существует стандарт SQL, установленный ANSI, но производители СУБД часто предлагают свои собственные «расширения» языка. Функции языков DML определяются первым словом в предложении (часто называемом **запросом**), которое почти

всегда является глаголом. В случае с SQL эти глаголы – «select» («выбрать»), «insert» («вставить»), «update» («обновить»), и «delete» («удалить»). Это превращает природу языка в ряд обязательных утверждений (команд) к базе данных.

Название SQL (Structured Query Language - язык структурированных запросов) отражает тот факт, что запросы являются наиболее часто используемым элементом SQL. Запрос - это оператор, который посылает команду Системе Управления Базой Данных (СУБД) произвести манипуляцию или отобразить определенную информацию. Все запросы по выборке данных в SQL конструируются с помощью оператора **SELECT**. Он позволяет выполнять довольно сложные проверки и обработку данных.

Запрос может выводить данные из определенного столбца или из всех столбцов таблицы. Чтобы создать простейших SELECT запрос, необходимо указать имя столбца и название таблицы. Также необходимо сказать, что SQL код является регистронезависимым. Пример запроса:

```
SELECT column_list
FROM table_name
[WHERE условие]
[GROUP BY условие]
[HAVING условие]
[ORDER BY условие]
```

Разберем каждую из инструкций:

SELECT - Ключевое слово, которое сообщает базе данных о том, что оператор является запросом. Все запросы начинаются с этого слова, за ним следует пробел. Column_list - Список столбцов таблицы, указанный через запятую, которые выбираются запросом. Столбцы, не указанные в операторе, не будут включены в результат. Если необходимо вывести данные всех столбцов, можно использовать сокращенную запись. Звездочка (*) означает полный список столбцов.

```
SELECT *  
FROM my_example_table;
```

```
SELECT column_1, column_2, column_3 AS myALIAS  
FROM my_example_table AS myTable;
```

MyALIAS — Псевдоним, который используется как название столбца при отображении результата запроса на экран. Псевдоним может быть как у столбца, так и у всей таблицы. Псевдонимы для таблиц используются, в основном, при соединении таблиц.

FROM - Ключевое слово, которое должно присутствовать в каждом запросе. После него через пробел указывается имя таблицы, являющейся источником данных.

Код в скобках является не обязательным в операторе SELECT. Он необходим для более точного определения запроса.

WHERE - Оператор SQL WHERE служит для задания дополнительного условия выборки, операций вставки, редактирования и удаления записей. После ключевого слова следует логическое выражение - *condition* (Принимает значение true/false). Условие может включать в себя предикаты AND, OR, NOT, LIKE, BETWEEN, IS, IN, ключевое слово NULL, операторы сравнения и равенства (<, >, =).

Для примера необходимо написать несложную таблицу и заполнить парой записей — на усмотрение семинариста. Обязательно указывать результат до и после запроса. По настроению можно рассказать про некоторые функции

GROUP BY + HAVING - Оператор SQL GROUP BY используется для объединения результатов выборки по одному или нескольким столбцам. Оператор SQL HAVING является указателем на результат выполнения агрегатных функций. Агрегатной функцией в языке SQL называется функция, возвращающая какое-либо одно значение по набору значений столбца. Такими функциями являются: COUNT(), MIN(), MAX(), AVG(), SUM(). Оператор SQL HAVING аналогичен оператору WHERE за тем исключением, что применяется не для всего набора столбцов таблицы, а для набора созданного оператором GROUP BY и применяется всегда строго после него.

```
SELECT Singer, SUM(Sale)
FROM Artists
GROUP BY Singer
HAVING SUM(Sale) > 2000000;
```

```
SELECT Singer, MIN(Year)
FROM Artists
GROUP BY Singer
HAVING MIN(Year) < 1995;
```

ORDER BY - Выполняет сортировку выходных значений. Оператор SQL ORDER BY можно применять как к числовым столбцам, так и к строковым. В последнем случае, сортировка будет происходить по алфавиту.

Оператор SQL ORDER BY имеет следующий синтаксис:

```
ORDER BY column_name [ASC | DESC]
```

Сортировка может производиться как по возрастанию, так и по убыванию значений.

- Параметр ASC (по умолчанию) устанавливает порядок сортирования во возрастанию, от меньших значений к большим.

- Параметр DECS устанавливает порядок сортирования по убыванию, от больших значений к меньшим.

```
SELECT *  
FROM Artists  
ORDER BY Singer;
```

```
SELECT Singer, Album, Year  
FROM Artists  
WHERE Year > 2005  
ORDER BY Year DESC;
```

Порядок написания инструкций: select -> from -> where -> group by -> having -> order by

Порядок, в котором обрабатывает инструкции СУБД: from -> where -> group by -> having -> select -> order by

* Именно поэтому если переименовывать переменные в select, новые имена не видны в остальных инструкциях (кроме order by)

Базы Данных

Семинар 3

DDL (Продолжение)

Убедительно прошу начать семинар с повторения материала предыдущего занятия. Возьмите предметную область и напишите 3-4 сложных линейных запроса (со всеми инструкциями). Без подзапросов и прочих наворотов. Дайте студентам время усвоить материал, пусть сами продитктуют запрос. Убедителсь, что простые запросы - не проблема. На лабах буду с пристрастием спрашивать чем отличается where от having.

После того, как понятно, как писать простые запросы, необходимо объяснить что такое подзапрос.

Подзапросы, внутренние или вложенные запросы - есть не что иное, как запрос внутри запроса. Обычно, подзапрос используется в конструкции WHERE или FROM. И, в большинстве случаев, подзапрос используется, когда вы можете получить значение с помощью запроса, но не знаете конкретного результата.

Подзапросы являются альтернативным путем получения данных из множества таблиц.

Вопрос в зал: Почему SQL допускает подзапросы? (Так как в основе лежит реляционная модель и реляционная алгебра, то результатом выполнения любой операции реляционной алгебры над отношениями также является отношением. Эта особенность называется свойством реляционной замкнутости.)

Подзапрос в WHERE обычно используют такие операторы, как IN, NOT IN

```

-- Когда список имен заранее определен
SELECT first_name, last_name, subject
FROM student_details
WHERE games NOT IN ('Cricket', 'Football');

-- Если список имен неизвестен
SELECT id, first_name
FROM student_details
WHERE first_name IN (SELECT first_name
FROM
student_details
WHERE subject
= 'Science');

```

Подзапрос в FROM. В таких запросах обязательно наличие псевдонима (Alias) после закрытой скобки подзапроса.

```

SELECT COUNT(made_only_recharge) AS
made_only_recharge
FROM
(
    SELECT DISTINCT (identifiant) AS
made_only_recharge
FROM cdr_data
WHERE CALLEDNUMBER = '0130'
EXCEPT
    SELECT DISTINCT (identifiant) AS
made_only_recharge
FROM cdr_data
WHERE CALLEDNUMBER != '0130'
) AS derivedTable

```

Наряду с операторами сравнения такими, как =, <, >, >=, <= и др., можно использовать подзапросы с конструкциями: SELECT, INSERT, UPDATE, DELETE.

JOIN

JOIN — оператор языка SQL, который является реализацией операции соединения реляционной алгебры. Входит в предложение FROM операторов SELECT, UPDATE и DELETE.

Операция соединения, как и другие бинарные операции, предназначена для обеспечения выборки данных из двух таблиц и включения этих данных в один результирующий набор. Отличительными особенностями операции соединения являются следующие:

- в схему таблицы-результата входят столбцы обеих исходных таблиц (таблиц-операндов), то есть схема результата является «сцеплением» схем операндов;
- каждая строка таблицы-результата является «сцеплением» строки из одной таблицы-операнда со строкой второй таблицы-операнда.

Определение того, какие именно исходные строки войдут в результат и в каких сочетаниях, зависит от типа операции соединения и от явно заданного *условия соединения*. Условие соединения, то есть условие сопоставления строк исходных таблиц друг с другом, представляет собой логическое выражение (предикат).

При необходимости соединения не двух, а нескольких таблиц, операция соединения применяется несколько раз (последовательно).

По механизму работы соединения различают три типа JOIN-ов. Далее рассматриваются внутренние соединения, на основе данных алгоритмов несложно написать внешние. Код выписывать не обязательно, достаточно прогнать пару примеров, чтобы были понятны основные различия:

Nested Loops Join

Самый базовый алгоритм объединения двух списков сейчас проходят в школах. Суть очень простая — для каждого элемента первого списка пройдемся по всем элементам второго списка; если ключи элементов вдруг оказались равны — запишем совпадение в результирующую таблицу. Для реализации этого алгоритма достаточно двух вложенных циклов, именно поэтому он так и называется.

```
public static <K, V1, V2> List<Triple<K, V1, V2>>
nestedLoopsJoin(List<Pair<K, V1>> left, List<Pair<K,
V2>> right) {
    List<Triple<K, V1, V2>> result = new
ArrayList<>();
    for (Pair<K, V1> leftPair: left) {
        for (Pair<K, V2> rightPair: right) {
            if (Objects.equals(leftPair.k,
rightPair.k)) {
                result.add(new Triple<>(leftPair.k,
leftPair.v, rightPair.v));
            }
        }
    }
    return result;
}
```

Основной плюс этого метода — полное безразличие к входным данным. Алгоритм работает для любых двух таблиц, не требует никаких индексов и переключиваний данных в памяти, а также прост в реализации. На практике это означает, что достаточно просто бежать по диску двумя курсорами и периодически выплёвывать в сокет совпадения.

Однако ж, фатальный минус алгоритма — высокая временная сложность $O(N \cdot M)$ (квадратичная асимптотика). Например, для джойна пары небольших таблиц в 100к и 500к записей потребуются сделать аж $100.000 \cdot 500.000 = 50.000.000.000$ (50 млрд) операций сравнения. Запросы с таким джойном будут выполняться невежливо долго, часто именно они — причина беспощадных тормозов кривеньких самописных

CMS'ок.

Современные РСУБД используют nested loops join в самых безнадежных случаях, когда не удастся применить никакую оптимизацию или затраты на оптимизация несравнимы с размерами таблиц.

Hash Join

Если размер одной из таблиц позволяет засунуть ее целиком в память, значит, на ее основе можно сделать хеш-таблицу и быстро искать в ней нужные ключи. Проговорим чуть подробнее.

Проверим размер обоих списков. Возьмем меньший из списков, прочтем его полностью и загрузим в память, построив HashMap. Теперь вернемся к большему списку и пойдем по нему курсором с начала. Для каждого ключа проверим, нет ли такого же в хеш-таблице. Если есть – запишем совпадение в результирующую таблицу.

Временная сложность этого алгоритма падает до линейной $O(N+M)$, но требуется дополнительная память.

```
public static <K, V1, V2> List<Triple<K, V1, V2>>
hashJoin(List<Pair<K, V1>> left, List<Pair<K, V2>>
right) {
    Map<K, V2> hash = new HashMap<>(right.size());
    for (Pair<K, V2> rightPair: right) {
        hash.put(rightPair.k, rightPair.v);
    }

    List<Triple<K, V1, V2>> result = new
ArrayList<>();
    for (Pair<K, V1> leftPair: left) {
        if (hash.containsKey(leftPair.k)) {
            result.add(new Triple<>(leftPair.k,
leftPair.v, hash.get(leftPair.k)));
        }
    }

    return result;
}
```

Что важно, во времена динозавров считалось, что в память нужно загрузить правую таблицу, а итерироваться по левой. У нормальных РСУБД сейчас есть статистика cardinality, и они сами определяют порядок джойна, но если по какой-то причине статистика недоступна,

то в память грузится именно правая таблица. Это важно помнить при работе с молодыми корявыми инструментами типа Cloudera Impala.

Merge Join

А теперь представим, что данные в обоих списках заранее отсортированы, например, по возрастанию. Так бывает, если у нас были индексы по этим таблицам, или же если мы отсортировали данные на предыдущих стадиях запроса. Как вы наверняка помните, два отсортированных списка можно склеить в один отсортированный за линейное время — именно на этом основан алгоритм сортировки слиянием. Здесь нам предстоит похожая задача, но вместо того, чтобы склеивать списки, мы будем искать в них общие элементы.

Итак, ставим по курсору в начало обоих списков. Если ключи под курсорами равны, записываем совпадение в результирующую таблицу. Если же нет, смотрим, под каким из курсоров ключ меньше. Двигаем курсор над меньшим ключом на один вперед, тем самым “догоняя” другой курсор.

```

public static <K extends Comparable<K>, V1, V2>
List<Triple<K, V1, V2>> mergeJoin(
    List<Pair<K, V1>> left,
    List<Pair<K, V2>> right
) {
    List<Triple<K, V1, V2>> result = new
ArrayList<>();
    Iterator<Pair<K, V1>> leftIter =
left.listIterator();
    Iterator<Pair<K, V2>> rightIter =
right.listIterator();
    Pair<K, V1> leftPair = leftIter.next();
    Pair<K, V2> rightPair = rightIter.next();

    while (true) {
        int compare =
leftPair.k.compareTo(rightPair.k);
        if (compare < 0) {
            if (leftIter.hasNext()) {
                leftPair = leftIter.next();
            } else {
                break;
            }
        } else if (compare > 0) {
            if (rightIter.hasNext()) {
                rightPair = rightIter.next();
            } else {
                break;
            }
        } else {
            result.add(new Triple<>(leftPair.k,
leftPair.v, rightPair.v));
            if (leftIter.hasNext() &&
rightIter.hasNext()) {
                leftPair = leftIter.next();
                rightPair = rightIter.next();
            } else {
                break;
            }
        }
    }
}

```

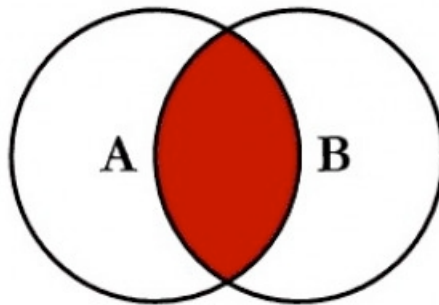
Если данные отсортированы, то временная сложность алгоритма линейная $O(M+N)$ и не требуется никакой дополнительной памяти. Если же данные не отсортированы, то нужно сначала их отсортировать. Из-за этого временная сложность возрастает до $O(M \log M + N \log N)$, плюс появляются дополнительные требования к памяти.

Виды JOIN-ов (по типу соединений)

Вместе составьте две таблицы, которые потом будете соединять. Для каждого вида нарисовать диаграмму Эйлера + написать выходную таблицу. Да много, но это правда НАДО.

Внутренне соединение:

Внутреннее объединение **INNER JOIN** (синоним JOIN, ключевое слово INNER можно опустить). Выбираются только совпадающие данные из объединяемых таблиц.



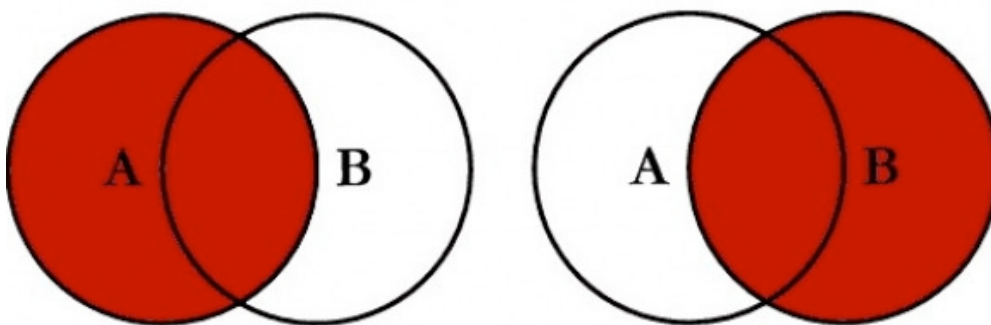
```
SELECT u.id, u.name, d.name AS d_name
FROM users u
INNER JOIN departments d ON u.d_id = d.id
```

Внешнее соединение:

Чтобы получить данные, которые подходят по условию частично, необходимо использовать внешнее объединение **OUTER JOIN - LEFT OUTER JOIN** и **RIGHT OUTER JOIN**.

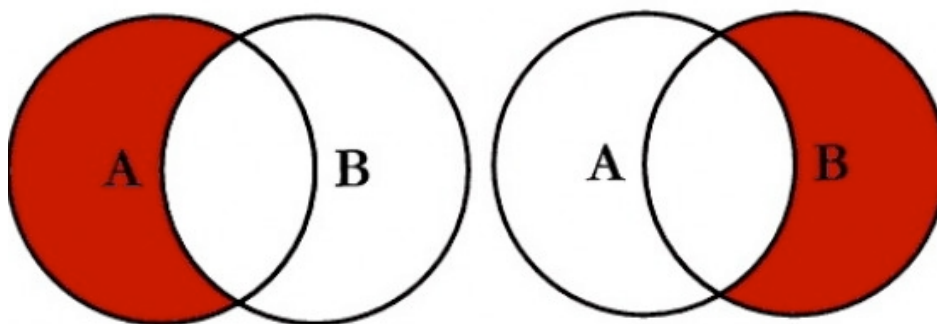
Работают они одинаково, разница заключается в том что **LEFT** - указывает что "внешней" таблицей будет находящаяся слева (в нашем примере это таблица **users**), а **RIGHT** — правая.

Ключевое слово **OUTER** можно опустить. Запись **LEFT JOIN** идентична **LEFT OUTER JOIN**.



```
SELECT u.id, u.name, d.name AS d_name  
FROM users u  
LEFT/RIGHT [OUTER] JOIN departments d ON u.d_id = d.id
```

Вопрос в зал: Как получить записи только из 1ой таблицы?



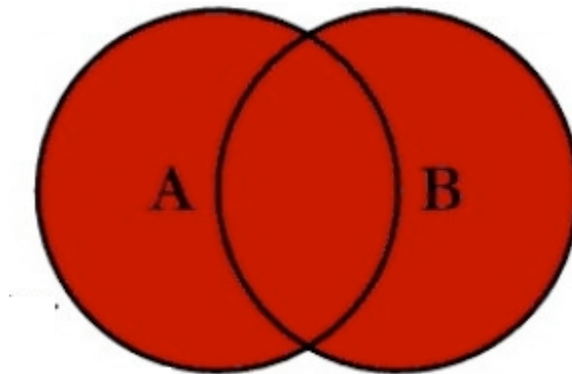
```
SELECT d.id, d.name  
FROM users u  
LEFT/RIGHT [OUTER] JOIN departments d ON u.d_id = d.id  
WHERE u.id IS null
```

Оператор **SQL FULL JOIN** осуществляет формирование таблицы из записей двух или нескольких таблиц. В операторе SQL FULL JOIN не важен порядок следования таблиц, он никак не влияет на окончательный результат, так как оператор является симметричным.

Оператор SQL FULL JOIN можно воспринимать как сочетание операторов SQL INNER JOIN + SQL LEFT JOIN + SQL RIGHT JOIN. Алгоритм его работы следующий:

1. Сначала формируется таблица на основе внутреннего соединения (оператор SQL INNER JOIN).
2. Затем, в таблицу добавляются значения не вошедшие в результат формирования из правой таблицы (оператор SQL LEFT JOIN). Для них, соответствующие записи из правой таблицы заполняются значениями NULL.
3. Наконец, в таблицу добавляются значения не вошедшие в результат формирования из левой таблицы (оператор SQL RIGHT JOIN). Для них, соответствующие записи из левой таблицы заполняются значениями NULL.

```
SELECT u.id, u.name, d.name AS d_name  
FROM users u  
FULL [OUTER] JOIN departments d ON u.d_id = d.id
```



Элемент **SELF JOIN** используется для объединения таблицы с ней самой таким образом, будто это две разные таблицы, временно переименовывая одну из них.

Базы Данных

Семинар 4

Операции над множествами

Добавим немного интерактива и узнаем, какие операции над множествами они знают.

Рассмотрим первую операцию - объединение (**UNION**):

Синтаксис оператора UNION:

```
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions]
UNION [ALL]
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions];
```

Предложение **UNION** объединяет вывод двух или более SQL запросов в единый набор строк и столбцов. Например чтобы получить всех продавцов и заказчиков размещенных в Лондоне и вывести их как единое целое вы могли бы ввести:

```
SELECT snum, sname
FROM Salespeople
WHERE city = 'London'

UNION

SELECT cnum, cname
FROM Customers
WHERE city = 'London';
```

Операция объединения может быть выполнена только при выполнении следующих условий:

- количество выходных столбцов каждого из запросов должно быть одинаковым;
- выходные столбцы каждого из запросов должны быть совместимы между собой (в порядке их следования) по типам данных;
- в результирующем наборе используются имена столбцов, заданные в первом запросе;
- предложение **ORDER BY** применяется к результату соединения, поэтому оно может быть указано только в конце всего составного запроса.

Пример с множествами:

Чему равно объединение множеств $\{1, 2\} \cup \{3\}$? - $\{1, 2, 3\}$

Чему равно объединение множеств $\{1, 2, 3\} \cup \{3\}$? - $\{1, 2, 3\}$

UNION работает по той же схеме: Если вы объединяете два запроса и в каждом из которых есть одинаковые данные, другими словами полностью идентичные, оператор `union` объединит их в одну строку для того чтобы не было дублей.

UNION ALL – это оператор SQL для объединения результирующего набора данных нескольких запросов, а вот данный оператор, выведет уже абсолютно все строки, даже дубли.

Далее рассмотрим оператор пересечения:

Оператор **INTERSECT** используется для возврата результатов 2-х или более запросов **SELECT**. Тем не менее, он возвращает строки, выбранные для всех запросов или наборов данных. Если запись существует в одном запросе, а в другом нет, то она будет исключена из результирующего набора **INTERSECT**.



Запрос **INTERSECT** будет возвращать записи в серой затененной области. Эти записи, которые существуют в обоих **SELECT1** и **SELECT2**.

Каждый оператор **SELECT** в **INTERSECT** должен иметь одинаковое количество полей в наборах результатов с одинаковыми типами данных.

Синтаксис оператора **INTERSECT**:

```
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions]
INTERSECT
SELECT expression1, expression2, ... expression_n
FROM tables
[WHERE conditions];
```

expression1, expression2, ... expression_n - Столбцы или расчеты, которые вы хотите получить.

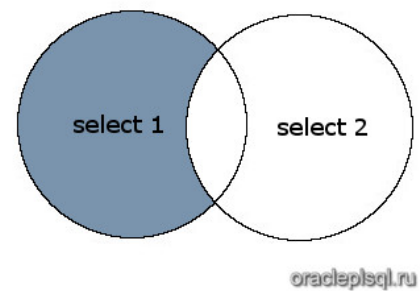
Tables - Таблицы из которых вы хотите получить записи.

WHERE conditions - Необязательный. Условия, которые должны быть выполнены для выбранных записей.

В обоих **SELECT** запросах должно быть одинаковое количество **expression** и иметь схожие типы данных.

Оператор вычитания - MINUS (Oracle)/ EXCEPT(MS SQL)

Oracle оператор **MINUS** используется для возврата всех строк первого запроса **SELECT**, не возвращаемых вторым **SELECT**. Каждый запрос **SELECT** будет определять набор данных. Оператор **MINUS** возвращает все записи из первого набора данных, а затем из результатов удалит все записи из второго набора данных.



Запрос **MINUS** вернет записи в серой затененной области. Эти записи, которые существуют в **SELECT1** и **SELECT2**. Каждый **SELECT** в запросе **MINUS** должны иметь одинаковое количество полей в результирующих наборах с одинаковыми типами данных.

Синтаксис оператора **MINUS**:

```
SELECT expression1, expression2, ... expression_n
  FROM tables
[WHERE conditions]
MINUS
SELECT expression1, expression2, ... expression_n
  FROM tables
[WHERE conditions];
```

expression1, expression2, ... expression_n - Столбцы или расчеты, которые вы хотите получить.

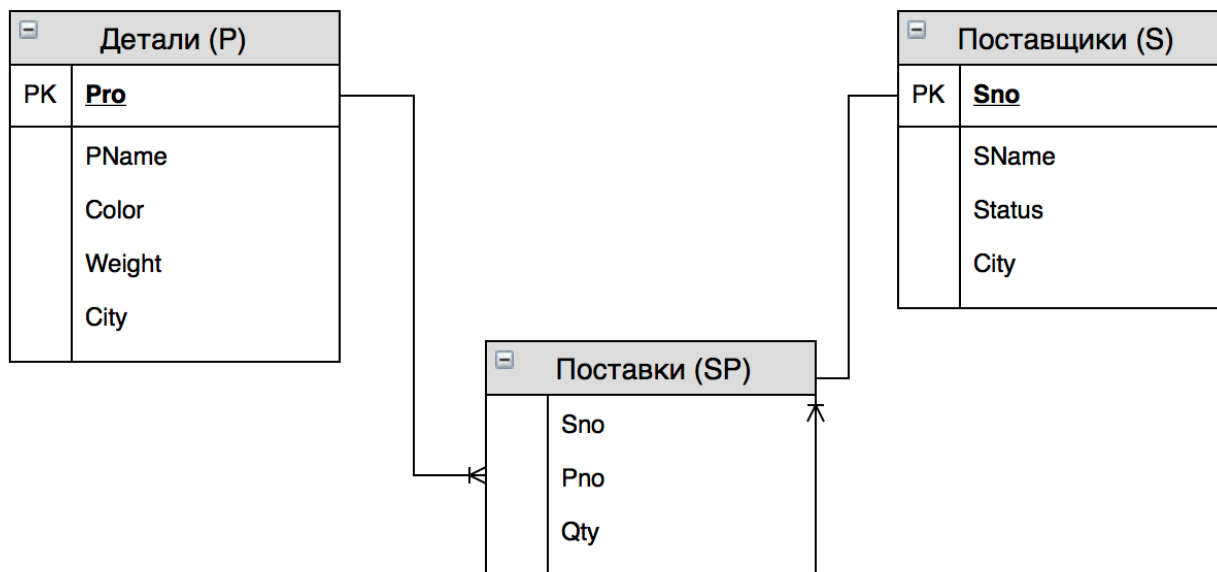
Tables - Таблицы из которых вы хотите получить записи.

WHERE conditions - Необязательный. Условия, которые должны быть выполнены для выбранных записей.

В обоих **SELECT** запросах должно быть одинаковое количество **expression** и иметь схожие типы данных.

Практическая часть

ER-модель, рассматриваемая на лекции:



1. Получить имена всех поставщиков, поставляющих деталь под номером 2

```
SELECT S.SName
FROM S inner join SP on S.Sno = SP.Sno
WHERE SP.Pno = 2
```

2. Получить имена поставщиков, поставляющих хотя бы одну деталь красного цвета

```
SELECT S.SName
FROM S inner join SP on S.Sno = SP.Sno
      inner join P on SP.Pno = P.Pno
WHERE SP.Color = 'Красный'
```

3. Получить пары имен всех поставщиков <SName1, SName2> из одного города

```
SELECT S.SName
FROM S AS FirstS inner join S AS SecondS
      on (FirstS.City = SecondS.City) AND (FirstS.Sno < SecondS.Sno)
```

Не говорить сразу ответ, пусть подумают, напишут, что будет если не будет ограничения на Sno. Спросить какой вид джойна СУБД выберет для соединения этих таблиц в данном запросе (Nested Loops Join), почему?

При составлении Хэш-таблицы ключ соединения хэшируется с помощью специальной функции, которая не гарантирует, что при $a < b$, $f(a) < f(b)$, поэтому данный вид соединения не подходит.

4. Найти имена всех поставщиков, не поставляющих деталь под номером 2:
Основная идея: Найти тех кто поставляет 2ую деталь (1-ый запрос) и вычесть из всех поставщиков

```
SELECT S.SName from S
EXCEPT
SELECT S.SName
FROM S inner join SP on (S.Sno = SP.Sno) AND (SP.Pno = 2)
```

Обратить внимание, что в условие соединения добавилось условие из WHERE.
Так тоже можно.

5. Найти всех поставщиков, поставляющих все детали:

Распишем поэтапно (После каждого запроса писать результат):

5.1. Сначала надо найти количество деталей:

```
SELECT count(distinct Pno) as cnt
FROM P
```

В результате - 1 ячейка с названием cnt и количеством различных деталей

5.2. Для каждого поставщика узнать, сколько различных деталей он поставляет

```
SELECT Sno, count(distinct Pno) as cntS
FROM SP
GROUP BY Sno
```

5.3 Собираем все вместе:

```
SELECT S.Sname
FROM (SELECT Sno, count(distinct Pno) as cntS
      FROM SP
      GROUP BY Sno) tmp
inner join S on tmp.Sno = S.Sno
WHERE cntS IN (SELECT count(distinct Pno)
              FROM P )
```

Если будет мало:

6. Найти поставщика, поставляющего больше всего различных деталей
(некрасивый способ, красивый будет после следующего семинара)

6.1. Для каждого поставщика узнать, сколько различных деталей он поставляет

```
SELECT Sno, count(distinct Pno) as cntS
FROM SP
GROUP BY Sno
```

6.2. Найти среди данной выборки MAX

```
SELECT MAX(cntS)
FROM( SELECT Sno, count(distinct Pno) as cntS
      FROM SP
      GROUP BY Sno)tmp
```

6.3. Найти тех, у кого количество деталей соответствует максимуму

```
SELECT S.SName
FROM (SELECT Sno, count(distinct Pno) as cntS
      FROM SP
      GROUP BY Sno) groupS
  inner join S on groupS.Sno = S.Sno
WHERE groupS.cntS = (SELECT MAX(cntS)
                    FROM (SELECT Sno, count(distinct Pno) as cntS
                          FROM SP
                          GROUP BY Sno) tmp)
```

Некрасиво, потому что два раза одна и та же группировка, от этого можно избавиться с помощью Обобщенного Табличного Выражения (ОТВ), которое будет рассмотрено на следующем семинаре.

Базы Данных

Семинар 5

Оконные функции

Семинар начинаем с напоминания, что такое группировка и что можно получить в результате запроса с GROUP BY. (Неплохо также узнать, усвоили ли различия WHERE и HAVING).

Как напоминание. Предложение **GROUP BY** используется для определения групп выходных строк, к которым могут применяться агрегатные функции (**COUNT**, **MIN**, **MAX**, **AVG** и **SUM**). Если это предложение отсутствует, и используются агрегатные функции, то все столбцы с именами, упомянутыми в **SELECT**, должны быть включены в агрегатные функции, и эти функции будут применяться ко всему набору строк, которые удовлетворяют предикату запроса. В противном случае все столбцы списка **SELECT**, не вошедшие в агрегатные функции, должны быть указаны в предложении **GROUP BY**. В результате чего все выходные строки запроса разбиваются на группы, характеризуемые одинаковыми комбинациями значений в этих столбцах. После чего к каждой группе будут применены агрегатные функции. Следует иметь в виду, что для **GROUP BY** все значения **NULL** трактуются как равные, то есть при группировке по полю, содержащему **NULL**-значения, все такие строки попадут в одну группу.

Реляционная модель исходит из того факта, что строки в таблице не имеют порядка, являющегося прямым следствием теоретико-множественного подхода. Поэтому наивными выглядят вопросы новичков, спрашивающих: "А как мне получить последнюю добавленную в таблицу строку?" Ответом на вопрос будет "никак", если в таблице не предусмотрен столбец, содержащий дату вставки строки, или не используется последовательная нумерация строк, реализуемая во многих СУБД с помощью столбца с автоинкрементируемым значением. Тогда можно выбрать строку с максимальным значением даты или счетчика.

Теоретически каждая строка запроса обрабатывается независимо от других строк. Однако на практике часто требуется при обработке строки соотносить ее с предыдущими или последующими строками (например, для получения нарастающих итогов), выделять группы строк, обрабатываемые независимо от других и т.д. В ответ на потребности практики в ряде СУБД в языке SQL появились

соответствующие конструкции, в частности, функции ранжирования и оконные (аналитические) функции.

Оконные функции не изменяют выборку, а только добавляют некоторую дополнительную информацию о ней. Т.е. для простоты понимания можно считать, что СУБД сначала выполняет весь запрос (кроме сортировки и limit), а потом только просчитывает оконные выражения.

Синтаксис примерно такой:

```
функция OVER окно
```

Окно — это некоторое выражение, описывающее набор строк, которые будет обрабатывать функция и порядок этой обработки. Причем окно может быть просто задано пустыми скобками (), т.е. окном являются все строки результата запроса.

Например, в этом селекте к обычным полям id, header и score просто добавится нумерация строк.

```
SELECT id, section, header, score,  
       row_number() OVER () AS num  
FROM news;
```

id	section	header	score	num
1	2	Заголовок	23	1
2	1	Заголовок	6	2
3	4	Заголовок	79	3
4	3	Заголовок	36	4
5	2	Заголовок	34	5
6	2	Заголовок	95	6
7	4	Заголовок	26	7
8	3	Заголовок	36	8

В оконное выражение можно добавить ORDER BY, тогда можно изменить порядок обработки.

```
SELECT id, section, header, score,
       row_number() OVER (ORDER BY score DESC) AS
rating
FROM news
ORDER BY id;
```

id	section	header	score	rating
1	2	Заголовок	23	7
2	1	Заголовок	6	8
3	4	Заголовок	79	2
4	3	Заголовок	36	4
5	2	Заголовок	34	5
6	2	Заголовок	95	1
7	4	Заголовок	26	6
8	3	Заголовок	36	3

Обратите внимание, что в конце всего запоса ORDER BY id, при этом рейтинг посчитан все равно верно. Т.е. посгрес просто отсортировал результат вместе с результатом работы оконной функции, один order ничуть не мешает другому.

В оконное выражение можно добавить слово PARTITION BY [expression], например row_number() OVER (PARTITION BY section), тогда подсчет будет идти в каждой группе отдельно:

```

SELECT id, section, header, score,
       row_number() OVER (PARTITION BY section
ORDER BY score DESC) AS
rating_in_section
FROM news
ORDER BY section, rating_in_section;

```

id	section	header	score	rating_in_section
2	1	Заголовок	6	1
6	2	Заголовок	95	1
5	2	Заголовок	34	2
1	2	Заголовок	23	3
4	3	Заголовок	36	1
8	3	Заголовок	36	2
3	4	Заголовок	79	1
7	4	Заголовок	26	2

Если не указывать партицию, то партицией является весь запрос.

В качестве функции можно использовать, так сказать, истинные оконные функции из мануала — это `row_number()`, `rank()`, `lead()` и т.д., а можно использовать функции-агрегаты, такие как: `sum()`, `count()` и т.д. Так вот, это важно, агрегатные функции работают слегка по-другому: если не задан `ORDER BY` в окне, идет подсчет по всей партиции один раз, и результат пишется во все строки (одинаков для всех строк партиции). Если же `ORDER BY` задан, то подсчет в каждой строке идет от начала партиции до этой строки.

```
SELECT transaction_id, change
FROM balance_change
ORDER BY transaction_id;
```

transaction_id	change
1	1.00
2	-2.00
3	10.00
4	-4.00
5	5.50

и мы хотим узнать заодно, как менялся остаток на балансе при этом:

```
SELECT transaction_id, change,
       sum(change) OVER (ORDER BY
transaction_id) as balance
FROM balance_change
ORDER BY transaction_id;
```

transaction_id	change	balance
1	1.00	1.00
2	-2.00	-1.00
3	10.00	9.00
4	-4.00	5.00
5	5.50	10.50

Т.е. для каждой строки идет подсчет в отдельном фрейме. В данном случае фрейм – это набор строк от начала до текущей строки (если было бы PARTITION BY, то от начала партиии).

Если же мы для агрегатной функции sum не будем использовать ORDER BY в окне, тогда мы просто посчитаем общую сумму и покажем её во всех строках. Т.е. фреймом для каждой из строк будет весь набор строк от начала до конца партиии.

```
SELECT transaction_id, change,
       sum(change) OVER () as result_balance
FROM balance_change
ORDER BY transaction_id;
```

transaction_id	change	result_balance
1	1.00	10.50
2	-2.00	10.50
3	10.00	10.50
4	-4.00	10.50
5	5.50	10.50

Вот такая особенность агрегатных функций, если их использовать как оконные. На мой взгляд, это довольно-таки странный, интуитивно неочевидный момент SQL-стандарта.

Оконные функции можно использовать сразу по несколько штук, они друг другу ничуть не мешают, чтобы вы там в них не написали.

```

SELECT transaction_id, change,
       sum(change) OVER (ORDER BY transaction_id) as
balance,
       sum(change) OVER () as result_balance,
       round( 100.0 * sum(change) OVER (ORDER BY
transaction_id) / sum(change)

OVER (), 2) AS percent_of_result,
       count(*) OVER () as transactions_count
FROM balance_change
ORDER BY transaction_id;

```

transaction_id	change	balance	result_balance	percent_of_result	transactions_count
1	1.00	1.00	10.50	9.52	5
2	-2.00	-1.00	10.50	-9.52	5
3	10.00	9.00	10.50	85.71	5
4	-4.00	5.00	10.50	47.62	5
5	5.50	10.50	10.50	100.00	5

Если у вас много одинаковых выражений после OVER, то можно дать им имя и вынести отдельно с ключевым словом WINDOW, чтобы избежать дублирования кода. Вот пример из мануала:

```

SELECT
    sum(salary) OVER w,
    avg(salary) OVER w
FROM empsalary
WINDOW w AS (PARTITION BY depname ORDER BY salary
DESC);

```

Здесь w после слова OVER идет без уже скобок.

Результат работы оконной функции невозможно отфильтровать в запросе с помощью WHERE, потому что оконные функции выполняются после всей фильтрации и группировки, т.е. с тем, что получилось. Поэтому чтобы выбрать, например, топ 5 новостей в каждой группе, надо использовать подзапрос:

```
SELECT *
FROM (
    SELECT id, section, header, score,
           row_number() OVER (PARTITION BY
section ORDER BY score DESC) AS
rating_in_section
    FROM news
    ORDER BY section, rating_in_section
) counted_news
WHERE rating_in_section <= 5;
```

Еще пример для закрепления. Помимо row_number() есть несколько других функций. Например lag, которая ищет строку перед последней строкой фрейма. К примеру мы можем найти насколько очков новость отстает от предыдущей в рейтинге:

```

SELECT id, section, header, score,
       row_number() OVER w AS rating,
       lag(score) OVER w - score AS score_lag
FROM news
WINDOW w AS (ORDER BY score DESC)
ORDER BY score desc;

```

id	section	header	score	rating	score_lag
6	2	Заголовок	95	1	
3	4	Заголовок	79	2	16
8	3	Заголовок	36	3	43
4	3	Заголовок	36	4	0
5	2	Заголовок	34	5	2
7	4	Заголовок	26	6	8
1	2	Заголовок	23	7	3
2	1	Заголовок	6	8	17

Прошу в комментариях накидать примеров, где особенно удобно применять оконные функции. А также, какие с ними могут возникнуть проблемы, если таковые имеются.

Обобщенное табличное выражение

Common Table Expression (CTE) или обобщенное табличное выражение (ОТВ) - это временные результирующие наборы (т.е. результаты выполнения SQL запроса), которые не сохраняются в базе данных в виде объектов, но к ним можно обращаться.

Главной особенностью обобщенных табличных выражений является то, что с помощью них можно писать рекурсивные запросы. Случаи в которых может пригодиться использование ОТВ:

- Основной целью ОТВ является написание рекурсивных запросов, можно сказать для этого они, и были созданы;
- ОТВ можно использовать также и для замены представлений (VIEW), например, в тех случаях, когда нет необходимости сохранять в базе SQL запрос представления, т.е. его определение;

- Обобщенные табличные выражения повышают читаемость кода путем разделения запроса на логические блоки, и тем самым упрощают работу со сложными запросами;
- Также ОТВ предназначены и для многократных ссылок на результирующий набор из одной и той же SQL инструкции.

Обобщенное табличное выражение определяется с помощью конструкции WITH, и определить его можно как в обычных запросах, так и в функциях, хранимых процедурах, триггерах и представлениях.

Синтаксис:

```
WITH [ ( column_name [ ,...n ] ) ]
AS
( CTE_query_definition )
```

Где,

- common_table_expression_name - это псевдоним или можно сказать идентификатор обобщенного табличного выражения. Обращаться к ОТВ мы будем, как раз используя этот псевдоним;
- column_name - имя столбца, который будет определен в обобщенном табличном выражении. Использование повторяющихся имен нельзя, а также их количество должно совпадать с количеством столбцов возвращаемых запросом CTE_query_definition. Указывать имена столбцов необязательно, но только в том случае, если всем столбцам в запросе CTE_query_definition присвоены уникальные псевдонимы;
- CTE_query_definition - запрос SELECT, к результирующему набору которого, мы и будем обращаться через обобщенное табличное выражение, т.е. common_table_expression_name.

После обобщенного табличного выражения, т.е. сразу за ним должен идти одиночный запрос SELECT, INSERT, UPDATE, MERGE или DELETE.

Какие бывают обобщенные табличные выражения?

Они бывают простые и рекурсивные.

Простые ОТВ не включают ссылки на самого себя.

```

WITH TestCTE (UserID, Post, ManagerID) AS
(
    SELECT UserID, Post, ManagerID
    FROM TestTable
)
SELECT * FROM TestCTE

```

В данном случае мы могли и не перечислять имена столбцов, так как они у нас уникальны. Можно было просто написать вот так:

```

WITH TestCTE AS
(
    SELECT UserID, Post, ManagerID
    FROM TestTable
)
SELECT * FROM TestCTE

```

Рекурсивные ОТВ используются для возвращения иерархических данных, например, классика жанра это отображение сотрудников в структуре организации (чуть ниже мы это рассмотрим).

Теперь допустим, что нам необходимо вывести иерархический список сотрудников, т.е. мы хотим видеть, на каком уровне работает тот или иной сотрудник. Для этого пишем рекурсивный запрос:

```

WITH TestCTE(UserID, Post, ManagerID, LevelUser)
AS
(
    -- Находим якорь рекурсии
    SELECT UserID, Post, ManagerID, 0 AS LevelUser
    FROM TestTable WHERE ManagerID IS NULL
    UNION ALL
    --Делаем объединение с TestCTE (хотя мы его еще не дописали)
    SELECT t1.UserID, t1.Post, t1.ManagerID, t2.LevelUser + 1
    FROM TestTable t1
    JOIN TestCTE t2 ON t1.ManagerID=t2.UserID
)
SELECT * FROM TestCTE ORDER BY LevelUser

```

В итоге, если мы захотим, мы можем легко получить список сотрудников определенного уровня, например, нам нужны только начальники отделов, для этого мы просто в указанный выше запрос добавим условие WHERE LevelUser = 1

При написании рекурсивного ОТВ нужно быть внимательным, так как неправильное его составление может привести к бесконечному циклу. Поэтому для этих целей есть опция MAXRECURSION, которая может ограничивать количество уровней рекурсии. Давайте представим, что мы не уверены, что написали рекурсивное обобщенное выражение правильно и для отладки применим после запроса инструкцию:

```
OPTION (MAXRECURSION 5)
```

Если запрос отработал корректно, значит проблем с рекурсией не возникло, запрос не был перманентной ошибкой.

Базы Данных

Семинар 6

Описание семинара: Функции и процедуры

Функции T-SQL

SQL Server содержит набор встроенных функций и предоставляет возможность создавать пользовательские функции (User Defined Function – UDF). Различают детерминированные и недетерминированные функции.

- Функция является **детерминированной**, если при одном и том же заданном входном значении она всегда возвращает один и тот же результат. Например, встроенная функция DATEADD является детерминированной; она возвращает новое значение даты, добавляя интервал к указанной части заданной даты.
- Функция является **недетерминированной**, если она может возвращать различные значения при одном и том же заданном входном значении. Например, встроенная функция GETDATE является недетерминированной; при каждом вызове она возвращает различные значения даты и времени компьютера, на котором запущен экземпляр SQL Server.

Все функции конфигурации, курсора, метаданных, безопасности и системные статистические – недетерминированные. Список этих функций приводится в справочнике. Функции, вызывающие недетерминированные функции и расширенные хранимые процедуры, также считаются недетерминированными.

Пользовательские функции в зависимости от типа данных возвращаемых ими значений могут быть скалярными и табличными. Табличные пользовательские функции бывают двух типов: подставляемые и многооператорные.

Скалярные пользовательские функции

Скалярные пользовательские функции обычно используются в списке столбцов инструкции SELECT и в предложении WHERE. Табличные пользовательские функции обычно используются в предложении FROM, и их можно соединять с другими таблицами и представлениями.

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )  
RETURNS скалярный-тип-данных  
[ WITH список-опций-функций ]  
[ AS ]  
BEGIN  
    тело-функции  
    RETURN скалярное-выражение  
END [ ; ]
```

Для создания скалярной функции используется инструкция CREATE FUNCTION, имеющая следующий синтаксис:

```
@имя-параметра [ AS ] [ имя-схемы. ] тип-данных [ = значение-по-умолчанию ] [ READONLY ]
```

- Тип данных параметра – любой тип данных, включая определяемые пользователем типы данных CLR и определяемые пользователем табличные типы, за исключением скалярного типа timestamp и нескаларных типов cursor и table.
- Значение по умолчанию
- Необязательное ключевое слово **READONLY** указывает, что параметр не может быть обновлен или изменен при определении функции. Если тип параметра является определяемым пользователем табличным типом, то должно быть указано ключевое слово **READONLY**.

где

- Список объявлений параметров является необязательным, но наличие скобок обязательно.
 - Объявление параметра в списке объявлений параметров имеет вид:
 - Возвращаемое значение может быть любого скалярного типа данных, включая определяемые пользователем типы данных CLR, за исключением типа данных timestamp.
 - В качестве опций функции используются:
 - ENCRYPTION — SQL Server шифрует определение функции.
 - SCHEMABINDING - привязывает функцию к схеме базовой таблицы или таблиц. Если аргумент SCHEMABINDING указан, нельзя изменить базовую таблицу или таблицы таким способом, который может повлиять на определение функции.
- RETURNS NULL ON NULL INPUT или| CALLED ON NULL INPUT - ...

- Предложение EXECUTE AS - указывает контекст безопасности, в котором может быть выполнена функция. Например, EXECUTE AS CALLER указывает, что функция будет выполнена в контексте пользователя, который ее вызывает. Также могут быть указаны параметры SELF, OWNER и имя-пользователя.
- Операторы BEGIN и END, которыми ограничивается тело функции, являются обязательными.
- Тело функции представляет собой ряд инструкций T-SQL, которые в совокупности вычисляют скалярное выражение.

Синтаксис вызова скалярных функций схож с синтаксисом, используемым для встроенных функций T-SQL:

имя-схемы.имя-функции ([список-параметров])

где

- Имя схемы (имя владельца) для скалярной функции является обязательным.
- Нельзя использовать синтаксис с именованными параметрами (@имя-параметра = значение).
- Нельзя не указывать (опускать) параметры, но можно применять ключевое слово DEFAULT для указания значения по умолчанию.

Для скалярной функции можно использовать инструкцию EXECUTE:

EXECUTE @возвращаемое-значение = имя-функции ([список-параметров])

где

- Не нужно указывать имя схемы (имя владельца).
- Можно использовать именованные параметры (@имя-параметра = значение).
- Если используются именованные параметры, они не обязательно должны следовать в том порядке, в котором указаны в объявлении функции, но необходимо указать все параметры; нельзя опускать ссылку на параметр для использования значения по умолчанию.

Примеры создания и вызова скалярных функций

```
CREATE FUNCTION dbo.AveragePrice() RETURNS smallmoney
WITH SCHEMABINDING AS
BEGIN
```

```
    RETURN (SELECT AVG(Price) FROM dbo.R)
END;
```

```
CREATE FUNCTION dbo.PriceDifference(@Price smallmoney) RETURNS smallmoney AS
BEGIN
```

```
    RETURN @Price - dbo.AveragePrice()
END;
```

— Вызов функции

```
SELECT Pname, Price, dbo.AveragePrice() AS Average, dbo.PriceDifference(Price) AS Difference FROM R
WHERE City='Смоленск'
```

Подставляемая табличная функция

Тело подставляемой табличной функции фактически состоит из единственной инструкции SELECT. Синтаксис создания подставляемой табличной функции выглядит так:

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )
RETURNS TABLE
```

```
[ WITH список-опций-функций ]
```

```
[ AS ]
```

```
RETURN [ ( [ выражение-выборки [ ] ] ) ] END [ ; ]
```

Пример создания и вызова подставляемой табличной функции

```
CREATE FUNCTION dbo.FullSPJ()
RETURNS TABLE
AS
    RETURN (SELECT S.Sname, P.Pname, J.Jname, SPJ.Qty
            FROM S INNER JOIN SPJ ON S.Sno=SPJ.Sno
                INNER JOIN P ON P.Pno=SPJ.Pno
                INNER JOIN J ON J.Jno=SPJ.Jno)

— Вызов функции
SELECT *
FROM dbo.FullSPJ()
```

Многооператорная табличная функция

Подобно скалярным функциям, в многооператорной табличной функции команды T-SQL располагаются внутри блока BEGIN-END. Поскольку блок может содержать несколько инструкций SELECT, в предложении RETURNS необходимо явно определить таблицу, которая будет возвращаться. Поскольку оператор RETURN в многооператорной табличной функции всегда возвращает таблицу, заданную в предложении RETURNS, он должен выполняться без аргументов. Синтаксис создания многооператорной табличной функции выглядит так:

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [ список-объявлений-параметров ] )
RETURNS @имя-возвращаемой-переменной TABLE определение-таблицы
[ WITH список-опций-функций ]
[ AS ]
BEGIN
RETURN END [ ; ]
```

Пример создания и вызова многооператорной табличной функции

```
CREATE FUNCTION dbo.fnGetReports ( @EmployeeID AS int )
RETURNS @Reports TABLE ( EmployeeID int NOT NULL, ReportsToID int NULL )
AS BEGIN
    DECLARE @Employee AS int
    INSERT INTO @Reports
        SELECT EmployeeID, ReportsTo FROM Employees
        WHERE EmployeeID = @EmployeeID
    SELECT @Employee = MIN(EmployeeID) FROM Employees
    WHERE ReportsTo = @EmployeeID
    WHILE @Employee IS NOT NULL
    BEGIN
        INSERT INTO @Reports
            SELECT *
    END RETURN
END
```

Хранимые процедуры T-SQL

Хранимая процедура – именованный объект базы данных, представляющий собой набор SQL-инструкций, который компилируется один раз и хранится на сервере. Хранимые процедуры похожи на обыкновенные процедуры языков высокого уровня, у них могут быть входные и выходные параметры и локальные переменные. В хранимых процедурах могут выполняться операторы DDL, DML, TCL, FCL. Процедуры можно создавать для постоянного использования, для временного использования в одном сеансе (локальная временная процедура), для временного использования во всех сеансах (глобальная временная

процедура). Хранимые процедуры могут выполняться автоматически при запуске экземпляра SQL Server.

Создание хранимых процедур

Для создания хранимой процедуры используется инструкция CREATE PROCEDURE, имеющая следующий синтаксис:

```
CREATE PROCEDURE [ имя-схемы. ] имя-процедуры [ список-объявлений-параметров ]  
[ WITH список-опций-процедуры ]  
[ FOR REPLICATION ]  
AS  
тело-процедуры  
[ ;]
```

где

- Если имя схемы не указано при создании процедуры, то автоматически назначается схема по умолчанию для пользователя, который создает процедуру.
- Список объявлений параметров является необязательным.
- Объявление параметра в списке объявлений параметров имеет вид:

```
@имя-параметра тип-данных [ VARYING ] [ = значение-по-умолчанию ] [ OUT | OUTPUT ] [ READONLY]
```

- Параметрами процедуры могут быть любые типы данных, за исключением table.
- Для создания параметров, возвращающих табличное значение, можно использовать определяемый пользователем табличный тип. Возвращающие табличное значение параметры могут быть только входными и должны сопровождаться ключевым словом READONLY.
- Тип данных cursor может быть использован только в качестве выходного параметра.
- VARYING применяется только к аргументам типа cursor.
- OUT или OUTPUT показывает, что параметр процедуры является выходным. Параметры типов text, ntext и image не могут быть выходными.
- READONLY указывает, что параметр не может быть обновлен или изменен в тексте процедуры.
- Опциями функции могут быть:
 - ENCRYPTION - SQL Server шифрует определение процедуры.
 - RECOMPILE - SQL Server перекомпилирует процедуру при каждом ее выполнении.
 - Предложение EXECUTE AS - определяет контекст безопасности, в котором должна быть выполнена процедура.
- Тело процедуры - одна или несколько инструкций T-SQL. Инструкции можно заключить в необязательные ключевые слова BEGIN и END.
- FOR REPLICATION указывает, что процедура создается для репликации.
- Тело процедуры может содержать оператор RETURN, возвращающий целочисленное значение вызывающей процедуре или приложению.

Примечания.

- Локальную временную процедуру можно создать, указав один символ номера (#) перед именем процедуры.
- Глобальную временную процедуру можно создать, указав два символа номера (##) перед именем процедуры.
- Временные процедуры создаются в базе данных tempdb.
- Рекомендуется начинать текст процедуры с инструкции SET NOCOUNT ON (она должна следовать сразу за ключевым словом AS). В этом случае отключаются сообщения, отправляемые SQL Server клиенту после выполнения любых инструкций SELECT, INSERT, UPDATE, MERGE и DELETE.

Выполнение хранимых процедур

При выполнении процедуры в первый раз она компилируется, при этом определяется оптимальный план получения данных. При последующих вызовах процедуры может быть повторно использован уже созданный план, если он еще находится в кэше планов компонента Database Engine.

Одна процедура может вызывать другую. Уровень вложенности увеличивается на 1, когда начинается выполнение вызванной процедуры, и уменьшается на 1, когда вызванная процедура завершается. Уровень вложенности процедур может достигать 32. Текущий уровень вложенности процедур можно получить при помощи функции @@NESTLEVEL.

Чтобы выполнить процедуру, надо использовать инструкцию EXECUTE. Также можно выполнить процедуру без использования ключевого слова EXECUTE, если процедура является первой инструкцией в пакете.

При выполнении процедуры (в пакете или внутри хранимой процедуры или функции) настоятельно рекомендуется уточнять имя хранимой процедуры указанием, по крайней мере, имени схемы.

Пример процедуры с входными и выходными параметрами

```
CREATE PROCEDURE dbo.Factorial @ValIn bigint, @ValOut bigint output
AS
BEGIN
    IF @ValIn > 20 BEGIN
        PRINT N'Входной параметр должен быть <= 20'
        RETURN -99
    END
    DECLARE @WorkValIn bigint, @WorkValOut bigint
    IF @ValIn != 1
    BEGIN
        SET @WorkValIn = @ValIn - 1
        PRINT @@NESTLEVEL
        EXEC dbo.Factorial @WorkValIn, @WorkValOut OUTPUT
        SET @ValOut = @WorkValOut * @ValIn
    END
    ELSE
        SET @ValOut = 1
END

-- Вызов процедуры
DECLARE @FactIn int, @FactOut int
SET @FactIn = 8
EXEC dbo.Factorial @FactIn, @FactOut OUTPUT

PRINT N'Факториал ' + CONVERT(varchar(3),@FactIn) +
      N' равен ' + CONVERT(varchar(20),@FactOut)
```

Изменение хранимых процедур

Если нужно изменить инструкции или параметры хранимой процедуры, можно или удалить (DROP PROCEDURE) и создать ее заново (CREATE PROCEDURE), или изменить ее за один шаг (ALTER PROCEDURE). При удалении и повторном создании хранимой процедуры все разрешения, связанные с ней, будут утеряны при восстановлении. При изменении хранимой процедуры ее определение или определение ее параметров меняются, но разрешения, связанные с ней, остаются и все зависящие от нее процедуры или триггеры не затрагиваются.

Базы Данных

Семинар 7

Описание семинара: Триггеры и курсоры

Триггеры T-SQL Триггеры DDL

Триггер DDL - это хранимая процедура особого типа, которая выполняет одну или несколько инструкций T-SQL в ответ на событие из области действия сервера или базы данных. Например, триггер DDL может активироваться, если выполняется такая инструкция, как ALTER SERVER CONFIGURATION, или если происходит удаление таблицы с использованием команды DROP TABLE.

Все события (кроме ALTER_SERVER_CONFIGURATION) разделены на две группы:

- события уровня базы данных (DDL_DATABASE_LEVEL_EVENTS)
- события уровня сервера (DDL_SERVER_LEVEL_EVENTS)

Следует обратить внимание на иерархическую природу групп событий. В отличие от триггеров DML, триггеры DDL не ограничены областью схемы. Поэтому для запроса метаданных о триггерах DDL нельзя воспользоваться такими функциями как OBJECT_ID, OBJECT_NAME, OBJECTPROPERTY и OBJECTPROPERTYEX. Используйте вместо них представления каталога.

Пример триггера DDL

```
CREATE TRIGGER safety
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE AS
    PRINT 'You must disable Trigger "safety" to drop or alter tables!'
    ROLLBACK;
```

Триггеры DML

Для поддержания согласованности и точности данных используются декларативные и процедурные методы.

Триггеры представляют собой особый вид хранимых процедур, привязанных к таблицам и представлениям. Они позволяют реализовать в базе данных сложные процедурные методы поддержания целостности данных. События при модификации данных вызывают автоматическое срабатывание триггеров. Как и в случае хранимых процедур, глубина вложенности триггеров достигает 32 уровней, также возможно рекурсивное срабатывание триггеров.

Прежде чем реализовывать триггер, следует выяснить, нельзя ли получить аналогичные результаты с использованием ограничений или правил. Для уникальной идентификации строк табличных данных используют целостность сущностей (ограничения primary key и unique). Доменная целостность служит для определения значений по умолчанию (определения default) и ограничения диапазона значений, разрешенных для ввода в данное поле (ограничения check и ссылочные ограничения). Ссылочная целостность используется для реализации логических связей между таблицами (ограничения foreign key и check). Если значение обязательного поля не задано в операторе INSERT, то оно определяется с помощью определения default. Лучше применять ограничения, чем триггеры и правила.

Триггеры применяются в следующих случаях:

- если использование методов декларативной целостности данных не отвечает функциональным потребностям приложения. Например, для изменения числового значения в таблице при удалении записи из этой же таблицы следует создать триггер;
- если необходимо каскадное изменение через связанные таблицы в базе данных. Чтобы обновить или удалить данные в столбцах с ограничением foreign key, вместо пользовательского триггера следует применять ограничения каскадной ссылочной целостности;
- если база данных денормализована и требуется способ автоматизированного обновления избыточных данных в нескольких таблицах;
- если необходимо сверить значение в одной таблице с неидентичным значением в другой таблице;
- если требуется вывод пользовательских сообщений и сложная обработка ошибок.

События, вызывающие срабатывание триггеров (типы триггеров)

Автоматическое срабатывание триггера вызывают три события: INSERT, UPDATE и DELETE, которые происходят в таблице или представлении. Триггеры нельзя запустить

вручную. В синтаксисе триггеров перед фрагментом программы, уникально определяющим выполняемую триггером задачу, всегда определено одно или несколько таких событий. Один триггер разрешается запрограммировать для реакции на несколько событий, поэтому несложно создать процедуру, которая одновременно является триггером на обновление и добавление. Порядок этих событий в определении триггера является несущественным.

Классы триггеров

В SQL Server существуют два класса триггеров:

- **INSTEAD OF.** Триггеры этого класса выполняются в обход действий, вызывавших их срабатывание, заменяя эти действия. Например, обновление таблицы, в которой есть триггер INSTEAD OF, вызовет срабатывание этого триггера. В результате вместо оператора обновления выполняется код триггера. Это позволяет размещать в триггере сложные операторы обработки, которые дополняют действия оператора, модифицирующего таблицу.
- **AFTER/BEFORE.** Триггеры этого класса исполняются после действия, вызвавшего срабатывание триггера. Они считаются классом триггеров по умолчанию. Между триггерами этих двух классов существует ряд важных отличий, показанных в таблице.

Характеристика	INSTEAD OF-триггер	AFTER/BEFORE-триггер
Объект, к которому можно привязать триггер	Таблица или представление. Триггеры, привязанные к представлению, расширяют список типов обновления, которые может поддерживать представление	Таблица. AFTER-триггеры срабатывают при модификации данных в таблице в ответ на модификацию представления
Допустимое число триггеров	В таблице или представлении допускается не больше одного триггера в расчете на одно действие. Можно определять представления для других представлений, каждое со своим собственным INSTEAD OF-триггером	К таблице можно привязать несколько AFTER- триггеров
Порядок исполнения	Поскольку в таблице или представлении допускается не больше одного такого триггера в расчете на одно событие, порядок не имеет смысла	Можно определять триггеры, срабатывающие первым и последним. Для этого служит системная хранимая процедура <code>sp_settriggerorder</code> . Порядок срабатывания других триггеров, привязанных к таблице, случаен

К таблице разрешено привязывать триггеры обоих классов. Если в таблице определены ограничения и триггеры обоих классов, то первым из них срабатывает триггер INSTEAD OF, затем обрабатываются ограничения и последним срабатывает AFTER-триггер. При нарушении ограничения выполняется откат действий INSTEAD OF-триггера. Если нарушаются ограничения или происходят какие-либо другие события, не позволяющие модифицировать таблицу, AFTER-триггер не исполняется.

Создание триггеров DML

```
CREATE TRIGGER имя_триггера
ON имя_таблицы_или_представления
[ WITH ENCRYPTION ]
класс_триггера тип(ы)_триггера
[ WITH APPEND ]
[ NOT FOR REPLICATION ]
AS sql_инструкции
```

В квадратных скобках указаны опции триггера, которые в данной теме не рассматриваются.

Пояснения к инструкции CREATE TRIGGER:

- Триггеры не допускают указания имени базы данных в виде префикса имени объекта. Поэтому перед созданием триггера необходимо выбрать нужную базу данных с помощью конструкции USE имя_базы_данных и ключевого слова GO. Ключевое слово GO требуется, поскольку оператор CREATE TRIGGER должен быть первым в пакете. Право на создание триггеров по умолчанию принадлежит владельцу таблицы.
- Триггер необходимо привязать к таблице или представлению. Любой триггер можно привязать только к одной таблице или представлению. Чтобы привязать к другой таблице триггер, выполняющий ту же самую задачу, следует создать новый триггер с другим именем, но с той же самой функциональностью и привязать его к другой таблице. AFTER-триггеры (этот класс задан по умолчанию) разрешено привязывать только к таблицам, а триггеры класса INSTEAD OF – как к таблицам, так и к представлениям.
- При создании триггера следует задать тип события, вызывающего его срабатывание: INSERT, UPDATE и DELETE. Один и тот же триггер может сработать на одно, два или все три события. Если необходимо, чтобы он срабатывал на все события, то после конструкций FOR, AFTER или INSTEAD OF следует поместить все три ключевых слова: INSERT, UPDATE и DELETE в любом порядке.
- Конструкция AS и следующие за ней команды языка T-SQL определяют задачу, которую будет выполнять триггер.

Пример простого триггера AFTER на событие UPDATE

```
CREATE TRIGGER AfterUpdateSPI
ON SPI
AFTER UPDATE
AS
BEGIN
RAISERROR(N'Произошло обновление в таблице поставок',1,1) END
```

Курсоры T-SQL

Операции в реляционной базе данных выполняются над множеством строк. Набор строк, возвращаемый инструкцией SELECT, содержит все строки, которые удовлетворяют условиям, указанным в предложении WHERE. Такой полный набор строк, возвращаемых инструкцией, называется результирующим набором. Приложения, особенно интерактивные, не всегда эффективно работают с результирующим набором как с единым целым. Им нужен механизм, позволяющий обрабатывать одну строку или небольшое их число за один раз. Курсоры предоставляют такой механизм.

Microsoft SQL Server поддерживает три способа реализации курсоров:

В SQL Server поддерживаются четыре типа серверных курсоров:

- **Статические курсоры (STATIC).** Создается временная копия данных для использования курсором. Все запросы к курсору обращаются к указанной временной таблице в базе данных tempdb, поэтому изменения базовых таблиц не влияют на данные, возвращаемые выборками для данного курсора, а сам курсор не позволяет производить изменения.
- **Динамические курсоры (DYNAMIC).** Отображают все изменения данных, сделанные в строках результирующего набора при просмотре этого курсора. Значения данных, порядок, а также членство строк в каждой выборке могут меняться. Параметр выборки ABSOLUTE динамическими курсорами не поддерживается.
- **Курсоры, управляемые набором ключей (KEYSET).** Членство или порядок строк в курсоре не изменяются после его открытия. Набор ключей, однозначно определяющих строки, встроены в таблицу в базе данных tempdb с именем keyset.
- **Быстрые последовательные курсоры (FAST_FORWARD).** Параметр FAST_FORWARD указывает курсор FORWARD_ONLY, READ_ONLY, для которого включена оптимизация производительности. Параметр FAST_FORWARD не может указываться вместе с параметрами SCROLL или FOR_UPDATE.

Статическими курсорами обнаруживаются лишь некоторые изменения или не обнаруживаются вовсе, но при этом в процессе прокрутки такие курсоры потребляют

сравнительно мало ресурсов. Динамические курсоры обнаруживают все изменения, но потребляют больше ресурсов при прокрутке. Управляемые набором ключей курсоры имеют промежуточные свойства, обнаруживая большинство изменений, но потребляя меньше ресурсов, чем динамические курсоры.

По области видимости имени курсора различают:

- **Локальные курсоры (LOCAL).** Область курсора локальна по отношению к пакету, хранимой процедуре или триггеру, в которых этот курсор был создан. Курсор неявно освобождается после завершения выполнения пакета, хранимой процедуры или триггера, за исключением случая, когда курсор был передан параметру OUTPUT. В этом случае курсор освобождается при освобождении всех ссылающихся на него переменных или при выходе из области видимости.
- **Глобальные курсоры (GLOBAL).** Область курсора является глобальной по отношению к соединению. Имя курсора может использоваться любой хранимой процедурой или пакетом, которые выполняются соединением. Курсор неявно освобождается только в случае разрыва соединения. Глобальность курсора позволяет создавать его в рамках одной хранимой процедуры, а вызывать его из совершенно другой процедуры, причем передавать его в эту процедуру необязательно.

По способу перемещения по курсору различают:

- **Последовательные курсоры (FORWARD_ONLY).** Курсор может просматриваться только от первой строки к последней. Поддерживается только параметр выборки FETCH NEXT. Если параметр FORWARD_ONLY указан без ключевых слов STATIC, KEYSET или DYNAMIC, то курсор работает как DYNAMIC. Если не указан ни один из параметров FORWARD_ONLY или SCROLL, а также не указано ни одно из ключевых слов STATIC, KEYSET или DYNAMIC, то по умолчанию задается параметр FORWARD_ONLY. Курсоры STATIC, KEYSET и DYNAMIC имеют значение по умолчанию SCROLL.
- **Курсоры прокрутки (SCROLL).** Перемещение осуществляется по группе записей как вперед, так и назад. В этом случае доступны все параметры выборки (FIRST, LAST, PRIOR, NEXT, RELATIVE, ABSOLUTE). Параметр SCROLL не может указываться вместе с параметром для FAST_FORWARD.

По способу распараллеливания курсоров различают:

- **READ_ONLY.** Содержимое курсора можно только считывать.
- **SCROLL_LOCKS.** При редактировании данной записи вами никто другой вносить в нее изменения не может. Такую блокировку прокрутки иногда еще называют «пессимистической» блокировкой. При блокировке прокрутки важным фактором является продолжительность действия блокировки. Если курсор не входит в состав некоторой транзакции, то блокировка распространяется только на текущую запись в курсоре. В противном случае все зависит от того, какой используется уровень изоляции транзакций.
- **OPTIMISTIC.** Означает отсутствие каких бы то ни было блокировок. «Оптимистическая» блокировка предполагает, что даже во время выполнения вами редактирования данных, другие пользователи смогут к ним обращаться. Если кто-то все-таки попытается выполнить модификацию данных одновременно с вами, генерируется ошибка 16394. В этом случае вам придется повторно выполнить доставку данных из курсора или же осуществить полный откат транзакции, а затем попытаться выполнить ее еще раз.

Для выявления ситуаций с преобразованием типа курсора используется опция TYPE_WARNING, которая указывает, что клиенту посылается предупреждающее сообщение при неявном преобразовании типа курсора из запрошенного типа в другой тип.

По умолчанию курсор, разрешающий изменение данных, позволяет выполнять модификацию любых столбцов в своем составе. Опция FOR UPDATE указывает столбцы курсора, которые можно изменять. Все остальные столбцы становятся доступными только для чтения. Если опция FOR UPDATE используется без списка столбцов, то обновление возможно для всех столбцов, за исключением случая, когда был указан параметр параллелизма READ_ONLY.

На содержательном уровне синтаксис инструкции DECLARE CURSOR будет иметь вид:

```
DECLARE имя-курсора CURSOR
[ область-видимости-имени-курсора ]
[ возможность-перемещения-по-курсору ]
[ типы-курсоров ]
[ опции-распараллеливания-курсоров ]
[ выявление-ситуаций-с-преобразованием-типа-курсора ]
FOR инструкция_select
[ опция-FOR-UPDATE ]
```

Формальный синтаксис инструкции DECLARE CURSOR имеет вид:

```
DECLARE имя-курсора CURSOR
[ LOCAL | GLOBAL ]
[ FORWARD_ONLY | SCROLL ]
[ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
[ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
[ TYPE_WARNING ]
FOR инструкция_select
[ FOR UPDATE [ OF список-имен_столбцов ] ]
```

Использование переменной типа cursor

Microsoft SQL Server поддерживает переменные с типом данных CURSOR. Курсор может быть связан с переменной типа CURSOR двумя способами, например:

- DECLARE @MyVariable CURSOR
DECLARE MyCursor CURSOR
FOR SELECT LastName FROM AdventureWorks.Person.Contact
SET @MyVariable = MyCursor;
- DECLARE @MyVariable CURSOR
SET @MyVariable = CURSOR SCROLL KEYSET
FOR SELECT LastName FROM AdventureWorks.Person.Contact;

После связи курсора с переменной типа CURSOR эта переменная может использоваться в инструкциях курсора языка T- SQL вместо имени курсора. Кроме того, выходным параметрам хранимой процедуры можно назначить тип данных CURSOR и связать их с курсором. Это позволяет управлять локальными курсорами из хранимых процедур.

Перемещение внутри курсора (прокрутка курсора)

Основой всех операций прокрутки курсора является ключевое слово FETCH. Оно может использоваться для перемещения по курсору в обоих направлениях, в том числе и для перехода к заданной позиции. В качестве аргументов оператора FETCH могут выступать:

- NEXT – возвращает строку результата сразу же за текущей строкой и перемещает указатель текущей строки на возвращенную строку. Если инструкция FETCH NEXT выполняет первую выборку в отношении курсора, она возвращает первую строку в результирующем наборе. NEXT является параметром по умолчанию выборки из курсора.
- PRIOR – возвращает строку результата, находящуюся непосредственно перед текущей строкой и перемещает указатель текущей строки на возвращенную строку. Если инструкция FETCH PRIOR выполняет первую выборку из курсора, не возвращается никакая строка и положение курсора остается перед первой строкой.
- FIRST – возвращает первую строку в курсоре и делает ее текущей.
- LAST – возвращает последнюю строку в курсоре, и делает ее текущей.
- ABSOLUTE { n | @nvar }. Если n или @nvar имеют положительное значение, возвращает строку, стоящую дальше на n строк от передней границы курсора, и делает возвращенную строку новой текущей строкой. Если n или @nvar имеют отрицательное значение, возвращает строку, отстоящую на n строк от задней границы курсора, и делает возвращенную строку новой текущей строкой. Если n или @nvar равны 0, не возвращается

никакая строка. n должно быть целым числом, а @nvar должна иметь тип данных smallint, tinyint или int.

- **RELATIVE** { n | @nvar }. Если n или @nvar имеют положительное значение, возвращает строку, отстоящую на n строк вперед от текущей строки, и делает возвращенную строку новой текущей строкой. Если n или @nvar имеют отрицательное значение, возвращает строку, отстоящую на n строк назад от текущей строки, и делает возвращенную строку новой текущей строкой. Если n или @nvar равны 0, возвращает текущую строку. Если при первой выборке из курсора инструкция **FETCH RELATIVE** указывается с отрицательными или равными нулю параметрами n или @nvar, то никакая строка не возвращается. n должно быть целым числом, а @nvar должна иметь тип данных smallint, tinyint или int. Опциями оператора **FETCH** являются:
 - **GLOBAL** – указывает, что параметр имя-курсора ссылается на глобальный курсор.
 - **INTO** – позволяет поместить данные из столбцов выборки в локальные переменные.

Каждая переменная из списка, слева направо, связывается с соответствующим столбцом в результирующем наборе курсора. Тип данных каждой переменной должен соответствовать типу данных соответствующего столбца результирующего набора, или должна обеспечиваться поддержка неявного преобразования в тип данных этого столбца. Количество переменных должно совпадать с количеством столбцов в списке выбора курсора.

Синтаксис инструкции **FETCH** имеет вид:

```
FETCH [ [ NEXT | PRIOR | FIRST | LAST | ABSOLUTE { n | @nvar } | RELATIVE { n | @nvar } ] FROM ]  
{ { [ GLOBAL ] имя-объявленного-курсора } | @имя-переменной-курсора }  
[ INTO список-имен-переменных ]
```

После выполнения каждой инструкции **FETCH** функция @@FETCH_STATUS обновляется, отражая состояние последней выборки. Функция @@FETCH_STATUS показывает такие состояния, как выборка за пределами первой или последней строк курсора. Функция @@FETCH_STATUS глобальна для соединения и обновляется после любой выборки в любом курсоре, открытом во время соединения. Если состояние нужно посмотреть позже, то перед выполнением следующей инструкции в пределах соединения необходимо сохранить значение функции @@FETCH_STATUS в пользовательской переменной. Даже если следующей инструкцией будет не **FETCH**, а **INSERT**, **UPDATE** или **DELETE**, сработает триггер, содержащий инструкции **FETCH**, что приведет к обновлению значения функции @@FETCH_STATUS.

```
-- Объявляем курсор  
DECLARE CursorTest CURSOR  
GLOBAL SCROLL STATIC FOR  
    SELECT OrderID, CustomerID  
    FROM CursorTable  
  
-- Объявляем переменные для хранения  
DECLARE @OrderID int  
DECLARE @CustomerID varchar(5)  
  
-- Откроем курсор и запросим первую запись  
OPEN CursorTest  
FETCH NEXT FROM CursorTest INTO @OrderID, @CustomerID -- Обрабатываем в цикле все записи курсора  
WHILE @@FETCH_STATUS=0  
BEGIN  
    PRINT CONVERT(varchar(5),@OrderID) + ' ' + @CustomerID  
    FETCH NEXT FROM CursorTest INTO @OrderID, @CustomerID  
END
```

Базы Данных

Семинар 8

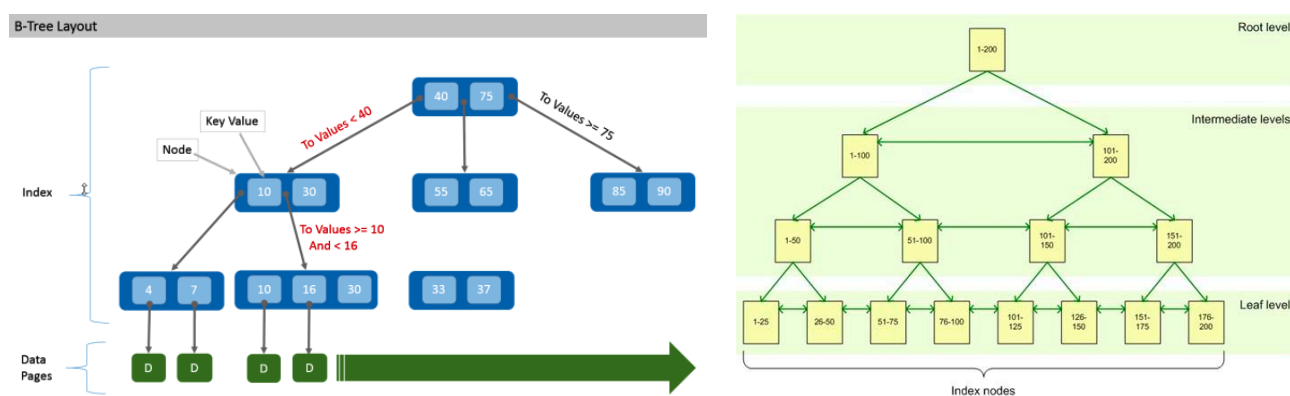
Описание семинара: Оптимизация запросов. Индексы, Партиционирование и сегментирование.

Индексы и проблемы производительности

Индекс — это объект базы данных, обеспечивающий дополнительные способы быстрого поиска и извлечения данных. Индекс может создаваться на одном или нескольких столбцах. Это означает, что индексы бывают простыми и составными. Если в таблице нет индекса, то поиск нужных строк выполняется простым сканированием по всей таблице. При наличии индекса время поиска нужных строк можно существенно уменьшить. К недостаткам индексов следует отнести:

- дополнительное место на диске и в оперативной памяти,
- замедляются операции вставки, обновления и удаления записей.

В SQL Server индексы хранятся в виде сбалансированных деревьев. Представление индекса в виде сбалансированного дерева означает, что стоимость поиска любой строки остается относительно постоянной, независимо от того, где находится эта строка.



Сбалансированное дерево состоит из:

- корневого узла (root node), содержащего одну страницу,
- нескольких промежуточных уровней (intermediate levels), содержащих дополнительные страницы,
- листового уровня (leaf level).

На страницах листового уровня находятся отсортированные элементы, соответствующие индексируемым данным. По мере добавления данных в таблицу индекс будет разрастаться, но по-прежнему оставаться в форме сбалансированного дерева.

Существует два типа индексов:

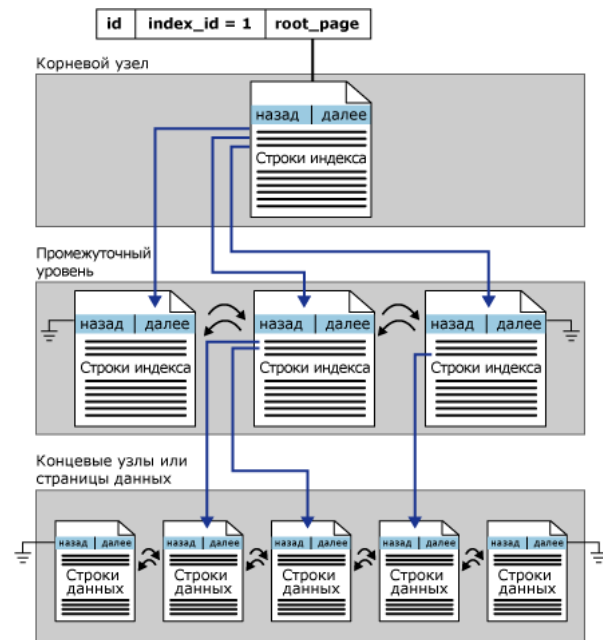
- Кластерные индексы;
- Некластерные индексы, которые включают:
 - i. на основе кучи;
 - ii. на основе кластерных индексов.

Кластерные индексы

В кластерном индексе таблица представляет собой часть индекса, или индекс представляет собой часть таблицы. Листовой узел кластерного индекса — это страница таблицы с данными. Поскольку сами данные таблицы являются частью индекса, то очевидно, что для таблицы может быть создан только один кластерный индекс. Кластерный индекс является уникальным индексом по определению. Это означает, что все ключи записей должны быть

уникальные. Если существуют записи с одинаковыми значениями, SQL Server делает их уникальными, добавляя номера из внутреннего (невидимого снаружи) 4-х байтного счетчика. Кластерный индекс использовать необязательно. Тем не менее, кластерные индексы часто используют в качестве первичного ключа. На уровне листьев кластерного индекса информация упорядочивается и физически сохраняется на диске в соответствии с заданными критериями сортировки. На каждом уровне индекса страницы организованы в виде двунаправленного связного списка. Вход в корень кластерного индекса дает поле `root_page` системного представления `sys.system_internals_allocation_units`.

При добавлении каждой новой записи хранимые данные пересортировываются, чтобы сохранить физическую упорядоченность данных. Если требуется создать новую запись в середине индексной структуры, то происходит обычное разбиение страницы. Половина записей из старой страницы переносится в новую, а новая запись добавляется либо в новую страницу, либо в старую страницу (для сохранения равновесия). Если же новая запись логически попадает в конец индексной структуры, тогда в новую создаваемую страницу помещается только новая строка (информация не делится поровну между двумя рассматриваемыми страницами).



Некластерные индексы на основе кучи

В листьях некластерного индекса на основе кучи хранятся указатели на строки данных. Указатель строится на основе идентификатора файла (ID), номера страницы и номера строки на странице. Весь указатель целиком называется идентификатором строки (RID). С точки зрения физического размещения данных – они хранятся в полном беспорядке. С точки зрения технической реализации нужные записи приходится искать по всему файлу. Вполне может стать, что в этом случае SQL Server придется несколько раз возвращаться к одной и той же странице для чтения различных строк, поскольку определить заранее, откуда придется физически считывать следующую строку нет никакой возможности.

Некластерные индексы, основанные на кластерных таблицах

В листьях некластерного индекса, основанного на кластерных таблицах, хранятся указатели на корневые узлы кластерных индексов. Поиск в таком индексе состоит из двух этапов:

- поиск в некластерном индексе
- поиск в кластерном индексе.

Упрощенный синтаксис явного создания индекса:

```
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ] INDEX имя-индекса
ON имя-таблицы-или-представления ( список-столбцов )
[ INCLUDE (список-столбцов) ]
[ WITH список-опций ]
[ ON файловая-группа ]
```

Замечания:

- Для каждого столбца в списке столбцов могут указываться две взаимоисключающие опции ASC/DESC, которые позволяют выбрать способ сортировки индекса. По умолчанию используется вариант ASC, что означает сортировку по возрастанию.
- Предложение INCLUDE указывает неключевые столбцы, добавляемые к ключевым столбцам некластеризованного индекса.
- Следующие за ключевым словом WITH опции являются необязательными. Их можно использовать в любом сочетании. Одни (как например PAD_INDEX, IGNORE_DUP_KEY, DROP_EXISTING, STATISTICS_NORECOMPUTE, SORT_IN_TEMPDB) используются довольно редко, другие (как например FILLFACTOR) существенно влияют на быстродействие и алгоритм работы системы. Описания опций индекса находятся по адресу
- С помощью предложения ON можно задать место хранения индекса. По умолчанию индекс помещается в ту же файловую группу, где находится таблица или представление, для которых он создан.

Пример создания индекса.

```
CREATE UNIQUE NONCLUSTERED INDEX [IX_Address]
ON [Person].[Address] ([AddressLine1] ASC,
                       [AddressLine2] ASC,
                       [City] ASC,
                       [StateProvinceID] ASC,
                       [PostalCode] ASC)
WITH (PAD_INDEX = OFF,
      STATISTICS_NORECOMPUTE = OFF,
      SORT_IN_TEMPDB = OFF,
      IGNORE_DUP_KEY = OFF,
      DROP_EXISTING = OFF,
      ONLINE = OFF,
      ALLOW_ROW_LOCKS = ON,
      ALLOW_PAGE_LOCKS = ON)
ON [PRIMARY]
```

Индексы, создаваемые вместе с ограничениям

Такой тип индексов часто называют «связанными индексами». Связанные индексы создаются при добавлении одного из следующих двух типов ограничений:

- ограничения первичного ключа (PRIMARY KEY);
- ограничения уникальности (UNIQUE).

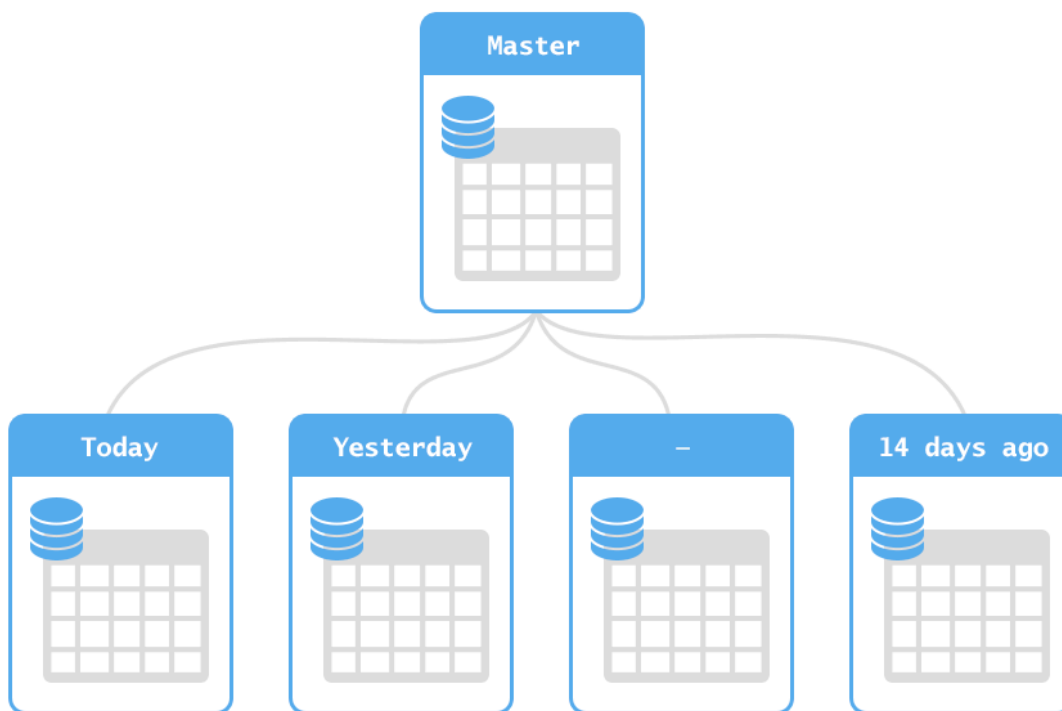
Пример создания индекса.

```
CREATE TABLE [Person].[Address] (
    [AddressID] int IDENTITY(1,1) NOT NULL,
    ...
    CONSTRAINT [PK_Address_AddressID] PRIMARY KEY
        CLUSTERED ([AddressID] ASC)
    WITH (PAD_INDEX = OFF,
          STATISTICS_NORECOMPUTE = OFF,
          IGNORE_DUP_KEY = OFF,
          ALLOW_ROW_LOCKS = ON,
          ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
) ON [PRIMARY]
```

Партиционирование

Партиционирование – это метод разделения больших (исходя из количества записей, а не столбцов) таблиц на много маленьких. И желательно, чтобы это происходило прозрачным для приложения способом.

Одной из редко используемых фич PostgreSQL является тот факт, что это объектно-реляционная база данных. И «объект» здесь ключевое слово, потому что объекты (или, скорее, классы) знают то, что называется «наследование». Именно это используется для партиционирования.



Давайте попробуем произвести настоящее партиционирование. Для начала нам нужно решить, каким будет ключ партиционирования – другими словами, по какому алгоритму будут выбираться партии.

Есть пара наиболее очевидных:

- партиционирование по дате – например, выбирать партии, основываясь на годе, в котором пользователь был создан
 - достоинства:
 - легко понять;
 - количество строк в данной таблице будет достаточно стабильным;
 - недостатки:
 - требует поддержки – время от времени нам придётся добавлять новые партии;
 - поиск по имени пользователя или id потребует сканирования всех партий;
- партиционирование по диапазону идентификаторов – например, первый миллион пользователей, второй миллион пользователей, и так далее
 - достоинства:

- легко понять;
- количество строк в партиции будет на 100% стабильным;
- недостатки:
 - требует поддержки – время от времени нам придётся добавлять новые партиции;
 - поиск по имени пользователя или id потребует сканирования всех партиций;
- партиционирование по чему-нибудь другому – например, по первой букве имени пользователя.
 - достоинства:
 - легко понять;
 - никакой поддержки – есть строго определенный набор партиций и нам никогда не придется добавлять новые;
 - недостатки:
 - количество строк в партициях будет стабильно расти;
 - в некоторых партициях будет существенно больше строк, чем в других (больше людей с никами, начинающимися на “t*”, чем на “y*»);
 - поиск по id потребует сканирования всех партиций;

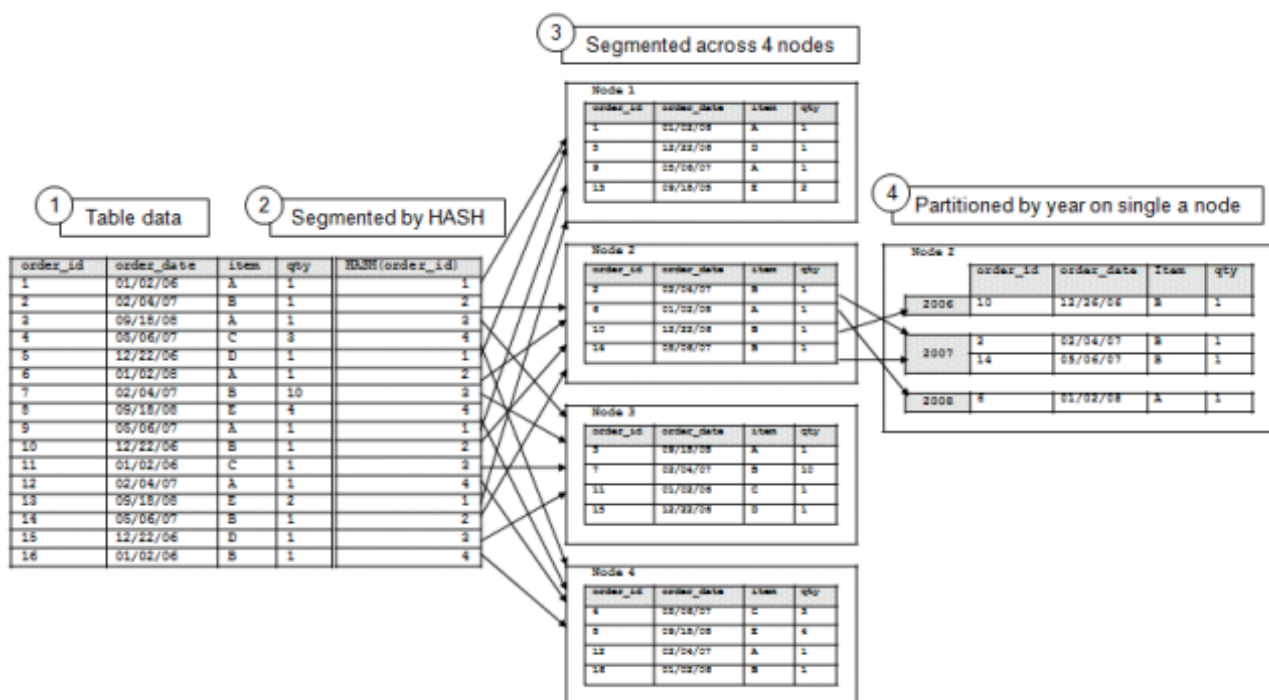
Есть еще пара других, не так часто используемых вариантов, вроде «партиционирования по хэшу от имени пользователя».

```
CREATE TABLE measurement (
  city_id      int not null,
  logdate      date not null,
  peaktemp     int,
  unitsales    int
) PARTITION BY RANGE (logdate);
```

Сегментирование

Для распределения данных по кластерам используется их сегментация(Segmentation), а точнее сегментация проекций(Projection) в которых они находятся. Задача разработчика — подобрать такой список полей и/или такую функцию(например, хэш-функцию), благодаря которым данные равномерно распределятся по нодам кластера. HP Vertica рекомендует использовать встроенные функции HASH и MODULARHASH для этих целей.

Для обеспечения K-Safety > 0 создаются сообщные проекции(Buddy Projection). Проекция называется сообщными, если они имеют в себе одинаковые наборы полей и одинаковое выражение сегментации, но хранятся на разных нодах. Сообщные проекции позволяют создать те самые реплики, которые позволяют работать БД в выбранном режиме K-Safety.



Рассмотрим K-Safety, сегментацию, и сообщные проекции на практическом примере. Дано: Кластер на 3-х нодах. K-Safety = 1.

Создадим простую таблицу:

```
CREATE TABLE test (  
  n integer primary key  
);
```

Вообще, синтаксис создания таблицы позволяет включать в себя выражения влияющие на проекцию(`order`, `hash`, `ksafe...`), для того чтобы проекции создавались автоматически в момент создания таблицы, но в этом примере я создаю проекции вручную.

Далее создадим две сообщные проекции.

```
CREATE PROJECTION p_test_b0 AS  
SELECT n FROM test_order BY n  
SEGMENTED BY hash(n) ALL NODES;
```

```
CREATE PROJECTION p_test_b1 AS  
SELECT n FROM test_order BY n  
SEGMENTED BY hash(n) ALL NODES offset 1;
```

Во-первых, выбрано сегментирование по функции hash от поля n. Во-вторых, те же данные, которые хранятся в проекции p_test_b0 на текущем ноде, хранятся в проекции p_test_b1(создана с offset 1) на следующем ноде. Иными словами текущий нод содержит в сегменте проекции p_test_b1 данные предыдущего нода.

Базы Данных

Семинар 9

Описание семинара: План запроса.

PostgreSQL

Унифицированный сервер баз данных, имеющий единый движок – storage engine. Постгрес использует клиент-серверную модель.

При подключении к серверу клиент соединяется с процессом postmaster. В задачи этого процесса входит:

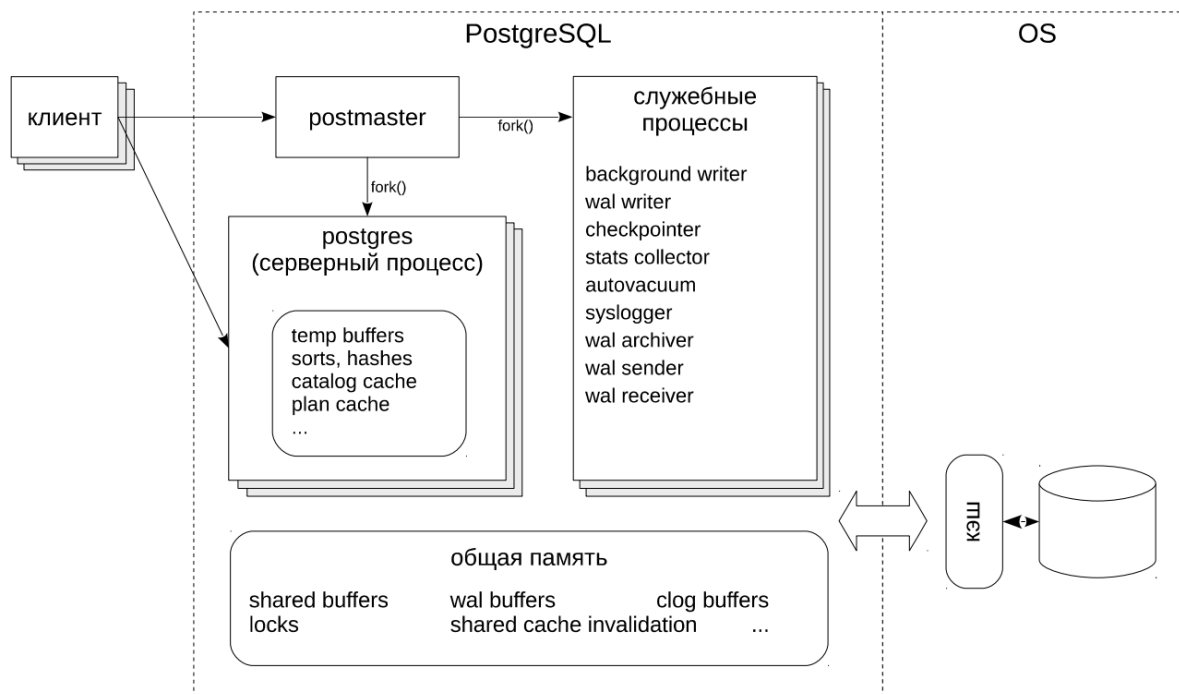
1. Порождение других процессов
2. Присмотр за созданными процессами.

Таким образом, postmaster порождает серверный процесс и дальше клиент работает уже с ним. На каждое соединение создается по серверному процессу, поэтому при большом числе соединений следует использовать пул (например, с помощью расширения pgbouncer). Postmaster также порождает ряд служебных процессов. Дерево процессов можно увидеть с помощью команды `ps fax`.

У экземпляра СУБД имеется общая для всех серверных процессов память. Большую ее часть занимает буферный кэш (shared buffers), необходимый для ускорения работы с данными на диске. Обращение к дискам происходит через операционную систему (которая тоже кэширует данные в оперативной памяти).

PostgreSQL полностью полагается на операционную систему и сам не управляет устройствами. В частности, он считает, что вызов `fsync()` гарантирует попадание данных из памяти на диск.

Кроме буферного кэша в общей памяти находится информация о блокировках и многое другое; через нее же серверные процессы общаются друг с другом. У каждого серверного процесса есть своя локальная память. В ней находится кэш каталога (часто используемая информация о базе данных), планы запросов, рабочее пространство для выполнения запросов и другое. Для работы с такими клиентскими процессами сервер использует семафоры.



Клиентский запрос проходит следующие стадии:

1. Прикладная программа устанавливает **подключение** к серверу PostgreSQL. Эта программа передаёт запрос на сервер и ждёт от него результатов.

2. На *этапе разбора запроса* сервер выполняет синтаксическую проверку запроса, переданного прикладной программой, и создаёт *дерево запроса*. Дерево разбора состоит из узлов двух типов:

- Атомы - лексические элементы следующих типов: ключевые слова (например, SELECT); имена атрибутов или отношений; константы; скобки; операторы (например, + или >); и др.
- Синтаксические категории - имена семейств, представляющих часть запроса. Заключаются в угловые скобки: <SFW>, <Condition>

Граматику языка SQL можно описать с помощью следующих правил:

```

<Query> ::= <SFW>
<Query> ::= (<Query>)
<SFW> ::= SELECT <SelList> FROM <FromList> WHERE <Condition>
<SelList> ::= <Attribute>, <SelList>
<SelList> ::= <Attribute>
<FromList> ::= <Relation>, <FromList>
<FromList> ::= <Relation>
<Condition> ::= <Condition> AND <Condition>
<Condition> ::= <Condition> OR <Condition>
<Condition> ::= NOT <Condition>
<Condition> ::= <Tuple> IN <Query>
<Condition> ::= <Attribute> = <Attribute>
<Condition> ::= <Attribute> > <Attribute>
<Condition> ::= <Attribute> < <Attribute>
<Condition> ::= <Attribute> LIKE <Pattern>
<Condition> ::= EXISTS <Query>
<Tuple> ::= <Attribute>

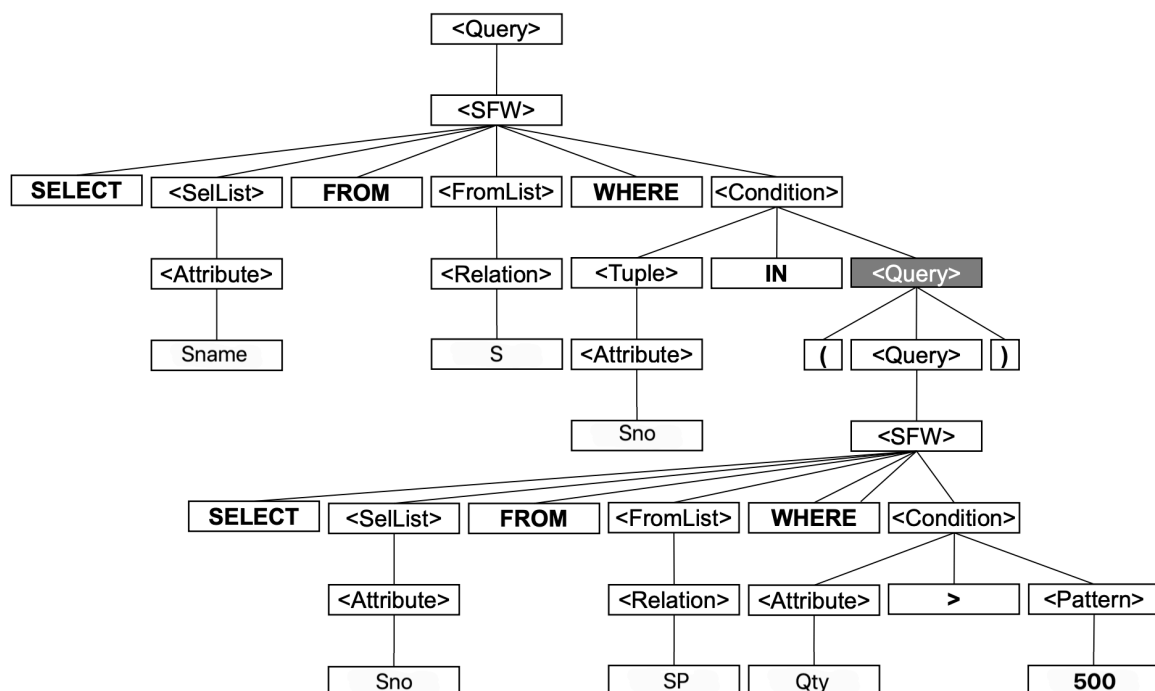
```

Рассмотрим задачу синтаксического анализа на примере запроса. Необходимо найти имена поставщиков, поставляющих хотя бы одну деталь в количестве более 500 штук (в рамках одной поставки).

```

SELECT Sname FROM S
WHERE Sno IN ( SELECT Sno FROM SP
               WHERE Qty > 500 );

```



3. Система правил принимает дерево запроса, созданное на стадии разбора, и ищет в **системных каталогах правила** для применения к этому дереву. Обнаружив подходящие правила, она выполняет преобразования, заданные в теле правил. Одно из применений системы правил заключается в реализации *представлений*. Когда выполняется запрос к представлению (т. е. *виртуальной таблице*), система правил преобразует запрос пользователя в запрос, обращающийся не к представлению, а к базовым таблицам из определения представления.

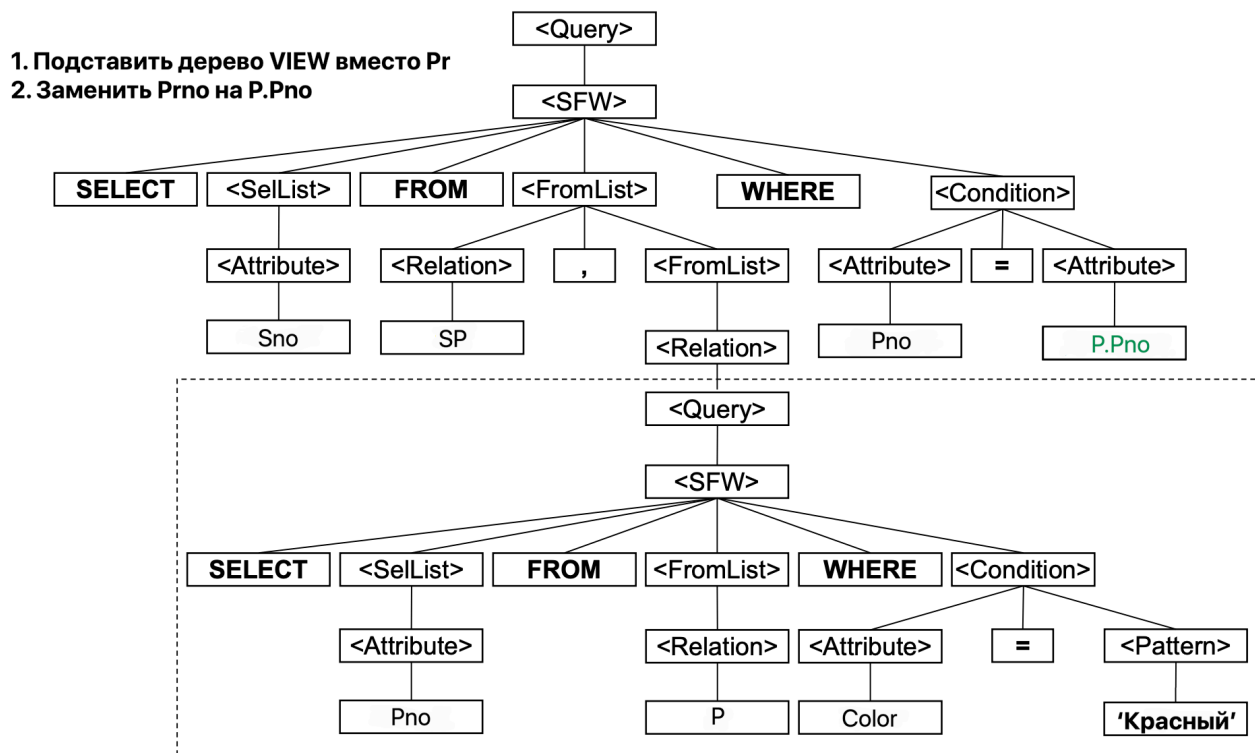
Рассмотрим пример:

Создадим представление, хранящую информацию о красных деталях:

```
CREATE VIEW Pr (Prno) AS
SELECT Rno FROM P
WHERE Color = 'Красный';
```

Теперь найдем все коды поставщиков, поставляющих хотя бы одну красную деталь:

```
SELECT Sno FROM SP, Pr
WHERE Pno= Prno;
```



4. **Планировщик/оптимизатор** принимает дерево запроса (возможно, переписанное) и создаёт *план запроса*, который будет передан *исполнителю*. Он выбирает план, сначала рассматривая все возможные варианты получения одного и того же результата. Например, если для обрабатываемого отношения создан индекс, прочитать отношение можно двумя способами. Во-первых, можно выполнить простое последовательное сканирование, а во-вторых, можно использовать индекс. Затем оценивается стоимость каждого варианта и выбирается самый дешёвый. Затем выбранный вариант разворачивается в полноценный план, который сможет использовать исполнитель.
5. Исполнитель рекурсивно проходит по *дереву плана* и **получает строки** тем способом, который указан в плане. Он сканирует отношения, обращаясь к *системе хранения*,

выполняет *сортировку и соединения*, вычисляет *условия фильтра* и, наконец, возвращает полученные строки.

План запроса

План выполнения показывает, как будут сканироваться таблицы, затрагиваемые оператором — просто последовательно, по индексу и т. д. — а если запрос связывает несколько таблиц, какой алгоритм соединения будет выбран для объединения считанных из них строк.

Наибольший интерес в выводимой информации представляет ожидаемая стоимость выполнения оператора, которая показывает, сколько, по мнению планировщика, будет выполняться этот оператор (это значение измеряется в единицах стоимости, которые не имеют точного определения, но обычно это обращение к странице на диске). Фактически выводятся два числа: стоимость запуска до выдачи первой строки и общая стоимость выдачи всех строк. Для большинства запросов важна общая стоимость, но в таких контекстах, как подзапрос в EXISTS, планировщик будет минимизировать стоимость запуска, а не общую стоимость (так как исполнение запроса всё равно завершится сразу после получения одной строки). Кроме того, если количество возвращаемых строк ограничивается предложением LIMIT, планировщик интерполирует стоимость между двумя этими числами, выбирая наиболее выгодный план.

Как вызвать план запроса в postgres:

EXPLAIN — показать план выполнения оператора

Синтаксис

EXPLAIN [(параметр [, ...])] оператор

EXPLAIN [ANALYZE] [VERBOSE] ... оператор

Здесь допускаются параметры:

- **ANALYZE** [boolean] - Выполнить команду и вывести фактическое время выполнения и другую статистику. По умолчанию этот параметр равен FALSE.
- **VERBOSE** [boolean]
- **COSTS** [boolean] - Вывести рассчитанную стоимость запуска и общую стоимость каждого узла плана, а также рассчитанное число строк и ширину каждой строки. Этот параметр по умолчанию равен TRUE.
- **BUFFERS** [boolean] – Включить информацию об использовании буфера. В частности, вывести число попаданий, блоков прочитанных, загрязненных и записанных в разделяемом и локальном буфере, а также число прочитанных и записанных временных блоков. *Попаданием* считается ситуация, когда требуемый блок уже находится в кеше и чтения с диска удастся избежать. Блоки в общем буфере содержат данные обычных таблиц и индексов, в локальном — данные временных таблиц и индексов, а временные блоки предназначены для краткосрочного использования при выполнении сортировки, хеширования, материализации и подобных узлов плана. Число *загрязнённых* блоков показывает, сколько ранее не модифицированных блоков изменила данная операция; тогда как число *записанных* блоков показывает, сколько ранее загрязнённых блоков данный серверный процесс вынес из кеша при обработке запроса. Значения, указываемые для узла верхнего уровня, включают значения всех его дочерних узлов. В текстовом формате выводятся только ненулевые значения. Этот параметр действует только в режиме ANALYZE. По умолчанию его значение равно FALSE.

- **TIMING** [boolean] - Включить в вывод фактическое время запуска и время, затраченное на каждый узел. Постоянное чтение системных часов может значительно замедлить запрос, так что если достаточно знать фактическое число строк, имеет смысл сделать этот параметр равным FALSE. Время выполнения всего оператора замеряется всегда, даже когда этот параметр выключен и на уровне узлов время не подсчитывается. Этот параметр действует только в режиме ANALYZE. По умолчанию его значение равно TRUE.
- **FORMAT** { TEXT | XML | JSON | YAML }

оператор - Любой оператор SELECT, INSERT, UPDATE, DELETE, VALUES, EXECUTE, DECLARE, CREATE TABLE AS и CREATE MATERIALIZED VIEW AS, план выполнения которого вас интересует.

Структура плана запроса представляет собой дерево *узлов плана*. Узлы на нижнем уровне дерева — это узлы сканирования, которые возвращают необработанные данные таблицы. Разным типам доступа к таблице соответствуют разные узлы: последовательное сканирование, сканирование индекса и сканирование битовой карты. Источниками строк могут быть не только таблицы, но и например, предложения VALUES и функции, возвращающие множества во FROM, и они представляются отдельными типами узлов сканирования. Если запрос требует объединения, агрегатных вычислений, сортировки или других операций с исходными строками, над узлами сканирования появляются узлы, обозначающие эти операции. И так как обычно операции могут выполняться разными способами, на этом уровне тоже могут быть узлы разных типов. В выводе команды EXPLAIN для каждого узла в дереве плана отводится одна строка, где показывается базовый тип узла плюс оценка стоимости выполнения данного узла, которую сделал для него планировщик. Если для узла выводятся дополнительные свойства, в вывод могут добавляться дополнительные строки, с отступом от основной информации узла. В самой первой строке (основной строке самого верхнего узла) выводится общая стоимость выполнения для всего плана; именно это значение планировщик старается минимизировать.

Взгляните на следующий простейший пример, просто иллюстрирующий формат вывода:

```
EXPLAIN SELECT * FROM tenk1;
```

QUERY PLAN

```
-----
Seq Scan on tenk1  (cost=0.00..458.00 rows=10000 width=244)
```

Этот запрос не содержит предложения WHERE, поэтому он должен просканировать все строки таблицы, так что планировщик выбрал план простого последовательного сканирования. Числа, перечисленные в скобках (слева направо), имеют следующий смысл:

- Приблизительная стоимость запуска. Это время, которое проходит, прежде чем начнётся этап вывода данных, например для сортирующего узла это время сортировки.
- Приблизительная общая стоимость. Она вычисляется в предположении, что узел плана выполняется до конца, то есть возвращает все доступные строки. На практике родительский узел может досрочно прекратить чтение строк дочернего (см. приведённый ниже пример с LIMIT).
- Ожидаемое число строк, которое должен вывести этот узел плана. При этом так же предполагается, что узел выполняется до конца.
- Ожидаемый средний размер строк, выводимых этим узлом плана (в байтах).

Стоимость может измеряться в произвольных единицах, определяемых параметрами планировщика. Традиционно единицей стоимости считается операция чтения страницы с

диска; то есть seq_page_cost обычно равен 1.0, а другие параметры задаётся относительно него. Примеры в этом разделе выполняются со стандартными параметрами стоимости.

Важно понимать, что стоимость узла верхнего уровня включает стоимость всех его потомков. Также важно осознавать, что эта стоимость отражает только те факторы, которые учитывает планировщик. В частности, она не зависит от времени, необходимого для передачи результирующих строк клиенту, хотя оно может составлять значительную часть общего времени выполнения запроса. Тем не менее планировщик игнорирует эту величину, так как он всё равно не сможет изменить её, выбрав другой план. (Мы верим в то, что любой правильный план запроса выдаёт один и тот же набор строк.)

Значение rows здесь имеет особенность — оно выражает не число строк, обработанных или просканированных узлом плана, а число строк, выданных этим узлом. Часто оно окажется меньше числа просканированных строк в результате применённой к узлу фильтрации по условиям WHERE. В идеале, на верхнем уровне это значение будет приблизительно равно числу строк, которое фактически возвращает, изменяет или удаляет запрос.

Примеры для демонстрации плана запроса:

<pre>explain select * from dbspj.spj s</pre>	 QUERY PLAN  Seq Scan on spj s (cost=0.00..28.50 rows=1850 width=16)
<pre>explain SELECT Sname FROM dbSPJ.S s JOIN dbSPJ.SPJ spj ON s.Sno = spj.Sno WHERE Qty > 500;</pre> <pre>explain SELECT Sname FROM dbSPJ.S s JOIN dbSPJ.SPJ spj ON s.Sno = spj.Sno AND Qty > 500;</pre>	 QUERY PLAN  Hash Join (cost=23.05..120.34 rows=1789 width=58) Hash Cond: (spj.sno = s.sno) -> Seq Scan on spj (cost=0.00..33.12 rows=617 width=4) Filter: (qty > 500) -> Hash (cost=15.80..15.80 rows=580 width=62) -> Seq Scan on s (cost=0.00..15.80 rows=580 width=6)
<pre>explain SELECT sp.pno, sp.jno, m.qty FROM dbspj.spj sp JOIN (SELECT spj.jno, MAX(qty) AS qty FROM dbspj.spj spj GROUP BY spj.jno) m ON sp.jno = m.jno AND sp.qty = m.qty</pre>	 QUERY PLAN  Hash Join (cost=44.75..82.96 rows=9 width=12) Hash Cond: ((sp.jno = m.jno) AND (sp.qty = m.qty)) -> Seq Scan on spj sp (cost=0.00..28.50 rows=1850 width=16) -> Hash (cost=41.75..41.75 rows=200 width=8) -> Subquery Scan on m (cost=37.75..41.75 rows=200 width=8) -> HashAggregate (cost=37.75..39.75 rows=200 width=8) Group Key: spj.jno -> Seq Scan on spj (cost=0.00..28.50 rows=1850 width=16)
<pre>explain SELECT pno, jno, qty FROM (SELECT pno, jno, qty, ROW_NUMBER() OVER(PARTITION BY jno ORDER BY qty DESC) AS rn FROM dbspj.spj spj) m WHERE m.rn = 1</pre>	 QUERY PLAN  Subquery Scan on m (cost=128.89..189.02 rows=9 width=16) Filter: (m.rn = 1) -> WindowAgg (cost=128.89..165.89 rows=1850 width=24) -> Sort (cost=128.89..133.52 rows=1850 width=12) Sort Key: spj.jno, spj.qty DESC -> Seq Scan on spj (cost=0.00..28.50 rows=1850 width=16)

Базы Данных

Семинар 10

Описание семинара: MPP системы и колоночное хранение

Массивно-параллельные системы обработки (massively parallel processing, MPP).

Хотя архитектуры отдельных поставщиков могут варьироваться, массивно-параллельная обработка — наиболее развитой, проверенный и широко используемый механизм хранения и анализа больших объемов данных. Так что же собой представляет массивно-параллельная архитектура и что в ней особенного?

При использовании массивно-параллельной архитектуры данные разделяются на фрагменты, обрабатываемые независимыми центральными процессорами (CPU) и хранящиеся на разных носителях. Это похоже на загрузку разных фрагментов данных на несколько объединенных в сеть персональных компьютеров. Таким образом устраняется ограничение, обусловленное наличием одного центрального сервера с одним процессором и диском. Данные в массивно-параллельной системе распределяются по нескольким дискам, управляемым процессорами разных серверов.

В чем преимущество такой архитектуры?

Представьте себе движение по шестиполосному шоссе. Если эти шесть полос сойдутся в одну, пусть даже на коротком участке дороги, движение будет сильно затруднено. Если шесть полос остаются открытыми на всем протяжении пути от отправной точки до места назначения, то поездка будет гораздо более комфортной. В часы пик на дороге могут возникать пробки, но они будут меньше и очень скоро рассосутся. В случае с традиционной архитектурой базы данных в процессе обработки существует по крайней мере несколько точек, в которых количество полос сокращается до одной. Одной полосы может быть достаточно, только если объем движения небольшой. Именно это делает архитектуру MPP незаменимой для анализа больших объемов данных: она позволяет всем полосам оставаться открытыми на протяжении всего процесса.



Рассмотрим пример из мира баз данных. Традиционная база данных будет опрашивать терабайтную таблицу по одной строке за раз. Однако при использовании массивно-параллельной системы с 10 обрабатывающими устройствами данные разбиваются на 10 независимых фрагментов по 100 гигабайт. Это означает, что одновременно выполняется 10 запросов. При необходимости в большей вычислительной мощности и более высокой скорости просто добавьте дополнительные обрабатывающие устройства.

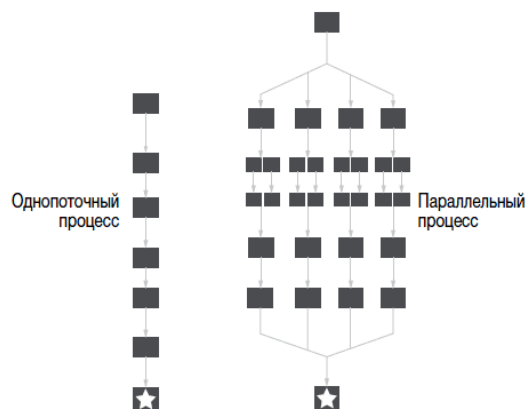
Разделяйте работу!

При использовании массивно-параллельной системы данные распределяются по разным процессорам и дискам. Подумайте о десятках или сотнях персональных компьютеров, каждый из которых содержит фрагмент большого набора данных. Это обеспечивает значительно более быстрое выполнение запроса, поскольку вместо одного большого запроса одновременно обрабатывается множество мелких независимых запросов.

Если бы система в нашем примере включала 20 обрабатывающих устройств, то вместо 10 фрагментов по 100 гигабайт одновременно обрабатывалось бы 20 независимых фрагментов по 50 гигабайт, что увеличило бы производительность. Дело несколько усложняется, когда выполнение запроса требует перемещения данных между процессорами, однако массивно-параллельные системы очень быстро справляются и с этой задачей (см. рис. 4.4).

При использовании МРР-систем данные не хранятся в одном месте, что облегчает восстановление в случае выхода оборудования из строя.

Эти системы также предусматривают инструменты управления ресурсами для управления процессорами и дисковым пространством, оптимизаторы запросов и другие средства, которые облегчают и повышают эффективность использования систем. Более подробное изложение этой темы выходит за рамки данной книги.



Массивно-параллельная система вместо использования однопоточного процесса обработки данных разделяет работу на части и позволяет различным процессорам и дискам производить обработку одновременно.

Аппаратная архитектура на примере VERTICA

Рассмотрим Vertica на уровне кластера. Эта СУБД обеспечивает массивно-параллельную обработку данных (МРР) в распределенной вычислительной архитектуре — «shared-nothing» — где, в принципе, любая нода готова подхватить функции любой другой ноды. Основные свойства:

- отсутствует единая точка отказа,
- каждый узел независим и самостоятелен,
- отсутствует единая для всей системы точка подключения,
- узлы инфраструктуры дублируются,
- данные на узлах кластера автоматически копируются.

Кластер без проблем линейно масштабируется. Мы просто ставим сервера в полку и подключаем их через графический интерфейс. Помимо серийных серверов, возможно развертывание на виртуальные машины. Что можно добиться с помощью расширения?

- Увеличения объема для новых данных
- Увеличение максимальной рабочей нагрузки
- Повышение отказоустойчивости. Чем больше нод в кластере, тем меньше вероятность выхода кластера из строя из-за отказа, а следовательно, тем ближе мы к обеспечению доступности 24/7.

Периодически ноды нужно вынимать из кластера для обслуживания. Еще довольно распространенный кейс в крупных организациях — сервера сходят с гарантии и переходят из продуктивной в какую-нибудь тестовую среду. На их место встают новые, которые находятся на гарантии производителя. По итогам всех этих операций нужно выполнять ребалансировку. Это процесс, когда данные перераспределяются между нодами — соответственно перераспределяется рабочая нагрузка. Это требовательный к ресурсам процесс, и на кластерах с большим объемом данных он может сильно снизить производительность. Чтобы этого избежать, нужно выбрать сервисное окно — время, когда нагрузка минимальна, и в этом случае пользователи этого не заметят.

Проекции

Для понимания, как хранятся данные в Vertica, требуется разобраться с одним из основных понятий — проекцией.

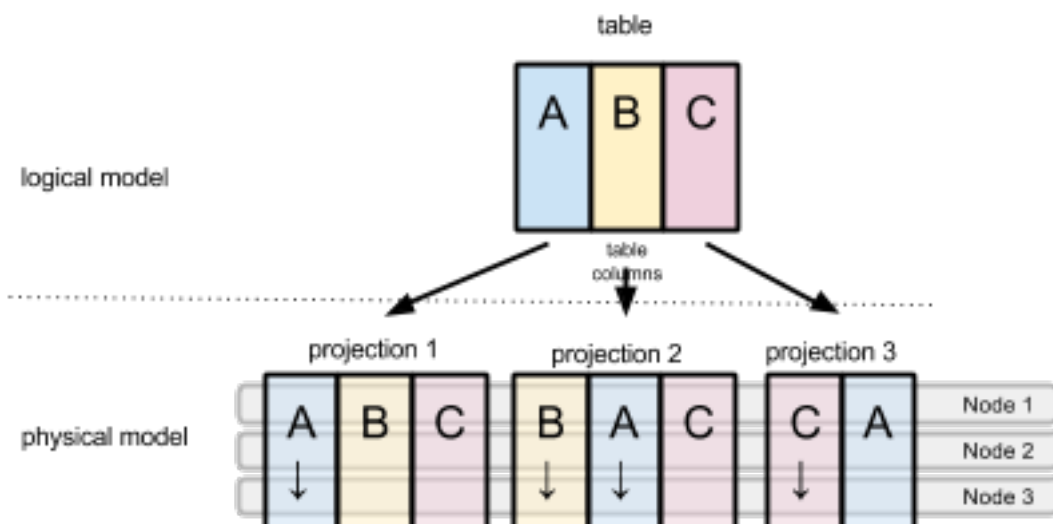
Логические единицы хранения информации — это схемы, таблицы и представления. Физические единицы — это проекции. Проекции бывают нескольких типов:

- суперпроекции (Superprojection),
- запрос-ориентированные проекции (Query-Specific Projections),
- агрегированные проекции (Aggregate Projections).

При создании любой таблицы автоматически создается суперпроекция, которая содержит все колонки нашей таблицы. Если нужно ускорить какой-то из регулярных процессов, мы можем создать специальную запрос-ориентированную проекцию, которая будет содержать, допустим, 3 столбца из 10.

Для ускорения предназначен и третий тип — агрегированные проекции. Не буду вдаваться в их подклассы — это не очень интересно. Сразу хочу предупредить, что постоянно решать свои проблемы с выполнением запросов через создание новых проекций не стоит. В конечном итоге кластер начнет тормозить.

Создавая проекции, нужно оценивать, хватает ли нашим запросам суперпроекций. Если мы все-таки хотим поэкспериментировать, добавляем строго по одной новой проекции. При возникновении проблем так будет проще найти первопричину. Для больших таблиц следует создавать сегментированную проекцию. Она разбивается на сегменты, которые распределяются по нескольким нодам, что повышает отказоустойчивость и минимизирует нагрузки на одну ноду. Если таблички маленькие, то лучше делать несегментированные проекции. Они полностью копируются на каждую ноду, и производительность таким образом увеличивается. Оговорюсь: в терминах Vertica «маленькая» таблица — это примерно 1 млн строк.

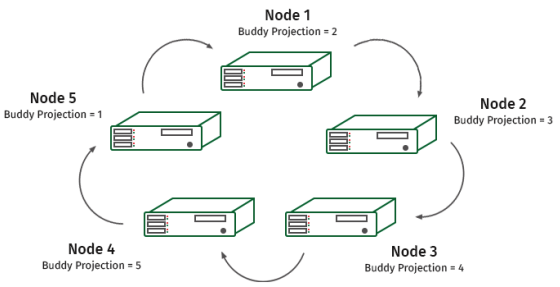
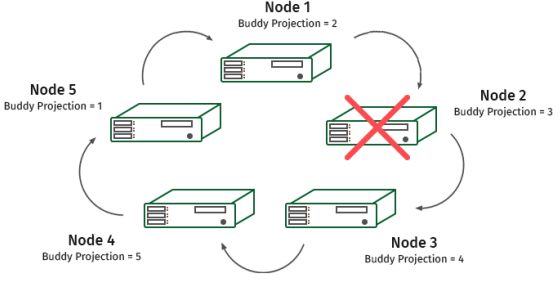
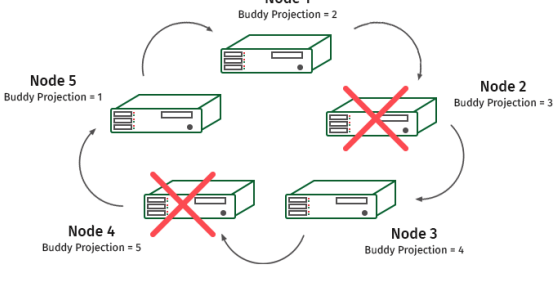
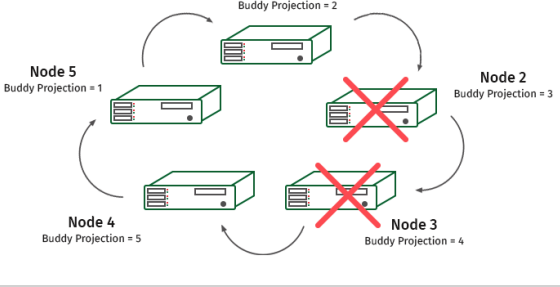


Отказоустойчивость

Отказоустойчивость в Vertica реализована при помощи механизма K-Safety. Он довольно простой с точки зрения описания, но сложный с точки зрения работы на уровне движка. Им можно управлять с помощью параметра K-Safety — он может иметь значение 0, 1 или 2. Этот параметр задает количество копий сегментированных проекционных данных.

Копии проекций называются buddy projections. Я попытался перевести это словосочетание через Яндекс-переводчик и получилось что-то вроде «проекции-кореша». Гугл предлагал варианты и интересней. Обычно данные проекции называют партнерскими или соседними, по их функциональному назначению. Это проекции, которые просто хранятся на соседних нодах и таким образом резервируются. У несегментированных проекций нет buddy projections — они копируются полностью.

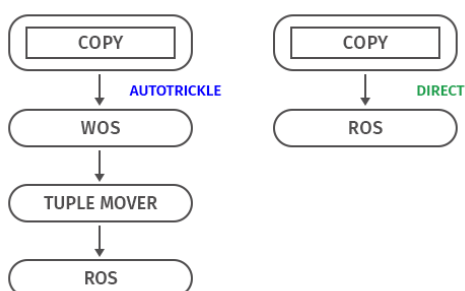
Как это работает?

Рассмотрим кластер из пяти машин. Пусть K-safety у нас равен 1. Ноды пронумерованы, а под ними написаны партнерские проекции, которые хранятся на них.	
Предположим, у нас отключилась одна нода. Что будет? Нода 1 содержит дружественную проекцию ноды 2. Поэтому на ноде 1 подрастет нагрузка, но работать кластер не перестанет.	
А теперь такая ситуация: Нода 3 содержит проекцию ноды 4, ноды 1 и 3 будут перегружены.	
Усложняем задачу. K-Safety = 2, отключим две соседние ноды. Здесь будут перегружены ноды 1 и 4 (нода 2 содержит проекцию ноды 1, а нода 3 содержит проекцию ноды 4).	

Логическое хранение данных

В Vertica есть области хранения данных, оптимизированные для записи, области, оптимизированные для чтения, и механизм Tuple Mover, который обеспечивает перетекание данных из первой области во вторую.

При использовании операции COPY, INSERT, UPDATE мы автоматически попадаем в WOS (Write Optimized Store) — область, где данные не оптимизированы для чтения и сортируются только при запросе, хранятся без сжатия или индексирования. Если объемы данных слишком велики для области WOS, то с помощью дополнительной инструкции DIRECT их стоит писать сразу в ROS. Иначе WOS будет переполнен, и у нас случится сбой.



По истечении заданного в настройках времени данные из WOS перетекают в ROS (Read Optimized Store) — оптимизированную, ориентированную на чтение структуру дискового хранилища. В ROS хранится основная часть данных, здесь она сортируется и сжимается. Данные в ROS разделены на контейнеры хранения. Контейнер представляет собой набор строк, созданных операторами трансляции (COPY DIRECT), и хранится в определенной группе файлов.

Вне зависимости от того, куда записаны данные — в WOS или в ROS — они доступны сразу. Но из WOS чтение идет медленнее, потому что данные там не сгруппированы.

Tuple Mover — это инструмент-уборщик, который выполняет две операции:

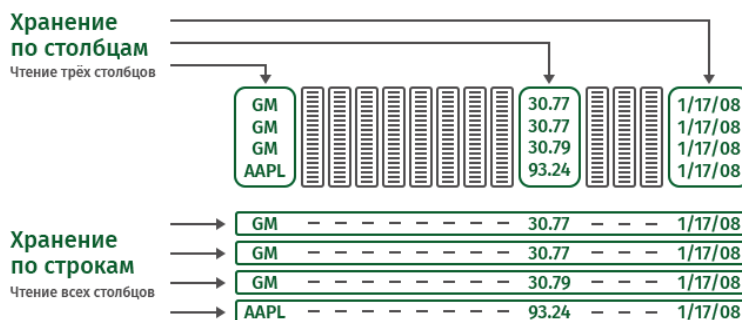
- Moveout — сжимает и сортирует данные в WOS, перемещает их в ROS и создает для них в ROS новые контейнеры.
- Mergeout — подметает за нами, когда мы используем DIRECT. Мы не всегда способны грузить столько информации, чтобы получались большие ROS-контейнеры. Поэтому он периодически объединяет небольшие контейнеры ROS в более крупные, очищает данные с пометкой на удаление, работая при этом в фоновом режиме (по времени, заданному в конфигурации).

В чем выгода колоночного хранения?

Если мы читаем строки, то, например, для выполнения команды

```
SELECT 1,11,15 FROM table1
```

нам придется читать всю таблицу. Это огромный объем информации. В данном случае колоночный подход выгоднее. Он позволяет считать только три нужных нам столбца, экономя память и время.



Что такое Parquet

Это бинарный, колоночно - ориентированный формат хранения данных, изначально созданный для экосистемы hadoop. Разработчики уверяют, что данный формат хранения идеален для big data (неизменяемых).



Parquet ведёт себя как неизменяемая таблица или БД. Значит для колонок определён тип, и если вдруг у вас комбинируется сложный тип данных (скажем, вложенный json) с простым (обычное строковое значение), то вся система разрушится. Например, возьмём два события и напомним их в формате Json:

```
{
  "event_name": "event 1",
  "value": "this is first value",
}

{
  "event_name": "event 2",
  "value": {"reason": "Ok"}
}
```

В parquet-файл записать их не получится, так как в первом случае у вас строка, а во втором сложный тип. Хуже, если система записывает входной поток данных в файл, скажем, каждый час. В первый час могут прийти события со строковыми value, а во второй — в виде сложного типа. В итоге, конечно, получится записать parquet файлы, но при операции merge schema вы наткнётесь на ошибку несовместимости типов.

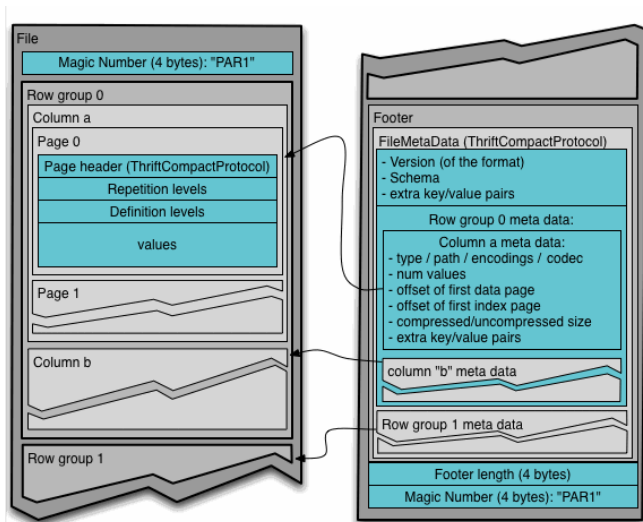
Как выглядит структура Parquet файлов

Parquet является довольно сложным форматом. Примечательно, что свои корни этот формат пустил даже в разработки Google, а именно в их проект под названием Dremel — об этом уже упоминалось на Хабре. Если коротко, Parquet использует архитектуру, основанную на “уровнях определения” (definition levels) и “уровнях повторения” (repetition levels), что позволяет довольно эффективно кодировать данные, а информация о схеме выносится в отдельные метаданные. При этом оптимально хранятся и пустые значения.

Файлы имеют несколько уровней разбиения на части, благодаря чему возможно довольно эффективное параллельное выполнение операций поверх них:

- **Row-group** — это разбиение, позволяющее параллельно работать с данными на уровне Map-Reduce
- **Column chunk** — разбиение на уровне колонок, позволяющее распределять IO операции
- **Page** — Разбиение колонок на страницы, позволяющее распределять работу по кодированию и сжатию

Если сохранить данные в parquet файл на диск, используя самую привычную нам файловую систему, вы обнаружите, что вместо файла создаётся директория, в которой содержится целая коллекция файлов. Часть из них — это метainформация, в ней — схема, а также различная служебная информация, включая частичный индекс, позволяющий считывать только необходимые блоки данных при запросе. Остальные части, или партиции, это и есть наши Row group.



Для интуитивного понимания будем считать **Row groups** набором файлов, объединённых общей информацией. Кстати, это разбиение используется HDFS для реализации data locality, когда каждая нода в кластере может считывать те данные, которые непосредственно расположены у неё на диске. Более того, row group выступает единицей Map Reduce, и каждая map-reduce задача в Spark работает со своей row-group. Поэтому worker обязан поместить группу строк в память, и при настройке размера группы надо учитывать минимальный объём памяти, выделяемый на задачу на самой слабой ноде, иначе можно наткнуться на OOM.

Column chunk (разбиение на уровне колонок) — оптимизирует работу с диском (дисками). Если представить данные как таблицу, то они записываются не построчно, а по колонкам.

Представим таблицу:

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

Тогда в текстовом файле, скажем, csv мы бы хранили данные на диске примерно так:

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

В случае с Parquet:

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

Благодаря этому мы можем считывать только необходимые нам колонки.

Из всего многообразия колонок на деле аналитику в конкретный момент нужны лишь несколько, к тому же большинство колонок остается пустыми. Parquet в разы ускоряет

процесс работы с данными, более того — подобное структурирование информации упрощает сжатие и кодирование данных за счёт их однородности и похожести.

Каждая колонка делится на страницы (**Pages**), которые, в свою очередь, содержат метайнформацию и данные, закодированные по принципу архитектуры из проекта Dremel. За счёт этого достигается довольно эффективное и быстрое кодирование. Кроме того, на данном уровне производится сжатие (если оно настроено). На данный момент доступны кодеки *snappy*, *gzip*, *lzo*.

Есть ли подводные камни?

За счёт “паркетной” организации данных сложно настроить их стриминг — если передавать данные, то полностью всё группу. Также, если вы утратили метайнформацию или изменили контрольную сумму для Страницы данных, то вся страница будет потеряна (если для Column chunk — то chunk потерян, аналогично для row group). На каждом из уровней разбиения строятся контрольные суммы, так что можно отключить их вычисления на уровне файловой системы для улучшения производительности.

Достоинства хранения данных в Parquet:

- Несмотря на то, что они и созданы для hdfs, данные могут храниться и в других файловых системах, таких как GlusterFs или поверх NFS
- По сути это просто файлы, а значит с ними легко работать, перемещать, бэкапить и реплицировать.
- Колончатый вид позволяет значительно ускорить работу аналитика, если ему не нужны все колонки сразу.
- Нативная поддержка в Spark из коробки обеспечивает возможность просто взять и сохранить файл в любимое хранилище.
- Эффективное хранение с точки зрения занимаемого места.
- Как показывает практика, именно этот способ обеспечивает самую быструю работу на чтение по сравнению с использованием других файловых форматов.

Недостатки:

- Колончатый вид заставляет задумываться о схеме и типах данных.
- Кроме как в Spark, Parquet не всегда обладает нативной поддержкой в других продуктах.
- Не поддерживает изменение данных и эволюцию схемы. Конечно, Spark умеет мерджить схему, если у вас она меняется со временем (для этого надо указать специальную опцию при чтении), но, чтобы что-то изменить в уже существующим файле, нельзя обойтись без перезаписи, разве что можно добавить новую колонку.
- Не поддерживаются транзакции, так как это обычные файлы а не БД.

Базы Данных

Семинар 11

Описание семинара: ETL и ELT

Kimball vs. Inmon

Два пионеры хранилищ данных: Билл Инмон и Ральф Кимбалл предлагают разные подходы к проектированию.

Подход **Ральфа Кимбалла** основывается на важности витрин данных, которые являются хранилищами данных, принадлежащих конкретным направлениям бизнеса. Хранилище данных — это просто **сочетание различных витрин данных**, которые облегчают отчетность и анализ. Проект хранилища данных по принципу Кимбалла использует подход «снизу вверх».

Подход **Билла Инмона** основывается на том, что хранилище данных является централизованным хранилищем всех корпоративных данных. При таком подходе организация сначала создает **нормализованную модель** хранилища данных. Затем создаются витрины размерных данных на основе модели хранилища. Это известно как нисходящий подход к хранилищу данных.

ETL – аббревиатура от Extract, Transform, Load.

Это системы корпоративного класса, которые применяются, чтобы привести к одним справочникам и загрузить в DWH и EPM данные из нескольких разных учетных систем. ETL – промежуточным слоем между OLTP системами и OLAP системой или корпоративным хранилищем. Хотя в принципе существуют ETL, который можно поставить между любыми системами, лучше интеграцию между учетными системами решать связкой MDM и ESB. Если же вам для интеграции двух зависимых учетных систем необходим функционал ETL, то это ошибка проектирования, которую надо исправлять доработкой этих систем.

Зачем нужна ETL система

Проблема, из-за которой в принципе родилась необходимость использовать решения ETL, заключается в потребностях бизнеса в получении достоверной отчетности из того бардака, который творится в данных любой ERP-системы. Этот бардак есть всегда, он бывает двух видов:

- Как случайные ошибки, возникшие на уровне ввода, переноса данных, или из-за багов;
- Как различия в справочниках и детализации данных между смежными ИТ-системами.

При этом если первый вид бардака побороть можно, то второй вид по большей части не является ошибкой – контролируемые различия в структуре данных, это нормальная оптимизация под цели конкретной системы.

Из-за этой особенности ETL-системы должны в идеале решать не одну, а две задачи:

- Привести все данные к единой системе значений и детализации, попутно обеспечив их качество и надежность;

- Обеспечить аудиторский след при преобразовании (Transform) данных, чтобы после преобразования можно было понять, из каких именно исходных данных и сумм собралась каждая строчка преобразованных данных.

Помнить об этих двух задачах бывает очень полезно, особенно если вы пишете ETL- процесс вручную, или делаете его с использованием фреймворков низкой готовности, в которых не задана готовая структура промежуточных таблиц. Легко упустить вторую задачу и иметь много проблем с поиском причин ошибок в трансформированных данных.

Как работает ETL система

Все основные функции ETL системы умещаются в следующий процесс:

В разрезе потока данных это несколько систем-источников (обычно OLTP) и система приемник (обычно OLAP), а так же пять стадий преобразования между ними:



Процесс загрузки – Его задача затянуть в ETL данные произвольного качества для дальнейшей обработки, на этом этапе важно сверить суммы пришедших строк, если в исходной системе больше строк, чем в RawData то значит — загрузка прошла с ошибкой;

Процесс валидации данных – на этом этапе данные последовательно проверяются на корректность и полноту, составляется отчет об ошибках для исправления;

Процесс мэппинга данных с целевой моделью – на этом этапе к валидированной таблице пристраивается еще n-столбцов по количеству справочников целевой модели данных, а потом по таблицам мэппингов в каждой пристроенной ячейке, в каждой строке проставляются значения целевых справочников. Значения могут проставляться как 1:1, так и *:1, так и 1:* и *:*, для настройки последних двух вариантов используют формулы и скрипты мэппинга, реализованные в ETL-инструменте;

Процесс агрегации данных – этот процесс нужен из-за разности детализации данных в OLTP и OLAP системах. OLAP-системы — это, по сути, полностью денормализованная таблица фактов и окружающие ее таблицы справочников (звездочка/снежинка), максимальная детализация сумм OLAP – это количество перестановок всех элементов всех справочников. А OLTP система может содержать несколько сумм для одного и того же набора элементов справочников. Можно было-бы убивать OLTP-детализацию еще на входе в ETL, но тогда мы потеряли бы «аудиторский след». Этот след нужен для построения Drill-down отчета, который показывает — из каких строк OLTP, сформировалась сумма в ячейке OLAP- системы. Поэтому сначала делается мэппинг на детализации OLTP, а потом в отдельной таблице данные «схлопывают» для загрузки в OLAP;

Выгрузка в целевую систему — это технический процесс использования коннектора и передачи данных в целевую систему.

Особенности архитектуры

Реализация процессов 4 и 5 с точки зрения архитектуры тривиальна, все сложности имеют технический характер, а вот реализация процессов 1, 2 и 3 требует дополнительного пояснения.

Процесс загрузки

При проектировании процесса загрузки данных необходимо помнить о том что:

- Надо учитывать требования бизнеса по длительности всего процесса. Например: Если данные должны быть загружены в течение недели с момента готовности в исходных системах, и происходит 40 итераций загрузки до получения нормального качества, то длительность загрузки пакета не может быть больше 1-го часа. (При этом если в среднем происходит не более 40 загрузок, то процесс загрузки не может быть больше 30 минут, потому что в половине случаев будет больше 40 итераций, ну или точнее надо считать вероятности:)) Главное если вы не укладываетесь в свой расчет, то не надейтесь на чудо — снесите и все, делать заново т.к. вы не впишитесь;
- Данные могут загружаться набегающей волной — с последовательным обновлением данных одного и того-же периода в будущем в течение нескольких последовательных периодов. (например: обновление прогноза окончания года каждый месяц). Поэтому кроме справочника «Период», должен быть предусмотрен технический справочник «Период загрузки», который позволит изолировать процессы загрузки данных в разных периодах и не потерять историю изменения цифр;
- Данные имеют обыкновение быть перегружаемыми много раз, и хорошо если будет технический справочник «Версия» как минимум с двумя элементами «Рабочая» и «Финальная», для отделения вычищенных данных. Кроме-того создание персональных версий, одной суммарной и одной финальной позволяет хорошо контролировать загрузку в несколько потоков;
- Данные всегда содержат ошибки: Перезагружать весь пакет в [50GB -> +8] это очень не экономно по ресурсам и вы, скорее всего, не впишитесь в

регламент, следовательно, надо грамотно делить загружаемый пакет файлов и так проектировать систему, чтобы она позволяла обновлять пакет по маленьким частям. По моему опыту лучший способ – техническая аналитика «файл-источник», и интерфейс, который позволяет снести все данные только из одного файла, и вставить вместо него обновленные. А сам пакет разумно делить на файлы по количеству исполнителей, ответственных за их заполнение (либо админы систем готовящие выгрузки, либо пользователи заполняющие вручную);

- При проектировании разделения пакета на части надо еще учитывать возможность так-называемого «обогащения» данных (например: Когда 12 января считают налоги прошлого года по правилам управленческого учета, а в марте-апреле перегружают суммы на посчитанные по бухгалтерскому), это решается с одной стороны правильным проектированием деления пакета данных на части так, чтобы для обогащения надо было перегрузить целое количество файлов (не 2,345 файла), а с другой стороны введением еще одного технического справочника с периодами обогащения, чтобы не потерять историю изменений по этим причинам).

Процесс валидации

Данный процесс отвечает за выявление ошибок и пробелов в данных, переданных в ETL. Само программирование или настройка формул проверки не вызывает вопросов, главный вопрос – как вычислить возможные виды ошибок в данных, и по каким признакам их идентифицировать? Возможные виды ошибок в данных зависят от того какого рода шкалы применимы для этих данных. (Ссылка на прекрасный пост, объясняющий, какие существуют виды шкал — <http://habrahabr.ru/post/246983/>).

Ближе к практике в каждом из передаваемых типов данных в 95% случаев возможны следующие ошибки:

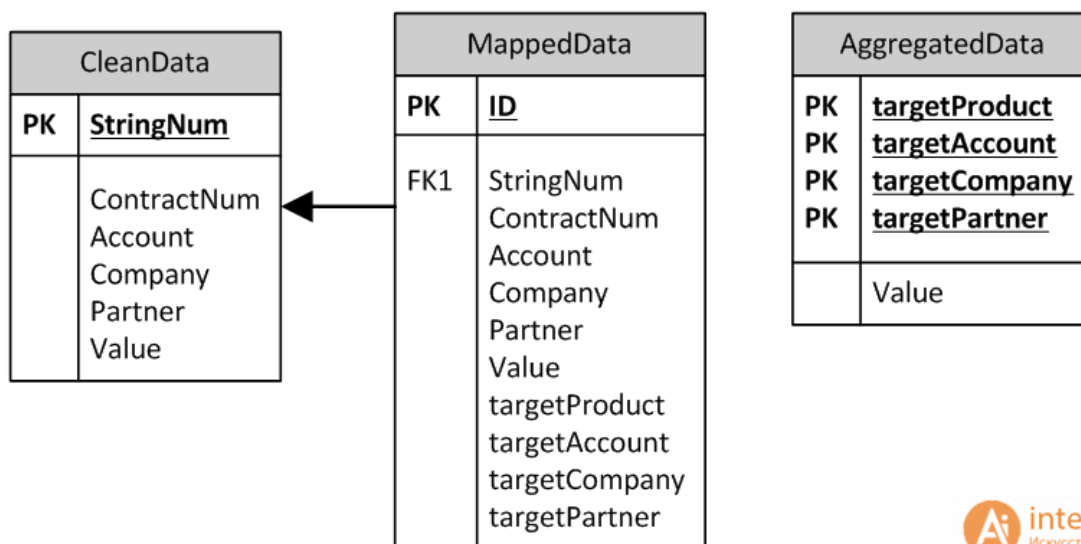
Типы данных	Внутри поля	По отношению к другим полям	Совместимость форматов при передаче между системами
Перечисления и текст	• Не из списка разрешенных значений • Отсутствие обязательных значений • Не соответствие формату (Все договора должны нумероваться «ДГВxxxx..»)	• Не из списка разрешенных значений для связанного элемента • Отсутствие обязательных элементов для связанного элемента • Не соответствие формату для связанного элемента (например: для продукта «АИС» все договора должны нумероваться «АИСxxxx..»)	• Символы допустимые в одном формате, недопустимы в другом • Кодировка • Обратная совместимость (Элемент справочника был изменен в целевой системе без добавления мэппинга) • Новые значения (нет мэппинга) • Устаревшие значения (не из списка разрешенных в целевой системе)
Числа	• Не число • Не в границах разрешенного интервала значений • Пропущено порядковое значение (например: данные не дошли)	• Не выполняется отношение $y=ax+b$ (например: НДС и Выручка, или Встречные суммы равны) • Элементу «А» присвоен неправильный порядковый номер • Разницы за счет разных правил округления значений (например: в 1С и SAP никогда не сходятся рассчитанный НДС)	• Переполнение • Потеря точности и знаков • Несовместимость форматов при конвертации в не число
Даты		• День недели не соответствует дате • Сумма единиц времени не соответствует из-за разницы рабочие/не рабочие/праздничные/сокращенные дни	• Несовместимость формата даты при передаче текстом (например: ISO 8601 в UnixTime, или разные форматы в ISO 8601) • Ошибка точки отсчета и точности при передаче числом (например: TimeStamp в Date Time)

Соответственно проверки на ошибки реализуются либо формулами, либо скриптами в редакторе конкретного ETL-инструмента. А если вообще по большому счету, то большая часть ваших валидаций будет на соответствие справочников, а это [select * from a where a.field not in (select...)] При этом для сохранения аудиторского следа разумно сохранять в системе две отдельные таблицы – rawdata и cleandata с поддержкой связи 1:1 между строками.

Процесс мэппинга

Процесс мэппинга так же реализуется с помощью соответствующих формул и скриптов, есть три хороших правила при его проектировании:

- Таблица замэпленных данных должна включать одновременно два набора полей – старых и новых аналитик, чтобы можно было сделать select по исходным аналитикам и посмотреть, какие целевые аналитики им присвоены, и наоборот:



- Таблица замэпленных элементов должна иметь отдельное PK-поле, чтобы при связи 1:* и *:1 можно было создать много строк в MappedData для одной строки в CleanData и сохранить аудиторский след
- Таблица MappedData должна быть отдельной от CleanData по тем же причинам что и пункт 2

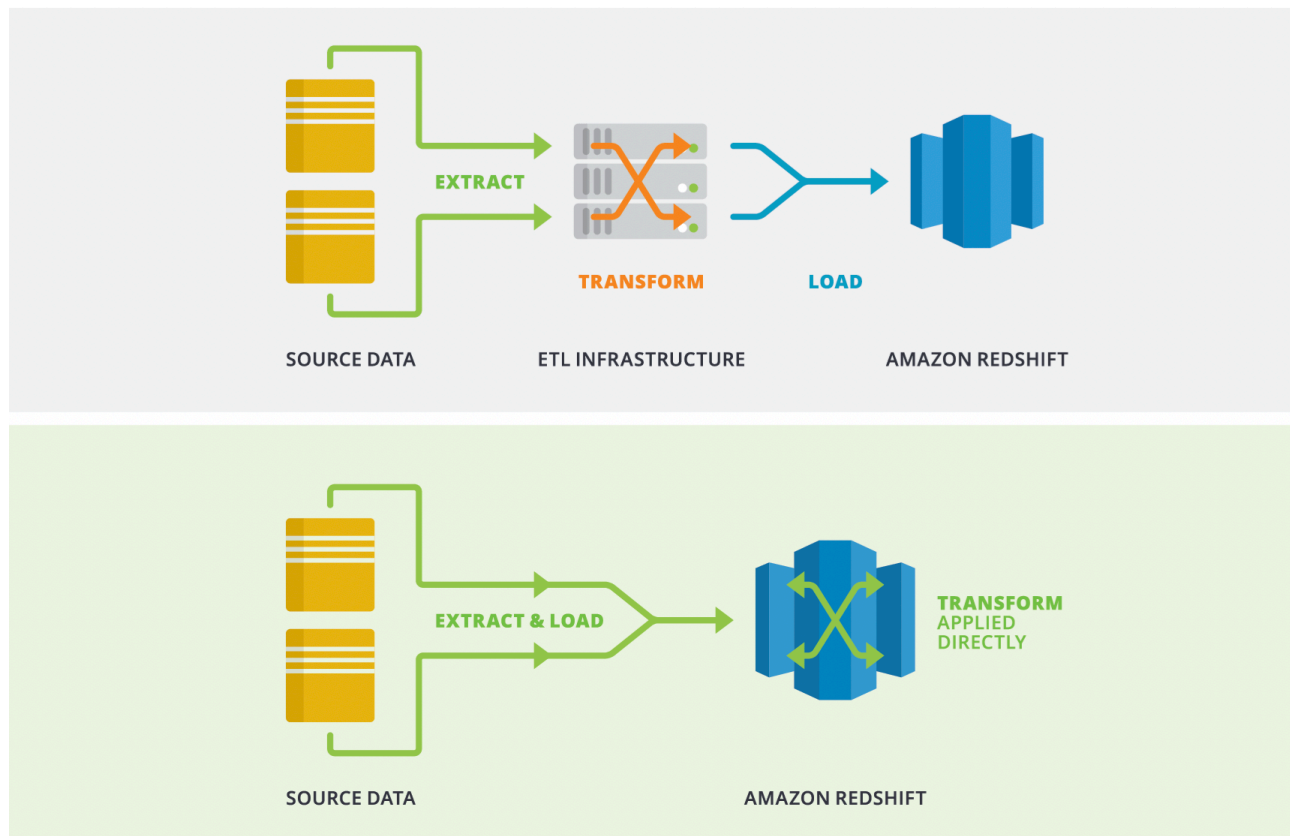
Заключение

В принципе это все архитектурные приемы, которые мне понравились в тех ETL инструментах, которыми я пользовался.

Кроме этого конечно в реальных системах есть еще сервисные процессы — авторизации, разграничения доступа к данным, автоматизированного согласования изменений, и все решения конечно являются компромиссом с требованиями производительности и предельным объемом данных.

Основное различие между ETL и ELT, где происходит обработка

Обработка ETL данных происходит в инструменте ETL (как правило, запись по времени и в памяти) Обработка ELT данных происходит в двигателе базы данных



Данные одинаковы, и конечные результаты данных могут быть достигнуты в обоих методах. Это очень зависит от вас и вашей среды. Если у вас сильный движок базы данных и хорошее аппаратное обеспечение, и вы можете выполнять тяжелую обработку, ELT хорош для вас. Если у вас есть загруженный механизм datawarehouse, и вам нужно освободить его от обработки, перейдите на ETL.

Обратите внимание, что наличие инструмента ETL дает вам обе опции, такие как ETL (T), вы можете выполнить преобразование в инструменте ETL, и вы также можете сделать преобразование в движке базы данных, но ELT у вас есть только возможность преобразования в движке базы данных, но вы должны знать, что базы данных лучше работают на основе набора, чем инструменты ETL по времени.

Базы Данных

Семинар 13

Описание семинара: In-Memory DataBase

In-Memory DataBase

Объемы баз данных и сложность запросов к ним всегда росли быстрее, чем скорость их обработки. Поэтому лучшие умы человечества много лет думали о том, что произойдет, когда оперативной памяти станет столько, что можно будет всю базу данных взять и поместить в кэш оперативной памяти.

В последние годы логический момент для этого, казалось бы, настал. Стоимость оперативной памяти падала, падала, и упала совсем. Еще в начале века казалось, что 256 МБ памяти для сервера — это нормально, и даже много. Сегодня нас не удивит параметром 256 ГБ оперативной памяти на сервере начального уровня, а с промышленными серверами вообще настал полный коммунизм, любой благородный дон может набрать хоть терабайт оперативной памяти на сервере, если захочет.

Но дело не только в этом — появились новые технологии индексирования данных, новые технологии компрессии данных — OLTP-компрессия, компрессия неструктурированных данных (LOB Compression). В Oracle Database 11g, например, появилась технология Result Cache, которая позволяет кэшировать не просто строки таблиц или индексы, но и сами результаты запросов и подзапросов.

То есть, с одной стороны, наконец-то можно использовать оперативную память по ее прямому назначению, но с другой стороны — не так все просто. Чем больше кэш, тем больше накладные расходы на его сопровождение, включая процессорное время. Вы ставите больше памяти, увеличиваете объем кэша, а система работает медленнее, и это, в общем-то, логично, потому что алгоритмы управления памятью, разработанные в Раннем средневековье нашими прапрадедушками, попросту не годятся для эпохи Возрождения, и все тут. Что же делать?

Давайте вспомним о том, что существует, по сути дела, две категории баз данных: строчные базы данных, которые и в буферном кэше в оперативной памяти, и на диске хранят информацию в строчном виде — Oracle Database, Microsoft SQL Server, IBM DB/2, MySQL и т.д.; и колоночные СУБД, в которых информация хранится по столбцам, и которые большого распространения в индустрии, к сожалению, не нашли. Строчные базы данных хорошо обрабатывают OLTP-операции, а вот для обработки аналитики больше подходят, вы будете смеяться, колоночные базы данных — зато DML-операции для них проблема, ну вы поняли, почему. Промышленность, как вы знаете, пошла по пути строчных баз данных, на которые в виде компромисса навешиваются аналитические возможности.

Как это обычно бывает, у термина In-Memory DataBase нет устойчивого русского перевода, это что-то вроде «базы данных в оперативной памяти».

Мы все помним известное сокращение ACID, обозначающее требования к транзакционной системе: атомарность, согласованность, изолированность, надежность. Но в мире больших данных, где постоянно растут требования к объему и скорости обрабатываемых данных, часто жертвуют одним из этих требований, чтобы обеспечить необходимую производительность. In-memory базы данных — типичный пример такого подхода.

Базы данных in-memory хранятся в оперативной памяти сервера. Очевидно, что это обеспечивает большие преимущества в скорости, поскольку операции с данными в памяти выполняются меньшим числом инструкций процессора, а время доступа к данным операциями типа seek вообще несравнимо — оперативная память глобально выигрывает у жесткого диска.

В терминах ACID базы данных in-memory конечно жертвуют надежностью, ведь при обесточивании или перезагрузке в них пропадают все данные. Существуют ряд методов, которые позволяют снизить этот риск:

- **Снэпшоты** (снимки) базы данных в разные моменты времени – фактически эквивалентны бэкапу базы данных на жесткий диск. Недостатком метода является гарантированная потеря части «свежих данных» при каких-то проблемах.
- **Логи транзакций** записывают все транзакции на жесткий диск и позволяют восстановить данные после перезагрузки.
- **Использование NVRAM или NVDIMM** – модулей памяти, которые сохраняют свое состояние при пропадании питания, хотя конечно NVDIMM и технология Flash-памяти – это уже какое-то гибридное и безусловно более медленное решение.

Redis

Redis – быстрое хранилище в памяти с открытым исходным кодом для структур данных «ключ-значение». Redis поставляется с набором разнообразных структур данных в памяти, что упрощает создание различных специальных приложений.

Благодаря высокой скорости и простоте Redis часто используется для мобильных и интернет-приложений, игр, рекламных платформ, «Интернета вещей», т. е. в тех случаях, когда необходима максимально возможная производительность.

Несколько фактов о ключах:

- Ключи в Redis — бинарно-безопасные (**binary safe**) строки.
- Слишком длинные ключи — плохая идея, не только из-за занимаемой памяти, но так же и в связи с увеличением времени поиска определенного ключа в множестве в связи с дорогостоящим сравнением.
- Хорошая идея — придерживаться схемы при построении ключей: «object-type:id:field».

Типы данных Redis

- **Строки** (strings). Базовый тип данных Redis. Строки в Redis бинарно-безопасны, *могут использоваться так же как числа*, ограничены размером 512 Мб.
- **Списки** (lists). Классические списки строк, упорядоченные в порядке вставки, которая возможна как со стороны головы, так и со стороны хвоста списка. Максимальное количество элементов — 232 — 1.
- **Множества** (sets). Множества строк в математическом понимании: не упорядочены, поддерживают операции вставки, проверки вхождения элемента, пересечения и разницы множеств. Максимальное количество элементов — 232 — 1.
- **Хеш-таблицы** (hashes). Классические хеш-таблицы или ассоциативные массивы. Максимальное количество пар «ключ-значение» — 232 — 1.

- **Упорядоченные множества** (sorted sets). Упорядоченное множество отличается от обычного тем, что его элементы упорядочены по особому параметру «score».

Топология Redis

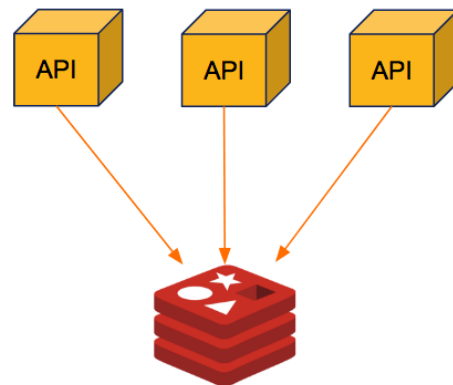
Один Redis-инстанс

Самый классический случай.

Простой в запуске и настройке.

Ограничен ресурсами хоста, на котором запущен — его процессором и памятью.

В случае выхода из строя хоста или самого сервера Redis — разумеется, упадут все зависящие от него службы, т.е. никакого обеспечения надёжности нет.



Master-slave репликация

Один мастер, к которому подключены несколько слейвов, на которые выполняется полная репликация всех данных.

Данные обновляются на мастере, после чего он отправляет информацию об обновлениях на слейвы.

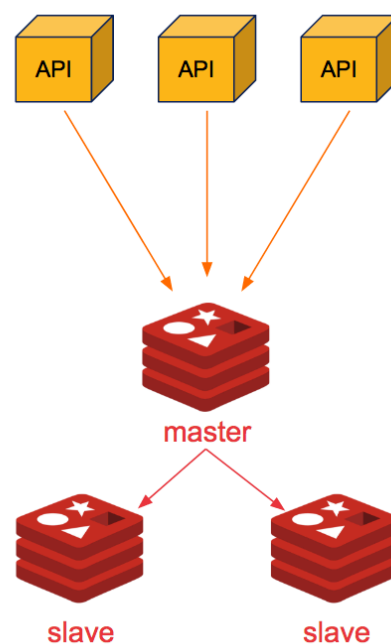
Слейвы общаются только с мастером (но могут иметь собственные слейвы).

Слейвы являются read-only нодами — никаких изменений на них не выполняется (если явно не задано другое, см. во второй части).

В случае выхода из строя одной из нод — все данные останутся доступными с других улов.

Прост в настройке, но операции записи ограничены доступными мастеру ресурсами.

В случае выхода из строя мастера — требует ручного вмешательства для переопределения нового мастера из одного из слейвов, кроме того — клиенты должны точно знать на какую ноду выполнять запросы чтения и записи.



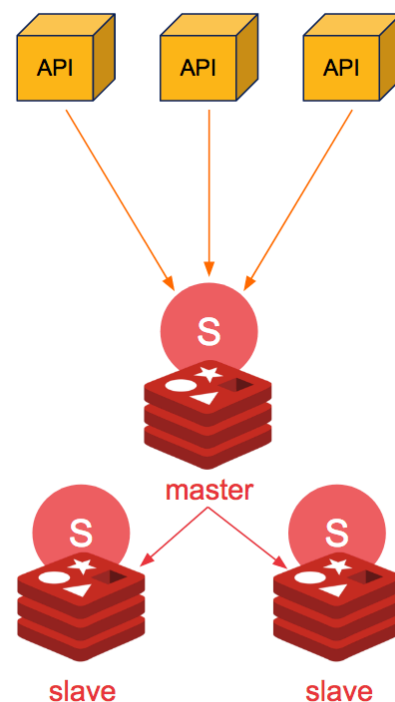
Redis Sentinel

Уже описывался выше, но ещё несколько слов.

Как и в случае с репликацией Redis — имеет одну Master Sentinel ноду, которая имеет приоритет при принятии решения о том, какую ноду Redis использовать в роли Master.

Т.е, в случае с одним Мастером Redis и двумя слейвами, и если Master Sentinel работает на том же хосте, где Мастер Redis-a, и этот хост умирает — Sentinel определяет свой новый мастер-инстанс, затем два оставшихся Sentinel-инстанса принимают решение о том, какой из оставшихся двух Redis-нод станет новым мастером. При этом у мастер-инстанса Sentinel будет приоритет при принятии решения.

Стоит учитывать, что не все клиенты умеют работать с моделью Redis Sentinel, список клиентов — тут>>>.



Redis Cluster

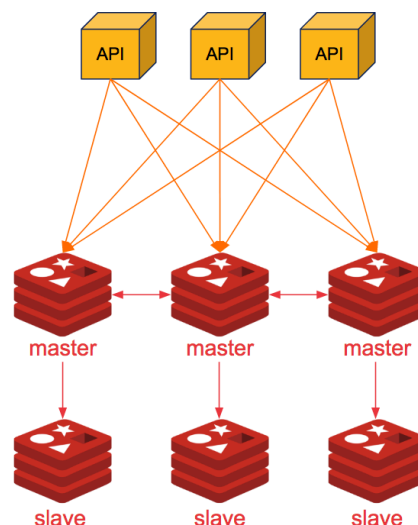
И самый полнофункциональный вариант — Redis Cluster.

Несколько мастер-инстансов, у каждого один или более слейвов (до 1000).

Выполняет все задачи по шардингу, репликации, failover, синхронизации данных.

Требует как минимум 6 нод Redis-a: три для мастеров, и три для слейвов.

Умеет перенаправлять запросы от клиентов на нужный мастер или слейв — но это требует поддержки кластера самими клиентами Redis.



Какие есть преимущества:

- **Высочайшая производительность.** Все данные Redis находятся в оперативной памяти сервера, в отличие от большинства систем управления базами данных, в которых данные хранятся на жестких дисках или твердотельных накопителях (SSD). Размещаемые в памяти базы данных, такие как Redis, не требуют доступа к дисковым накопителям, что позволяет избежать потерь времени на поиск. Доступ к данным можно организовать при помощи более простых алгоритмов, в которых используется меньшее количество инструкций ЦПУ. Для выполнения типовых операций требуется менее миллисекунды.
- **Структуры данных в памяти.** Redis позволяет пользователям хранить ключи, привязанные к различным типам данных. Основной тип данных — это строка, которая может состоять из текстовых или двоичных данных размером до 512 МБ. Redis также поддерживает списки строк (List of Strings), упорядоченные в порядке вставки; множества неупорядоченных

строк (Sets of unordered Strings); упорядоченные по результату множества (Sorted Sets); хеш-таблицы (Hashes), содержащие список полей и значений, и тип данных HyperLogLogs для подсчета уникальных элементов в наборе данных. С помощью Redis в памяти можно хранить практически любые типы данных.

- **Универсальность и простота использования.** Redis поставляется с несколькими инструментами, ускоряющими и облегчающими разработку и эксплуатацию, включая шаблон «издатель-подписчик», для публикации сообщений в логических каналах, доставляемых подписчикам (хорошо подходит для чатов и систем обмена сообщениями); TTL: ключи могут иметь определенное время жизни (TTL), после чего они самостоятельно уничтожаются, это используется, чтобы избежать заполнения базы данных ненужными данными; атомарные счетчики, которые обеспечивают непротиворечивость результатов в ситуации состязания; а также Lua, мощный, но несложный язык скриптов.
- **Репликация и сохранность.** В Redis применяется архитектура «ведущий–подчиненный» и поддерживается асинхронная репликация, при которой данные могут реплицироваться на несколько подчиненных серверов. Это обеспечивает как улучшенные характеристики чтения (так как запросы могут быть распределены между серверами), так и восстановление при отключении основного сервера. Для обеспечения надежности Redis поддерживает как снимки состояния на момент времени (копирование наборов данных Redis на диски), так и создание файла только для добавления (AOF) для последовательной записи на диск всех изменений данных. Любой из этих методов обеспечивает быстрое восстановление данных Redis в случае сбоя.
- **Поддержка удобного языка разработки.** Для разработчиков под Redis доступны более ста клиентов с открытым исходным кодом. Поддерживаемые языки программирования включают Java, Python, PHP, C, C++, C#, JavaScript, Node.js, Ruby, R, Go и многие другие.

Примеры использования Redis:

- **Кэширование.** Если Redis поместить «перед» другой базой данных, создается высокопроизводительный кэш в памяти, который уменьшает задержку доступа, увеличивает пропускную способность и уменьшает нагрузку на реляционную базу данных или базу данных NoSQL.
- **Управление сессиями.** Redis отлично подходит для задач управления сессиями. Можно просто использовать Redis как быстрое хранилище пар «ключ-значение» с соответствующим временем жизни (TTL) ключей сессии для управления информацией сессии. Управление сессиями обычно требуется для интернет-приложений, включая игры, сайты электронной коммерции и платформы социальных сетей.
- **Таблицы лидеров в режиме реального времени.** С помощью структуры данных Redis Sorted Set элементы хранятся в списке, отсортированном по их результатам. Это позволяет легко создавать динамические таблицы лидеров, показывающие, кто побеждает в игре, или размещать наиболее понравившиеся сообщения, или использовать структуру в других случаях, когда необходимо показать, кто является лидером.
- **Ограничение интенсивности.** Redis может измерять и при необходимости ограничивать скорость наступления событий. Используя счетчик Redis, связанный с ключом API клиента, можно подсчитать количество запросов на доступ за определенный период времени и принять меры при превышении лимитов. Ограничители интенсивности обычно используются для установки лимита по количеству публикаций на форуме, лимитирования использования ресурсов и уменьшения влияния спамеров.

- **Очереди.** Структура данных список Redis позволяет легко создавать упрощенные постоянные очереди. Списки Redis List обеспечивают выполнение элементарных операций, а также возможности блокировки, поэтому они подходят для различных приложений, в которых требуется надежный брокер сообщений или циклический список.
- **Чат и обмен сообщениями.** Redis поддерживает стандарт «издатель-подписчик», а также сопоставления с шаблоном. Это позволяет использовать Redis для создания высокопроизводительных комнат чата, лент комментариев, работающих в режиме реального времени, и систем взаимодействия серверов. Стандарт «издатель-подписчик» также можно использовать для запуска действий на основе опубликованных событий.

Заключение

Резидентные СУБД отлично подходят для оперативной аналитики больших данных в режиме онлайн. Из-за их архитектурных особенностей, когда данные для распределенных вычислений хранятся в оперативной памяти кластерных узлов, а не считываются с жесткого диска, ключевыми преимуществами In-Memory СУБД считаются следующие:

- быстрая выполнения операций;
- эффективное сохранение зафиксированных данных, которые используются не часто, на жестком диске;
- высокая пропускная способность систем, критичных к производительности.

Обратной стороной этих достоинств являются следующие недостатки:

- однопоточность и эффективная утилизация только одного ядра ЦП, что не позволяет в полной мере воспользоваться возможностями современных многоядерных серверов. Поэтому при разработке решения на базе IMDB следует проектировать его с перспективой распараллеливания на десятки экземпляров (инстансов), что эффективно утилизировать hardware;
- энергозависимость и привязка к размеру оперативной памяти. Для решения этой проблемы используется запись с предварительным журналированием на энергонезависимом устройстве для обеспечения целостности базы данных при внезапной перезагрузке.

Поскольку существуют надежные и достаточно простые способы устранения указанных недостатков, неудивительно, что резидентные СУБД, такие как Tarantool, Arenadata Grid, Redis, Apache Ignite, InfinityDB, Memcached и прочие коммерческие и open-source продукты, очень востребованы в современных Big Data системах.

Базы Данных

Семинар 14

Описание семинара: Redis. Практика

С одной стороны есть традиционные ACID реляционные базы данных такие как MySQL, PostgreSQL, Oracle и др. Они надежны и стабильны. Сама аббревиатура ACID описывает требования к транзакционной системе (Atomicity — Атомарность, Consistency — Согласованность, Isolation — Изолированность, Durability — Долговечность). Их основная задача не просто хранить данные, а хранить с максимальной надежностью. Но их основной недостаток, они очень медленные.

С другой стороны есть очень быстрые хранилища в памяти типа ключ-значение, например memcached. Они очень быстрые за счет предельной простоты и отказа от надежности. Несложно привести несколько примеров задач где нужна производительность даже за счет надежности:

- система мониторинга с n-ым количеством датчиков, которые должны постоянно отправлять данные.
 - система логирования действий пользователя на каком нибудь сайте
 - контроль трафика сети в реалтайм
 - хранение сессий пользователей
- и т.д.

Redis относится к третьему типу хранилищ. Когда нужна быстрая обработка данных и при этом сохранялось бы определенный уровень надежности и возможности масштабирования.

Redis (REmote DIctionary Server) — это не реляционная структура данных в памяти, используемая в качестве базы данных. Данные хранятся в виде пары ключ-значение. И при этом хранилище умеет масштабироваться путем репликации между серверами. Redis сохраняет все данные в памяти, что позволяет сделать доступ к данным максимально быстрым по сравнению с другими базами данных. Почему Redis известен своей исключительной высокой производительностью даже среди других key-value хранилищ.

Redis позволяет нам хранить данные в высокоуровневых структурах данных, такие как строки, хэши, списки, наборы. Это дает нам больше гибкости в отношении типа и объема информации, которую мы можем хранить в хранилище данных Redis.

Он также довольно дружелюбен для разработчиков, поскольку поддерживает большинство языков высокого уровня, таких как Python, JavaScript, Java, C / C ++ и PHP.

Установка Redis

Redis распространяется по лицензии BSD. Написан на ANSI C, поэтому легок и не имеет внешних зависимостей. Официальной версии для Window нет, но есть версия под Open Server. Но его можно запустить под Docker .

Для дальнейшего изучения Redis нам необходимо скачать и установить сервер Redis. Можно использовать инструкции с официальной веб-страницы. Так же под MacOS можно использовать Homebrew, а для Linux что типа apt. Для запуска Redis используется команда redis-server. Установка сервера Redis:

```
$ redis-server /usr/local/etc/redis.conf
```

У Redis есть Redis-CLI (Common Line Interface), который можно использовать для взаимодействия с данными напрямую на сервере Redis.

Чтобы проверить правильность установки Redis, запустите `redis-cli`, а затем введите команду `ping` в появившейся подсказке:

\$ redis-cli
127.0.0.1:6379> ping
PONG
127.0.0.1:6379>

Если сервер отвечает ответом PONG значит он готов к работе. По умолчанию сервер Redis работает на порту 6379, что видно в нашем приглашении. Чтобы выйти из `redis-cli` используйте команду `quit`.

Для вывода помощи по списку команд в консоли можно использовать команду `HELP @string`. Для вывода помощи по конкретной команде можно использовать команду `HELP APPEND` Например:

127.0.0.1:6379> HELP APPEND
APPEND key value
summary: Append a value to a key
since: 2.0.0
group: string

Далее приведен краткий список самых необходимых команд. Для изучения полного списка команд, если в этом есть необходимость, обратитесь к официальной документации.

Прежде чем начать использовать Redis в любом языке программирования нужно узнать о базовых командах и структурах используемых в Redis.

Команды

Прежде всего Redis это хранилище типа ключ: значение. И самые первые команды которую все изучают, это команды SET и GET:

SET - Команда используется для установки ключа и его значения, с дополнительными необязательными параметрами для указания срока действия записи значения ключа. Давайте установим ключ `foo` со значением «hello world». Параметр `EX` указывает время жизни объекта в секундах, `PX` в миллисекундах:

127.0.0.1:6379> SET foo "hello world"
OK
127.0.0.1:6379> SET foo1 "hello world" ex 5
OK

GET - Команда используется для получения значения, связанного с ключом. Если запись значения ключа превысила срок действия, будет возвращено `nil`:

По умолчанию все значение в Redis сохраняются как строки.

127.0.0.1:6379> GET foo
"hello world"
если истечет время жизни записи
127.0.0.1:6379> GET foo
(nil)

EXISTS - Эта команда проверяет, существует ли что то с данным ключом. Она возвращает 1 если объект существует или 0 если нет. Boolean типа в Redis нет.

127.0.0.1:6379> EXISTS foo
(integer) 1

FLUSHALL - Эта команда полностью удаляет все данные в текущем сеансе.

GETSET - Команда возвращает текущее значение и устанавливает новое. Используется для атомарного управления данными.

127.0.0.1:6379> SET foo "hello"
OK
127.0.0.1:6379> GETSET foo "world"
"hello"
127.0.0.1:6379> GET foo
"world"

DEL - Команда удаляет ключ и соответствующее значение:

127.0.0.1:6379> DEL foo
(integer) 1

APPEND - Команда добавляем в соответствующий ключ дополнительное значение. Возвращает количество символов итогового значения.

127.0.0.1:6379> SET foo "hello"
OK
127.0.0.1:6379> APPEND foo " world"
(integer) 11
127.0.0.1:6379> GET foo
"hello world"

KEYS - Возвращает все ключи из базы по указанному шаблону. Есть предостережение что в реальных приложениях эту команду лучше не использовать из-за того что она очень медленная.

127.0.0.1:6379> KEYS *
1) "boo"
2) "foo"

INCR / DECR - Инкремент / декремент. Если значение ключа integer (хотя в базе храниться все равно строка) можно увеличить или уменьшить значение на 1. Если использовать команду INCR с несуществующим значением то создаться новый ключ со значением 1.

127.0.0.1:6379> SET foo 1
OK
127.0.0.1:6379> INCR foo
(integer) 2
127.0.0.1:6379> INCR foo
(integer) 3
127.0.0.1:6379> DECR foo
(integer) 2

TTL - Когда ключ установлен с истечением срока действия (например SET foo EX 10), эту команду можно использовать для просмотра оставшегося времени:

127.0.0.1:6379> SET foo 10 EX 10
OK
127.0.0.1:6379> TTL foo
(integer) 6
127.0.0.1:6379> TTL foo
(integer) 5
127.0.0.1:6379> TTL foo
(integer) 2
127.0.0.1:6379> TTL foo
(integer) -2

PERSIST - Если мы передумаем об истечении срока действия ключа, мы можем использовать эту команду, чтобы удалить период истечения срока действия:

127.0.0.1:6379> PERSIST foo
(integer) 1
127.0.0.1:6379> TTL foo
(integer) -1
127.0.0.1:6379> GET foo
"bar"

RENAME - Эта команда используется для переименования ключей на нашем сервере Redis:

127.0.0.1:6379> RENAME foo foo2
OK
127.0.0.1:6379> GET foo
(nil)
127.0.0.1:6379> GET foo2
"bar"

Комплексные типы данных

Хеш таблицы

Redis позволяет в качестве значения так же использовать ключ: значение. Что по сути будет почти аналогией объектов из JavaScript или словари в Python. Для записи объекта используется команда HSET в следующем формате HSET имя_ключа имя_атрибута значение. Для чтения объекта используется команда HGET в формате HGET имя_ключа имя_атрибута. Команда HGETALL используется для получения

127.0.0.1:6379> HSET obj attr1 value1
(integer) 1
127.0.0.1:6379> HSET obj attr2 value2
(integer) 1
127.0.0.1:6379> HGET obj attr1
"value1"
127.0.0.1:6379> HGETALL obj
1) "attr1"
2) "value1"
3) "attr2"
4) "value2"

Множества

Не упорядоченная коллекция уникальных элементов. Аналог set в Python. Для добавление нового элемента во множество используется команда SADD. Для получения все элементов используется команда SMEMBERS. UNION используется для объединение множеств. SDIFF используется для вычитания из первого множества второго. SINTER возвращает общие элементы указанных множеств. SPOP удаляет и возвращает случайный элемент множества.

127.0.0.1:6379> SADD objects obj1
(integer) 1
127.0.0.1:6379> SADD objects obj2
(integer) 1
127.0.0.1:6379> SADD objects obj3
(integer) 1
127.0.0.1:6379> SADD objects obj1
(integer) 0
127.0.0.1:6379> SMEMBERS objects
1) "obj2"
2) "obj3"
3) "obj1"

Упорядоченные множества

Упорядоченная коллекция уникальных элементов. Для добавление нового элемента в упорядоченное множество используется команда ZADD. Формат ZADD имя_ключа порядковое_число_упорядочивания_множества значение

Команда ZRANGE возвращает срез данных множества

127.0.0.1:6379> ZADD objects 1500 value1
(integer) 1
127.0.0.1:6379> ZADD objects 1600 value2
(integer) 1
127.0.0.1:6379> ZADD objects 1700 value3
(integer) 1
127.0.0.1:6379> ZRANGE objects 0 -1
1) "value1"
2) "value2"
3) "value3"

Списки

Список это последовательность значений упорядоченных по порядку их создания. Аналог двух-стороннего стека (то есть стека в который можно добавлять с двух сторон). Списки обычно применяются для создания очередей.

Команда добавления элемента в список LPUSH

LRANGE — возвращает срез списка с левой стороны. Формат LRANGE имя_списка индекс_начало индекс_конца Указание -1 означает до конца списка.

LPOP — вернуть одно значение с левой стороны и удалить его из списка. Формат LPOP имя_списка. Аналогичные команды RPUSH, RRANGE, RPOP только для правой стороны.

127.0.0.1:6379> LPUSH obj1 value1
(integer) 1
127.0.0.1:6379> LPUSH obj1 value2
(integer) 2
127.0.0.1:6379> LPUSH obj1 value3
(integer) 3
127.0.0.1:6379> LRANGE obj1 0 -1
1) "value3"
2) "value2"
3) "value1"
127.0.0.1:6379>

Транзакции в Redis

Обычное определение транзакций для реляционных баз данных означает следующее: транзакции это группа команд с базой данных, которые должны либо полностью выполняться или в случае возникновения ошибки вернуть состояние базы данных в исходное состояние. В

Redis то же есть такое понятие как транзакции. Но означает немного другое. Транзакции в Redis это просто последовательное выполнение ранее записанных команд без возможности полноценного возвращения исходного состояния в случае ошибки исполнения.

С помощью команды MULTI можно начать запись команд. Далее введенные команды не исполняются а записываются в буфер. Это будет происходить до ввода команды на исполнения транзакции EXEC. Далее все ранее введенные команды будут исполнены один за другим. Команда DISCARD отмена записи команд транзакций. Если возникнет ошибка в процессе ввода команд вся транзакция не будет выполнена.

127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> SET obj1 value1
QUEUED
127.0.0.1:6379> SET obj2 value2
QUEUED
127.0.0.1:6379> EXEC
1) OK
2) OK
127.0.0.1:6379> GET obj1
"value1"

Механизм подписок PUS-SUB

Одно из основных преимуществ Redis от других key-value хранилищ заключается в том, что в Redis есть механизм подписок. То есть Redis можно использовать как сервер сообщений.

Одни клиенты подписываются на определенные каналы используя команду SUBSCRIBE имя_канала

Другие клиенты могут отправлять сообщения в этот канал используя команду PUBLISH имя_канала значение

Допустим один клиент подписывается на канал

SUBSCRIBE channel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel"
3) (integer) 1

Другой клиент что то отправляет в этот канал

PUBLISH channel "hello world"
(integer) 1

И в этот момент первый клиент получит это сообщение

SUBSCRIBE channel
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "channel"
3) (integer) 1
1) "message"
2) "channel"
3) "hello world"

Основы применение Redis в Python

Redis очень широко применяется в современной разработке ПО. Библиотеки поддержки есть для любого языка программирования.

Его используют в:

- Кеширование данных
- Чаты и системы обмена сообщенийДиспетчеризация любых данных
- Различные очереди задач
- Мониторинг различных данных

Кратко рассмотрим использование Redis в Python. Для этого первым делом загрузим библиотеку поддержки:

<code>#!/usr/bin/env python</code>
<code>import redis</code>

Далее подключимся к серверу

<code>client = redis.Redis(host='127.0.0.1')</code>

И далее можно уже начать попробовать использовать все ранее рассмотренные команды. Надеюсь они будут понятны без дополнительных пояснений:

<code>client.set('Language', 'Python')</code>
<code>print client.get('Language')</code>
<code>from time import sleep</code>
<code>client.set('Language', 'Python', ex=5)</code>
<code>print client.get('Language')</code>
<code>sleep(3)</code>
<code>print client.ttl('Language')</code>

Пример использования множеств
client.sadd('pythonlist', 'value1', 'value2', 'value3')
print client.smembers('pythonlist')
client.sadd('redislist', 'value1', 'value5', 'value6', 'value7', 'value8')
print "sinter", client.sinter('pythonlist', 'redislist')
print "sunion", client.sunion('pythonlist', 'redislist')
print client.scard('pythonlist')
Пример использования хеш таблиц
client.hset('Person', 'Name', 'Person1')
client.hset('Person', 'Health', '600')
client.hset('Person', 'Mana', '200')
print client.hgetall('Hero')

Заключение

Redis — это мощный и быстрый вариант хранения данных, который при правильном использовании может принести много преимуществ. Он не имеет крутой кривой обучения, поэтому с ним легко начать работать. Также поставляется с удобным инструментом CLI, который помогает нам взаимодействовать с ним с помощью простых и интуитивно понятных команд.